



中国研究生创新实践系列大赛
“华为杯”第二十三届中国研究生
数学建模竞赛

学 校 东南大学

参赛队号 26102860224

1. 周泽华

队员姓名 2. 单文杰

3. 苏永强

题 目： 算力约束下提升大语言模型能力的资源配置建模

摘要

扩大模型、增加训练数据和改善语料质量都需要消耗算力，其收益又受投入起点与处理费用影响。本文从语料评价出发，建立资源投入与训练损失的关系，求解有限预算下的配置，并利用历史评测分析能力前沿。

针对问题一：对 272,505 条记录的 22 个质量指标按字段含义统一方向，以分位截断、中位数填补和熵权法合成评分。成对得分差与 Kendall 系数分别衡量样本内分歧和总体排序一致性：冲突比例为 21.9%，低于独立参照的 28.9%，但 Kendall 系数仅为 0.076；域内中位数标签只表示相对位置。领域均分以 arxiv 最高、github 最低。对于配比与损失，线性岭保留系数解释，平方根嵌入核岭用于预测及训练配方凸包内推荐。在 1M 检验集，平均逐域 R^2 从 0.6186 提高至 0.9454，综合 RMSE 从 0.5232 降至 0.2256。核岭在固定域和配比上限约束下给出预测加权损失 4.6444，参考配方为 4.8173；这一模型内收益尚需训练实验验证。跨尺度绝对误差较大，故后续仅将配比损失差作为条件修正。质量指数与领域占比线性相关，其重复加入不能识别质量的独立作用。

针对问题二：采用稳健非线性最小二乘拟合经典标度律，并在四种质量项中选择加性缺口形式。经典模型与广义联合模型的 R^2 分别为 0.9999998、0.9896，跨族和文献记录的 MAPE 则为 8.8%、8.0%，显示同源拟合精度不能直接推广到其他来源。模型导数用于计算弹性与局部替代率，完整等损失方程用于较大幅度的投入调整。质量均为 0.6 时，1B 参数、100B tokens 与 12B 参数、300B tokens 两个起点的质量-参数替代率分别为 2.844 和 76.838 B/单位质量。领域系数的中心化相似度描述作用方向，核岭相对参考配方的损失差另作为配比修正；质量效应与跨尺度迁移仍受半合成实验和尺度假设限制。

针对问题三：将基础训练、质量处理及注意力费用计入预算，以广义模型的预测损失为目标，采用 SLSQP 求解。幂函数型质量成本、参考配比及 2048 tokens 上下文下， 10^{19} FLOPs 对应 0.225B 参数、6.9B tokens，质量维持在基线 0.549； 10^{22} 、 10^{24} FLOPs 下质量均为 1，参数与数据配置分别为 6.091B、229.4B tokens 和 44.939B、3417.8B tokens。扫描显示维持基线、增加质量和达到上界三个阶段，首次投入与饱和的有效网格点为 1.58×10^{19} 、 7.08×10^{19} FLOPs。成本式给出 30,000 tokens 的注意力与基础训练等费用长度。固定 10^{22} FLOPs，上下文增至 131072 tokens 时参数量降至 2.705B，损失约增加 6.0%。不同质量成本与尺度映射改变投入门槛，配比收益的预算等价也依赖迁移幅度。

针对问题四：从许可与字段筛选后的 2,346 条记录出发，以六项评测均值描述能力。损失与得分桥接的 R^2 仅为 0.26，因此只解释关联方向。合并横截面回归中，参数量每扩大十倍对应 13.84 分差异，控制规模后的年度组差为 4.84 分，后者不表示同一模型的年增长。将前沿评分与相应规模配对后，2019—2025 年窗口的规模关联项占 51.4%，剩余项占 48.6%；2024 年 6—9 月窗口的剩余项占 95.4%，其中仍含样本构成等未观测影响。以 2025 年 3 月为预测基点，logistic 的 12、24 个月条件预测为 56.3、56.6 分，曲线重拟合的 5%—95% 分位范围为 53.6—64.5、53.8—66.5。时间留出未显示稳定预测优势，窗口删减也会改变结果。增长参数减半时两点为 55.6、56.3 分；另一结构情景将设定年增量减半时为 53.4、55.8 分，两类情景的基期水平不同。C8 子任务与 C4 资源记录进一步用于比较能力短板和数据覆盖。

各项配置和外推均对应明确的成本、评分与观测窗口。质量处理应随预算和边际费用安排，模型扩容则须结合可用数据判断；跨来源标定不足的部分保留为条件分析。

关键词： 大语言模型；标度律；数据质量；资源配置优化；能力前沿预测

目录

一、引言	3
1.1 问题背景	3
1.2 问题重述	3
二、总体分析	3
三、模型假设	4
四、符号说明	5
五、模型的建立与求解	6
5.1 问题一：数据质量评价与领域配比建模	6
5.1.1 具体分析	6
5.1.2 模型准备	7
5.1.3 模型建立	11
5.1.4 模型求解	14
5.2 问题二：跨维度数据融合与广义标度律	24
5.2.1 具体分析	24
5.2.2 模型准备	24
5.2.3 模型建立	26
5.2.4 模型求解	29
5.3 问题三：算力约束下的资源配置与结构性转移	36
5.3.1 问题分析与模型准备	36
5.3.2 模型建立	37
5.3.3 模型求解	40
5.4 问题四：技术演进分析与能力前沿条件预测	44
5.4.1 能力度量与数据组织	44
5.4.2 损失与评测得分的分层联系	47
5.4.3 规模关系与历史增量分解	48
5.4.4 前沿曲线与条件预测	50
5.4.5 从综合前沿到具体任务瓶颈	53
5.4.6 宏观资源证据与预测边界	55
六、模型检验与分析	56
6.1 误差分析	56
6.2 配方决策与时间外推检验	58
6.3 灵敏度分析	59
6.4 各问结果的衔接与使用条件	61
七、模型的评价	61
7.1 模型的优点	61
7.2 模型的缺点	62
八、模型的改进与推广	62
8.1 模型的改进	62
8.2 模型的推广	63
参考文献	63

一、引言

1.1 问题背景

在给定算力预算 C 下，大语言模型的参数量 N 、训练数据量 D 、语料质量 Q 与领域配比 p 需要共同安排。扩容和增加数据直接增加计算量，筛选语料也有成本；配置问题因而取决于各项投入能够换取多少损失下降。标度律提供了规模与损失的经验关系 [1, 2]，但同一函数形式并不保证相同的配置结论：Chinchilla 的复现研究发现，数据重建和参数估计均会影响结果 [3]。本文据题目附件重新拟合，并分别报告同源拟合与跨来源检验。

继续扩大训练还面临语料供给问题。随着数据需求增长，高质量文本是否足够，成为“数据墙”研究关注的内容 [4]。语料的组织和使用方式也会影响任务表现。Gunasekar 等以教材式文本和练习组织代码训练数据，在较小模型上取得了有竞争力的代码评测结果；这一证据针对代码任务，不能直接推广到全部能力维度 [5]。DataComp-LM 则通过受控的数据筛选、去重和混合实验，比较数据组织对下游评测的影响 [6]。这些研究为将质量和领域构成纳入资源分析提供了依据，但具体收益仍需结合本文附件估计。语料处理、模型扩容和上下文延长各有费用，最终还要放到同一预算下比较。

附件分别提供质量信号、配方实验、训练日志与能力评测。总数据量决定训练使用多少文本，领域配比决定各来源所占份额，两者不能相互替代。同一领域内部的文本质量也有差异，某领域的损失较低不等于其语料质量更高。为此，本文先按观测对象确定附件用途，再以领域映射和损失模型连接语料评价与资源配置。

预算模型将交叉熵作为连续优化目标，同时计入扩容、数据处理与注意力费用。任务评测还受题目构成和评分尺度影响，故能力分析直接使用相应评测记录，并另行检验损失与得分的联系。资源配置与前沿预测由此各有数据依据，历史分解描述条件关联，不将未解释增量直接归为某种技术的因果收益。

1.2 问题重述

问题 1：用 22 个质量指标构建评分 Q ，使其能够分别评价文本、语料及领域。在统一指标方向的同时，说明评价冲突的识别和处理办法，并分析 17 个训练领域的占比如何对应交叉熵损失。

问题 2：在经典标度律中补充质量 Q 和领域配比 p 的作用，得到损失关于 N 、 D 、 Q 、 p 的关系。由此计算边际效用与弹性，比较不同投入，并确定质量提升能够与参数扩张等效的条件。

问题 3：给定算力预算，求参数量 N 、数据量 D 、配比 p 及质量 Q 的配置方案；再把预算扩大若干数量级，观察资源投入的重点是否发生转移。

问题 4：利用历史记录区分能力增长中的规模扩张项与非规模技术进步项，计算各自的增量和占比；在此基础上估计开源大语言模型未来 12 至 24 个月的能力前沿。

二、总体分析

我们按“语料评价—损失建模—预算配置—能力分析”的顺序处理四问：先建立质量和配比的度量，再比较各项投入的作用，最后在费用约束下求解配置，并分析评测能力的变化。图1列出了各问使用的资料、计算方法和结果去向。

问题一先合成文本质量评分，再分析配方与损失。成对标准化得分差刻画样本内的评价分歧，Kendall 协同系数衡量指标排序的一致程度。配方实验另用线性岭解释领域作用，用核岭完成预测与训练凸包内推荐。问题二、三按各自用途接收质量基线、领域系数及核岭配比损失差。

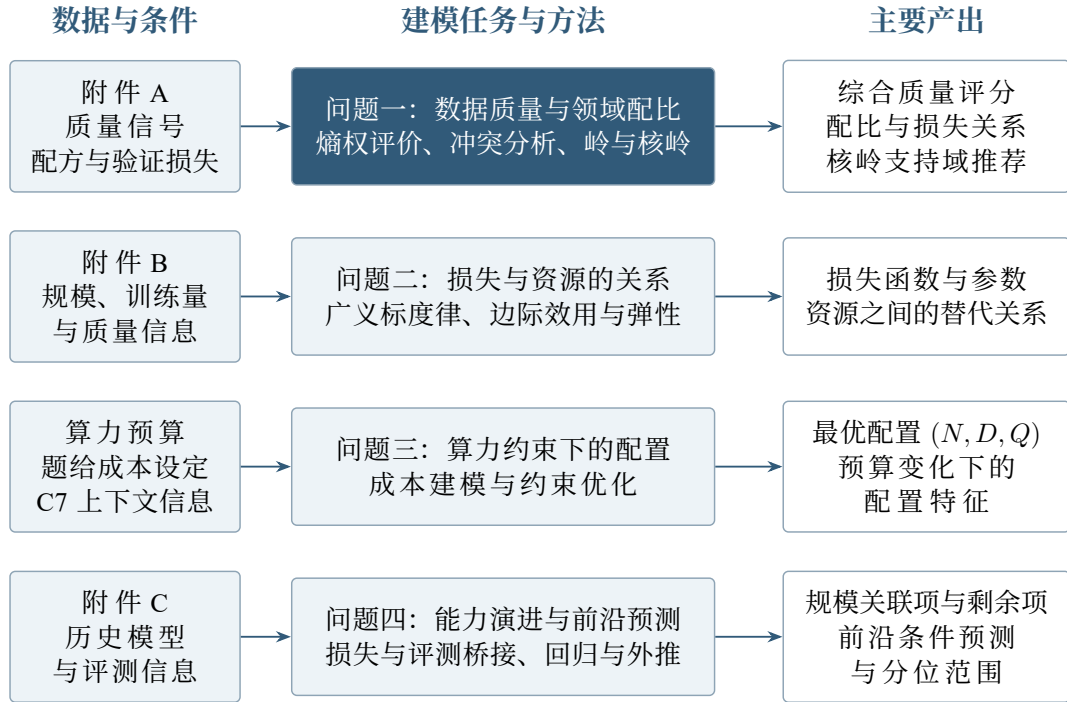


图 1 四问建模流程

问题二在 Chinchilla 形式中加入质量缺口项，在同一损失尺度上比较三类投入，写成 $L(N, D, Q) = E + AN^{-\alpha} + BD^{-\beta} + C(1 - Q)^{\gamma}$ 。取 $Q = 1$ 时，新增项为零，函数回到经典形式。接着分别对参数、数据和质量求导，比较它们带来的损失降幅，并求解等损失条件下的替代关系。

对于问题三，我们将题设费用写入约束，使配置同时满足损失目标与预算限制。基础训练计为 $6ND$ ，质量处理计为 $D[g(Q) - g(Q_0)]_+$ ，注意力计算计为 ηNDL_{ctx} 。给定配比后，用 SLSQP 计算不同预算的方案，再通过预算扫描寻找质量投入发生变化的位置。上下文的影响则从费用比入手，以 $L_{\text{crit}} = 6/\eta$ 确定注意力与基础训练开销相等的长度。

对于问题四，我们先按数据可比程度分别拟合损失与得分的关系，再以含年份控制项的合并横截面回归描述规模关联，核算前沿增量中的规模关联项与剩余项，再比较 logistic 曲线与结构情景的预测。综合分数之外，C8 用于查看哪些子任务仍较薄弱，C4 用于观察宏观资源变化和记录覆盖情况。

问题三使用问题一给出的 Q_0 、配比修正 $M(\mathbf{p})$ ，以及问题二的标度律参数。这里暂按两类质量变量数值相同、配比效应在损失尺度上可迁移处理，并分别检查质量映射和配比收益折减的影响。问题四另检验训练损失与评测能力能否对应：两者虽大体同向改善，换算误差却较大。因此，历史分解和未来预测直接从评测得分计算，不把前三问的损失降幅直接折成能力分数。

估计、检验与应用的区分。问题一、二从对应实验估计响应关系，在留出数据和其他来源上检验；问题三接收估计参数与费用设定，核对配置的预算可行性；问题四直接使用历史评测建立条件关联与前沿情景。质量评分、损失和能力得分各有尺度，接口传递须保留原有含义。第六、节集中比较预测误差、约束残差和敏感性，它们分别回答预测是否准确、方案是否可行以及结果依赖哪些假设。

三、模型假设

结合附件覆盖范围与题设成本，我们作出以下假设：

- **假设 1:** 在附件 A 的训练配方范围内, 17 个领域占比的线性组合能够近似描述验证损失; 换到其他参数规模时, 允许损失的整体尺度发生变化。
- **假设 2:** B6、B7 的质量随评分提高而提高, 质量不足带来的损失可作为单独一项加到经典标度律中。其他来源须先核对评分含义和方向, 再确定用法。
- **假设 3:** 按题设将总算力分为基础训练、质量提升和长文本注意力三项; 质量费用采用附录 B 的函数, 上下文采用 C7 的给定长度, 不参与自由优化。
- **假设 4:** 当前窗口的能力前沿可由 logistic 函数近似, 规模与得分的条件关联可由含年份项的合并横截面回归描述; 沿用这些关系的未来结果视为条件情景。

线性近似用于比较领域系数, 核岭用于同尺度预测和训练凸包内的配方选择。相同配方迁移到其他参数规模时, 仍需检验其损失水平; 模型选择与支持范围约束均不能替代新配方的训练实验。

加性质量项便于比较各类投入的边际作用, 其与规模可分离的形式属于建模假设。附件 A 的综合评分与附件 B 的实验质量变量并无配对标定, 问题三暂按两者数值相同传递基线, 并另用单调映射作敏感性对照。评分方向一致只能支持排序, 不能证明两种尺度相同。

预算优化沿用题设费用体系。参数量和数量共同决定基础训练开销, 质量费用随处理量累计, 上下文长度则作为给定情景进入注意力开销。由此得到的配置是这些条件共同作用的结果, 更换费用关系时应同时观察配置与损失的变化。

C1 的不同年份记录没有形成同一模型的跨年追踪, 因而回归系数描述的是模型之间的条件差异。前沿分解将规模关联项以外的增量记为剩余项, 其中也含样本构成和遗漏因素。前沿曲线则描述高分记录的累积变化; 我们保留其观测窗口, 并通过窗口删减和时间留出检查外推对平台段的依赖。

四、符号说明

主要符号列于表1。问题二拟合时以十亿参数和十亿 tokens 计量规模, 问题三计算成本时转换为参数个数和 token 数。

表 1 主要符号及含义

符号	含义
N	模型包含的参数数量
D	用于模型训练的数据总量
L, E	验证交叉熵损失、参考配比下的渐近损失基准
Q	当前模型采用的质量变量, 分值越高则质量越好
Q_A, Q_B, h	附件 A 的综合评分、附件 B 的实验质量变量及假设映射 $Q_B = h(Q_A)$
Q_0	质量基线; 问题三沿用既定参考配比的加权质量, 作为外生成本基线
p	各领域训练占比构成的向量, 各项非负且总和为 1
θ, λ_k	配比核函数的距离参数与核岭正则强度
A, B	参数规模项、数据规模项的标度系数
α, β	参数规模项、数据规模项的幂指数
C, C_q	质量缺口系数, 问题三记为 C_q
γ	质量缺口的幂指数
$\varepsilon_N, \varepsilon_D, \varepsilon_Q$	损失对参数量、数据量和质量的弹性

表 1 主要符号及含义 (续)

符号	含义
$M(\mathbf{p})$	假设响应可迁移时, 相对参考配比的损失平移修正
$C_{\text{train}}, C_Q, C_{\text{attn}}$	基础训练、质量提升和注意力开销
C_{budget}, C_b	总算力预算, 问题三简记为 C_b
$g(Q)$	单个 token 的质量处理成本
η	注意力开销系数; 参数个数作为计数代入成本公式
$L_{\text{ctx}}, L_{\text{crit}}$	上下文长度、注意力与训练开销相等时的长度
$\lambda(L_{\text{ctx}})$	注意力开销与基础训练开销的比值
$\tau(Q)$	质量处理成本折算的等效附加参数规模
$\Theta(C_b)$	质量提升完成度
$F(t), K$	能力前沿及当前窗口拟合的 logistic 渐近水平
t, t_0	以 2019 年为原点的时间、logistic 拐点时间
r	logistic 增长率
b_P	参数量每扩大十倍对应的回归得分变化
b_t	控制规模后, 不同年份记录的条件得分差异系数
α_t	合并横截面回归的年份控制项
$\bar{B}$	Leaderboard 六维平均得分
$Q_i, Q_d, \bar{Q}(\mathbf{p})$	依次表示样本评分、领域平均评分和按训练配比加权的质量指数
$\mathbf{P}, \mathbf{L}$	分别由各组训练配比及其对应的验证损失构成的矩阵
$\mathbf{A}, \mathbf{b}$	配比回归中的系数矩阵和截距向量
λ	配比回归的岭惩罚系数, 设为 10^{-3}
τ	判断两个指标的标准化得分是否冲突的阈值, 设为 1.5
w_j, e_j	指标 j 对应的熵权系数与信息熵值
W	Kendall 协同系数, 用于衡量指标排序的一致程度
ρ_c, ρ_0	指标评价的冲突比例及指标独立条件下的参考冲突比例
R^2	决定系数
RMSE、MAE	均方根误差、平均绝对误差
MAPE	平均绝对百分比误差

五、模型的建立与求解

本文使用腾讯 WorkBuddy 接入的 DeepSeek-V4-Flash 辅助文段相关工作, 并使用 OpenAI Codex (GPT-5.6-Luna, 2026-07-09) 辅助图表展示、排版检查、程序整理、数据整理与结果复核, 所用工具、模型及版本信息见各问的辅助分析说明。正文表达与段落衔接另经 OpenAI Codex 辅助润色。

5.1 问题一：数据质量评价与领域配比建模

5.1.1 具体分析

面对 22 个取值形式和评价侧重点不同的质量指标, 我们先统一评分方向与尺度, 再分析同一文本在各指标下的评价差异。具体采用熵权法计算文本质量并汇总语料和领域

评分，以成对标准化得分差和 Kendall 协同系数分别刻画局部分歧与整体排序一致性。对于配方实验，我们采用线性岭回归解释 17 个训练领域与 13 个验证领域之间的配比-损失关系 [7]，并以平方根嵌入核岭增强同尺度预测。线性有界解作为解释对照，核岭在闭合训练配方的凸包内给出正式配方；二者均按相同加权目标接受检验。

图2给出了质量评价与配比建模的分析流程。质量评价以 A1–A3 为输入，依次完成缺失值填补、列表值标量化、指标同向化与归一化，再计算质量得分和冲突情况。配比建模先按索引匹配训练配方与验证损失，比较线性岭与核岭的训练五折误差，并在 1M、60M、1B 记录上检验预测；线性模型用于系数解释，核岭在训练配方支持范围内给出有界推荐，10B、70B 外推表仅作参考。两部分涉及的领域分类通过映射表建立对应关系。

我们将质量评价与配比回归分别计算，是因为二者的观测单位不同：前者的一行记录对应一段文本，后者则对应一次配方实验。文本评分不能替代实验损失，领域平均损失也不能代表该领域全部文本的质量。完成两部分计算后，再通过领域映射比较文本评价与训练难度的关系。

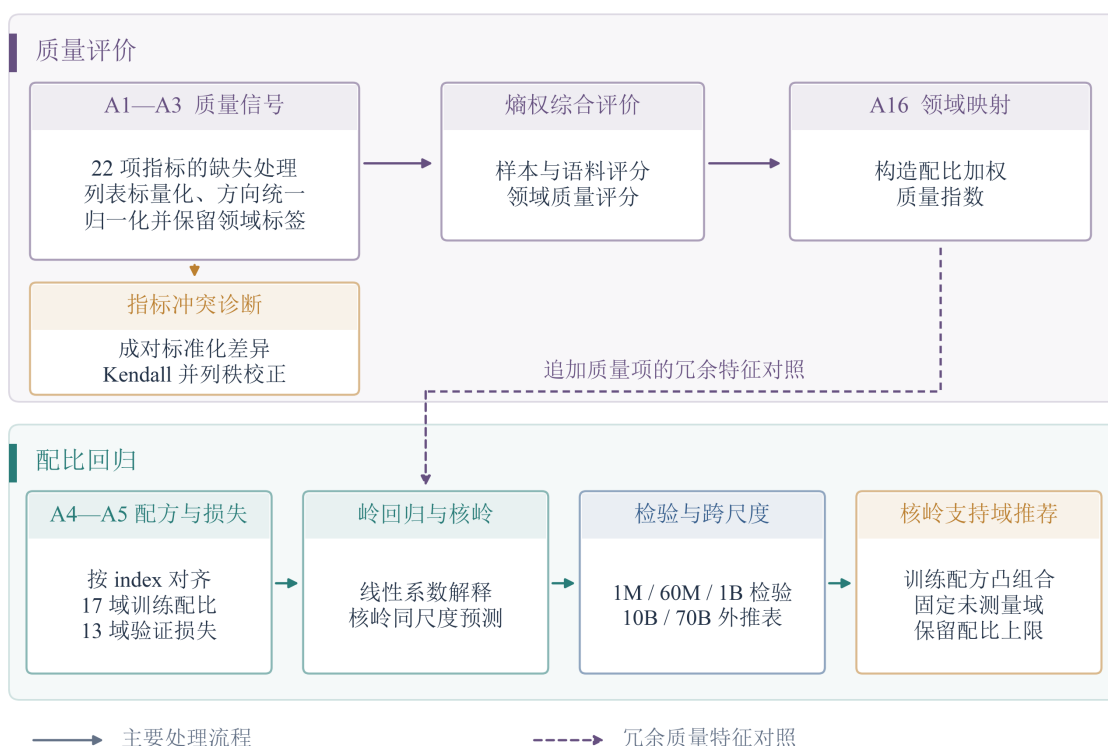


图 2 问题一分析流程

5.1.2 模型准备

1. 数据来源与预处理

A4 与 A5 分别记录训练配方及其对应的验证损失，通过 index 关联。A4 包含 The Pile 的 17 个领域，各项占比非负，理论上每行之和为 1，附件小数舍入使训练行和最大偏差为 0.004[8]；A5 提供 13 个领域的交叉熵损失。其中，nih_exporter、enron_emails、europarl 和 philpapers 参与训练配方，但缺少对应的验证列。我们将这部分领域记为“间接影响域”：拟合时保留其占比，但不将其作为独立验证目标。

按索引匹配后，同一次实验的配方与损失逐行对应。训练领域比验证领域多，说明训练来源的覆盖范围大于评价任务的覆盖范围；缺少验证列的领域仍可能影响其他任

务。因此，我们在回归中保留这些领域，在候选优化模型中则固定其参考占比，以区分能够直接评价与只能间接分析的调整。

A1–A3 提供 SlimPajama 语料的 22 个质量指标 [9]。列表值先转为标量，非有限值视为缺失并按全量记录中该指标的中位数填补。设处理后的值为 x_{ij} 。对于过长或过短均不合适的长度型指标，先以稳健中心接近度作变换；其余指标保留原值：

$$u_{ij} = \begin{cases} \exp\left(-\frac{|\ell_{ij} - m_j|}{2.5(\text{MAD}_j + 10^{-9})}\right), & \text{长度型指标,} \\ x_{ij}, & \text{其余指标,} \end{cases} \quad (1)$$

其中 $\ell_{ij} = \ln(1 + \max(x_{ij}, 0))$, $m_j = \text{median}_i \ell_{ij}$, $\text{MAD}_j = \text{median}_i |\ell_{ij} - m_j|$ 。随后对每个变换后指标，取 A1–A3 全量记录的第 1 和第 99 百分位数 a_j 、 b_j ，进行分位数截断归一化，再对负向指标取补：

$$v_{ij} = \text{clip}_{[0,1]}\left(\frac{u_{ij} - a_j}{\max(b_j - a_j, 10^{-12})}\right), \quad x'_{ij} = \begin{cases} 1 - v_{ij}, & \text{负向指标,} \\ v_{ij}, & \text{其余指标.} \end{cases} \quad (2)$$

其中 $\text{clip}_{[0,1]}(v) = \min(1, \max(0, v))$ ；分位数按相邻顺序统计量线性插值得到。长度型指标也经过这一步缩放。各指标最终均取值于 $[0, 1]$ ，且越大越好。与直接使用最小、最大值相比，分位数端点降低了极端值对缩放区间的影响，代价是端点外的差异不再保留。各领域及抽样、扩展集沿用同一组全量端点与权重，保证评分可比较。含多个有序类别的列表字段通过 softmax 将类别 logits 转为概率，再计算归一化期望级别；单元素列表直接取其数值。qurater 的四个分量分别为写作风格、所需专业知识、事实知识和教育价值，属于不同维度而非互斥等级 [9]。设其原始值为 q_{ir} ($r = 1, \dots, 4$)，各维缺失先用该维全量中位数填补，再以对应的第 1、99 百分位数 a_r^q, b_r^q 计算

$$x_{i,\text{qurater}} = \frac{1}{4} \sum_{r=1}^4 \text{clip}_{[0,1]}\left(\frac{q_{ir} - a_r^q}{\max(b_r^q - a_r^q, 10^{-12})}\right). \quad (3)$$

聚合值作为 22 个指标之一，继续参与上述外层分位数归一化。我们将四维同向、等权聚合，把知识密度与专业要求纳入广义质量；这一设定并不意味着专业要求越高就越适合所有训练任务。数据卡只规定字段含义，聚合权重属于本文的评价假设。表2列出各字段的含义、方向及权重。

表 2 质量指标的含义、评价方向及熵权

指标字段	指标含义与判向依据	类型	方向	熵权 w_j
fineweb_edu	文本用于教育的价值	列表 (1)	正向	0.0433
fluency_en	英语流畅度（索引 1= 流畅，列表 (2) 已核验）	列表 (2)	正向	0.0675
modernbert_cleanliness	文本内容的洁净程度	列表 (6)	正向	0.1420
modernbert_readability	文本的易读程度	列表 (6)	正向	0.0562
modernbert_reasoning	内容体现的推理程度	列表 (6)	正向	0.1502
modernbert_professionalism	内容的专业程度	列表 (6)	正向	0.0392
dsir_books	相对于书籍领域的重要性 权重	标量	正向	0.0081

表 2 质量指标的含义、评价方向及熵权 (续)

指标字段	指标含义与判向依据	类型	方向	熵权 w_j
dsir_wiki	相对于维基领域的重要性权重	标量	正向	0.0080
dsir_math	相对于数学领域的重要性权重	标量	正向	0.0086
qurater	四维质量: 分维归一化后等权聚合	列表 (4)	正向	0.0586
ad_en	广告含量 (索引 1= “无广告”, 正向; 依据: 原文抽查)	列表 (2)	正向	0.0071
rps_doc_word_count	文档包含的词总数	标量	长度型	0.0511
rps_doc_num_sentences	文档包含的句子总数	标量	长度型	0.0518
rps_doc_unigram_entropy	用一元词熵衡量词汇多样性	标量	正向	0.0343
rps_doc_frac_unique_words	不同词项所占比例	标量	正向	0.0533
rps_doc_frac_no_alph_words	不含字母的词项所占比例	标量	负向	0.0284
rps_doc_frac_chars_top_2gram	最高频词 2-gram 覆盖的字符比例	标量	负向	0.0128
rps_doc_frac_chars_top_3gram	最高频词 3-gram 覆盖的字符比例	标量	负向	0.0118
rps_lines_uppercase_letter_fraction	大写字符占全部字符的比例	标量	负向	0.0313
rps_lines_ending_with_terminal_punctution_mark	以终止标点结束的行所占比例	标量	正向	0.0871
rps_lines_numerical_chars_fraction	数字字符占全部字符的比例	标量	负向	0.0124
rps_doc_mean_word_length	词项长度的平均值	标量	长度型	0.0369

在统一指标方向后, 以加权和计算样本质量 Q_i , 再对同一领域内的样本取均值, 得到领域质量 Q_d :

$$Q_i = \sum_{j=1}^{22} w_j x'_{ij}, \quad Q_d = \frac{1}{n_d} \sum_{i \in \mathcal{D}_d} Q_i, \quad \sum_j w_j = 1 \quad (4)$$

表2中, 推理与洁净程度对综合评分的影响较大: modernbert_reasoning 的权重为 0.150, modernbert_cleanliness 为 0.142。这里的 w_j 依据样本分布中的信息差异计算, 不能直接代表指标本身的重要程度。极端左偏的分布也可能获得较高熵权, 因此第6.3节将通过更换赋权方式, 考察评分结果的变化。

二分类字段的类别索引从 0 开始, 表中的索引 1 对应列表第二项。

字段缺失情况见表3。逐项检查后, 采用对应指标的中位数填补缺失值, 使填补值处于分布的中间水平, 避免将缺失记录直接赋为高分或低分。

表3 A1-A3 中 272,505 条记录的质量字段缺失统计

缺失字段	缺失情况	涉及记录数
modernbert_reasoning	该记录中此字段的 6 个分量均为 NaN	13
modernbert_professionalism	该记录中此字段的 6 个分量均为 NaN	6
其余 20 个指标	未发现缺失值	0

2. 方法选择及其依据

选择综合评分方法时，我们同时考察权重含义与计算结果。表4对熵权法、等权平均及首主成分非负载荷加权对照进行比较。评价分歧从两个层面考察：成对标准化得分差用于识别单条文本的指标分歧，Kendall 协同系数用于衡量整体排序的一致程度。配比拟合以线性岭回归为解释基线，以平方根嵌入核岭为非线性预测模型，通过共同五折检验其误差差异。

表4 不同赋权方式下的样本得分及领域排序比较

赋权方法	与熵权得分的相关系数	领域排序相关性 Spearman 系数，参照熵权法	样本平均得分
熵权法	1.000	1.000	0.491
等权平均	0.875	0.643	0.637
首主成分非负载荷加权	0.927	0.679	0.496

表4显示，熵权与等权评分的样本相关系数为 0.875，而领域均分排序的 Spearman 系数为 0.643。两种赋权方式的总体趋势接近，但具体领域的名次有所变化。据此，我们采用熵权法，以样本中的信息差异确定权重，同时通过赋权对照保留对领域名次变化的检验。对照方案先对标准化指标矩阵取首主成分载荷，统一符号使载荷总和非负，再将负载荷截为零并归一化，最后对原归一化指标加权。因此它是首主成分非负载荷加权评分，不是标准 PCA 的投影得分；这种处理保证各项权重非负，但改变了主成分原有的方差最大化含义，本文仅将其作为赋权对照。

我们同时比较样本得分和领域名次，因为两者的稳定性并不等价。赋权改变后，多数文本的相对得分可能仍然接近，但均值相近的领域会交换名次。因此，样本相关用于判断评分的共同趋势，领域秩相关用于判断汇总排序是否保持。

配比预测的方法取舍围绕误差、解释与计算组织展开。普通最小二乘在占比和为 1 的设计中存在截距与系数不可唯一识别的问题；线性岭通过正则化保留稳定的相对系数，但难以充分表达非线性响应。平方根核岭把配方之间的距离纳入预测，在保留零占比的同时，13 个验证域共用一套核矩阵和两个待选超参数，便于统一验证。高斯过程也能刻画非线性，并提供模型条件下的预测不确定性；若逐域采用 ARD 核，则还需分别估计各维长度尺度、核幅度及噪声，并检查边际似然优化的收敛情况。

我们保留线性岭解释领域间的相对作用，以核岭提高同尺度预测精度，并在训练配方支持范围内搜索配方。本问以点预测为主，未估计高斯过程后验区间；选择依据是共同五折及同尺度检验的实测改善、核矩阵共享的计算便利与跨问角色划分，并不意味着核岭在所有数据或不确定性估计任务中更优。

配比回归需考虑占比约束对系数估计稳定性的影响。17 项占比之和为 1，任一占比增加均需由其他占比的减少补偿，普通回归可能因此产生不稳定估计。岭回归通过引入 L_2 惩罚缓解这一问题，同时保留线性系数的可解释性。

5.1.3 模型建立

1. 综合质量评分

将预处理后的 m 条文本表示为矩阵 $\mathbf{X} = (x'_{ij})$ ，每行包含 22 项质量值。以 Shannon 信息熵 [10] 描述各列得分的分布，再用归一化熵 e_j 的信息差异确定权重 w_j ：

$$e_j = -\frac{1}{\ln m} \sum_{i=1}^m f_{ij} \ln f_{ij}, \quad f_{ij} = \frac{x'_{ij} + \varepsilon}{\sum_i (x'_{ij} + \varepsilon)}, \quad w_j = \frac{1 - e_j}{\sum_{j=1}^{22} (1 - e_j)} \quad (5)$$

式中，每列的 f_{ij} 之和为 1，表示该指标的归一化得分在样本间如何分配； ε 用来使对数项有定义。当一列在全部样本上的取值相同， $f_{ij} = 1/m$ ，代入即可得到

$$e_j = -\frac{m(1/m) \ln(1/m)}{\ln m} = 1.$$

此时该列没有区分当前样本的能力，其信息差异 $1 - e_j$ 为零。若得分更多集中于部分样本，信息熵会降低，该列在加权时获得更大份额。这个过程把权重与当前样本分布联系起来，而指标应取正向还是负向，仍须由指标含义决定。

权重非负且总和为 1，因而综合得分是各项同向评分的凸组合。高权重指标的变化会更明显地影响总分，但一项指标的高分不会自动抵消对其他指标含义的核对。领域均值则是在同一评分规则下进行第二次汇总，其作用是比较整类语料的总体位置，不能描述其中每条文本的情况。因此，我们同时报告领域均值和标准差，保留领域内部的评分差异。

除文本指标外，验证损失还可用于考察领域拟合难度。在平均配方下，损失较低的领域更易被当前模型拟合。为将这一差异与附件 B 联系起来，定义基于损失的质量代理值：

$$Q_d^{(\text{loss})} = 1 - \frac{\bar{L}_d - \min_d \bar{L}_d}{\max_d \bar{L}_d - \min_d \bar{L}_d} \quad (6)$$

$\bar{L}_d$ 为领域 d 的平均交叉熵损失。代理值的计算范围为具有损失列的 13 个领域，文本质量评价则覆盖 A1–A3 的 7 类语料。本文依据 A16 建立两套分类的对应关系，并在结果部分比较其排序，避免直接混用含义不同的评分。

这里的端点由本次参与比较的领域确定：平均损失最小者对应代理值上端，平均损失最大者对应下端。因此，代理值提供的是当前验证范围内的相对位置。它与从文本指标得到的 Q_d 都采用便于比较的数值范围，但生成过程不同。前者随验证模型和任务而变化，后者由文本信号与赋权规则决定。将二者并列展示，可以检查语料评价与训练难度是否给出相近的领域次序。

2. 质量冲突的定义与处理

同一文本在两个指标上的得分差异较大时，需进一步检查其综合评价。为保证指标差值具有可比性，先对同向指标进行标准化，再定义样本 i 在指标 j 、 k 之间的距离：

$$\delta_i(j, k) = |z_{ij} - z_{ik}|, \quad z_{ij} = \frac{x'_{ij} - \mu_j}{\sigma_j}, \quad \mathbf{1}\{\delta_i(j, k) > \tau\} \quad (7)$$

以 $\tau = 1.5$ 标记冲突，将全部样本中的指标对一并统计，得到全样本冲突率：

$$\rho_c = \frac{1}{m \binom{22}{2}} \sum_{i=1}^m \sum_{j < k} \mathbf{1}\{\delta_i(j, k) > \tau\} \quad (8)$$

为比较实际冲突比例，假定各指标为相互独立的标准正态变量，得到参照值 $\rho_0 = 2\Phi(-\tau/\sqrt{2})$ 。当 $\rho_c/\rho_0 < 1$ 时，实际冲突比例低于该参照。另将 22 个指标视为评价者、 m 条文本视为评价对象，采用 Kendall 协同系数 [11] 衡量整体排序的一致程度：

$$W = \frac{12 \sum_i (R_i - \bar{R})^2}{22^2(m^3 - m) - 22 \sum_{j=1}^{22} T_j}, \quad T_j = \sum_g (t_{jg}^3 - t_{jg}) \quad (9)$$

其中 r_{ij} 为指标 j 对样本 i 给出的平均秩， $R_i = \sum_{j=1}^{22} r_{ij}$ ； t_{jg} 为指标 j 的第 g 个并列组所含记录数。分位截断和离散取值会产生并列，故在分母中扣除相应修正项；无并列时各 $T_j = 0$ 。辅助统计量取 $\chi^2 = 22(m-1)W$ ，以自由度 $m-1$ 的卡方分布计算名义近似概率。由于记录存在重复、指标之间也可能相关，该概率不作为独立样本下的显著性证据，解释以 W 的大小为主。

对于已标记的文本，进一步比较加权评分 Q_i 与本领域中位数，记录其相对位置。这一标签汇总了指标的加权信号，不是逐项多数表决，也不能裁定发生分歧的某个指标是否失真。本文将冲突处理分为分歧诊断与相对定位，聚合评分仍按统一规则进入后续分析；缺少人工或任务效度依据时，不把相对标签解释为已经消除了评价矛盾。

冲突率中的统计单位是“文本与指标对”的组合。对同一条文本，各指标对可以给出不同结果；把这些结果平均后，才得到总体比例。独立参照的推导也在这个层面进行：两个独立标准正态变量之差的方差为 2，将差值除以 $\sqrt{2}$ 后转为标准正态变量，双侧尾部概率便给出 ρ_0 。该参照是为便于比较而设置的理想情形，实际指标是否严格服从正态分布并不由这一公式证明。

Kendall 系数使用的是各指标对全部文本给出的排序。若两个指标的数值尺度不同、但对文本高低的排列接近，排序一致性仍可能较高；成对标准化差值则保留了每条文本偏离各指标均值的程度。两项统计量分别从局部评分差异和整体排列关系描述分歧，后续结果需要结合两者解释。

3. 领域配比的线性混合回归

将训练配方整理为矩阵 $\mathbf{P} \in [0, 1]^{n \times 17}$ ，将各组对应的验证损失整理为矩阵 $\mathbf{L} \in \mathbb{R}^{n \times 13}$ 。理想配方的占比之和为 1，附件原始份额近似满足这一约束；线性基线沿用原始份额，核岭则逐行闭合。对 13 个验证领域分别建立线性关系：

$$L_d(\mathbf{p}) = \sum_{k=1}^{17} a_{dk} p_k + b_d, \quad d = 1, \dots, 13 \quad (10)$$

令系数矩阵的第 k 行、第 d 列为 $A_{kd} = a_{dk}$ ，其矩阵形式为

$$\hat{\mathbf{L}} = \mathbf{P}\mathbf{A} + \mathbf{1}\mathbf{b}^\top \quad (11)$$

在理想行和约束下，普通回归无法唯一识别截距与配比系数；舍入偏差虽可打破严格共线性，仍会形成近似共线。为获得稳定估计，对系数矩阵 $\mathbf{A} \in \mathbb{R}^{17 \times 13}$ 施加岭惩罚 [12]，并与截距 $\mathbf{b}$ 联合求解：

$$\min_{\mathbf{A}, \mathbf{b}} \|\mathbf{L} - \mathbf{P}\mathbf{A} - \mathbf{1}\mathbf{b}^\top\|_F^2 + \lambda \|\mathbf{A}\|_F^2 \quad (12)$$

相应的系数估计为

$$\begin{bmatrix} \mathbf{A} \\ \mathbf{b}^\top \end{bmatrix} = \left(\tilde{\mathbf{P}}^\top \tilde{\mathbf{P}} + \lambda \mathbf{D} \right)^{-1} \tilde{\mathbf{P}}^\top \mathbf{L} \quad (13)$$

式中，扩展矩阵为 $\tilde{\mathbf{P}} = [\mathbf{P}, \mathbf{1}]$ ，惩罚矩阵为 $\mathbf{D} = \text{diag}(1, \dots, 1, 0)$ 。最后一个对角元取零，表示不惩罚截距；本次计算取 $\lambda = 10^{-3}$ 。

为说明理想占比约束造成的识别问题，记 $\mathbf{u}$ 为 17 维全 1 向量，则 $\mathbf{P}\mathbf{u} = \mathbf{1}$ 。对任意 13 维向量 $\mathbf{v}$ ，有

$$\mathbf{P}(\mathbf{A} + \mathbf{u}\mathbf{v}^\top) + \mathbf{1}(\mathbf{b} - \mathbf{v})^\top = \mathbf{P}\mathbf{A} + \mathbf{1}\mathbf{b}^\top.$$

也就是说，在没有惩罚或附加约束时，可以把同一列系数共同增加一个量，再从相应截距中减去这个量，而预测损失保持不变。岭惩罚通过限制系数的平方和，从这些相同预测的表示中选择稳定估计。由于截距不受惩罚，损失的总体水平仍可由截距承担，配比系数主要描述构成差异。

求解式也可以由目标函数的一阶条件得到。将系数和截距合写为 $\Theta = [\mathbf{A}^\top, \mathbf{b}]^\top$ ，则

$$\tilde{\mathbf{P}}^\top (\tilde{\mathbf{P}}\Theta - \mathbf{L}) + \lambda \mathbf{D}\Theta = \mathbf{0}.$$

移项后得到前述正规方程。这个写法还说明，13 个验证任务共用同一配方矩阵，但各自对应不同的损失列和系数列；一个领域在不同验证任务上的系数可以具有不同方向。

由于占比总和受约束，我们按完整的份额转移解释系数。设某个可行调整为 $\Delta \mathbf{p}$ ，其各分量之和为零，则预测损失变化为

$$\Delta \hat{\mathbf{L}} = (\Delta \mathbf{p})^\top \mathbf{A}.$$

将任意一列系数共同平移不会改变该变化量。因而，中心化系数适合展示相对作用，而具体资源转移仍须比较被增加领域与被减少领域的系数差。仅凭一个系数的正负，无法同时确定所有验证任务上的调整收益。

4. 平方根嵌入核岭预测

线性模型便于解释领域间的份额转移，但可能遗漏配方对损失的非线性影响。核岭回归将岭惩罚与核函数结合，可用于非线性预测 [13]。为此，我们先对存在舍入误差的配方逐行闭合，再作平方根变换，在保留零占比的同时构造核函数：

$$\tilde{p}_k = \frac{p_k}{\sum_l p_l}, \quad k_\theta(\mathbf{p}, \mathbf{q}) = \exp \left[-\theta \sum_{k=1}^{17} (\sqrt{\tilde{p}_k} - \sqrt{\tilde{q}_k})^2 \right]. \quad (14)$$

平方根距离利用了比例非负的结构，无需为零占比添加伪计数；核参数 $\theta > 0$ 控制响应随配方距离变化的范围。线性基线仍按原始附件份额、固定正则系数 10^{-3} 计算，以便与系数解释及线性对照对应。

设训练核矩阵为 $\mathbf{K}$ ， $\mathbf{H} = \mathbf{I} - \mathbf{1}\mathbf{1}^\top/n$ ，训练损失的逐域均值为 $\bar{\mathbf{L}}$ 。对核矩阵和响应同时中心化，使截距不受惩罚，求得核系数矩阵：

$$\mathbf{K}_c = \mathbf{H}\mathbf{K}\mathbf{H}, \quad \mathbf{U} = (\mathbf{K}_c + \lambda_k \mathbf{I})^{-1} \mathbf{H}\mathbf{L}, \quad \hat{\mathbf{L}}(\mathbf{p}) = \bar{\mathbf{L}} + \mathbf{k}_c(\mathbf{p})^\top \mathbf{U}. \quad (15)$$

新配方的第 i 个中心化核值为

$$[\mathbf{k}_c(\mathbf{p})]_i = k_\theta(\mathbf{p}, \mathbf{p}_i) - \frac{1}{n} \sum_j k_\theta(\mathbf{p}, \mathbf{p}_j) - \frac{1}{n} \sum_j K_{ji} + \frac{1}{n^2} \sum_{j,l} K_{jl}. \quad (16)$$

训练与预测共用训练核的均值；交叉验证时所有这些均值均只由当前训练折计算。13 个输出共享同一核矩阵，但各自具有响应均值与系数列。

固定随机种子 20260926，将 512 组训练配方分为共同的五折，折大小依次为 103、103、102、102、102。核岭在 $\theta \in \{2^{-6}, \dots, 2^2\}$ 、 $\lambda_k \in \{10^{-6}, \dots, 10^2\}$ 的 81 组预设组合中，选择综合折外 RMSE 最小者：

$$\text{RMSE}_{\text{OOF}} = \sqrt{\frac{1}{13n} \sum_{i=1}^n \sum_{d=1}^{13} (L_{id} - \hat{L}_{id}^{(-f(i))})^2}. \quad (17)$$

其中 $f(i)$ 为记录所属验证折。该指标先汇总全部折外平方误差，再开平方；不能用逐域 RMSE 的算术平均代替。最小交叉验证误差用于调参，不作为嵌套验证的无偏估计；既有检验集此前已参与方法分析，故后续结果也不称为新的盲测成绩。

5. 混合质量指数与冗余特征对照

我们先考察混合质量与领域配比之间的关系。按占比计算混合质量 $\bar{Q}(\mathbf{p}) = \sum_k p_k Q_k$ 。各 Q_k 依据 A16 确定：direct、near_direct 采用 A1–A3 中对应的领域评分，inferred 采用全局均值。将 $\bar{Q}$ 加入回归，得到

$$\hat{\mathbf{L}} = \mathbf{P}\mathbf{A} + \mathbf{1}\mathbf{b}^\top + (\mathbf{P}\mathbf{q})\mathbf{c}^\top \quad (18)$$

其中 $\mathbf{q} = (Q_1, \dots, Q_{17})^\top$ ， $\mathbf{c}$ 为 13 维质量系数向量。由于 $\mathbf{q}$ 固定， $\mathbf{P}\mathbf{q}$ 严格位于 $\mathbf{P}$ 的列空间内，上式等价于 $\mathbf{P}(\mathbf{A} + \mathbf{q}\mathbf{c}^\top) + \mathbf{1}\mathbf{b}^\top$ ，并未增加独立的解释信息。新增系数参与岭惩罚后，原有效系数所受的惩罚发生变化，故拟合值仍可能略有改变。这里仅作冗余特征加入后的拟合对照，不能据此识别质量的独立作用。问题二、三再利用质量实验考察固定配比下的质量变化。

6. 核岭预测与训练支持范围内的配比优化

参考配方先由 512 组原始份额取列均值，再闭合为 $\mathbf{p}_{\text{ref}}$ 。令 $\mathcal{M}$ 为 13 个有验证损失的领域，权重取 $w_d = p_{\text{ref},d} / \sum_{j \in \mathcal{M}} p_{\text{ref},j}$ ；它与质量指标的熵权不同。四个无对应验证损失的领域固定参考占比，其余领域仍不超过参考占比的 2.5 倍。线性目标容易把份额集中到少数有利系数上，故其有界解保留作对照；正式方案采用检验表现较好的核岭目标，并限制在闭合训练配方的凸包内：

$$\begin{aligned} \min_{\mathbf{a}} \quad & \sum_{d \in \mathcal{M}} w_d \hat{L}_d^K(\mathbf{p}), \\ \text{s.t.} \quad & \mathbf{p} = \sum_{i=1}^{512} a_i \tilde{\mathbf{p}}_i, \quad a_i \geq 0, \quad \sum_i a_i = 1, \\ & p_k = p_{\text{ref},k} \quad (k \notin \mathcal{M}), \\ & 0 \leq p_k \leq 2.5 p_{\text{ref},k} \quad (k \in \mathcal{M}). \end{aligned} \quad (19)$$

其中 $\hat{L}_d^K$ 为核岭预测， a_i 是训练配方的凸组合权重，与线性回归系数不同。该约束保证输出由已有训练配方组合得到，避免仅限制各域上限却进入训练凸包之外；但凸包内部仍可能观测稀疏，不能将支持范围约束当作真实训练收益证明。

我们采用四个固定初值的全校正条件梯度法求解。每步以线性规划寻找可行下降顶点，再在已选顶点的凸组合上修正权重。程序以方向间隙小于 10^{-6} 作为停止条件，另核对份额、上限与固定域残差。四次求解得到相近的可行解，目标差小于 10^{-8} 。零份额处的平方根导数采用数值保护，故停止量不作为光滑驻点证明；多初值一致也不构成全局最优证明。

5.1.4 模型求解

辅助分析说明：本问的文段相关辅助参考腾讯 WorkBuddy 接入的 DeepSeek-V4-Flash (2026-07-31 公开测试)，图表展示、程序整理与复现核对使用 OpenAI Codex (GPT-5.6-Luna, 2026-07-09) [14, 15, 16]。正文表达与段落衔接另经 OpenAI Codex 辅助润色。

本问分别利用 A1–A3 和 A4–A15 开展质量评价与配比计算。质量评分包含 A1 的 51,230 条抽样记录及 A2、A3 的 221,275 条扩展记录，共计 272,505 条。按附件 ID 核对，其中有 261,086 个唯一 ID，11,419 个 ID 在不同文件中各出现两次。本文的全量熵权、领域均分和冲突率均按附件记录计权，重复 ID 的记录也分别参与计算；因此这些记录不应解释为同等数量的独立文本，抽样与扩展集之间也不是完全独立的验证。

1. 质量评分与冲突分析

图3汇总了各领域的质量均值和离散程度，柱高和误差线分别表示领域均值与标准差， n 为对应记录数。对于同时出现在抽样集与扩展集中的领域，表5进一步比较两套评分。

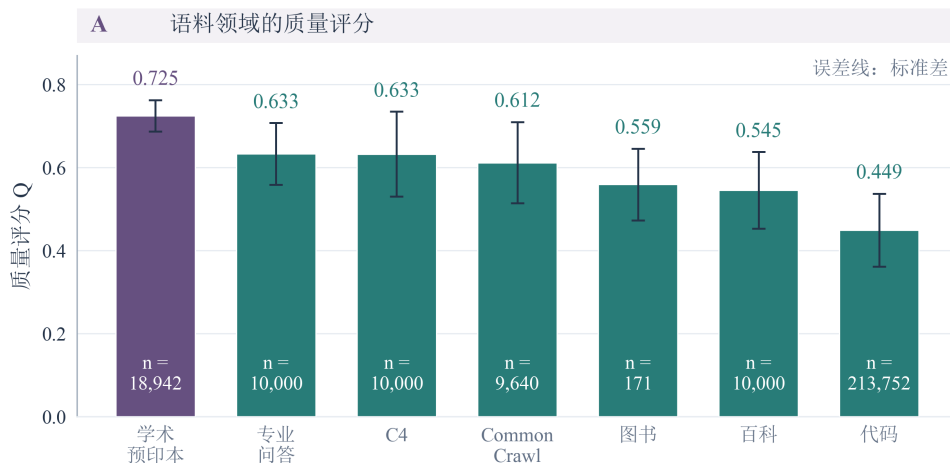


图3 领域质量评分

表5 各领域在 A1 抽样集与 A2/A3 扩展集中的质量评分

质量信号领域	训练配方对应领域	A1 评分 Q	扩展集评分 Q	评分差值
arxiv	arxiv	0.7272	0.7248	+0.0024
stackexchange	stackexchange	0.6333	—	—
c4	(配方侧无同名列)	0.6327	—	—
commoncrawl	pile_cc	0.6119	—	—
book	gutenberg_pg_19	0.5592	—	—
wikipedia	wikipedia_en	0.5450	—	—
github	github	0.4498	0.4492	+0.0006

arxiv 与 github 均包含抽样和扩展数据。表5中，两者的评分差值分别为 +0.0024、+0.0006。差值较小，说明在可直接比较的领域内，抽样集与扩展集的平均评分接近。

为解释领域内部的分数差异，我们结合原文抽查进行对照。抽查中，arxiv 的一个低分片段主要由章节格式定义和排版命令组成，评分约为 0.350；一个高分片段则以连贯语句介绍计算机实验的用途，评分约为 0.849。这一差异说明，即使同属 arxiv，文本内容也可能明显不同，不能只凭领域名称判断质量。对预训练语料而言，正文叙述与格式代码在语言信号上差异明显，综合评分会把这种差异反映到单条记录上。

网页语料中也存在可辨认的对照。c4 的一个低分片段反复出现同一家具商品短语，评分约为 0.284；另一个高分片段介绍细胞内不同细胞器的空间分工，评分约为 0.914。这些对照使我们能够从重复程度、句子组织和内容连续性解释评分差异。不过，少量抽查不能替代全量人工标注，总分也不能单独证明某一分类器的方向。

在全量领域均分中，arxiv 的 0.725 与 github 的 0.449 分别位于评分两端。stackexchange、c4、commoncrawl 依次为 0.633、0.633、0.612；book 与 wikipedia 较为接近，分别为 0.559、0.545。这一排序由 22 个质量信号加权得到。例如，rps_doc_frac_unique_words 与 rps_doc_unigram_entropy 均可能随文档长度变化。因此，网页类评分高于书籍、百科，仅表示其在本评分体系下得分较高，不能直接推断其对所有训练任务都更有利。

评分标尺对来源构成的依赖 全量标尺把缺失填充值、长度中心、分位端点和熵权都固定在 A1–A3 的合并分布上。这样便于按同一规则评分，但扩展集会改变规则本身：github 在 A1 中占 19.52%，在合并记录中占 78.44%。为保留 A1 主样本的参照作用，我们另做标尺敏感性诊断，只用 A1 估计上述参数，再冻结参数评价全部记录；变换公式、指标方向和四维聚合方式保持相同。该诊断独立保存，不替换原评分或后问的质量基线。

表 6 相同评分公式在全量标尺与 A1 标尺下的领域均分

领域	全量标尺	A1 标尺
arxiv	0.7250	0.7297
book	0.5592	0.5094
c4	0.6327	0.5852
commoncrawl	0.6119	0.5884
github	0.4492	0.4927
stackexchange	0.6333	0.6320
wikipedia	0.5450	0.5047

两套标尺下的领域排序 Spearman 系数为 0.9643，arxiv 与 github 仍居两端，但部分均分相差约 0.04 ~ 0.05。词数中心由全量标尺的 152 变为 A1 标尺的 253，平均词长中心由 8.057 变为 5.323。因此，领域排序相近并不意味着评分间距不变。本文的 Q 应理解为指定标尺下的综合指标，长度中心接近度和四维同向聚合属于评价假设；将其数值传入后问时，需要一并保留标尺及领域映射条件。

表7比较各集合的冲突比例和排序一致性；图4则给出分歧较多的指标对及对应的熵权分布。图4左侧展示冲突率最高的 12 对指标关系，边旁数值为冲突率，线宽与其成正比；右侧列出全部 22 项熵权。M01–M22 按表2的指标顺序编号，紫色与青绿色分别对应列表型与标量型指标。

表 7 各数据集的指标冲突比例与排序一致性

统计范围	记录数	冲突比例 ρ_c	独立基准 ρ_0	ρ_c/ρ_0	Kendall W
A1 抽样集	51,230	0.2219	0.2888	0.768	0.0854
A2+A3 扩展集	221,275	0.2162	0.2888	0.748	0.0718
全量	272,505	0.2190	0.2888	0.758	0.0761

表7显示，抽样集、扩展集及全量记录的冲突率均处于 0.216 ~ 0.222。统计集合的变化未明显改变这一比例。

全量排序经并列秩校正后的 Kendall 系数为 0.076，表明排序一致程度有限。卡方近似的名义概率为 $p < 0.001$ ，但受重复记录和指标相关性的影响，不据此作独立样本下的显著性判断。实际冲突率为 21.9%，低于独立参照的 28.9%，二者比值为 0.758。据此，我们判断指标之间存在共同倾向，但整体一致程度仍然有限。

这些分歧既来自指标含义，也来自计算尺度。rps_doc_unigram_entropy 与 rps_doc_frac_unique_words 都关注词汇多样性，却不使用同一尺度；modernbert_professionalism 对学术文本和论坛内容的反应也可能不同。另如 ad_en，长文档中的二分类概率容易靠近两端。由此，同一文本在不同指标下得到不同评价是可以预期的。

若以“至少一对指标超过阈值”标记文本，则 272,505 条记录中有 271,458 条被标记，占 99.62%；这与 21.9% 的样本–指标对冲突率不是同一口径，也说明该标记不宜直

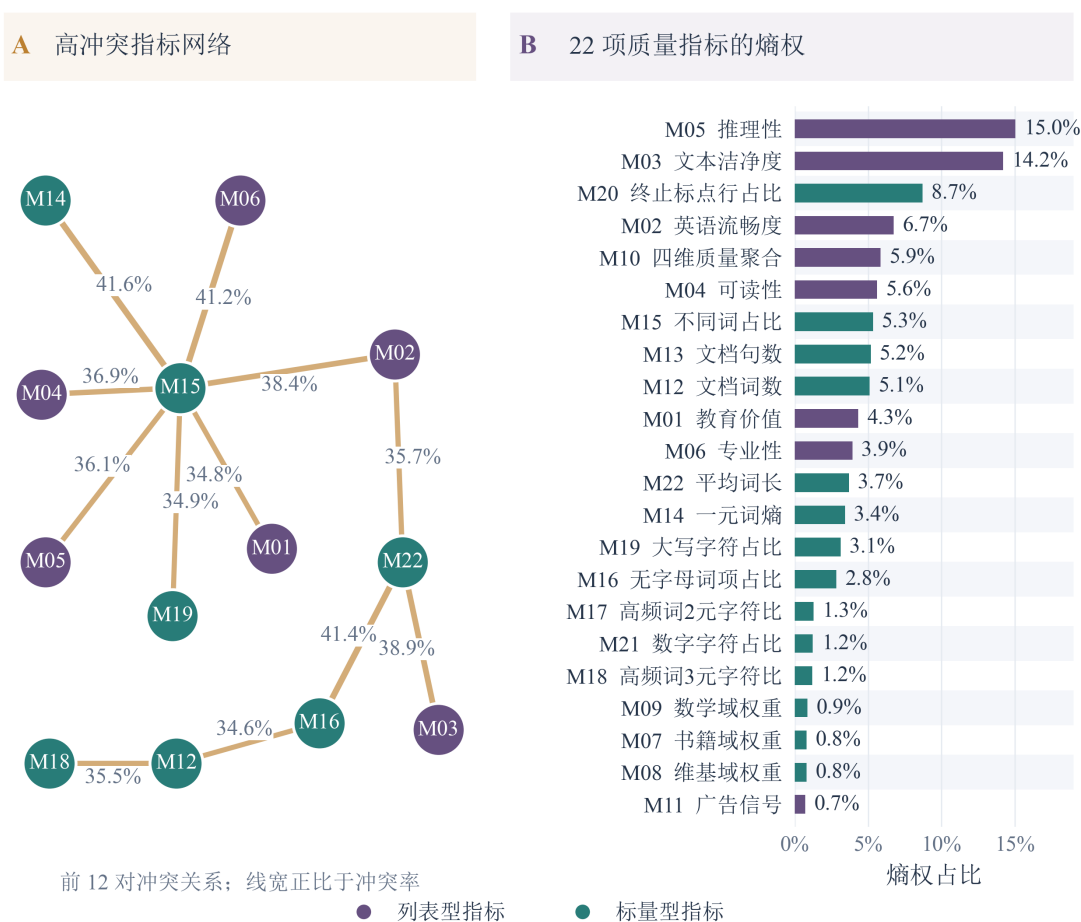


图 4 指标冲突与熵权

接作为剔除规则。相对定位使用 Q_i 与领域中位数的差，计算领域均值时仍纳入这些记录，不删样本或降权。后两问采用聚合评分，不将该标签视为独立的质量真值。

对既有标签按领域汇总可见，arxiv、book、c4、commoncrawl 的记录全部触发“至少一对冲突”。其中 arxiv 的 18,942 条记录恰好分为低于和高于域中位数各 9,471 条，c4 同样为各 5,000 条。即使省略冲突标记，直接按总分划分，这些领域也会得到相同标签。可见标签回答的是文本处于域内何种位置，而不是哪项指标应被纠正。专业要求高而易读性较低可能属于合理的多维取舍；是否存在提取噪声或分类器误判，还需要逐条内容依据。因此，当前完成了分歧量化与相对定位，尚不能以标签比例证明冲突消解有效。

2. 领域配比与损失的拟合及验证

配比拟合的结果依次展示训练配方构成、线性基线与核岭的验证表现、不同训练领域的相对系数，以及同尺度预测和观测损失的对应关系。

图5将前 8 组训练配方的领域占比绘为热图，仅标注占比不低于 10% 的格子；右侧按同一领域顺序列出全部 512 组配方的平均占比。

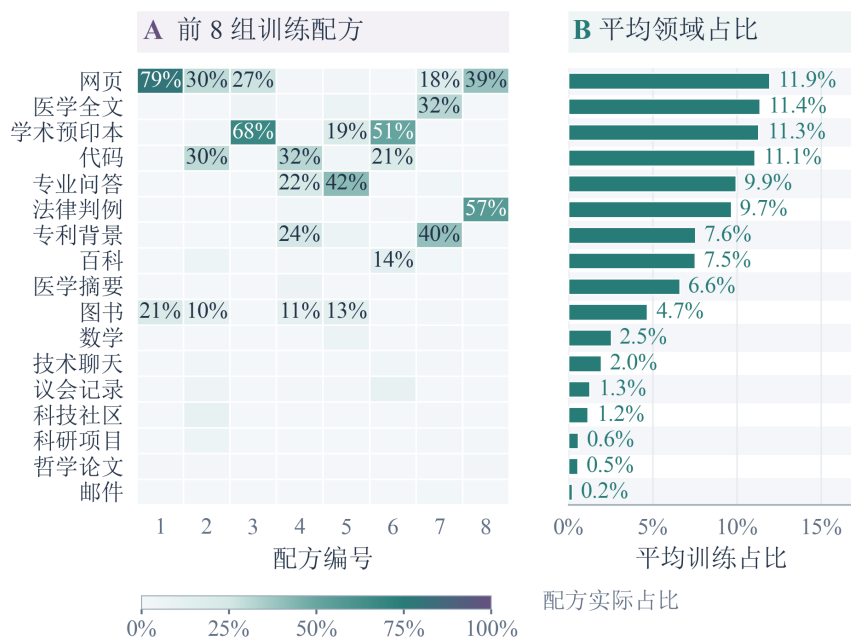


图 5 训练配方分布

图6分别展示 1M 训练与检验的决定系数，以及 60M、1B 检验的 Pearson 相关系数；连线连接同一验证领域的两个结果。

同尺度下，线性岭回归的训练与检验表现接近。1M 检验的平均 R^2 为 0.619，训练均值为 0.629，训练集各验证域介于 0.42 ~ 0.77。在这一对照中，未出现训练表现明显优于检验的现象。

平均值之外，各验证域的可预测程度仍有区别。现有检验结果中，arxiv 和 dm_mathematics 的 R^2 分别约为 0.430、0.428，pubmed_abstracts 和 pile_cc 则约为 0.788、0.771。我们比较的是配方变化对损失变异的解释程度，并不据此评价四类文本的优劣。例如，数学领域的平均损失较低，并不意味着其损失随配方变化的部分更容易由线性式解释。因此，我们分别报告损失水平、拟合优度和文本质量，避免将三者混为一谈。

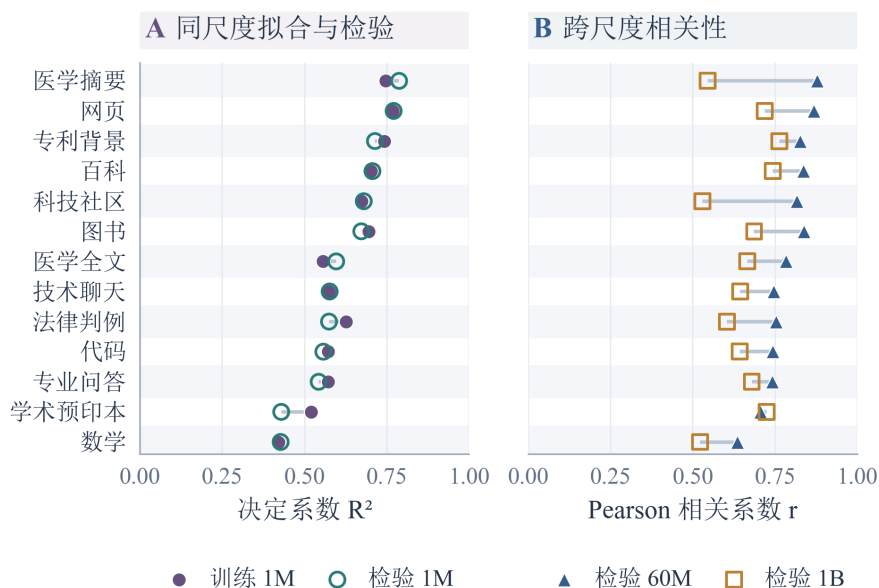


图 6 线性岭配比回归检验

在相同五折下，线性基线的综合折外 RMSE 为 0.5761；核岭选择 $\theta = 0.0625$ 、 $\lambda_k = 10^{-4}$ ，对应 0.2558。表8同时列出同尺度与跨尺度结果，所有 RMSE 都在“配方 × 验证域”上汇总， R^2 则先逐域计算再平均。

表 8 线性基线与核岭的配比预测比较

检验尺度	模型	平均域 R^2	综合 RMSE	MAE
1M (256 组)	线性岭	0.6186	0.5232	0.3858
	平方根核岭	0.9454	0.2256	0.1344
60M (256 组)	线性岭	-7.9526	1.6217	1.5142
	平方根核岭	-7.3068	1.5536	1.4721
1B (64 组)	线性岭	-810.7435	3.2569	3.1900
	平方根核岭	-555.8823	2.8676	2.7952

核岭在 1M 检验上的综合误差较线性基线下降 56.9%，13 域平均排序相关由 0.8311 提高至 0.9725。图7左侧按域比较决定系数，右侧每个点对应一组配方的 13 域等权平均损失，虚线表示预测等于观测。先对 13 域取平均后，配方均值 RMSE 为 0.0733，排序相关为 0.9644；因域间误差可能抵消，这一数值不能替代综合 RMSE 0.2256。数学域 RMSE 仍为 0.5817，说明总体改善后仍有局部误差。上述计算与图表由 Codex 辅助实现和核对，运行过程使用附件原始记录及保存的预测明细 [16]。

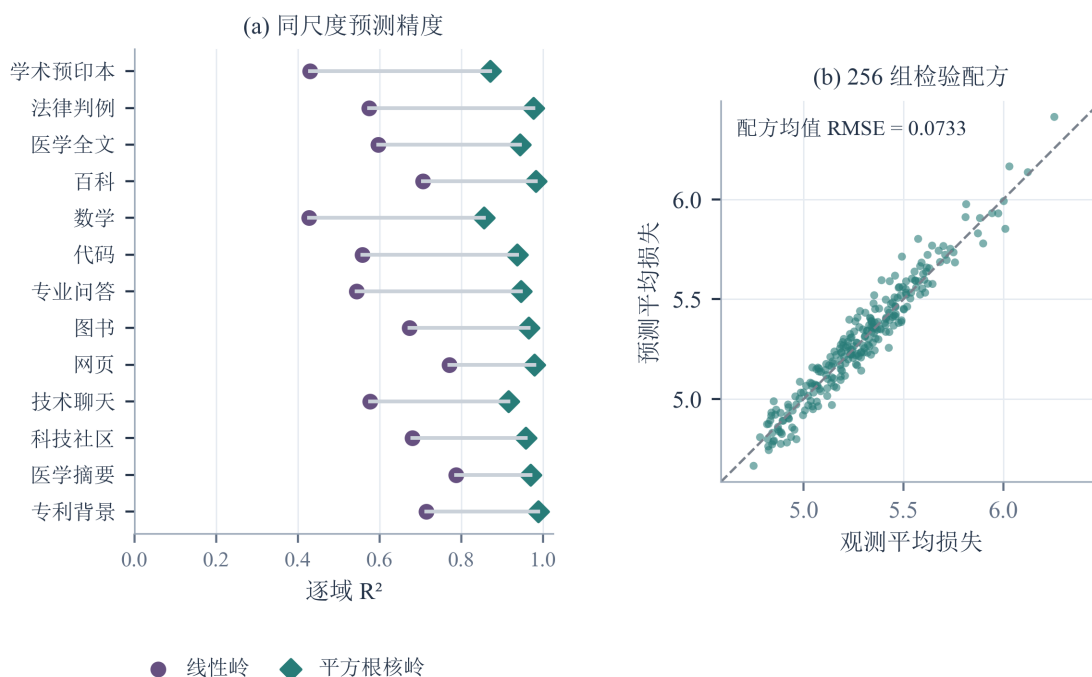


图 7 核岭同尺度预测与逐域对照

跨尺度时，两种模型的绝对损失均有明显偏差，核岭在 60M、1B 的负 R^2 不能用较高排序相关抵消。因此，核岭用于 1M 同尺度预测及支持范围内的配方选择；系数解释和冗余特征对照仍来自线性近似。问题二、三对核岭损失差的跨来源传递是一项条件假设，而非已经验证的大模型绝对损失规律。

训练与检验表现接近，说明拟合关系在同尺度配方中具有一定稳定性；各任务的误差差异则表明，候选结果仍依赖评价任务的组成。若应用更重视某个预测精度较低的领域，还需单独检查该领域的偏差，不能只依据平均逐域指标作配方决策。

线性基线的跨规模比较还需考虑损失基准的变化。60M 相对 1M 的损失约为 0.68 倍，直接沿用原损失尺度会影响误差评价。因此，采用 Pearson 相关系数比较变化趋势：60M 检验均值为 0.782，1B 为 0.651。配比变化对应的部分趋势得以保留，但绝对损失预测仍需调整尺度。

我们另行报告 Pearson 相关，是为了检查跨尺度后配方变化趋势是否保留。整体平移或正比例变化可能保留较高相关，却产生较大的绝对误差。因此，趋势相似只能支持方向比较，跨尺度使用系数仍需校准损失水平。

图8以 17 个训练领域为行、13 个验证领域为列，展示全部 221 项中心化回归系数。颜色表示系数的正负及大小，色标在 $[-1, 1]$ 内为线性映射，其余区间采用对称对数映射；每列仅标注绝对值最大的系数。

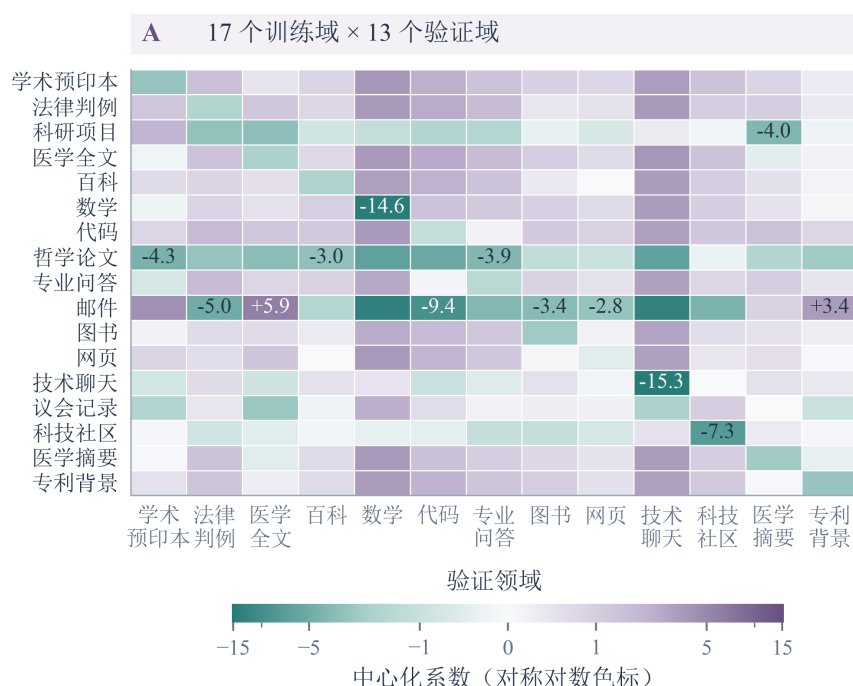


图 8 线性岭领域回归系数

图8显示，各训练领域对验证任务的作用并不相同。dm_mathematics、arxiv 等领域在部分任务上对应较低损失，pile_cc、ubuntu_irc 等领域则表现出不同方向。配比选择因而需要结合验证任务权重，不能仅看某一个系数。

图9汇总 1M 检验集的 3,328 个实测与预测数据点，六边形颜色表示分箱内记录数，上方与右侧分别为实测损失和预测损失的频数分布。

图中仍有记录偏离预测值等于实测值的参考线，因此平均检验精度接近训练精度，并不表示每条记录的预测都准确。

加入 $\bar{Q}$ 后，13 个验证域的平均 R^2 增加 9.8×10^{-5} 。这一差异反映了冗余参数化对岭惩罚的影响，不是质量提供独立信息的证据。我们据此保留原配比回归；质量投入的作用则由问题二、三结合附件 B 的半合成实验另行分析。

文本质量与损失难度并非同一个概念。图10利用 6 个可映射领域，对照 A1–A3 的熵权评分 Q 与损失代理 $Q^{(\text{loss})}$ 的排序。右侧连线连接同一领域在两套口径下的名次，名次 1 表示得分最高。

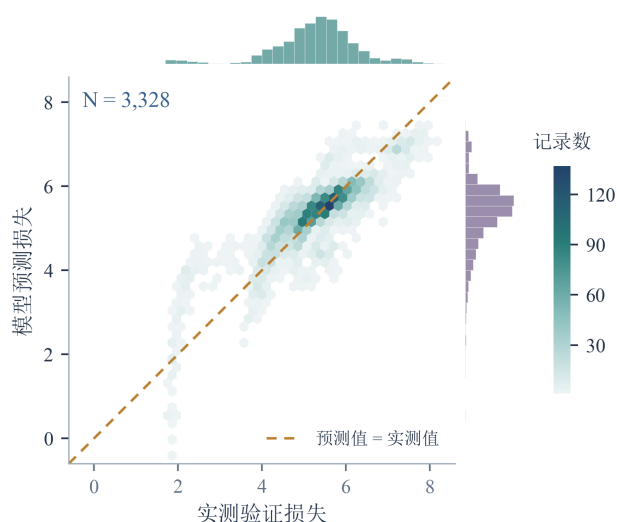


图9 线性岭预测与实测损失

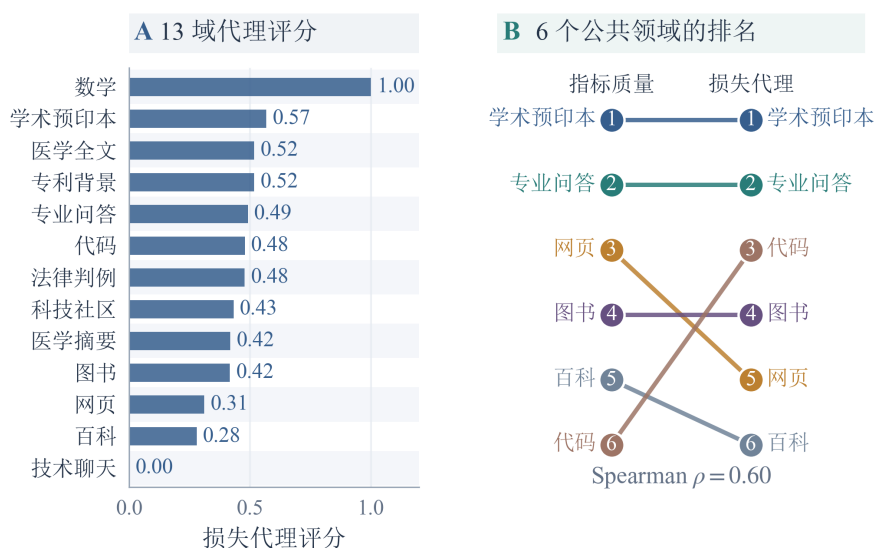


图10 质量与损失难度

图10(A) 的评分端点由损失高低确定：dm_mathematics 为 1.000，对应 13 域中的低损失端；ubuntu_irc 为 0.000，对应高损失端。依据 A16 的 direct/near_direct 映射比较两类评分，图10(B) 中的 Spearman 系数为 0.60。arxiv、stackexchange、github 的相对位置较一致，wikipedia、book、pile_cc 则存在差异。较高的文本质量不必然对应较低的拟合难度，因此语料治理采用文本评分，损失预测则单独考虑领域难度。

领域评分在两套分类之间的衔接

A16 将配方领域与质量信号领域建立对应。现有映射中，arxiv、github、stackexchange 为直接对应；wikipedia_en 与 wikipedia、gutenberg_pg_19 与 book、pile_cc 与 commoncrawl 为近似对应。两类映射都利用已有领域评分，但近似对应还依赖语料来源与内容相近这一条件，名称相近本身并不能证明两份语料的质量分布完全相同。

其余 11 个配方领域在质量数据中没有直接对应，现有计算统一使用全局均值 0.4912。这样可以为混合质量指数提供完整的领域向量，但这些领域之间的质量差别并没有由该

向量单独辨认。对它们的配比作用，仍须查看回归系数和验证损失，而不能依据相同的代入分数作高低排序。

我们只在 6 个可映射领域中比较两种质量口径的名次，因为这些领域具有明确的文本评分来源。其余领域使用全局均值代入，若一并排序，会混合代入规则与实际观测。因此，映射后的质量参照与全部验证任务的损失结果分别展示。

混合质量指数随配方变化而变化，其信息由领域评分和占比共同决定。领域评分固定以后，改变配方才会改变该指数；在相同配方内部提升文本质量，则属于另一种投入变化，需要附件 B 的质量实验提供依据。这与前述冗余特征对照的解释一致，也解释了问题一的配比分析如何衔接到后续的质量投入模型。

3. 支持范围内的配比推荐与跨尺度分析

按式(19)选择配方后，核岭预测由参考配方的 4.8173 降至 4.6444，模型内损失差为 -0.1730 。保存的非负凸组合权重可重建新配方，约束残差小于 10^{-10} ；表9列出主要调整。相同目标的候选排序检验见第6.2节，推荐的实际训练收益仍需另行验证。

表 9 训练支持范围内的核岭方案与参考方案

训练领域	核岭占比	参考占比	占比变化量
freelaw	0.2421	0.0968	+0.1452
uspto_backgrounds	0.1801	0.0756	+0.1045
dm_mathematics	0.0637	0.0255	+0.0382
wikipedia_en	0.0994	0.0751	+0.0243
hackernews	0.0291	0.0116	+0.0175
stackexchange	0.0640	0.0994	-0.0354
pubmed_abstracts	0.0268	0.0662	-0.0394
github	0.0591	0.1107	-0.0516
arxiv	0.0579	0.1130	-0.0551
pile_cc	0.0000	0.1195	-0.1195

表9中，法律、专利背景、数学和百科份额增加，网络爬取、学术论文与代码份额减少。这是对 13 个验证任务的加权损失所作的条件选择，不是文本质量名次：例如 arxiv 份额下降，不能据此判断学术文本质量较差。表中只列出主要变化，其余领域依约束分配；线性有界对照的完整份额也保存在相应结果表中。

核岭比较的参考配方和推荐配方均闭合为 1；线性基线保留附件原始份额，列均值之和约为 0.999881。这一舍入差异在核岭预测与推荐约束中统一处理。后续算力情景仍采用既有原始均值计算的共同质量基线 $Q_0 = 0.5487$ ，因此只考察配比损失差，不把配方变化同时解释为质量处理成本变化。

旧线性方案为何只作对照 同一加权目标下的排序与选优结果见表35。1M 检验的目标 RMSE 为线性 0.2305、核岭 0.0701；线性选中项的观测目标比该集合最小值高 0.2635，核岭选中项达到该集合最小值。这里比较既有检验配方，不能当作连续优化方案的新增训练验证。

固定已选定的 $\theta = 0.0625$ 、 $\lambda_k = 10^{-4}$ 后，我们再由核岭评价原线性有界对照，结果见表10。重建核岭对已保存的 256 组 1M 检验预测的最大绝对差小于 5×10^{-12} 。线性模型沿用原始参考份额，核岭按既定规则将其逐行闭合，其他目标权重保持相同。

两模型对旧线性方案的收益方向相反，且该方案到闭合训练配方凸包的最小 L_1 距离为 0.08205。结合表35的排序表现，我们将旧方案保留为线性对照，正式方案改由核岭在训练凸包内选择；代理间分歧本身不能证明真实训练收益。

表 10 两个代理对旧线性方案的预测对照

代理模型	参考配方	旧线性方案	参考减候选
线性岭	5.334831	5.136032	+0.198799
平方根核岭	4.817338	4.988626	-0.171288

线性解释模型的正则敏感性 已有正则扫描主要比较 R^2 ，这里进一步沿用 10^{-4} 至 10^{-1} 的四档取值，分别检查中心化系数和旧线性候选配方。设 $\mathbf{A}_c(\lambda)$ 为列中心化后的系数矩阵， $\lambda_0 = 10^{-3}$ 为主值，比较量定义为

$$r_A(\lambda) = \frac{\|\mathbf{A}_c(\lambda) - \mathbf{A}_c(\lambda_0)\|_F}{\|\mathbf{A}_c(\lambda_0)\|_F}, \quad d_p(\lambda) = \|\mathbf{p}^*(\lambda) - \mathbf{p}^*(\lambda_0)\|_1.$$

表11同时记录中心化系数的变号项数。各档沿用线性对照的原始参考份额、固定域和 2.5 倍上限，以等价线性规划求解。这一检验仅针对线性解释模型，不代表核岭推荐的参数敏感性。

表 11 岭正则变化对相对系数与线性对照配方的影响

λ	中心化系数相对变化	变号项数 (共 221 项)	配方 L_1 差
10^{-4}	3.90%	2	0.00000
10^{-3}	0.00%	0	0.00000
10^{-2}	22.65%	5	0.00000
10^{-1}	56.33%	16	0.34277

前三档得到相同候选；当 $\lambda = 0.1$ 时，医学全文份额由 28.43% 降至 11.29%，百科由 1.63% 升至 18.77%。将后一配方放回原线性模型评价，其预测目标仅比原线性候选高 0.00228。这表明近似相同的损失可以对应明显不同的配置：拟合精度较稳定，不代表每个相对系数或详细份额都稳定。上述百分比只能作为线性对照的代表解，不能据此认定核岭方案具有相同的配置波动。

图11比较相同配方下的跨尺度损失比。60M/1M 的领域损失比均值处于 0.51 ~ 0.82；外推记录中，10B 的 est/train 因子均值为 0.338，标准差为 0.038，70B 对应的均值与标准差分别为 0.266、0.036。10B 和 70B 没有新增实测训练记录，这些数值仅用于检查估算比例的稳定性。

图11(A) 的各领域在每个外推尺度下均包含 63 条记录，箱体对应第 25 至第 75 百分位，箱内竖线为中位数，两端须为最小值和最大值；图中比值为外推损失与同配方 1M 训练损失之比。图11(B) 的圆点与误差线分别表示同配方 60M/1M 损失比的均值和标准差。

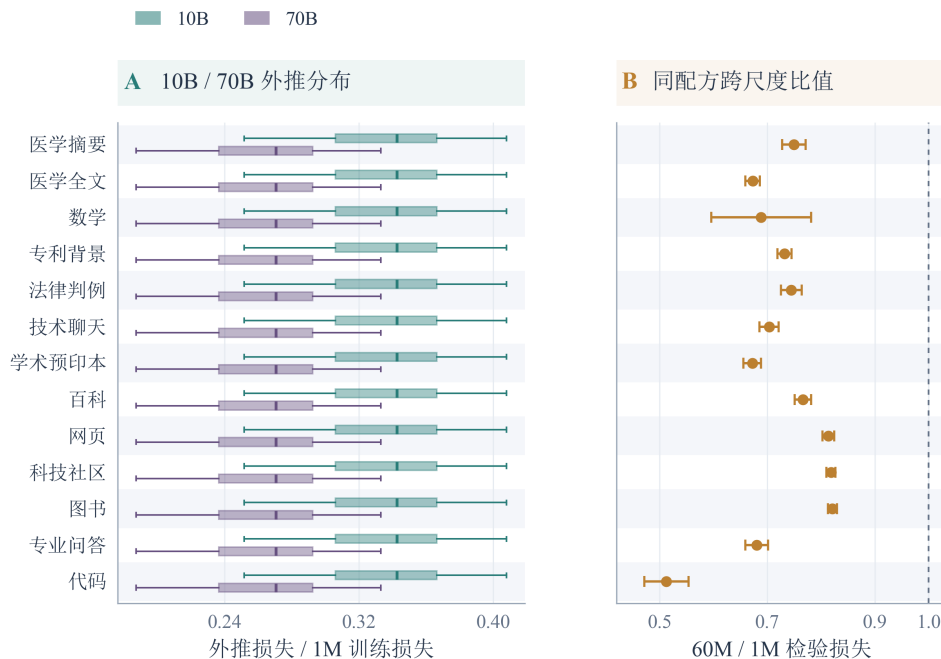


图 11 跨尺度损失比较

本问输出指定标尺下的领域质量、用于解释的线性系数，以及训练凸包内的核岭配方。后续配置沿用共同质量基线及核岭损失差，因而属于固定接口下的条件分析；质量标尺的跨来源对应、冲突消解效度和新配方的真实训练收益仍需进一步验证。

5.2 问题二：跨维度数据融合与广义标度律

5.2.1 具体分析

在参数量和数据量相同时，语料质量与领域构成仍可能影响训练损失。我们将这两类因素纳入规模关系，用以计算质量提升能替代多少参数投入。由于附件 A 主要记录配比，附件 B 主要记录规模与质量，现有资料没有同时覆盖全部变量，因此先核对评分的可比性，再明确配比结果跨来源使用的条件。

图12给出计算顺序：先用 Pythia 记录估计规模关系，留出较大模型检查预测，再由质量实验选择损失修正项。确定函数后，以弹性比较相对变化，用等损失条件换算质量与参数投入。领域关系沿用问题一的线性系数解释，配比收益则采用核岭预测损失差，并单独说明跨来源迁移条件。

5.2.2 模型准备

1. 数据组织与变量口径

规模项的估计数据取自 B1，覆盖 Pythia 的 8 种模型，每种模型保留 147 次检查记录，总计 1,176 条 [17]。B6 半合成实验补充了质量维度，B7 则在剔除与 B6 重复的记录后检查扩展表现。其余附件各有用途：B2 提供另一模型族的训练轨迹，B3 检查插值轨迹与拟合关系是否一致；B4 的收敛点和 B5 的文献记录用于比较跨来源预测；B8 用于诊断质量方向；B9、B10 配对后核对大规模条件下的估算损失。

我们按各组材料能够识别的关系组织拟合。B1 中同一模型随训练推进形成的检查点，能够描述数据量增加时损失怎样下降；不同参数规模的模型则提供了估计规模项的依据。B6 在固定 (N, D) 的条件下改变质量，补充了仅靠训练轨迹无法区分的质量影响。

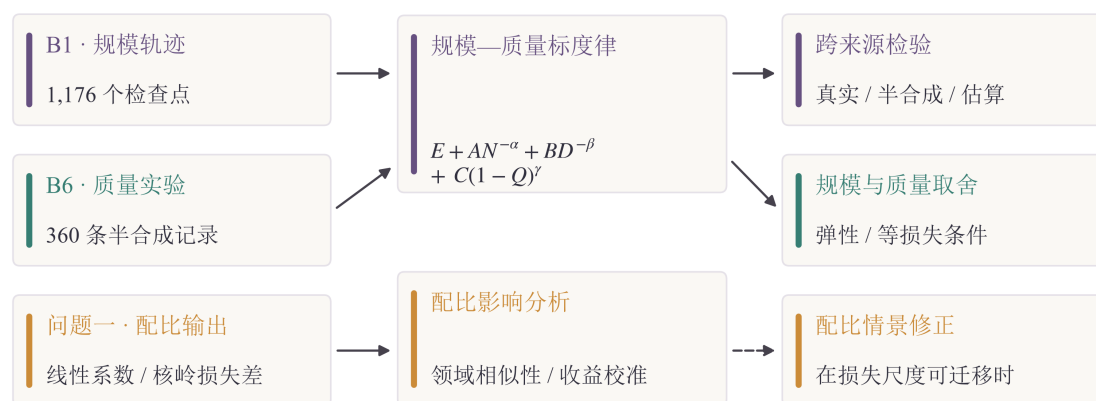


图 12 问题二分析流程

因此，我们先用 B1 估计经典模型，再合并 B1 与 B6 估计广义模型；B2、B4、B5 等来源用于拟合之外的对照，检查跨来源使用这些参数时的偏差。

同一模型的多个检查点共享训练过程，B3 也来自已有检查点的插值，记录增多不等于独立实验增多。B7 与 B6 的重复部分同样不能重复计作证据。因此，后文分别报告拟合、留出和跨来源对照结果。

记验证集交叉熵损失为 L ，模型变量为参数量 N 、数据量 D 和质量 Q 。本节将 N 、 D 都按十亿为单位记录，问题三计算 FLOPs 时再换回实际参数数和 token 数，即各乘 10^9 。拟合保留常数项 E ，不通过双对数变换处理，以免把参考配方下的渐近损失并入其他项。

统一规模单位也是复用拟合参数的前提。幂函数项中的系数随输入尺度改变，若直接把实际参数数代入以 B 为单位估计的函数，就会改变预测值。本文因此将单位换算放在数据输入与费用计算之间处理：损失预测始终沿用本节的规模单位，训练费用再使用实际数量。不同来源的记录在进入同一函数前，也先核对字段所指的是模型参数数、累计训练 token 数还是其他数量，避免把计量差异误作标度关系的差异。

2. 质量字段方向检验

我们先检验质量高分是否对应较低损失。为控制规模差异，在每个固定 (N, D) 的实验组内计算 Q_{score} 与损失的 Pearson 相关，再求组间算术平均。均值为负时，方向与本文的质量定义相符；均值为正时，则需要进一步检查。表12据此说明方向，不据此作显著性判断。

固定规模后计算相关，可减少参数扩容和数据增加的混杂。各有效组等权参与平均，不按组内记录数加权。这里检查的是质量方向，评分尺度另作讨论。

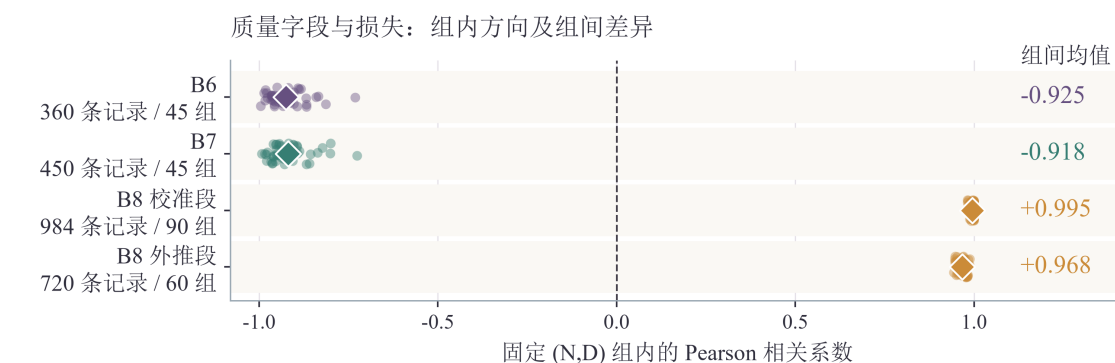
表 12 质量评分方向与样本使用方式

数据分组	平均组内 相关系数	记录数 / 实验组数	评分解释与使用方式
B6	-0.925	360 / 45	沿用原评分； $Q = 1$ 为最优参考
B7	-0.918	450 / 45	与 B6 同向，仅新增记录用于检验
B8 校准段	+0.995	984 / 90	方向相反，翻转评分后作诊断
B8 外推段	+0.968	720 / 60	方向相反，不纳入主拟合

注：实验组固定 N, D 。B7 包含 B6 的全部 360 条记录，新增 90 条；均值只描述方向，不表示显著性。

B6、B7 的组内相关均值分别为 -0.925 、 -0.918 ，高分对应较低损失，因此保留字段原方向，取 $Q = Q_{\text{score}}$ 。记录核对同时表明，B7 收录了 B6 的全部 360 条数据，另外只有 90 条新增记录，两者不能分别计作独立证据。B8 校准段与外推段都表现为高分对应较高损失，部分记录还越过了模型所能达到的损失下限。对此仅在方向诊断中使用 $1 - Q_{\text{score}}$ 作对照，主拟合不使用 B8。

图13列出各实验组的相关系数。每个圆点为一组固定 (N, D) 的相关，菱形为组间平均；B6/B7 的点多数为负，B8 多数为正，方向差异并非个别组造成。另外，B6 与 B7 点群接近部分来自记录重复，不能把相似图形再算作一次独立验证。



圆点：一个 (N, D) 组；菱形：组间均值。
均为半合成数据；B7 含 B6，不构成独立证据。

图 13 质量方向检验

确认方向后，还需区分评分尺度。问题一的综合评分与附件 B 的 Q_{score} 计算方法不同，同为某个分值未必代表同等质量。因此，本节估计质量参数、解释质量增量时，统一使用附件 B 的评分尺度。现有资料没有提供两种评分的共同标定样本；问题三将恒等映射作为基准情景，并在表20中检查改变这一假设的影响。

5.2.3 模型建立

1. 经典标度律

我们先固定参考质量，采用 Chinchilla 标度律 [2]，把验证损失 L 写成参数量与数据量的函数：

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta} \quad (20)$$

当 $N, D \rightarrow \infty$ 时，两项规模损失逐渐消失，剩下的就是 E 。参数或数据增加得越多，相应幂函数项越小；下降幅度由 A 、 B 决定，快慢由 α 、 β 决定。这样，在固定参考质量的前提下，便能分别计算扩参数和加数据的收益。

2. 质量项的选择与损失修正

质量可以通过两种方式进入模型：一是改变有效参数或有效数据量，二是单列质量不足带来的损失。我们据此比较四个候选形式，使用同一组联合样本，并查看 B6 拟合、B7 新增点预测相关及残差信息评分，结果列于表13。经典模型与候选模型都保留常数和幂函数结构，参数搜索限定在给定区间内；残差经 $\text{soft-}L_1$ 处理后求和，以减轻少数大残差对估计结果的影响。

四种形式的差别在于如何解释质量不足。将质量乘入数据量，相当于认为同样多的 token 能提供多少有效训练信息取决于质量；同时乘入参数与数据量，则让两类规模收益都随质量调整。两个加性形式保留了规模项，其中 $CQ^{-\gamma}$ 在 $Q = 1$ 时仍保留常数 C ，而 $C(1 - Q)^\gamma$ 在参考质量处恰为零。我们需要在同一规模基线上比较质量增量，因此后者的参考点更便于解释；最终仍以同样本拟合和扩展表现作为选择依据。

表 13 不同质量作用形式的拟合与扩展检验指标

比较指标	加性质量缺口 $C(1-Q)^\gamma$	质量同时乘 N 与 D	加性质量亏损 $CQ^{-\gamma}$	质量作为 有效数据乘子
联合样本 R^2	0.9896	0.9882	0.9848	0.9745
B6 样本 R^2	0.9556	0.9495	0.9349	0.8905
B7 新增点 r	0.991	0.991	0.991	0.983
待估参数个数	7	7	7	6
残差信息评分 I_{RSS}	-10236	-10042	-9660	-8866

注：相关系数保留三位小数；加粗的最优指标按未舍入值确定。 I_{RSS} 越小越优，仅用于同样本比较。

表13中，四种质量项形式对 B7 新增点的相关都达到 0.983 以上，单看相关很难取舍。进一步比较联合样本与 B6 的 R^2 后，我们发现加性质量缺口形式均为最高，因而采用该形式。相关系数之外，再用 RMSE、MAPE 衡量预测值的偏差。

辅助比较量记为 $I_{\text{RSS}} = n \ln(\text{RSS}/n) + 2k$ ， n 表示样本数， k 表示参数数。估计采用 soft- L_1 ，并非高斯极大似然，因此不将该值严格解释为 AIC，只在相同样本之间比较。此外，B7 新增点参与过模型形式选择，其扩展检查仍属于开发阶段验证，不是完全独立的最终测试。

参数估计的目标与约束。为统一各候选形式的比较，我们记第 i 条记录的残差为 $r_i(\boldsymbol{\theta}) = \hat{L}_i(\boldsymbol{\theta}) - L_i$ ， $\boldsymbol{\theta}$ 为相应形式的待估参数。现有计算所用目标可写为

$$\min_{\boldsymbol{\theta} \in \Theta} \frac{s^2}{2} \sum_{i=1}^n \rho\left(\frac{r_i(\boldsymbol{\theta})^2}{s^2}\right), \quad \rho(z) = 2(\sqrt{1+z} - 1), \quad s = 0.05. \quad (21)$$

残差较小时，该目标近似平方损失；残差较大时，其增长接近线性，因而不会按残差平方的速度持续放大单条记录的影响。其中 s 为求解时设定的残差尺度，不表示验证损失的标准差或置信水平。联合拟合对 B1 与 B6 的记录逐条使用同一目标，没有另外按来源重新加权；各来源的记录数因此仍会影响其在拟合中的作用。结果解释同时保留 B6 单独的评价指标，便于看到加入质量项后质量实验本身的拟合情况。

对选定的加性质量缺口形式， $\boldsymbol{\theta} = (E, A, \alpha, B, \beta, C, \gamma)$ ，已有搜索范围为

$$\begin{aligned} 0 \leq E \leq 5, \quad 10^{-8} \leq A, B \leq 10^3, \quad 0.01 \leq \alpha, \beta \leq 2, \\ 10^{-8} \leq C \leq 50, \quad 0.05 \leq \gamma \leq 10. \end{aligned} \quad (22)$$

正系数与正指数使参数项、数据项随各自规模增加而减小，也使质量缺口项在 $0 \leq Q \leq 1$ 内保持非负。上界用于限定数值搜索，不能当作观测得出的参数区间。广义模型以经典拟合的五个参数为起点，再为 C, γ 赋初值；这提供了从规模基线进入联合估计的衔接方式，并不把经典参数固定下来。最终仍以整个联合样本重新估计七个参数。

将选定的质量缺口项加入规模基线后，损失关系写为：

$$L(N, D, Q) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta} + C(1-Q)^\gamma, \quad C > 0, \gamma > 0 \quad (23)$$

质量的参考点设在 $Q = 1$ 。在此处质量项为零，函数便回到经典标度律形式，但联合拟合重新估计各项参数，并不要求数值也回到经典拟合结果。Pythia 没有实测质量，给它取 $Q = 1$ 只是一项参考约定。在参考配方下固定 Q ，再让参数和数据量同时增大，则有

$$\lim_{N, D \rightarrow \infty} L(N, D, Q) = E + C(1 - Q)^\gamma \geq E \quad (24)$$

只要固定的质量仍低于 1，上式就会保留质量缺口。扩参数、加数据能够压低各自的规模项，却无法在这个加性模型中完全抵消质量不足。该结论来自所选函数的结构，实际使用时仍须限制在有数据支持的范围内。

3. 边际效用与弹性

为刻画各因素微小变化对损失的影响，对广义模型求偏导，并在参考点 (N_0, D_0, Q_0) 处评价：

$$\frac{\partial L}{\partial N} = -\alpha AN^{-\alpha-1}, \quad \frac{\partial L}{\partial D} = -\beta BD^{-\beta-1}, \quad \frac{\partial L}{\partial Q} = -\gamma C(1 - Q)^{\gamma-1} \quad (25)$$

各偏导的单位不同。令参考点的预测损失为 L_0 ，按变量与损失的比例变化定义弹性：

$$\varepsilon_N = \frac{\partial L}{\partial N} \frac{N_0}{L_0}, \quad \varepsilon_D = \frac{\partial L}{\partial D} \frac{D_0}{L_0}, \quad \varepsilon_Q = \frac{\partial L}{\partial Q} \frac{Q_0}{L_0} \quad (26)$$

4. 质量提升与参数扩张的等效条件

我们从同一起点比较两条改善路径：保持 D 与配比不变，分别小幅增加质量或参数；若损失降幅相同，就以所需正向增量之比 R_{NQ} 描述这两种方案：

$$\begin{aligned} \left| \frac{\partial L}{\partial Q} \right| dQ &= \left| \frac{\partial L}{\partial N} \right| dN_{\text{eq}}, \\ R_{NQ} &= \frac{dN_{\text{eq}}}{dQ} = \frac{\gamma C(1 - Q)^{\gamma-1}}{\alpha AN^{-\alpha-1}} = \frac{\gamma C}{\alpha A} (1 - Q)^{\gamma-1} N^{\alpha+1}. \end{aligned} \quad (27)$$

这里比较的是两种改善方案的投入增量。如果沿着同一损失水平移动，则应取 $dN/dQ = -R_{NQ}$ ，含义是质量提高后可减少参数。在 Q 不变时， R_{NQ} 按 $N^{\alpha+1}$ 增长，这是因为大模型再扩容的边际收益较小。上述比较只要求损失效果相等，是否节省算力，还要计算语料处理和训练各自的费用。

对于较大的质量增量 ΔQ ，我们直接计算完整损失差，避免将起点导数用于整个变化区间。记质量改善带来的降幅为 δ_Q ，令参数扩张取得相同降幅，解得

$$\begin{aligned} \delta_Q &= C[(1 - Q)^\gamma - (1 - Q - \Delta Q)^\gamma], \\ \Delta N_{\text{eq}} &= N \left[\left(1 - \frac{\delta_Q N^\alpha}{A} \right)^{-1/\alpha} - 1 \right]. \end{aligned} \quad (28)$$

有限等效解有两个条件。质量增量先要满足 $0 < \Delta Q \leq 1 - Q$ ，保证不越过上界；损失改善还要满足 $\delta_Q < AN^{-\alpha}$ ，因为参数扩张最多只能消除当前的参数项 $AN^{-\alpha}$ 。若两者相等，需要无限参数才能等效；若改善超过这一项，单靠扩容便无解。随着 $\Delta Q \rightarrow 0$ ，完整关系退回 $\Delta N_{\text{eq}} \approx R_{NQ} \Delta Q$ 。即使公式有解，超出拟合规模的数值也仍属外推。

式(28)可从两条改善路径直接得到。质量路径保持参数量为 N ，参数路径保持质量为 Q ，而两者的数据量相同。比较终点损失时，常数项和数据项相消，因此等效条件只剩

$$\begin{aligned} A[N^{-\alpha} - (N + \Delta N_{\text{eq}})^{-\alpha}] &= \delta_Q, \\ \left(1 + \frac{\Delta N_{\text{eq}}}{N} \right)^{-\alpha} &= 1 - \frac{\delta_Q}{AN^{-\alpha}}. \end{aligned} \quad (29)$$

右端表示质量改善量相对于当前参数项的比例扣减。它为正时才能反求有限的正参数增量；越接近零，反求的规模越大。这样，等效关系的可行条件便与参数项本身能够下降的空间对应起来。局部公式和有限公式的差别也在此：局部换算只使用起点导数，有限换算则计入扩容过程中参数边际收益持续降低的影响。两者服务于不同大小的改变量，不能把局部替代率乘以任意质量增量后都视为精确结果。

5. 领域效应的相似性

同一训练领域在不同验证任务上的作用也可能不同。问题一给出了领域 i 、 j 在 13 个任务上的系数向量 $\mathbf{a}_i, \mathbf{a}_j$ 。先减去平均向量，再用余弦相似度比较它们的相对作用：

$$s_{ij} = \frac{\langle \mathbf{a}_i - \bar{\mathbf{a}}, \mathbf{a}_j - \bar{\mathbf{a}} \rangle}{\|\mathbf{a}_i - \bar{\mathbf{a}}\| \|\mathbf{a}_j - \bar{\mathbf{a}}\|} \quad (30)$$

用 17 个领域系数向量的均值 $\bar{\mathbf{a}}$ 作中心后， s_{ij} 接近 1 表示两域对不同任务的相对作用接近，接近 -1 表示方向相反，可能存在任务取舍。但余弦值只保留方向，没有保留幅度，原回归也没有交互项。因此，它能帮助寻找可比较的领域，却不足以证明等量替换无损或混合后有额外收益。

5.2.4 模型求解

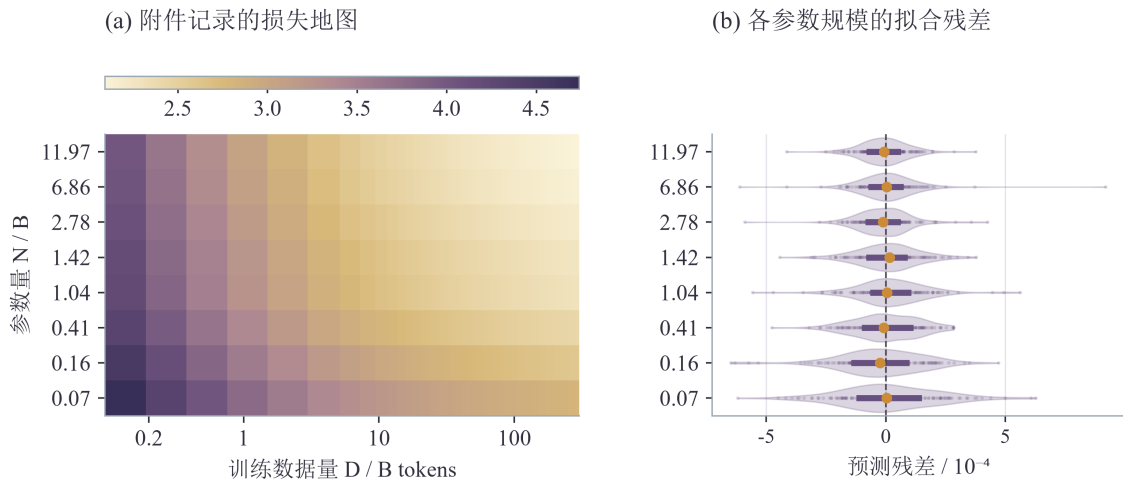
辅助分析说明：本问的文段相关辅助参考腾讯 WorkBuddy 接入的 DeepSeek-V4-Flash (2026-07-31 公开测试)，图表展示、程序整理与复现核对使用 OpenAI Codex (GPT-5.6-Luna, 2026-07-09) [14, 15, 16]。正文表达与段落衔接另经 OpenAI Codex 辅助润色。

1. 经典标度律拟合与留出验证

首先用 B1 的 1,176 条记录估计经典模型。图14左侧展示损失与两类规模的关系，右侧比较不同规模模型的残差，拟合得到

$$L(N, D) = 1.6898 + \frac{0.3540}{N^{0.3400}} + \frac{1.2403}{D^{0.2799}} \quad (31)$$

在 Pythia 族内，留出较大模型后预测仍较准确。将最小的 4 个模型用于重估参数，另外 4 个留作预测，检验结果为 $R^2 = 0.9999997$ 、 $\text{MAPE} = 0.005\%$ 。全量拟合的对应结果是 $R^2 = 0.9999998$ 、 $\text{MAPE} = 0.004\%$ ，其 RMSE 为 1.5×10^{-4} 。两种结果接近，说明所考察族内的规模关系较稳定。



B1: 8 个模型 $\times$ 147 个检查点；左图颜色表示验证损失。
右图：记录点、四分位区间及金色中位点；轮廓表示平滑密度。

图 14 经典标度律拟合

这组记录的损失标准差为 0.342，拟合残差标准差仅 1.5×10^{-4} ，二者相差较大。这样的吻合程度只对应当前来源，不能直接推广到其他训练资料。因此，下一步仍以 B4、B5 的外部记录及 B2 半合成轨迹作对照。

还可从 B3 检查数据量的幂指数。对 8 条插值轨迹分别固定 N ，扣除 $E + AN^{-\alpha}$ 后，将剩余损失 $L - (E + AN^{-\alpha})$ 对 D 作双对数回归。指数落在 $-0.2806 \sim -0.2803$ ，接近模型的 $-\beta = -0.2799$ 。B3 的生成依赖 Pythia 检查点插值，故这项对照只检验同一资料内部的曲线一致性，并未引入独立训练证据。

2. 广义标度律参数估计

再将 Pythia 的规模记录与 B6 质量实验合并，估计表13选出的广义形式。对没有质量实测值的 Pythia 仍按约定取 $Q = 1$ ，各项系数和指数列于表14。

表 14 广义标度律联合拟合的系数与指数

损失组成	系数符号	系数估计	指数符号	指数估计
参考下界	E	1.6558	—	—
参数规模项	A	0.3808	α	0.3266
训练数据项	B	1.2515	β	0.2767
质量缺口项	C	0.3544	γ	0.9779

注：联合拟合 $R^2 = 0.9896$ ； E 为参考配方在 $Q = 1$ 时的模型渐近下界，不含衰减指数。

表14中的 $\gamma = 0.9779 \approx 1$ ，使质量缺口对应的渐近损失近似线性增加。联合拟合的 RMSE 为 0.036，MAPE 为 0.66%， $R^2 = 0.9896$ 。联合样本加入了质量实验，与经典拟合使用的样本不同，因此两个 R^2 不能直接作差来衡量质量项的收益。

我们不直接比较这两个 R^2 ，因为其分母取决于评价样本相对均值的总平方差。加入 B6 后，损失分布、样本均值和分母均已改变，不能将联合拟合的数值下降归因于质量项。表13在同一联合样本上比较候选形式，才使残差大小具有一致的评价参照；其中再单列 B6，是为了避免仅凭规模记录较多的联合指标判断质量项。

估计结果还与质量参考点有关。对取 $Q = 1$ 的 B1 记录， $C(1 - Q)^\gamma$ 恒为零，因此这部分记录主要约束常数与规模项； C, γ 的质量变化信息来自 B6。联合估计允许规模参数随 B6 的加入一同调整，不能把表14理解为在完全固定的经典模型上单独追加两个质量参数。后续弹性与替代率采用这组联合参数，才与所选损失函数保持一致。

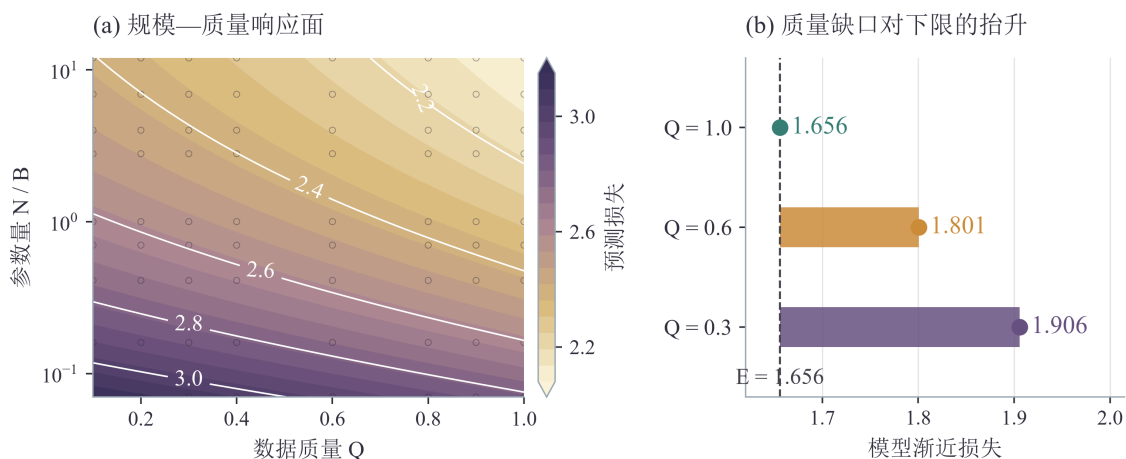
在 B6 的 45 个固定 (N, D) 组内，质量从 0.1 提高到 1 的给定损失降幅为 0.1067–0.6091，中位数 0.3142，与参数量的 Spearman 相关为 -0.7489 ；加性模型却对各组统一给出 $C 0.9^\gamma = 0.3197$ 。这表明整体拟合指标会掩盖部分规模差异。后续配置保留该加性近似；是否需要交互项，还须通过按规模组留出等比较判断，半合成附件中的组内差异本身不能代替这种检验。

图15说明质量对损失的影响。左图固定 $D = 300\text{B tokens}$ ，用底色和等值线表示不同规模、质量组合的预测损失，空心点为 B6 实验位置；右图以 E 为起点，展示质量不足使损失下限增加多少。

从 $E + C(1 - Q)^\gamma$ 可直接看出损失下限随质量的变化：满分参考 $Q = 1$ 对应 1.656；若只达到 $Q = 0.6$ ，下限为 1.801；降至 $Q = 0.3$ 时，下限为 1.906。

3. 分层检验与适用范围

为判断拟合参数能够用于哪些配置情景，我们进一步比较不同来源上的预测误差。表15汇总 6 组检验的 MAPE、RMSE 和相关系数，图16展示 $100(\hat{L} - L)/L$ 的分布：负值为低估，正值为高估。为同时看清小误差和大偏差，图中横轴在 $\pm 10\%$ 内采用线性刻度，外侧改用对称对数刻度。



D=300 B tokens; 空心点: B6 实验位置; 底色与等值线: 模型预测。

图 15 质量项影响

我们联合使用这三类指标，以同时检查变化趋势与数值偏差。相关系数用于查看预测是否跟随样本损失的变化，整体偏高或偏低并不一定明显降低相关；RMSE 保留损失本身的量纲，反映绝对偏差；MAPE 则以各条记录的损失为参照，把误差换成相对比例。相同的绝对误差在低损失记录上对应更大的百分比，因此跨来源比较不能只依赖其中一个指标。图中的带符号相对误差进一步保留了偏差方向，补充了表中取绝对值或平方后无法直接表达的信息。

表 15 按数据性质区分的预测误差与相关性

数据性质	对照样本	记录数	MAPE/%	RMSE	r
真实	B4 跨模型族收敛点	57	8.8	0.293	0.975
	B5 文献标度记录	44	8.0	0.193	0.910
	B7 新增质量实验	90	1.6	0.056	0.991
半合成	B2 Cerebras 轨迹	1,029	33.7	1.245	0.666
	B8 校准段（方向变换）	984	133.4	1.294	0.638
估算	B10 大规模估算损失	128	1.0	0.020	1.000

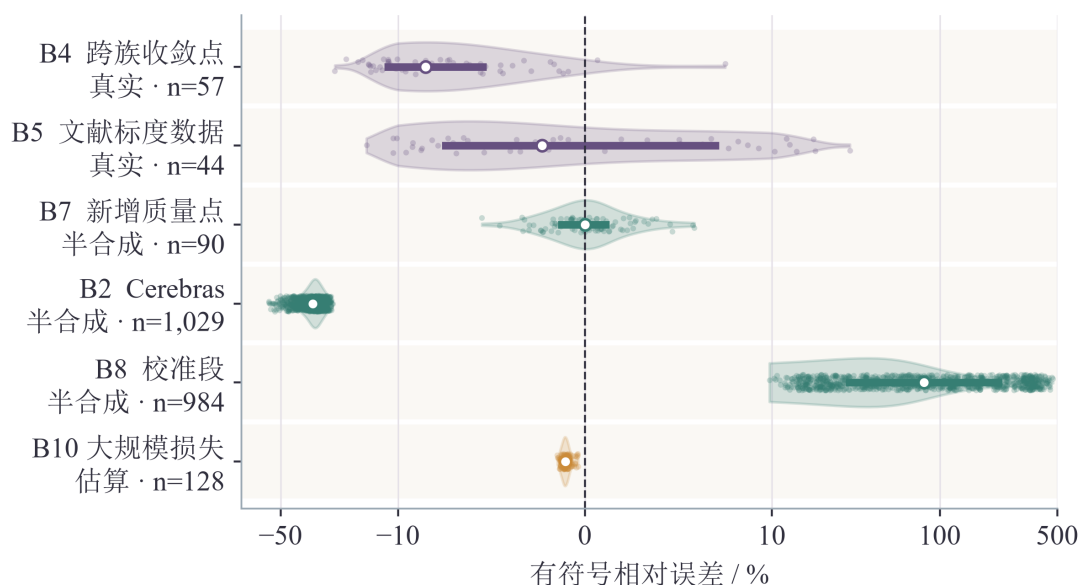
注：B8 使用 $1 - Q_{\text{score}}$ ；B10 统一取 $Q = 1$ 。B7 属于开发阶段验证，B10 不构成独立实验验证。

表15中，B4 的相关为 0.975，B5 为 0.910；相应 MAPE 仍有 8.8% 和 8.0%。由此，我们认为模型保留了部分跨族变化趋势，但其绝对损失预测仍存在迁移偏差。

对于同一半合成设定，B7 的 90 个新增点有 1.6% 的 MAPE，支持质量关系在该设定内作一定扩展。B10 的含义不同：损失本就由估算得到，比较时又统一设为 $Q = 1$ ，所以只检查两种估算是否接近，不检验质量效应，也不属于独立实验验证。

B2 的 MAPE 达到 33.7%；B8 校准段在翻转质量评分后仍有 133.4%，部分损失甚至低于模型渐近下限。可见问题不止是分数方向；损失尺度和数据生成方式不同，也会使这些来源无法直接合并使用。

上述判断还受样本覆盖范围限制。B1 的参数约为 0.07–12B，数据量为 0.134–299.893B tokens；B6 的相应范围为 0.07–11.97B 和 10–600B tokens。对质量效应的判断依赖 B6 的半合成设定及 B7 新增记录的对照，更大规模的估算资料仅能检查外推关系之间的一致性。



相对误差 = $100 \times (\text{预测值} - \text{给定值}) / \text{给定值}$ ；横轴在 $\pm 10\%$ 内线性。
B8 使用 1-Q；空心点：中位数；粗线：四分位区间；轮廓：平滑密度。

图 16 分层验证误差

4. 大规模模型的对应与外推范围

B9 提供大模型元数据，B10 提供估算损失，两者先按模型名称和模型族名称配对。B10 的 128 条记录都在 B9 中找到唯一对应，且每对的 N 、 D 相同。B9 共 132 条，剩余 4 条的数据量为 0，又没有 B10 损失，故未进入对照。表 16 按规模汇总了匹配结果。

表 16 大规模匹配样本的覆盖范围与估算偏差

参数规模区间 N/B	[100, 300)	[300, 1000)	[1000, 10000]
成功匹配记录数	65	43	20
最小数据量 / B tokens	1.3	0.3	0.1
最大数据量 / B tokens	36,000	30,000	36,000
MAPE / %	0.94	1.08	1.12

注：B9 元数据与 B10 损失按名称匹配；统一取 $Q = 1$ ，MAPE 比较本文预测与 B10 估算值。数据量上下限为各组记录的实际极值。

这批记录的规模为 100–10,000B，已经超过主拟合范围。分组误差约 1%，表示本文预测与 B10 估算较接近；但 B10 同样由标度律产生，因此低误差不能代替真实训练检验，更不能说明质量项在这些规模下已经得到验证。

5. 质量投入的局部收益与规模替代

我们选取两个参考配置，比较各项投入小幅变化时的收益。表 17 同时列出弹性和质量–参数等效增量比，前者比较相同比例变化的影响，后者给出相同损失降幅下的局部换算。

比较两种局部指标时，需要区分质量的相对变化与评分的绝对增量。弹性比较的是 dQ/Q_0 和 dN/N_0 ，而 R_{NQ} 的分母为 dQ ；两者采用的增量口径不同。质量评分又有上界，其相对变化所对应的绝对增量取决于起点评分。因而应用表 17 时，应先给定起点和改变量，再选择弹性近似或有限等效关系。换到其他 (N, D, Q) 后，表中的数值需要重新评价，不能把某个参考点的换算比例作为固定系数贯穿所有规模。

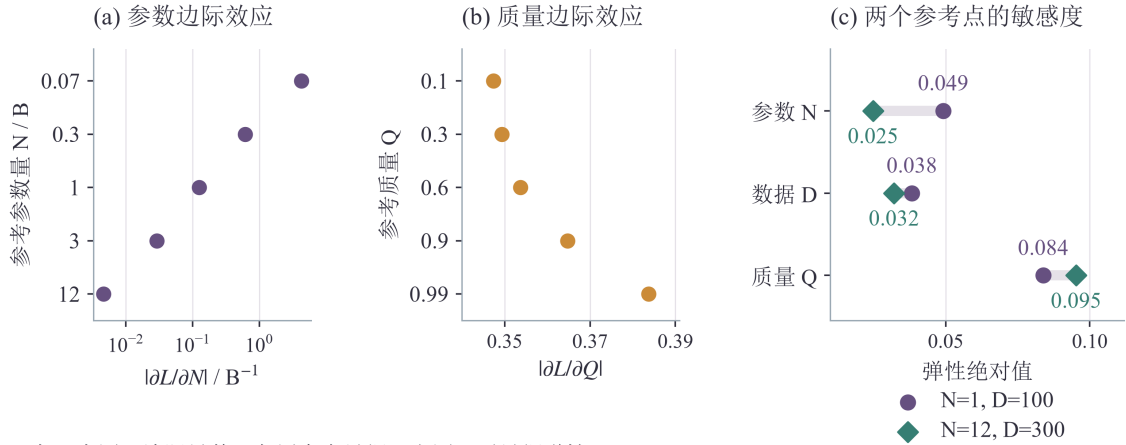
表 17 两个参考情景下的局部敏感度与等效增量比

指标	参考情景一	参考情景二
	$N_0 = 1, D_0 = 100$ $Q_0 = 0.6$	$N_0 = 12, D_0 = 300$ $Q_0 = 0.6$
参数量弹性 ε_N	-0.0491	-0.0248
数据量弹性 ε_D	-0.0383	-0.0321
质量弹性 ε_Q	-0.0838	-0.0953
局部等效增量比 R_{NQ}	2.844	76.838

注： N_0 以 B 参数计， D_0 以 B tokens 计；弹性无量纲， R_{NQ} 的单位为 B/单位质量。弹性保留方向，图中展示其绝对值。

在 $(N_0, D_0, Q_0) = (1, 100, 0.6)$ 处，质量弹性绝对值约为参数弹性的 1.7 倍。因此，参数量与质量分别提高同一较小比例时，提高质量对应的损失降幅更大。两个参考点的局部替代率为 2.844 和 76.838 B/单位质量；按一阶近似，0.1 的质量增量分别对应 0.284B、7.684B 参数。这里只比较损失降幅，有限变化仍须用式(28)计算并检查拟合范围，不能据此判断成本。

图17把参考位置的变化展开。前两图分别展示参数、质量的边际导数绝对值，由于单位不同，两图数值不能直接比较。第三图使用无量纲弹性，比较两个参考点上参数量或质量发生相同微小相对变化时的损失响应。



左、中图：边际导数，各用自身量纲；右图：无量纲弹性。
仅右图两个参考点均取 $Q=0.6$ ； N 、 D 以 B 为单位。

图 17 边际效用与弹性

固定 $D = 300$ B tokens 后，还可以反求达到目标损失需要的质量，见图18。三个子图各对应一个目标，横轴为参数量，纵轴为质量；实线在主拟合范围内，虚线为外推。对于 $L = 1.95$ ，图中特别放大了高质量区间，并将参数轴延至 10^4 B 以展示理论边界，这不表示该规模在工程上可行。

我们将边际比较限定在远离质量满分的区间。拟合得到 $\gamma = 0.9779 < 1$ ，使 $|\partial L / \partial Q|$ 在 $Q < 1$ 内逐渐增大，并在 $Q \rightarrow 1^-$ 发散。因此，边际数值只在离端点较远的位置解释，不能直接用满分附近的导数建议实际投入。

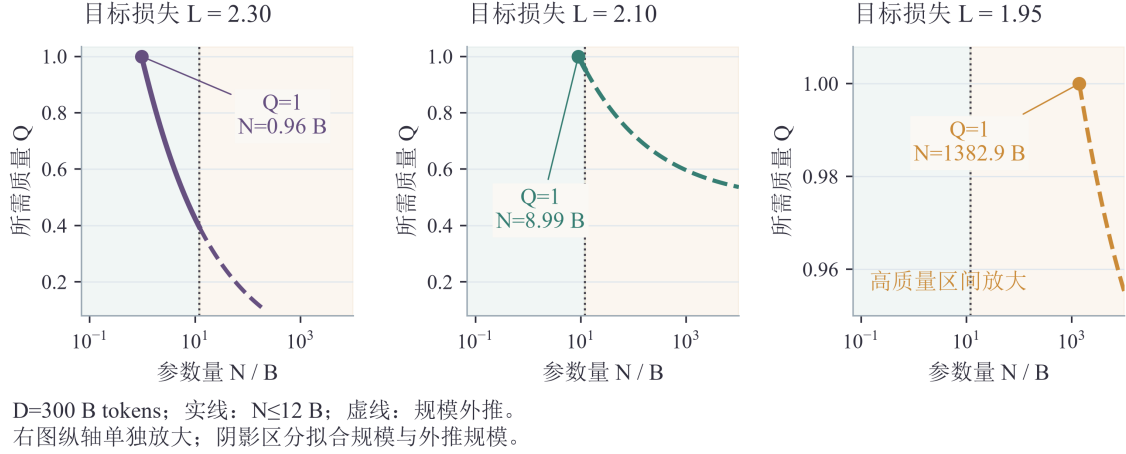


图 18 规模与质量替代

这一端点性质也会影响预算配置。按问题三的费用消去数据量，固定可行参数量，并在规模边界不活跃、质量成本光滑且有限的条件下，从 $Q = 1$ 向内移动 $\varepsilon > 0$ ，损失差可写为

$$\Delta L = C\varepsilon^\gamma - \kappa\varepsilon + o(\varepsilon), \quad \kappa > 0. \quad (32)$$

其中 κ 表示略降质量后，释放预算、增加数据量带来的局部降损系数。当 $0 < \gamma < 1$ 时，第一项在足够小的邻域内占优，使满分端成为一侧局部极小，但不保证它在所有预算下都是全局最优。为检查该性质对结果的影响，我们固定 $\gamma = 1$ ，在相同 B1+B6 样本、边界和 soft- L_1 目标下仅重估其余六个参数，结果见表18。

表 18 质量指数受限拟合与原饱和预算处的配置对照

参数情景	γ	稳健目标值	联合 RMSE	剖面 Q
原拟合参数	0.9779	0.576864	0.035557	1.0000
固定指数的受限拟合	1.0000	0.577003	0.035553	0.9640

注：剖面比较采用问题三的幂函数成本、参考配比及 2048 tokens 上下文，预算为原扫描首次饱和点 7.07946×10^{19} FLOPs；显式比较质量端点与局部候选。受限结果仅用于诊断，不替换表14的参数。

受限拟合的稳健目标仅增加 0.0241%，原饱和预算处的质量却降至 0.9640，说明拟合接近并不保证阶段位置稳定。这是函数形状对照，不是参数置信区间。另在 10^{19} 、 6.31×10^{19} 、 7.08×10^{19} 和 10^{22} FLOPs 四处独立求质量剖面，损失与保存解的差均小于 10^{-7} ；该抽查不构成全局最优证明。即使指数略高于 1，高预算内点仍可能在数值精度内接近满分，所以这一对照主要限制饱和边界的解释。

6. 领域效应与配比衔接

为观察领域之间的关系，图19左侧按 $d_{ij} = \sqrt{2 - 2s_{ij}}$ 对 17 个训练领域作平均连接聚类；右侧列出相似度最高和最低的各四对领域，用绿色和金色分别表示作用趋同和相反。该图比较的是各域影响验证任务的方式，并不检验混合使用是否产生额外增益。

在图19中，作用接近的例子有百科与网页、医学全文与医学摘要，相似度分别为 0.925、0.903；医学全文与邮件为 -0.967 ，则偏向不同的验证任务。判断领域能否替换时，我们进一步计入系数幅度与任务权重。若从领域 i 转移占比 h 到 j ，保证总量不变且 $0 \leq h \leq p_i$ ，预测加权损失的变化为

$$\Delta \hat{L}_A = h \mathbf{w}^\top (\mathbf{a}_j - \mathbf{a}_i), \quad (33)$$

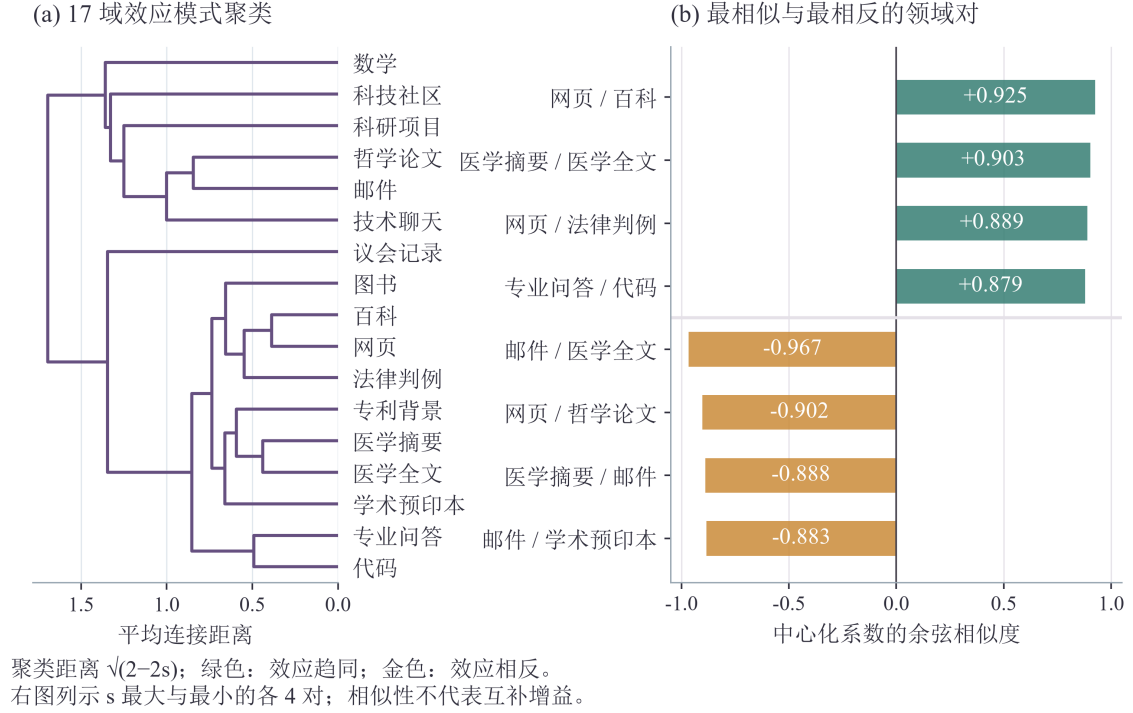


图 19 领域相似性

w 为验证领域权重。上式为负，表示转移后总体损失下降；为零表示相当；为正则表示变差。相似度接近的两域仍可能有幅度差，方向相反也只说明任务取舍。由于没有建立交互项，不能从这个线性模型再推断额外的混合收益。

配比收益采用问题一的核岭模型计算。保持验证领域和权重相同，训练配方凸包内的推荐配比相对参考配比给出

$$M = \hat{L}(\mathbf{p}_{\text{rec}}) - \hat{L}(\mathbf{p}_{\text{ref}}) = -0.1730.$$

这个差值来自 1M 规模的核岭预测。将其用于广义标度律，需要配比效应可与规模、质量分离，并允许绝对损失差跨来源迁移。RegMix 的跨尺度检验主要支持排序关系 [7]，不能据此认定同一损失差在目标规模仍然成立。问题三因此将 M 作为固定平移情景，并检查收益幅度改变时的结果。

在上述分离与迁移条件下，规模、质量和配比的关系可合写为

$$L(N, D, Q, \mathbf{p}) = E + AN^{-\alpha} + BD^{-\beta} + C(1 - Q)^{\gamma} + M(\mathbf{p}),$$

$$M(\mathbf{p}) = \sum_{d \in \mathcal{M}} w_d \left[\hat{L}_d^K(\mathbf{p}) - \hat{L}_d^K(\mathbf{p}_{\text{ref}}) \right]. \quad (34)$$

其中 $\hat{L}_d^K$ 是第 d 个验证任务的核岭预测， $\mathcal{M}$ 为有验证记录的任务集合。在参考配比处 $M(\mathbf{p}_{\text{ref}}) = 0$ ；配比固定后，该项不改变 N, D, Q 的偏导或等损失替代条件，弹性仍按参考配比报告。相应渐近下界为 $E(\mathbf{p}) = E + M(\mathbf{p})$ ，故 E 仅是参考配方的基准。若把当前 M 固定延伸到无限规模，将得到约 1.4829 的下界；这是加性模型的推论，绝对收益能否保持仍需跨规模验证。

本问的计算表明，参数规模越大，继续扩容的边际收益越小；质量能够等效替代的参数量则随起点变化。我们将所选损失函数用于下一问的预算配置。跨族和文献检验仍分别存在 8.8%、8.0% 的 MAPE，故后续使用拟合参数时，也要一并保留质量尺度、数据来源和外推范围的条件。

5.3 问题三：算力约束下的资源配置与结构性转移

5.3.1 问题分析与模型准备

我们在同一训练预算下联合配置参数量、数据量与质量。扩大参数规模会降低参数项损失，同时提高每个 token 的训练费用；改善质量会缩小质量缺口，同时增加整份语料的处理成本。两项投入都会挤占数据训练预算，因此配置时需要比较直接收益与数据量减少带来的损失。

已有研究给出了固定算力下参数和数据的配置关系 [2]。考虑到题设还包含质量处理与长上下文计算，我们以问题二的损失函数为目标，加入相应费用约束，先求不同预算下的配置，再检验结论对成本形式的依赖。

图20梳理了本问的计算过程。先汇入前两问的质量基线、配比修正与损失参数，再将训练、质量处理和注意力费用写入预算约束。多初值求解后，分别比较三档预算配置、连续预算中的质量投入阶段，以及成本形式和上下文改变时的配置差异。

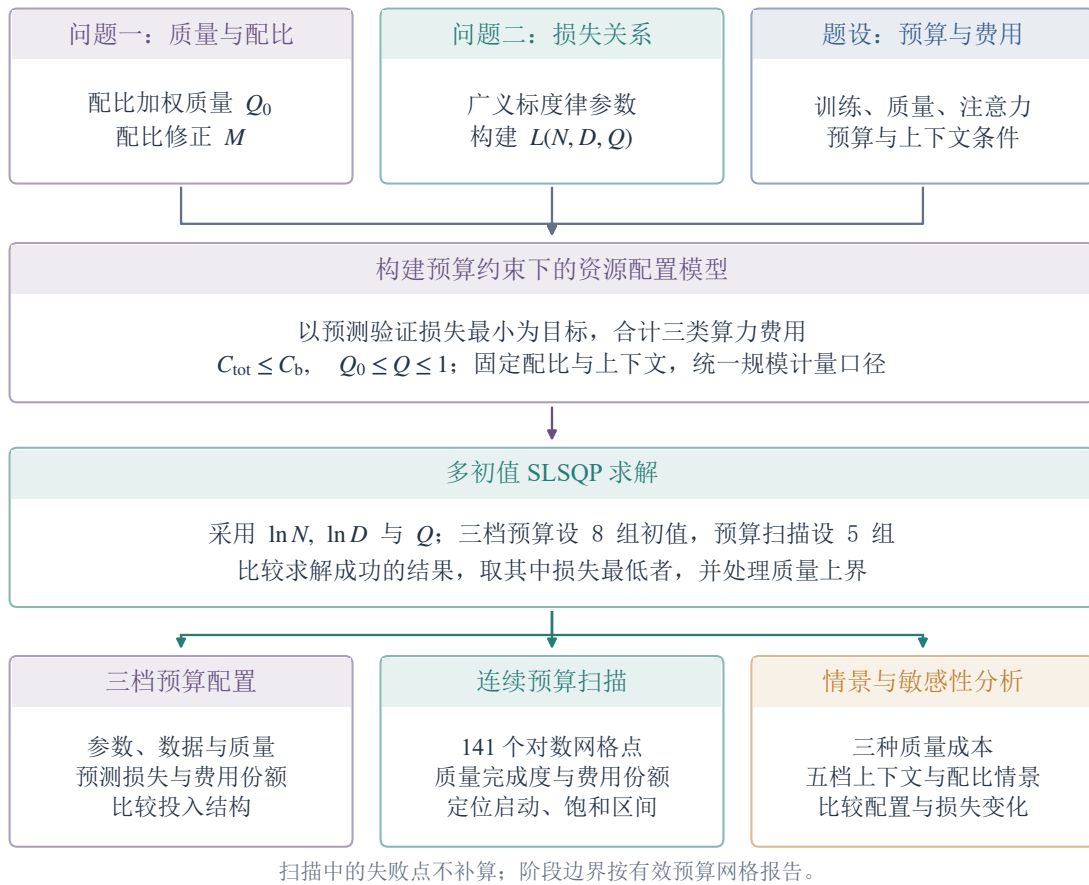


图 20 问题三资源配置与求解流程

数据与计算口径

本问直接使用问题二估计的损失参数。其中规模项为 $A = 0.3808$ 、 $\alpha = 0.3266$ 、 $B = 1.2515$ 、 $\beta = 0.2767$ ，质量项为 $C_q = 0.3544$ 、 $\gamma = 0.9779$ ，常数项为 $E = 1.6558$ 。质量系数改记 C_q ，以区别预算 C_b 。计算费用时， N 、 D 分别用参数个数与 token 个数；代入损失式时再换成 $n = N/10^9$ 、 $d = D/10^9$ 。下列表格的 B 均表示十亿。

求解器以绝对数量的对数为变量，计算目标和约束时还原规模，再分别用于损失与费用计算。后文的一阶条件也先换算系数，再对绝对数量求导，保持两种计量口径一致。

我们先以参考配方确定质量处理的起点。用 A4 的平均配比 $\bar{p}_k$ 加权问题一经 A16 映射的领域评分 Q_k ，得到基线

$$Q_0 = \sum_{k=1}^{17} \bar{p}_k Q_k = 0.5487.$$

这里沿用 A4 原始平均份额得到外生成本基线 $Q_0 = 0.548688$ 。问题一求核岭配比时将参考份额归一化至和为 1，其加权质量为 0.548753；两种口径分别记录，不因配比归一化而重设本问的费用起点。全量文本均值 0.4912 另作灵敏度对照。

记问题一评分为 Q_A 、附件 B 评分为 Q_B ，本问采用 $Q_B = h(Q_A) = Q_A$ ，并在共同数值轴上计算费用。恒等映射是缺少配对标定资料时的情景假设。优化范围为 $Q_0 \leq Q \leq 1$ ，满分只表示模型质量上界。

配比比较保持共同 Q_0 ，只对超过该基线的质量计费。核岭候选自身的加权质量为 0.5182，换配方后原料质量并不相同。若按各配方质量 $Q_0(\mathbf{p})$ 分别计费，就需要另行求解配比与处理成本的耦合；本节先单独比较固定损失平移，随后给出更换费用基线的对照。

基准情景取参考配比 $M = 0$ 、幂函数成本及 2048 tokens 上下文；成本类型和上下文依次改变。附件 C7 的 2048、4096、8192、32768、131072 tokens 均作为给定条件，求解器只调整参数、数据与质量。此外，将核岭配对应应的 $M = -0.1730$ 单列为收益情景，检查损失与等效预算如何变化。

5.3.2 模型建立

1. 每个训练 token 的算力代价

基础训练使用 $6ND$ 的成本近似 [2]，质量与注意力费用按照题目第三问及题目附录 B 的成本设定计算。由于三类 $g(Q)$ 在允许范围内均递增，可记质量提升的单位增量费用为 $\Delta g(Q) = g(Q) - g(Q_0) \geq 0$ ，从而将总成本写为

$$C_{\text{tot}} = 6ND + D\Delta g(Q) + \eta NDL_{\text{ctx}} = D \underbrace{[(6 + \eta L_{\text{ctx}})N + \Delta g(Q)]}_{\text{单位 token 的合计算力成本}}, \quad \eta = 2 \times 10^{-4}. \quad (35)$$

三项费用都随数据量 D 累计。给定 N, Q 时，每个新增 token 的单价不变；提高质量则增加整份语料的处理费用，压缩可用于训练的数据量。即使质量已达上界，新增数据仍需支付处理费用。

这里的 D 按累计训练 token 数使用，质量费用也按相同数量核算，尚未扣除重复使用语料可能带来的处理成本复用。若一份语料仅预处理一次、随后多轮训练，应区分独立语料量与累计训练量；当前算例沿用题设费用，不对这类复用作额外修正。

表19列出题设的三种质量成本。由于比较的是相对基线的增量 $\Delta g(Q)$ ，三种函数在 Q_0 处均不产生额外费用。

表 19 质量处理成本函数 (g 的单位为 FLOPs/token)

成本类型	$g(Q)$	本节用途
幂函数型	$5 \times 10^9 Q^4$	基准配置
指数型	$10^7 e^{6Q}$	成本形式对照
对数渐进型	$2 \times 10^9 \ln(1 + 10Q)$	成本形式对照

2. 预算约束下的配置模型

记 $k = 6 + \eta L_{\text{ctx}}$ 。给定配比和上下文长度后，以预测验证损失最小为目标，得到

$$\begin{aligned} \min_{N,D,Q} \quad & E + An^{-\alpha} + Bd^{-\beta} + C_q(1-Q)^\gamma + M, \\ \text{s.t.} \quad & D[kN + \Delta g(Q)] \leq C_b, \quad Q_0 \leq Q \leq 1, \\ & 10^5 \leq N \leq 10^{15}, \quad 10^6 \leq D \leq 10^{16}. \end{aligned} \quad (36)$$

末尾界限限定数值搜索范围，不代表实际数据供给。共同 Q_0 固定、配比可行域独立且 $M(\mathbf{p})$ 可加分离时，先最小化 M 再优化 N, D, Q ，与联合最小化等价；若基线或费用随配比变化，这种分离便不再成立。本文使用问题一多初值求得的核岭候选，不据此保证全局最优或真实训练收益。因此，“配比改变而配置不变”来自模型设定。

固定质量且规模界限不活跃时，增加 N 或 D 都能降低损失，预算约束因而取等号。将其写为

$$D = \frac{C_b}{kN + \Delta g(Q)} \quad (37)$$

预算式说明，提高 N 或 Q 后，能够负担的 D 都会减少。因此，我们比较一项投入的净收益时，同时计入其直接降低的损失与数据缩减增加的损失。

预算用尽可由数据项的单调性说明：在 N, Q 固定、数据上界未达到时，增加 D 只会减小 $Bd^{-\beta}$ 。若仍有余款，就还可降低损失。消去数据量后，可在预算边界上比较参数与质量；以下用该化简解释一阶条件，正式程序仍求解三变量约束模型。

由式(37)还可分别求得预算固定时的数据变化：

$$\left. \frac{\partial D}{\partial N} \right|_Q = -\frac{kD}{kN + \Delta g(Q)}, \quad \left. \frac{\partial D}{\partial Q} \right|_N = -\frac{Dg'(Q)}{kN + \Delta g(Q)}. \quad (38)$$

两项均为负，分别对应扩大模型和提高质量带来的数据缩减。质量成本的影响还取决于当前位置的斜率 $g'(Q)$ ，因而不能只比较不同成本函数在某一个质量水平上的取值。这也说明，费用形式相近的情景仍可能产生不同的质量选择。

3. 参数量与数据量的平衡

为统一求导单位，先令 $\tilde{A} = A(10^9)^\alpha$ 、 $\tilde{B} = B(10^9)^\beta$ ，把规模项改写为绝对数量形式。固定 Q 后，将预算等式(37)代入损失函数，可得

$$F(N) = \tilde{A}N^{-\alpha} + \tilde{B} \left[\frac{kN + \Delta g(Q)}{C_b} \right]^\beta.$$

此处省略的常数项和质量项在固定质量后不随 N 变化，不影响导数。参数项的导数为负，数据项则因预算挤占而产生正导数，两者相加为

$$F'(N) = -\alpha \tilde{A}N^{-\alpha-1} + \frac{\beta \tilde{B}k}{C_b^\beta} [kN + \Delta g(Q)]^{\beta-1}. \quad (39)$$

令该式为零，再乘以 $[kN + \Delta g(Q)]/k$ ，并用预算等式替换数据项，即可得到参数与数据损失的平衡关系。对应的平衡式为

$$\alpha \tilde{A}N^{-\alpha} \left(1 + \frac{\tau(Q)}{N} \right) = \beta \tilde{B}D^{-\beta}, \quad \tau(Q) = \frac{\Delta g(Q)}{k}. \quad (40)$$

为解释式(40)，可把 $\tau(Q)$ 看作质量费用折算出的附加参数规模，它与 N 单位相同。因而，因子 $1 + \tau/N$ 表示额外质量处理对规模平衡的影响。一旦为质量付费，原本只在参数与数据之间分配的方案就需要随之调整。

作为对照，保持基线质量时有 $\tau = 0$ ，于是 $\alpha \tilde{A} N^{-\alpha} = \beta \tilde{B} D^{-\beta}$ 。将其与 $kND = C_b$ 联立，在上下文固定时得到

$$N^* \propto C_b^{\beta/(\alpha+\beta)}, \quad D^* \propto C_b^{\alpha/(\alpha+\beta)}. \quad (41)$$

不额外提高质量时，数据量随预算增长稍快于参数量：两者的指数分别约为 0.541、0.459。这只是基线质量下的参照；开始投入质量后，应回到包含处理费用的完整模型计算。

基线幂律的比例系数也可由同一组方程求出。将 $D = C_b/(kN)$ 代入平衡条件，得到

$$\begin{aligned} (N^*)^{\alpha+\beta} &= \frac{\alpha \tilde{A}}{\beta \tilde{B}} \left(\frac{C_b}{k} \right)^\beta, \\ N^* &= \left(\frac{\alpha \tilde{A}}{\beta \tilde{B}} \right)^{1/(\alpha+\beta)} \left(\frac{C_b}{k} \right)^{\beta/(\alpha+\beta)}, \\ D^* &= \left(\frac{\beta \tilde{B}}{\alpha \tilde{A}} \right)^{1/(\alpha+\beta)} \left(\frac{C_b}{k} \right)^{\alpha/(\alpha+\beta)}. \end{aligned} \quad (42)$$

因此，预算通过幂指数决定规模的增长速度，系数比值决定具体配置水平。上下文固定时， k 只进入有效预算因子；质量费用不为零时，预算中多出无法并入 kN 的处理项，基线表达式便不再适用于完整配置。因此，该解析式只适用于没有质量增量费用的参照情形，用来解释规模权衡；完整配置仍由含质量费用的预算模型求解。

质量选择也可在同一预算边界上解释。当预算约束活跃、数据量未触及搜索上下界且 $Q_0 < Q < 1$ 时，固定 N 并对消去数据量后的损失求导，质量最优的必要条件为

$$\frac{\beta \tilde{B} D^{-\beta} g'(Q)}{kN + \Delta g(Q)} = \gamma C_q (1 - Q)^{\gamma-1}. \quad (43)$$

式中左侧是质量提升挤占数据预算后增加的损失，右侧是质量改善直接减少的损失。我们据此将内部配置解释为两项边际作用相抵的结果。若解位于基线或质量上界，应结合允许的变化方向判断，不能要求上述等式成立。尤其在上界处，质量项的导数性质与内部不同。因此，本问的阶段位置仍依据后文的预算扫描确定，一阶条件用于解释内部配置的局部权衡。

评分尺度的条件对照。为检查恒等映射的影响，我们保留 Q_A 轴上的幂函数费用，分别令 $Q_B = h(Q_A)$ 取 $\sqrt{Q_A}$ 、 Q_A 、 Q_A^2 。三者方向及端点相同，但固定 N, D 时，质量项关于处理评分的损失导数变为

$$-\frac{\partial L}{\partial Q_A} = C_q \gamma [1 - h(Q_A)]^{\gamma-1} h'(Q_A). \quad (44)$$

在参考配比、2048 tokens 上下文及基线规模最优曲线上，以质量收益与预算挤占损失的一阶平衡确定下界释放点，得到表20。该点说明何时从基线小幅提高质量开始有利，存在竞争端点时不一定等于全局阶段边界。

这些映射用于检验同向评分并不唯一决定投入阈值，没有被当作新的标定结果。基准情景仍用恒等映射；其他处理评分或实际费用若定义在不同尺度上，应先同步转换损失与费用，再使用相应配置。

4. 数值求解与阶段识别

我们采用 SciPy 的 SLSQP 实现 [18] 求解式(36)，以 $\ln N$ 、 $\ln D$ 缩小变量尺度差异，并将预算不等式等价写为 $1 - C_{\text{total}}/C_b \geq 0$ ，避免约束绝对量级过大；停止条件取 $\text{ftol}=10^{-14}$ ，单次求解最多迭代 1500 次。三档预算各设 8 组初值，连续预算扫描各设 5 组，从求解成功的结果中选取损失最低的一组，记作 (N^*, D^*, Q^*) 。这样可降低初值和局部解的影响，

表 20 评分映射改变时的局部质量投入释放点

映射 $h(Q_A)$	参考基线在 B 轴的值	释放预算 (10^{19} FLOPs)
$\sqrt{Q_A}$	0.7407	2.94
Q_A	0.5487	1.57
Q_A^2	0.3011	1.37

但不能保证全局最优。另因 $\gamma < 1$ 时 $Q = 1$ 的导数无界，将达到该上界的配置作为边界解处理。将保存的配置重新代入成本函数，三档预算、上下文敏感性与全部 141 个预算扫描点的最大相对预算残差均小于 10^{-12} 。这些数值核验说明记录的配置满足预算约束，但只验证可行性，不构成对全局最优性的证明。

对数变量使参数量与数据量的正值约束自然满足，原有规模上下界相应转换为对数区间。实际初值设置让参数量与数据量沿相反方向变化，同时使质量起点分布在允许区间内部，以比较不同资源组合出发后的求解结果。每次求解都保留总预算不等式和质量边界，而不是先独立确定三项投入后再缩放预算。成功状态用于筛选候选结果，边界配置则保留其实际质量值；未记录成功结果的预算点不参与后续阶段定位。多初值比较用于同一预算下的候选选择，相邻预算点仍分别求解，阶段变化依据有效输出识别。

质量投入的阶段用完成度 $\Theta = (Q^* - Q_0)/(1 - Q_0)$ 表示，质量处理占预算的份额用 $f_Q = D^* \Delta g(Q^*)/C_b$ 表示。完成度为 0 时维持基线，介于 0、1 时正在提升，等于 1 时达到上界。计算在 10^{18} – 10^{25} FLOPs 内设置 141 个对数等距点，相邻预算之比约为 1.122，逐点寻找首次投入与首次饱和的位置。采用无量纲预算约束后，141 个网格点均取得有效配置，阶段分析使用全部输出。

我们同时观察完成度与费用份额，以区分质量提升的进度和资源支出。前者以可提升的质量区间为分母，后者以总预算为分母；质量不变时，规模调整仍可能改变费用份额。因此，质量曲线变平不表示处理费用消失，阶段位置也只能由有效预算网格点定位。

5.3.3 模型求解

图中坐标、费用份额及网格判定的计算口径见附录 F。

辅助分析说明：本问的文段相关辅助参考腾讯 WorkBuddy 接入的 DeepSeek-V4-Flash (2026-07-31 公开测试)，图表展示、程序整理与复现核对使用 OpenAI Codex (GPT-5.6-Luna, 2026-07-09) [14, 15, 16]。正文表达与段落衔接另经 OpenAI Codex 辅助润色。

1. 预算增加后，质量与规模如何分配

比较表21中的三档配置，我们发现各项投入并未按相同比例增长。参数和数据都在增加，但数据更快， D/N 由 30.9 升到 76.1。质量在低预算档停留于基线，中、高预算档则达到上界，不能简单地按固定比例同时扩张三项投入。

表 21 三档预算的基准配置（幂函数型成本， $L_{\text{ctx}} = 2048$ ， $M = 0$ ）

预算 (FLOPs)	N^* (B)	D^* (B tokens)	Q^*	L^*	D^*/N^*	f_Q (%)
10^{19}	0.225	6.9	0.549	3.171	30.9	0.0
10^{22}	6.091	229.4	1.000	2.145	37.7	10.4
10^{24}	44.939	3417.8	1.000	1.897	76.1	1.6

为定位质量投入的阶段变化，我们继续沿用幂函数型成本、 $L_{\text{ctx}} = 2048$ 、 $M = 0$ 作预算扫描。图21左侧为完成度，右侧为三类费用份额，内嵌小图放大了质量提升区间。两条竖线对应首次启动和首次饱和的有效网格点；曲线连接逐点求解得到的记录。

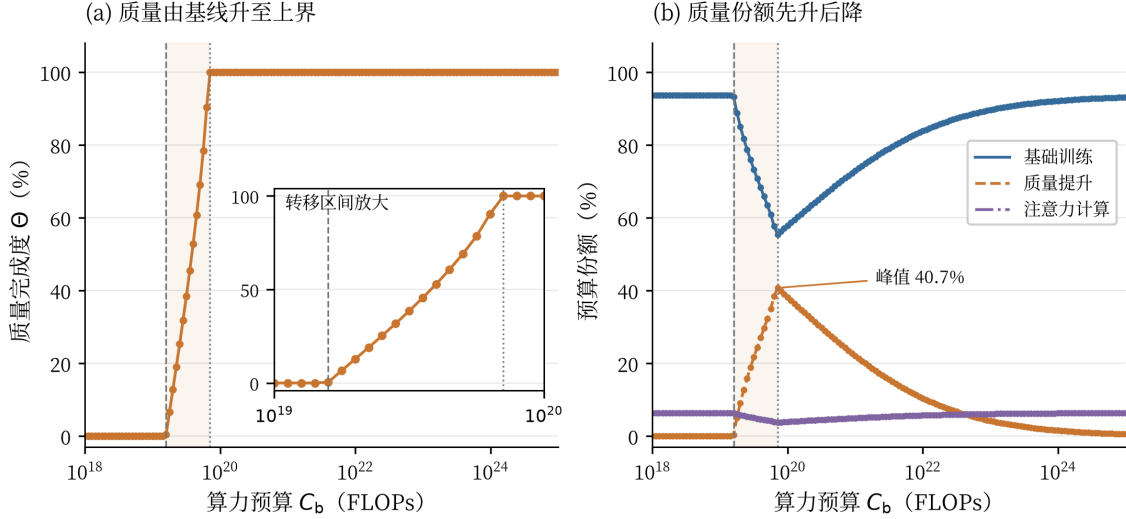


图 21 给定标度律参数下的预算与费用份额

低预算的有效解都保持 $Q^* = Q_0$ 。此时 93.6% 用于基础训练，6.4% 用于注意力计算。虽然提高质量可降低质量项损失，但它会占用原本可用于数据训练的费用；在当前成本设定下，数值比较选择维持基线。

费用份额的变化可进一步从预算等式得到解释。记基础训练和注意力的份额分别为 f_{train} 、 f_{attn} ，预算充分使用时有

$$f_{\text{train}} = \frac{6N^*}{kN^* + \Delta g(Q^*)}, \quad f_{\text{attn}} = \frac{\eta L_{\text{ctx}} N^*}{kN^* + \Delta g(Q^*)}, \quad f_{\text{train}} + f_{\text{attn}} + f_Q = 1. \quad (45)$$

上下文固定时，基础训练与注意力的费用比保持不变。质量投入出现后，这两项在总预算中的份额会共同受到压缩，但彼此的比例仍由上下文决定。因此，图中的三类份额并非能够任意调整的三个独立变量，其变化受到成本结构的约束。

质量投入的两个变化位置可以从扫描点看出。到 1.58×10^{19} FLOPs，首次出现额外质量费用；到 7.08×10^{19} FLOPs，质量达到上界，费用份额也升至峰值 40.7%。在后一位置之前，最近有效点的质量约为 0.956。结合相邻点，启动区间为 $[1.41, 1.58] \times 10^{19}$ FLOPs，饱和区间为 $[6.31, 7.08] \times 10^{19}$ FLOPs。这些是当前网格的定位范围，精确临界值并未由扫描确定。

上述区间仅反映原参数和当前网格。表18中， $\gamma = 1$ 的受限拟合与原拟合目标接近，却使原首次饱和点的质量变为 0.9640，说明网格定位误差之外还存在函数形状的不确定性。因此，三阶段图用于解释现有情景，不将两个网格区间当作已经验证稳健的投入门槛。

达到质量上界后， Q 已不能继续增加，新增预算主要用于扩参数和数据。此时单位质量费用固定为 $\Delta g(1)$ ，预算关系给出

$$f_Q = \frac{\Delta g(1)}{kN^* + \Delta g(1)}.$$

达到上界后，模型越大，质量费用份额越小，至 10^{24} FLOPs 约为 1.6%。这是训练费用增长更快造成的，不能理解为质量处理总支出减少，因为每个新增 token 仍承担同样的处理费用。质量不再增加也来自上界约束，而非边际收益归零；问题二估计的 γ 仍小于 1。

2. 低预算的质量选择取决于成本形式

我们进一步更换质量成本形式，检查上述阶段是否依赖幂函数设定。保持 $L_{\text{ctx}} = 2048$ 、 $M = 0$ ，用表19的三种函数分别求解。图22对照九组结果的质量及费用份额，表22列出对应规模配置。

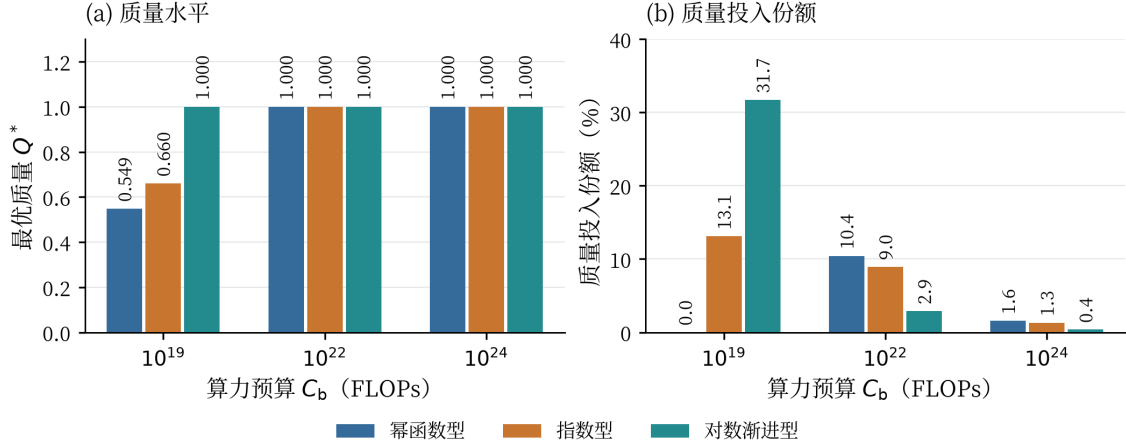


图 22 成本函数比较

表 22 三类质量成本下的配置对比 ($L_{\text{ctx}} = 2048$, $M = 0$)

预算 (FLOPs)	成本类型	N^* (B)	D^* (B tokens)	Q^*	L^*	f_Q (%)
10^{19}	指数型	0.266	5.1	0.660	3.163	13.1
	幂函数型	0.225	6.9	0.549	3.171	0.0
	对数渐进型	0.355	3.0	1.000	3.113	31.7
10^{22}	指数型	5.972	237.8	1.000	2.144	9.0
	幂函数型	6.091	229.4	1.000	2.145	10.4
	对数渐进型	5.528	274.1	1.000	2.139	2.9
10^{24}	指数型	44.797	3437.6	1.000	1.897	1.3
	幂函数型	44.939	3417.8	1.000	1.897	1.6
	对数渐进型	44.300	3508.8	1.000	1.897	0.4

在 10^{19} FLOPs 时，幂函数成本得到的质量是 0.549，指数型为 0.660，对数渐进型直接达到上界。三种预测损失只差约 0.058，质量费用份额却从 0 跨到 31.7%，其中对数渐进型最高。因此，不能只凭损失接近就认为投入方案也相同。

我们从累计费用和增量费用两方面解释情景差异。累计费用决定给定质量水平下还剩多少预算用于训练，成本斜率则进入式(43)中的局部权衡。三种函数的系数和变化形态共同作用，不能仅凭“指数型”或“对数型”的名称判断哪种方案始终更贵。表中结果均在相同预算、上下文和配比下取得，能够用于比较题设成本情景；它们并未提供实际清洗流程的计费测量，也不能据此确定某种具体处理技术更优。

中、高预算档都出现 $Q^* = 1$ ，但这并没有消除成本形式的影响。例如 10^{22} FLOPs 下，三类成本对应的数据量从 229.4B 到 274.1B tokens，相差约 19.5%。所以，低预算基准方案不能直接照搬到其他成本条件；即使质量都达到上界，参数与数据也仍需重新计算。

3. 长上下文如何改变配置

上下文延长会使式(35)的 k 增大。把注意力费用与基础训练费用相除，得到 $\lambda = \eta L_{\text{ctx}}/6$ ，令这一比值为 1，可求出两项费用相等的位置：

$$L_{\text{crit}} = \frac{6}{\eta} = 30,000 \text{ tokens.} \quad (46)$$

该长度只表示两项训练费用相等。实际配置还受到质量费用影响，故进一步固定 10^{22} FLOPs 预算，比较表23中的五档上下文结果。

表 23 上下文长度敏感性 ($C_b = 10^{22}$ FLOPs, 幂函数型成本, $M = 0$)

L_{ctx} (tokens)	λ	N^* (B)	D^* (B tokens)	Q^*	L^*	注意力份额 (%)
2048	0.068	6.091	229.4	1.000	2.145	5.7
4096	0.137	5.898	223.4	1.000	2.149	10.8
8192	0.273	5.563	212.6	1.000	2.157	19.4
32768	1.092	4.319	170.2	1.000	2.194	48.2
131072	4.369	2.705	109.1	1.000	2.273	77.3

在五档上下文下, 最优质量均为 1, 额外注意力费用主要压缩了规模投入。上下文由 2048 增至 131072 tokens 时, 注意力份额由 5.7% 升至 77.3%, 参数量和数量分别减少约 55.6%、52.5%, 损失增加约 0.128。32768 tokens 时注意力费用虽已超过基础训练费用, 仍只占总预算的 48.2%, 因为质量处理也占用预算。

我们将这一损失变化解释为满足更长上下文要求的预算代价。当前损失式中没有单独描述长上下文对任务能力的收益, 上下文通过计算费用影响可行的规模与质量配置。因此, 表中损失随上下文增长, 不能用于评价长文本任务是否值得采用更长上下文。实际应用时, 应先根据任务确定上下文要求, 再在给定长度下选择配置; 不同情景的损失不能直接用于判断任务能力的全面优劣。

图23给出更密集的上下文扫描, 仍使用幂函数型成本和 $M = 0$ 。左图固定 10^{22} FLOPs, 方形点标明 C7 的五档; 右图比较三个较低预算, 其中 10^{20} 与 10^{21} FLOPs 的质量曲线重合。竖线表示 L_{crit} 。连续上下文扫描的 40 个点和三档预算下的 42 个质量配置点均求解有效, 曲线连接相邻扫描记录。

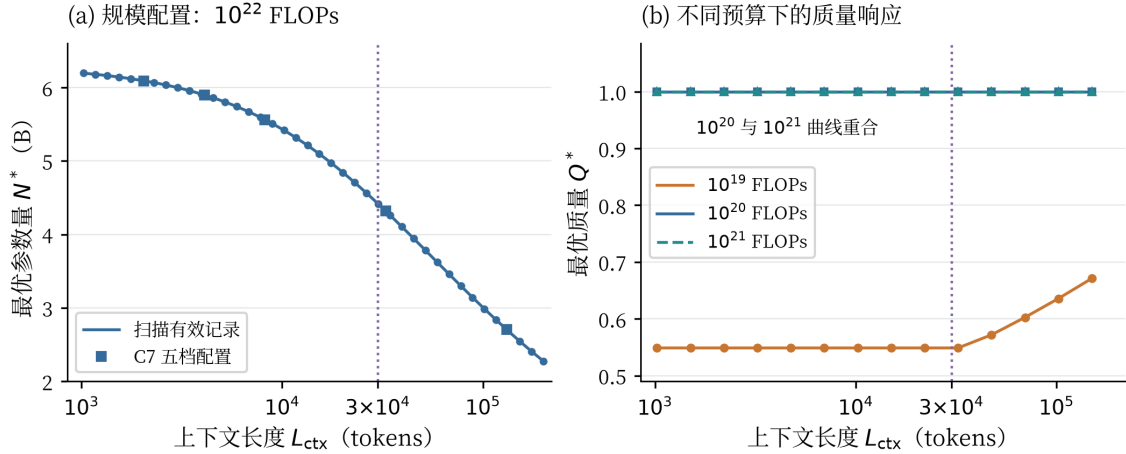


图 23 上下文敏感性

低预算的质量变化则不同。图23右图中, 10^{19} FLOPs 的质量由 0.549 上升到约 0.671。按题设, 质量处理总费用与数据量 D 成正比; 上下文延长后, 可负担的数据变少, 处理整份语料所需的费用也改变了, 因此质量与规模之间的取舍随之变化。这一解释依赖当前的数据量成本关系。

4. 固定配比平移的条件算例与适用范围

领域配比会影响预训练效果 [19]。按问题一核岭预测的 $M = -0.1730$, 保持共同费用基线, 在 10^{22} FLOPs 下得到相同的 N, D, Q , 损失从 2.1451 平移至 1.9721。参考配比达到后一损失的模型等效预算约为 1.691×10^{23} FLOPs, 即 16.9 倍。该换算依赖绝对损失差跨尺度不变, 不能解释为实际训练加速比。

表24将收益幅度依次保留 0、25%、50%、75% 和 100%，在满分配置下独立反解预算。全部幅度对应的 16.90739 倍与正式计算一致；仅保留一半时降至 3.49 倍。可见，等效预算对迁移幅度较敏感，这些人为设定的比例也没有置信区间含义。

表 24 固定损失平移幅度的条件算例

假定收益保留比例	0	25%	50%	75%	100%
等效预算倍数	1.00	1.81	3.49	7.30	16.91

固定配比项只平移目标函数，成本或上下文变化则会移动预算边界，两类情景因此产生不同作用。另将核岭候选自身质量 0.5182 仅作为费用基线，在 10^{22} FLOPs 下得到约 6.104B 参数、228.5B tokens，变化较小但不再完全相同。闭合参考质量 0.548753 对应的对照则约为 6.09049B、229.44095B。两项均另列保存，正式配置继续使用外生 Q_0 。

质量未到上界时，新增预算同时影响质量和规模；达到上界后，质量处理继续付费，预算份额随模型扩大而降低。低预算尤其依赖处理成本，上下文则通过训练单价改变可用的数据量。把这些数值用于实际训练，还需核对评分尺度和外推范围，并确认高预算所需语料能否取得。数据受限训练研究已指出，重复使用数据的收益会递减 [20]。本文尚未加入供给与重复训练约束，高预算解因此作为理论情景使用。

各情景共同采用问题二的点估计参数，结果表刻画的是这些参数给定时的配置差异。参数估计误差尚未转换为配置区间，因而表中精确到小数位的规模不应被理解为同等精度的工程需求。本文能够支持的是在既定评分、损失与费用关系下比较投入方向；实际资源安排还需结合可用语料、模型架构及任务要求判断这些条件是否成立。

5.4 问题四：技术演进分析与能力前沿条件预测

5.4.1 能力度量与数据组织

前三问研究训练损失，本问研究模型的实际评测得分。评分方式可能使连续的能力变化呈现为表观突变 [21]，因此不能把分数曲线的形状直接解释为模型机制的变化。我们先检验损失与得分的关系，再将历史增量核算为规模关联项与未解释剩余项，最后在给定增长形式下外推能力前沿。由于评测体系和样本来源并不完全一致，综合分数与单项任务结果同时保留：前者用于比较整体水平，后者用于寻找具体短板。

图24按数据用途列出四条分析路径。损失与得分的分层回归用于判断变化方向；历史增量分解与未来预测直接使用评测得分，其中规模、年度回归的系数也用于结构情景。子任务结果用于定位能力短板，宏观资料则单独提供资源趋势参照。

辅助分析说明：本问的文段相关辅助参考腾讯 WorkBuddy 接入的 DeepSeek-V4-Flash (2026-07-31 公开测试)，图表展示、程序整理与复现核对使用 OpenAI Codex (GPT-5.6-Luna, 2026-07-09) [14, 15, 16]。正文表达与段落衔接另经 OpenAI Codex 辅助润色。

1. 综合得分、许可筛选与时间变量

综合分数 $\bar{B}$ 定义为 Open LLM Leaderboard v2 六项评测的平均值，满分 100[22]。六项包括 IFEval、BBH、MATH Lvl 5、GPQA、MUSR 和 MMLU-PRO，分别对应指令遵循、综合推理、数学、研究生级问答、多步推理和专业知识。其中，IFEval 用可验证指令衡量遵循要求的程度 [23]；MMLU-Pro 增加推理要求并扩展选项数量，考察较高难度的知识与推理表现 [24]。本文沿用附件中的评测结果，用平均值追踪整体变化，再用单项得分识别相对薄弱的能力维度。

我们对六项得分赋予相同权重，以形成可比较的综合分数。不过，均值会掩盖能力构成：总分接近的模型可能分别擅长数学和指令遵循。因此，前沿曲线仅描述综合水平，六维分布与子任务统计则用于补充各项能力差异，不改变综合分数的定义。

近期变化采用 C1，长期趋势用 C3 作为补充。时间变量按评测记录的时间字段定义：年度回归取 $t = \text{年份} - 2019$ ，月度前沿用月中点 $t = \text{年份} - 2019 + (\text{月} - 0.5)/12$ ，C3 的年度记录放在年中。这样既能比较规模和年度变化，也不把年度资料误当成精确到月的观测。

C1 的时间字段为 Submission Date，表示记录提交的时间。因此，同一月份的样本是这一时间口径下可观察到的评测记录，不能据此确定各模型完成训练或实际发布的先后。月度聚合有助于减少具体提交日的影响，年度回归则进一步概括较粗的时间差异；两者都沿用原始记录的时间含义。将年份改写为从 2019 年起算，只是改变时间原点，便于与 C3 的历史段连接。

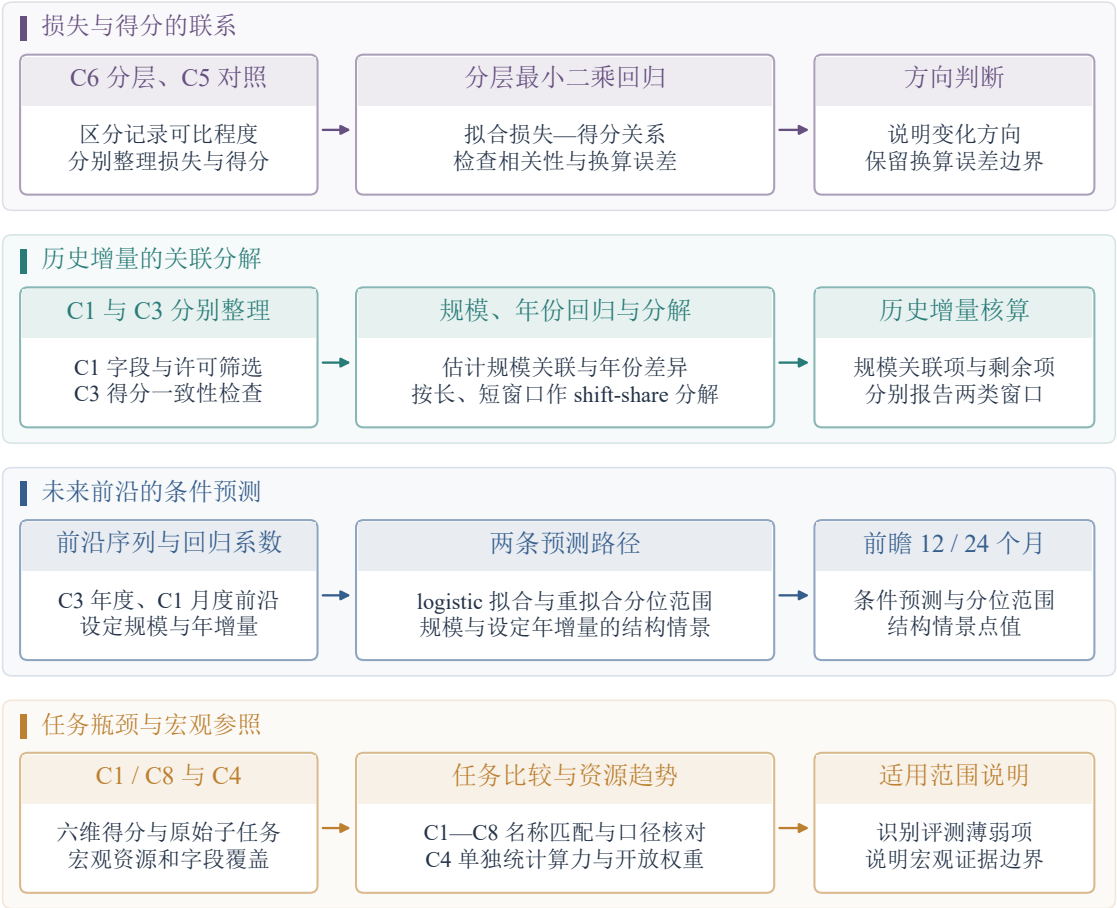


图 24 问题四技术演进与能力前沿分析流程

表25展示了主样本的筛选过程。先去掉关键字段不完整的记录，再按许可白名单保留本文讨论的开放权重模型。白名单包括 apache-2.0、mit、gpl-3.0、cc-by-4.0、mpl-2.0、bsd-3-clause、cc-by-sa-4.0、openrail，以及 llama2、llama3、llama3.1、llama3.2、llama3.3、gemma。其他或未知许可不进入主样本；进入白名单只说明符合本研究的筛选条件，不表示这些许可的具体要求完全相同。

表 25 主样本的形成及模型类型构成

数据层次	记录数	判定依据或构成
C1 原始记录	4,576	六维得分均完整；日期覆盖 2024-06 至 2025-03
完整性筛选后	4,561	日期、参数量和六维得分齐备，排除 15 条记录
许可白名单主样本	2,346	以 Hub License 字段匹配许可集合
预训练类	231	pretrained 与持续预训练模型
对话及微调类	1,497	chat 与领域微调模型
其他类型	618	包含 611 条合并模型记录和 7 条多模态记录

三类模型均保留在主样本中，类型只用于分组比较。为了解预训练与后训练的差别，再分别拟合预训练类、对话及微调类的规模和时间系数。C3 的历史记录按六维得分一致性及参数有效性筛选，没有重复使用 C1 的许可白名单，因而长期样本不是同一批开放权重模型的连续追踪。

我们先检查完整性，使规模、时间和得分逐行对应，再用许可条件限定研究范围。排除不满足条件的记录，并不表示这些模型能力较低。表25所列数量也均以记录为单位，同一模型的多次提交可能形成多条记录，故不能将主样本数量解释为独立训练实验的数量。按类型分组后，比较对象仍是相应记录集合。

2. 从原始评测记录到子任务样本

C8 用来补充综合分数看不到的任务细节。其 1,863 个模型目录共有 1,958 个 JSON 文件，优先读取每个目录的最新文件，失败后再回退，最终 1,860 个目录得到有效结果，3 个没有可用记录。每个任务先检查 `acc_norm` 是否为数值；若无法取用，再依次检查 `acc`、`prompt_level_loose_acc`、`inst_level_loose_acc` 和 `exact_match`，取首个可用值并换算为百分制。名称规范化后，与主样本对应 964 个目录，汇总 BBH 24 项、MATH-Hard 7 项、GPQA 3 项、MUSR 3 项，共 37 项任务，各项有效样本数为 961–964。

文件回退与指标回退处理的是两个层次的问题。前者在文件读取异常，或结果字段缺失、为空、结构不符时，尝试同一目录中的其他文件；后者应对不同任务采用不同指标字段的情况。每个目录保留一份通过结构检查的结果，每个任务按固定优先级取值，而不在多个字段中挑选最高分。文件被接受后，某项任务的指标缺失不会再触发文件回退；该任务值在相应统计中排除，不作为零分加入均值或排序。

C8 保留的是任务原始指标，C1 则对汇总分数作过基准调整，两者不能直接混算。GPQA 的随机基线为 0.25，MMLU-PRO 为 0.10，原始比例 s 分别换成 $(s - 0.25)/0.75$ 、 $(s - 0.10)/0.90$ ；IFEval 还对严格判定下的提示级和指令级结果求平均。因此，C1 用来比较综合能力，C8 用来比较任务内部表现。

两套结果的名称匹配见表26。C1 中同名模型可能有多条记录，配对数因此不等于独立模型数。六项的 Spearman 系数均超过 0.81，多数超过 0.97，说明排序大致相近；BBH、GPQA、MUSR 的平均绝对差却较大，表明任务构成和归一化口径仍会影响绝对分数。

我们按分析目的区分两种匹配集合。表26以完整 C1 记录与 C8 对照，用来检查两个来源在排序和分值上的关系；后续子任务分析则限定在许可筛选后的主样本所对应的目录中。前者按有效记录配对计数，后者按可用目录及任务计数，因此表中的配对数量大于子任务目录数量并不矛盾。名称规范化只是统一匹配键，无法把重复提交自动转化为相互独立的观测。

表 26 C8 原始指标与 C1 汇总得分的记录级相关性及绝对偏差

评测维度	有效配对 记录数	秩相关 ρ	线性相关 r	平均绝对 偏差 / 分
IFEval	1,896	0.987	0.987	2.88
BBH	1,898	0.997	0.998	21.06
MATH Lvl 5	1,897	0.812	0.803	3.47
GPQA	1,894	0.992	0.987	23.38
MUSR	1,894	0.977	0.973	30.73
MMLU-PRO	1,896	0.998	0.998	7.56

5.4.2 损失与评测得分的分层联系

为判断前三问的损失变化能否对应评测能力，我们先在可比数据中检验二者的关系。设验证损失为 L ，采用线性映射估计变化方向及预测误差：

$$\bar{B} = a + bL \quad (47)$$

C6 的高可比层主要由同模型、同验证集的 Pythia 记录构成，可与附件 B 对照；中可比层来自不同模型族和技术报告，验证集并不统一。两层分别采用最小二乘拟合，再把合并结果与 C5 对照，以区分同模型、同验证集记录与跨模型族记录的拟合结果。图25中的散点为 C6 记录，直线为各层拟合结果；两个面板的纵轴范围不同。

我们分层拟合，是因为交叉熵损失依赖验证语料和计算口径，不同来源的相同损失值未必具有相同含义。高可比层减少了部分条件差异，中可比层扩大了模型范围。分别报告斜率和误差后，能够检查扩大范围带来的换算偏差，避免合并拟合掩盖来源差异。各层的样本量、斜率与误差汇总于表27。

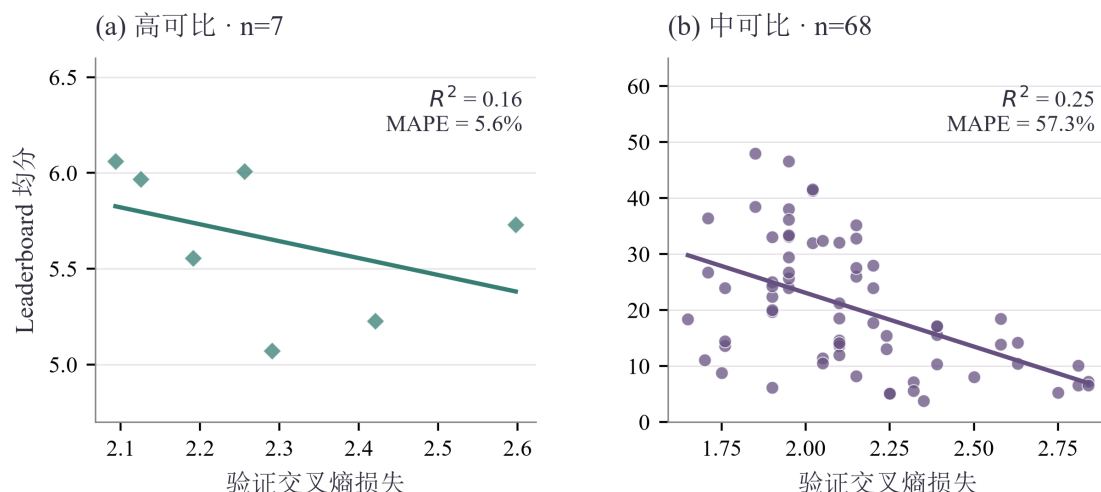


图 25 损失与得分映射

图25与表27呈现了负相关，但换算精度有限。合并样本有 $R^2 = 0.26$ 、 $MAPE 67.8\%$ ， $Pearson r = -0.510$ 、 $Spearman \rho = -0.549$ 。中可比层混用了不同验证集，低分记录还会放大相对误差；高可比层虽仅有 5.6% 的 MAPE，却只有 7 条记录，得分集中于 5.07 ~ 6.06。这一窄范围尚不足以支持高能力模型的损失-得分换算。

表 27 不同可比性样本中损失-得分回归的斜率、相关性与误差

评价量	C6 高可比 $n = 7$	C6 中可比 $n = 68$	C6 合并 $n = 75$	C5 对照 $n = 43$
损失斜率 b	-0.882	-19.185	-20.417	-16.155
Pearson r	-0.397	-0.502	-0.510	-0.502
Spearman ρ	-0.607	-0.509	-0.549	—
决定系数 R^2	0.16	0.25	0.26	—
MAPE / %	5.6	57.3	67.8	—

相关系数、决定系数与相对误差在这里分别回答不同问题。相关系数用于判断共同变化的方向，决定系数衡量直线对当前得分差异的解释程度，MAPE 则把预测偏差相对于各自得分进行比较。高可比层得分集中，即使预测值靠近样本均值，相对误差也可能较小；这不足以说明斜率在更宽的能力范围内仍然适用。因此，我们结合样本量和得分范围判断表27中的误差，不以较低 MAPE 单独认定映射有效。

基于上述误差，我们仅用该关系判断损失降低与得分提高的共同趋势，不将具体损失降幅换算为能力增量。跨模型的相关也不能证明资源调整具有因果效果。历史增量与未来前沿都直接从 Benchmark 得分计算，以免继续传递换算误差。

5.4.3 规模关系与历史增量分解

1. 控制年份后的规模关联

令 P_{it} 表示第 t 年第 i 条模型记录的参数量，单位为十亿参数。这里的 i 是记录索引，并非跨年保持不变的模型个体。我们对不同年份的记录作合并横截面回归，同时放入规模和时间变量，并以年份固定效应作对照：

$$\text{Score}_{it} = b_0 + b_P \log_{10} P_{it} + b_t t + \varepsilon_{it} \quad \text{或} \quad \text{Score}_{it} = \alpha_t + b_P \log_{10} P_{it} + \varepsilon_{it} \quad (48)$$

在控制年份后， b_P 表示参数扩大十倍对应的得分差；在当前只有 2024、2025 两个年份的样本中， b_t 表示控制规模后的年份组间得分差， α_t 则吸收某一年度共同存在的差异。它们是条件关联，不能进一步拆成数据工程、网络结构或对齐技术各自带来的因果贡献。

参数量取常用对数后，相同扩张倍数在对数轴上具有相同距离，便于比较不同起点的规模关联。年度项还可能包含模型类型与收录构成的变化，不能解释为同一模型多训练一年的收益。

主样本中，2024 年有 1,588 条记录，2025 年有 758 条，共涉及 2,294 个原始模型名称，没有同名模型跨年出现，故采用合并横截面回归。这里只有两个年份，线性年度变量与年份固定效应给出等价拟合，相同的 R^2 不构成独立验证。图26按年份展示记录及拟合直线，两条直线共用规模系数；表28另列模型类型子样本的结果。

表 28 全样本与类型子样本的规模关联及年份组差

样本与时间设定	n	b_P 分/十倍参数	b_t 年份组差/分	R^2
主样本，年份固定效应	2,346	13.842	—	0.475
主样本，线性年度项	2,346	13.842	4.844	0.475
预训练类，线性年度项	231	6.414	1.552	0.460
对话及微调类，线性年度项	1,497	14.853	4.680	0.484

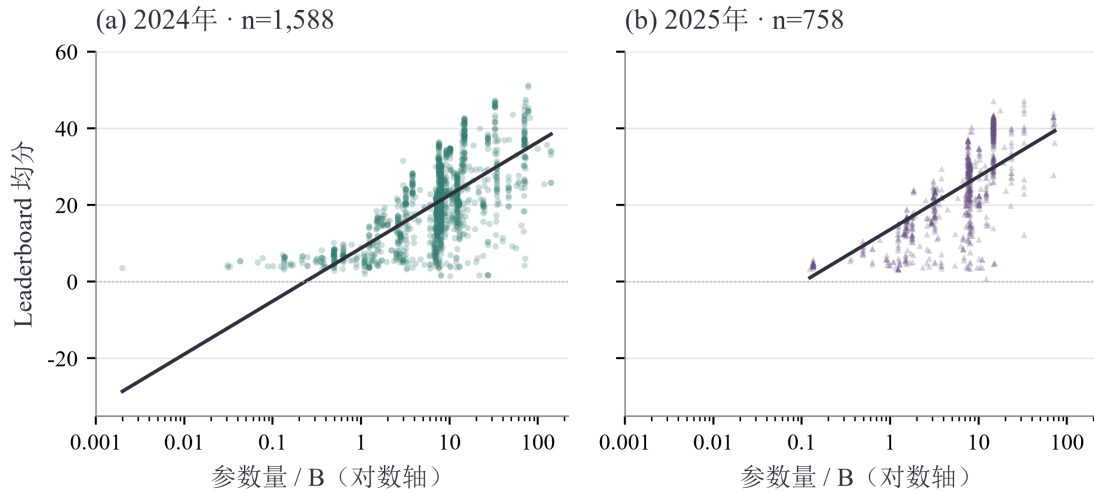


图 26 合并横截面中的规模关系与年份组差异

预训练类的规模和年度系数均小于对话及微调类，提示全样本结果受类型构成影响。由于缺少同一模型微调前后的受控对照，这些差异不能归因于微调效率。线性式在极小参数端还可能给出负分，本文仅用它概括当前样本的关联并作增量核算。

表29进一步比较同名去重、类型控制和时间编码。两组记录的平均提交日分别约为2024年9月21日、2025年2月6日，相隔仅0.379年，不能将年份标签相差1当成平均观测间隔整整一年。

表 29 合并横截面回归的事后敏感性诊断

诊断设定	n	b_P	时间项系数	R^2
年份标签，原规格	2,346	13.842	4.844	0.475
同名记录保留首条	2,294	13.977	4.858	0.473
同名记录保留末条	2,294	13.960	4.875	0.473
年份标签并控制类型	2,346	12.894	4.462	0.496
按提交日计连续年数	2,346	14.168	13.803	0.516

注：前四行的时间项是年份组差，单位为分；末行按实际提交日折合年数，斜率单位为分/年。首条、末条按附件原始行序确定。各行仅改变注明的处理，均未形成受控的个体追踪。

去重后系数变化较小，说明同名重复记录不是当前回归差异的主要来源；加入类型后仍然不能排除同类型内部的选择差异。按提交日编码得到的时间斜率为13.803分/年，与年份组差4.844分属于不同估计口径，不能直接用前者替换后者，或把任一数值当作因果增长率。后文若将4.844分逐年累加，需要另加“该组间差异可作为未来年增量”的情景假设。

2. 长周期与近端窗口的增量核算

我们采用 shift-share 形式核算历史增量，将前沿得分变化拆为规模关联项与未解释剩余项：

$$\Delta F = \underbrace{b_P \Delta \log_{10} P_{\text{frontier}}}_{\text{规模关联项}} + \underbrace{[\Delta F - b_P \Delta \log_{10} P_{\text{frontier}}]}_{\text{未解释剩余项}} \quad (49)$$

规模关联项由该窗口的参数对数变化乘以回归系数得到，其余增量记为未解释剩余项；两项分别除以总增量即为核算占比。剩余项混合了技术变化、样本替换、遗漏因素及测

量差异，分解式本身没有识别各因素的效果。因此，以下结果均需连同前沿定义和规模代表值理解。

长期和近期采用不同的前沿定义。C3 取截至当年的最高分，并配对产生该分数的模型参数量；跨年同分时保留较早记录，同年并列时依次按较小参数量、模型名排序。C1 取当月前 1% 记录的平均得分，规模取同组参数量的几何均值，等价于先平均对数参数量，再比较首月与峰值月。配对使规模与得分对应同一记录或记录组，但前沿模型仍会更替，并非受控的个体增长。两组占比同时依赖时间窗口、样本和评分口径，不能合并为平均比例或据此识别技术增长来源。

年度最高分和用于预测的月度前列均值均取累计最大值，因而新记录偏低时，前沿仍停留在此前峰值。曲线的单调性由定义保证，并不说明每个模型都在持续改进。月度前列组数量较少时，其均值还容易受到个别入选记录影响。

C3 的 4,599 条记录中，13 条的 Average 与六维均分相差超过 5 分，另有 7 条参数量为零，筛选后保留 4,579 条。差异最大的 GPT-3 175B 记录中，Average 为 50.0，六维均分为 7.67，相差约 42.3 分。长期回归得到 $b_P = 14.10$ 、 $b_t = 3.30$ 、 $R^2 = 0.42$ 。2019–2025 年累计前沿由 5.00 升至 52.08，总增量为 47.08 分；规模关联项为 24.20 分，占 51.4%，未解释剩余项为 22.88 分，占 48.6%。2025 年的累计峰值仍由 2024 年的 77.965B 模型保持，规模也沿用该模型，不能另取 2025 年最大参数量。图27列出两个窗口的核算结果；历史评分来源的差异仍限制其可比性。

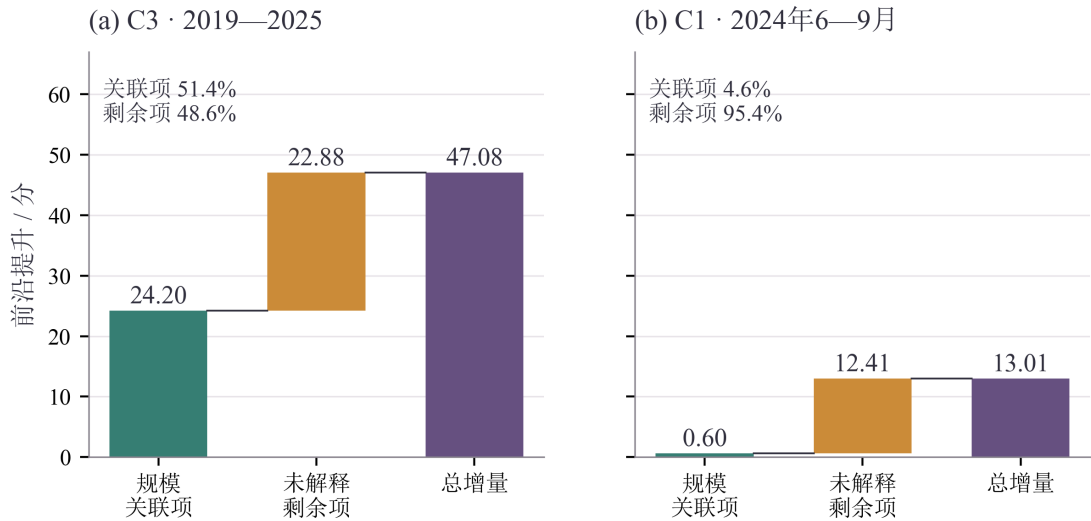


图 27 两类前沿定义下的规模关联项与未解释剩余项

近端比较使用评测标准较一致的 C1。2024 年 6–9 月，前列组参数量的几何均值由 70.6B 增至 78.0B， $\Delta \log_{10} P = 0.043$ ，平均得分由 37.99 升至 51.00。按同一组记录核算，规模关联项为 0.60 分，占 4.6%；未解释剩余项为 12.41 分，占 95.4%。该比例说明当前回归及前列组定义下的规模关联项较小，不能将其余部分归给某一种算法，也不能沿用为全年或未来占比。

5.4.4 前沿曲线与条件预测

1. 饱和曲线及重拟合分位范围

前沿数据由 C3 的 2019–2023 年间 4 个年度累计最高分和 C1 的 10 个月度前 1% 均值累计最大值连接而成，共 14 个点。每月前列组数量按记录数的 1% 向下取整，但最少

保留 1 个。两个来源未作历史评分量表校准，连接后的序列用于构造条件曲线。我们以 logistic 函数表示先快后慢的增长形式，并把上界限制在 100 以内：

$$F(t) = \frac{K}{1 + e^{-r(t-t_0)}}, \quad K \text{ 为饱和上界, } r \text{ 为增长率, } t_0 \text{ 为拐点} \quad (50)$$

估计结果为 $K = 56.61$ 、 $r = 2.523$ 、 $t_0 = 5.11$ ，样本内 $R^2 = 0.975$ 。按照 bootstrap 思路 [25]，固定时间点，将有放回抽取的残差叠加到原曲线上，尝试重拟合 600 次，再汇总成功拟合曲线在两个未来时点的第 5、第 95 百分位。这里没有增加独立评测记录，也未加入未来观测的新扰动，所得范围不能解释为对未来实际得分具有 90% 覆盖率的预测区间。

逐点残差抽样还要求残差可交换，但累计前沿中的相邻月份可能共享同一峰值，这一条件尚未得到验证。分位范围因此只描述既定函数与抽样规则下的曲线变化；历史来源、前沿定义及技术路线改变带来的不确定性不在其中。

我们进一步构造放缓情景，假设从预测基点 t_h （2025 年 3 月）起，logistic 动力学中的增长参数由 r 降为 $r/2$ ，饱和上界 K 不变。以连续的有效时间表示该假设：

$$\tau(t) = \min(t, t_h) + \frac{1}{2} \max(t - t_h, 0), \quad F_{\text{slow}}(t) = \frac{K}{1 + e^{-r(\tau(t)-t_0)}}. \quad (51)$$

该曲线在 t_h 处与原曲线相接，之后满足 $dF_{\text{slow}}/dt = (r/2)F_{\text{slow}}(1 - F_{\text{slow}}/K)$ ，仍随时间增长。减半指增长参数，不表示每个未来时点的得分增量都恰好减半。此处将算力放缓的影响表示为条件性增长参数变化，未从附件估计算力增速到该参数的传导关系。

2. 基于规模和年度系数的结构情景

为考察不预设短期饱和的增长路径，我们利用规模和年度系数作结构外推：

$$\hat{F}(t + \Delta) = F(t) + [b_P \Delta \log_{10} P + b_t] \cdot \Delta \quad (52)$$

式中， Δ 为向前预测的年数， $\Delta \log_{10} P$ 为假设的年均参数对数增量。设置三种条件进行比较：基准情景保持规模不变，并将年份组差 $b_t = 4.844$ 分作为每年的增量；放缓情景同样固定规模，但把这一设定年增量减半；扩容情景则令规模每年增加 0.3 个数量级，同时保留该增量。这一步是在横截面回归之外增加可延续的年增量假设，并非由两组年份记录识别出的动态规律；减半幅度也不是从算力增速估计出的传导关系。

在结构外推内部，各情景均从 2025 年 3 月的累计观测前沿 50.998 分起算，保持预测时点一致，仅改变规模增长与年度增量的设定。logistic 原曲线与增长参数减半曲线则在同月的拟合值 53.267 分处相接，两类方法的数值起点相差 2.269 分。因此，跨方法的终点差异同时受到起点和增长形式影响；只有各自内部的情景比较保持了相同基点。每个结果都是给定条件下的点预测，结构情景最大值与最小值的跨度只反映设定差异。由于未对这些条件赋予发生概率，该跨度不是概率区间，也不覆盖所有可能的未来结果。图 28 将条件曲线及其重拟合分位范围与三种结构情景分别展示。

两个预测时点分别为 2026 年 3 月、2027 年 3 月，结果见表 30。logistic 条件预测为 56.33、56.59 分；结构情景分别落在 53.42 ~ 59.99 和 55.84 ~ 68.99 分，向外取整为 53 ~ 60、55 ~ 69 分。结构情景的最大值与最小值仅表示设定差异，不能按概率区间理解。

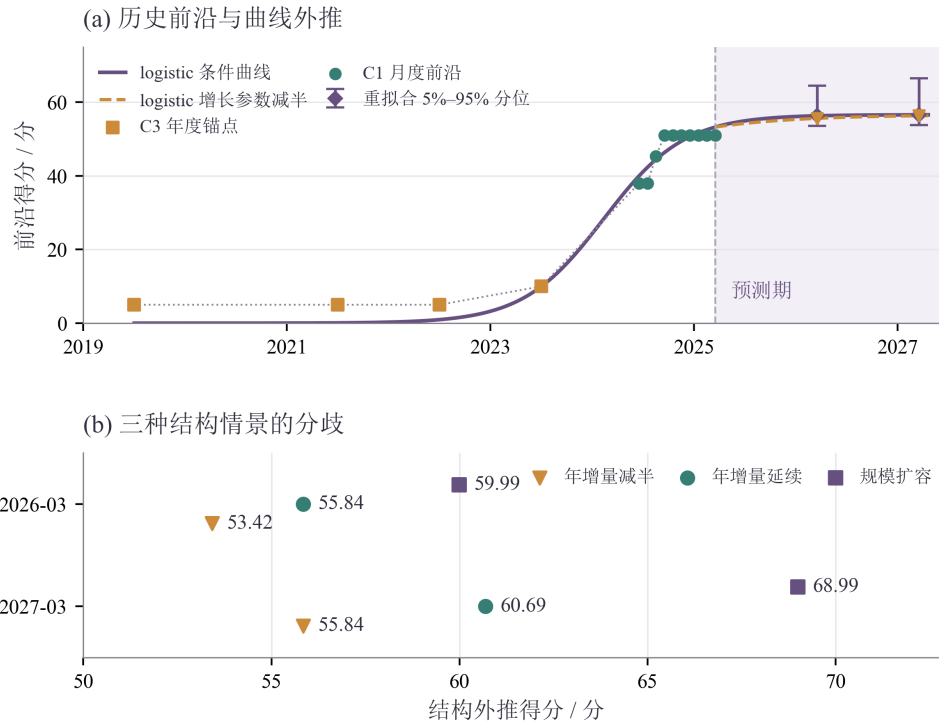


图 28 能力前沿的条件曲线与结构情景

表 30 以 2025 年 3 月数据为基点的能力前沿条件预测

预测口径	前瞻 12 个月 2026-03	前瞻 24 个月 2027-03
logistic 点预测	56.33	56.59
重拟合曲线第 5 百分位	53.62	53.75
重拟合曲线第 95 百分位	64.52	66.52
logistic 增长参数减半	55.62	56.33
结构外推：规模持平	55.84	60.69
结构外推：设定年增量减半	53.42	55.84
结构外推：每年扩容 0.3 dex	59.99	68.99

图28上图的紫色误差棒为两个预测时点的重拟合分位范围，金色虚线与三角形为增长参数减半的曲线及点值，下图列出结构情景。logistic 已趋近饱和，结构基准仍逐年累加设定增量；增长参数减半只使前者放缓，并未停止增长。C1 可比窗口约 10 个月，前瞻达到其 1.2 ~ 2.4 倍，早期 C3 又存在评分差异，故样本内的高拟合度不能替代时间外检验。

曲线参数 K 是当前前沿样本和所选函数共同确定的渐近位置，其含义不同于评测量表的满分。观察期末曲线变平，会使近端预测靠近这一位置；结构外推则保留按年累计的增量。我们保留两类预测，用以比较起点及增长形式设定下的结果；二者并不构成相互独立的外部验证。

3. 前沿窗口与时间留出的事后诊断

为检查饱和位置是否依赖所选记录，我们保持 logistic 函数、初值及参数界限不变，分别改变历史锚点、末端窗口和持平点权重，统一在 2026 年 3 月、2027 年 3 月计算曲线值。表31中的诊断结果不替换表30。仅用 C1 月度点时，增长参数到达预设上界 $r = 5$ ；

相同累计值仅留首次时，饱和参数到达上界 $K = 100$ 。这表示局部最小二乘拟合受到参数界限约束，不能将触界值解释为已观测到的能力极限。

表 31 固定未来时点下的前沿窗口敏感性诊断

拟合记录设定	点数	K	2026-03	2027-03	触及上界
原 14 点	14	56.61	56.33	56.59	无
仅保留 C1 月度点	10	52.93	52.93	52.93	r
去掉末 3 个月	11	71.40	69.31	71.09	无
相同累计值仅留首次	5	100.00	94.17	99.04	K

原序列从 2024 年 9 月至 2025 年 3 月连续 7 个点均为 50.998 分，这一持平包含“尚未出现更高记录”的信息。删除重复值会改变这些信息在目标函数中的权重，删除末端月份则缩短了观测窗口；两者都不是更优的数据清理方法。表31说明当前 K 与未来曲线值对窗口和权重较敏感，高训练拟合度不足以稳健识别饱和上界。由于 C3 与 C1 的评分来源尚未校准，仅保留 C1 的诊断还同时改变了来源构成与历史覆盖，不能把差异全部归于单一原因。

时间留出结果统一列于表36：分别截至 2024 年 7、8、9 月拟合，再与保持最后累计峰值的参照比较，logistic 未表现出稳定优势。三组截点均为观察完整序列后补充的诊断；尤其 9 月之后恰为平台，持平参照的零误差不能推广至增长阶段。结合窗口敏感性，现有证据支持报告条件曲线与情景值，尚不足以确认单一外推方法的可靠性。

5.4.5 从综合前沿到具体任务瓶颈

1. 六维能力的分布差异

综合分数只能说明整体位置，不能说明六项能力是否同步。图29中，每一维均有 $n = 2,346$ 条记录；箱体覆盖第 25 至第 75 百分位，箱内线标出中位数，两端须线延伸至最小值和最大值。表32同时列中位数、前列均值和最高分。距满分的差值按 $100 - \text{最优得分}$ 计算，只表示当前评分体系中的剩余空间，不是确定可获得的提升量。

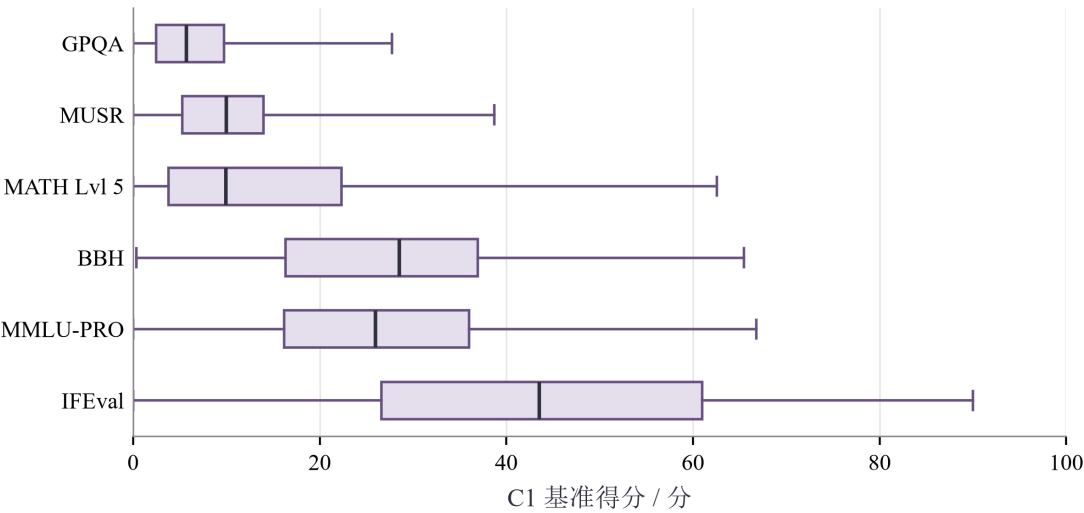


图 29 六维能力分布

IFEval 的中位数为 43.51，最高分则达到 89.98，距满分约 10.0 分。前沿与中间水平仍有距离，不能因少数模型接近满分就判断整体已经饱和。GPQA、MUSR 最高仅 27.74、

表 32 六项评测的中位数、前列得分与距满分差值

任务维度	中位数	前 1% 均值	最优得分	距满分差值 / 分
GPQA	5.70	22.80	27.74	72.3
MUSR	9.97	28.06	38.69	61.3
MATH Lvl 5	9.93	58.02	62.54	37.5
BBH	28.52	59.95	65.47	34.5
MMLU-PRO	25.98	55.26	66.80	33.2
IFEval	43.51	84.45	89.98	10.0

38.69, 中位数更低, 是当前较明显的短板。MATH Lvl 5 的中位数为 9.93, 最高分为 62.54, 也表现出这种分化; 单看最高分会高估大多数模型的水平。

我们同时列出中位数、前列均值和最高分, 分别描述通常水平、高分组表现和当前观察到的单项上界, 以区分少数模型领先与较多模型共同达到较高水平的情况。六个维度分别排序, 入选前列的记录并不必然相同, 因此不能把六项前列均值拼成一个已经存在的模型得分。

2. 原始子任务中的瓶颈位置

C8 的 37 项子任务进一步把这种差异展开。图30采用各任务的原始指标, 每项 $n = 961 \sim 964$; 圆点、短线和菱形分别标出中位数、前 5% 均值和最高分, 对应通常水平、前列表现和单项前沿。表33列出距满分差值最大的六项, 中文名称与图中保持一致。

表 33 距满分差值最大的六个子任务

所属类别	子任务	n	前 5% 均值	最优得分	距满分差值 / 分
MATH	中级代数	962	20.11	31.79	68.2
MATH	预备微积分	962	22.89	36.30	63.7
MATH	几何	962	28.72	40.91	59.1
BBH	位置跟踪, 7 个对象	964	32.67	41.20	58.8
BBH	位置跟踪, 5 个对象	964	31.24	41.60	58.4
GPQA	Extended 子集	961	39.74	44.87	55.1

在具体任务层面, MATH-Hard 的中级代数、预备微积分和几何最高分为 31.8 ~ 40.9; BBH 的 5、7 对象位置跟踪约 41 分, GPQA Extended 为 44.9 分。这些最高分仍留有较大余量, 但任务难度和随机基线不同, 不能把原始分数视为完全同尺度的能力值。

类别汇总也反映了前沿与通常水平的距离。MATH 的任务最高分均值为 55.5, 合并全部模型和子任务后的中位数为 3.7; BBH 两项分别为 81.8、48.8。这里平均的是不同任务的最高得分, 它们可能分属不同模型, 不能合成为某个模型的整套能力。

对子任务排序时, 我们同时保留有效样本数与评分方式。各项缺失情况不同, 参与某一任务统计的目录集合可能略有变化; 不同任务的原始得分也有各自的难度背景。表33用于指出当前记录中表现较弱的具体位置, 便于把综合能力的短板落实到任务, 而不据此给出统一的训练收益排序。

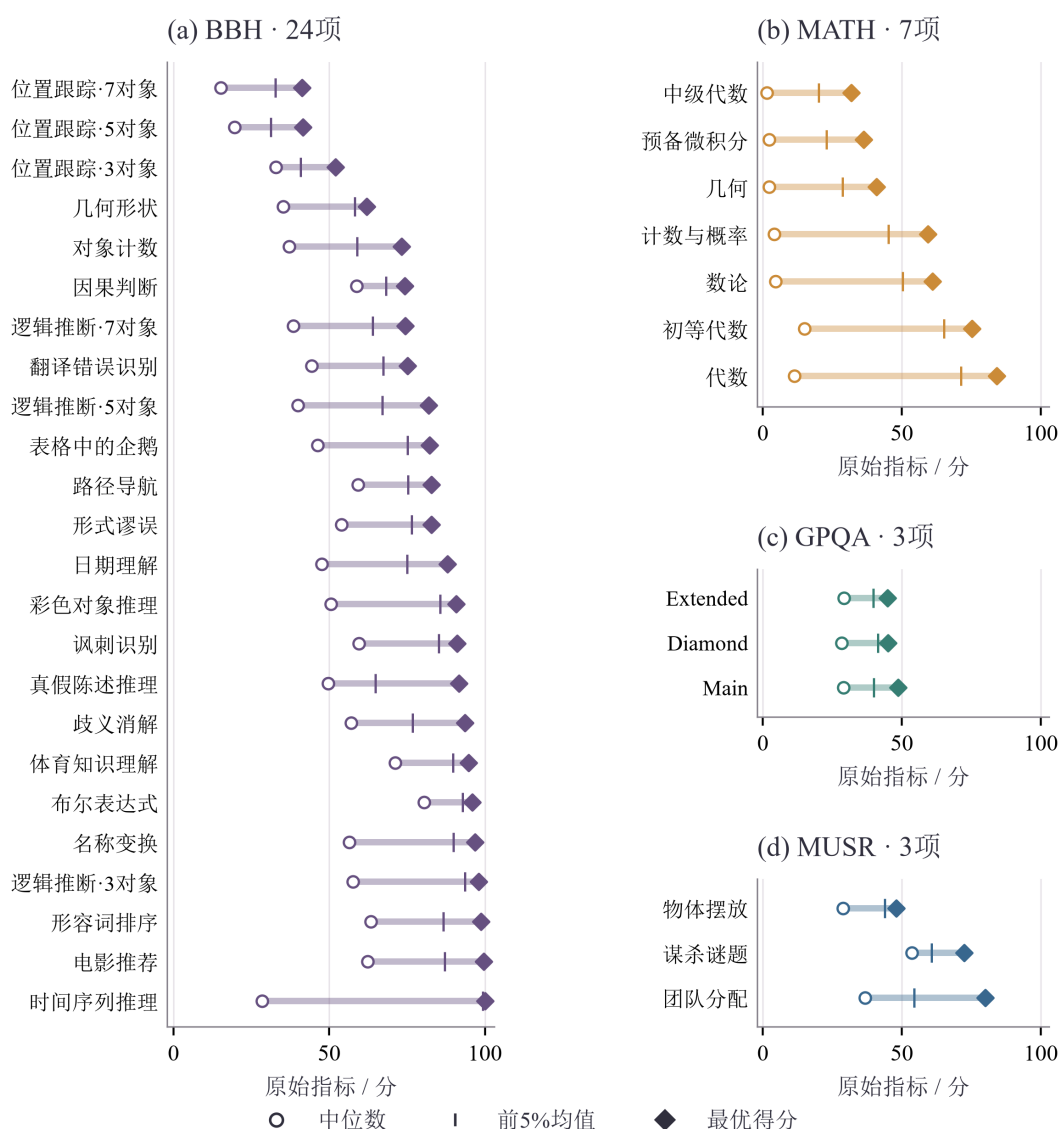


图 30 子任务得分

5.4.6 宏观资源证据与预测边界

Epoch AI 数据库 C4 与 C1 的名称规范化后只有 1 条匹配，不能组成逐模型联合样本，因此这里只把 C4 作为宏观资料 [26]。图31上图连接各年 Language 记录的算力中位数和最大值，左下图给出开放权重标记 Yes 的比例，右下图以全量 3,523 条记录为背景展示字段覆盖；浅色部分对应缺失或状态未明确的记录。

2018–2026 年，Language 子集共有 1,783 条记录。对九个年度的最大算力取对数拟合，斜率约为 0.589 个数量级/年，即相邻年度的拟合算力之比约为 3.88。记录整体呈扩张趋势，但并非每年递增，2024 年最大值低于 2023 年。2026 年仅有 40 条 Language 记录，其中 5 条带算力值，该点容易受收录不足影响，不能据此认定趋势反转。

以全部 Language 记录为分母，权重标记 Yes 在 2018 年占 26.7%，2025 年占 59.4%，中间各年有波动。由于分母也含权重状态缺失的记录，这只能反映数据库的收录结构，不能解释为市场份额，也不能据此认定 C1 能代表总体模型。

C4 的 3,523 条记录中，1,390 条含算力值，1,424 条含训练数据量；另有 2,653 条明确记录了权重状态，其中 Yes 为 1,325 条、No 为 1,328 条。训练数据量缺失较多且来源

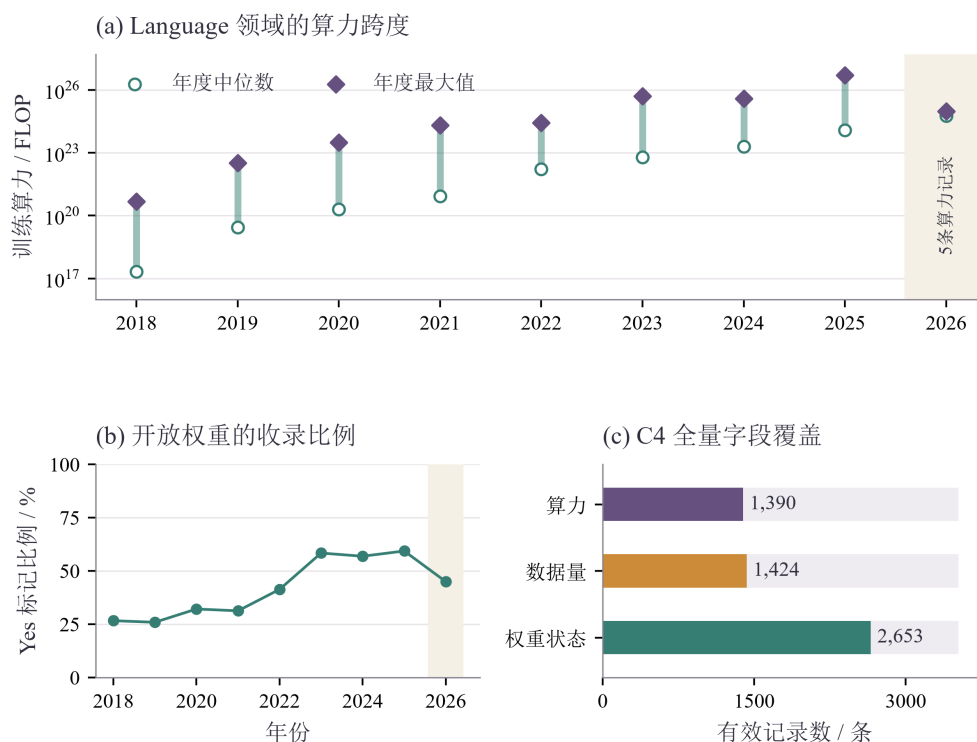


图 31 资源趋势与字段覆盖

不统一，因此只报告该字段的覆盖情况，不估计其年度增长。C4 可以为宏观资源情景提供参照，但不能确定算力放缓会使非规模进步按何种比例下降。

宏观记录与能力评测在对象范围和字段覆盖上存在差别，因而本问没有把 C4 的年度算力趋势直接代入 C1 的得分回归。年度最大值用于观察数据库中已记录的高算力水平，中位数用于描述当年有算力记录样本的中间位置，两者都依赖收录情况。因此，我们同时报告趋势与字段覆盖，明确各项统计的样本范围，缺失字段不按零投入处理。

综合本问分析，我们观察到损失与得分大体反向变化，并在明确各自前沿定义后核算了长期与近期增量。按当前代理指标，长期窗口中的规模关联项与未解释剩余项接近各半；2024 年 6–9 月的短窗口则以剩余项为主，二者均不能直接认定技术进步的来源比例。未来预测保留原条件曲线、重拟合分位范围和结构情景，补充诊断表明其饱和位置和时间外表现仍受窗口选择影响。任务层面则提示数学、研究生级问答和多步推理是当前评测中的薄弱部分。

六、模型检验与分析

我们从预测误差和条件变化两方面检验模型：前者考察对检验记录的预测，后者考察赋权、成本与预算改变后配置及结论是否保持。

6.1 误差分析

表34汇总各问使用的检验指标。问题一的线性岭与核岭、问题二的经典和广义标度律，以及问题四的损失–得分桥接和合并横截面回归，分别报告 R^2 、RMSE、MAE、MAPE。我们据此同时评价拟合解释度与预测偏差，避免仅凭相关性判断效果。

设同一评价集合中的观测值为 y_i ，预测值为 $\hat{y}_i$ ，记录数为 n ，均值为 $\bar{y}$ 。几类指标分别由残差的平方、绝对值及相对大小构成：

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2},$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}, \quad \text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|,$$

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|.$$

其中, R^2 需要观测值具有非零变异, MAPE 的逐条计算则要求观测值非零。RMSE 对较大的偏差给予更多权重, MAE 描述平均绝对偏离, 两者与被预测量具有相同量纲; MAPE 便于阅读相对误差, 但观测值靠近零时, 少量记录也可能显著影响其大小。因此, 我们结合样本取值范围共同报告这些指标。

我们还区分决定系数与相关系数的用途。 R^2 将残差与观测值围绕均值的变异作比较, 要求预测值在实际尺度上接近观测值; Pearson 的 r 则衡量两者共同变化的程度。预测序列整体偏高或偏低时, 仍可能保持较高相关。表中的跨尺度相关用于评价趋势, 其余结果按各问的评价集合解释, 不据同列数值直接排列所有模型的优劣。

表 34 各问拟合、预测误差与预算约束残差汇总

模型（数据集）	R^2 / r 或残差	RMSE / 相对残差	MAE	MAPE(%)
问题一岭回归（训练 1M, $n = 512$ ）	$R^2 = 0.629$	0.551	0.403	8.54
问题一岭回归（检验 1M, $n = 256$ ）	$R^2 = 0.619$	0.523	0.386	8.43
问题一核岭（检验 1M, $n = 256$ ）	$R^2 = 0.9454$	0.2256	0.1344	—
问题一线性岭跨尺度 Pearson（60M / 1B）	$r: 0.782 / 0.651$	—	—	—
问题二经典标度律（B1, $n = 1,176$ ）	$R^2 = 0.9999998$	1.5×10^{-4}	1.05×10^{-4}	0.004
问题二经典标度律留出（4 小模型 $\rightarrow$ 4 大模型）	$R^2 = 0.9999997$	1.5×10^{-4}	—	0.005
问题二广义标度律（B1+B6 联合, $n = 1,536$ ）	$R^2 = 0.990$	0.036	0.017	0.66
问题二跨族验证（B4, $n = 57$ ）	$r = 0.975$	0.293	—	8.83
问题二文献验证（B5, $n = 44$ ）	$r = 0.910$	0.193	—	7.99
问题四 Loss—Benchmark 桥接（C6, $n = 75$ ）	$R^2 = 0.26$	—	—	67.79
问题四合并横截面回归（C1 开源筛选, $n = 2,346$ ）	$R^2 = 0.475$	—	—	—

表 34 各问拟合、预测误差与预算约束残差汇总 (续)

模型 (数据集)	R^2 / r 或残差	RMSE / 相对 残差	MAE	MAPE(%)
问题四合并横截面回归 (C3 长周期, $n = 4,579$)	$R^2 = 0.424$	—	—	—
问题四前沿 logistic (历史 拟合)	$R^2 = 0.975$	—	—	—
问题三优化约束残差 ($ C_{\text{total}}/C_{\text{budget}} - 1 $)	—	$< 10^{-6}$	—	—

问题一的同尺度预测与训练表现接近。表34中, 13 个验证域的平均训练 R^2 为 0.629, 1M 检验为 0.619, 相差 0.010, MAPE 约 8.5%。这一对照没有显示明显的训练与检验落差, 但不能据此排除线性欠拟合。核岭的同尺度平均域 $R^2 = 0.9454$ 、综合 RMSE 为 0.2256; 相应训练五折误差为 0.2558。跨尺度绝对误差见表8, 较高趋势相关不代表绝对损失准确。

问题二的精度取决于检验来源。对 B1, 经典拟合有 $R^2 \approx 1.000$ 、MAPE 0.004%, 仅用小模型估计后, 对大模型的留出预测仍有 $R^2 = 0.9999997$; 残差标准差 1.5×10^{-4} 相比损失标准差 0.342 很小。这些结果说明对 B1 的贴合程度高, 尚不能说明跨来源同样准确。B4 和 B5 的 MAPE 分别为 8.8%、8.0%, 相关分别为 $r = 0.975$ 、 $r = 0.910$ 。广义模型的 1,536 条联合记录给出 $R^2 = 0.990$ 、MAPE 0.66%; 由于样本组成改变, 不能将其拟合优度与经典模型直接作差来判断泛化收益。

问题四的换算精度主要受样本可比性限制: 中可比层的验证集不同, 高可比层的得分跨度有限, 合并后的 $R^2 = 0.26$ 、MAPE 为 67.8%, 故结果仅作方向解释。问题三检查的是预算可行性; 表中最后一行的数值是成本回代得到的相对预算残差, 虽与 RMSE 同列展示, 不能解释为预测误差。有效解的残差均 $< 10^{-6}$, 符合成本约束。

各问的评价集合不同, 也决定了误差数字的使用范围。问题一的检验针对新配方下的领域损失, 问题二的对照涉及不同规模、质量和资料来源, 问题四的桥接还要跨越损失与能力两种指标。较小误差能够支持相应集合内的描述, 却不能自动扩大到其他来源。因此, 我们按记录来源及其是否参与估计, 限定表34中各项误差的解释范围。

6.2 配方决策与时间外推检验

逐域预测误差与配方选优并非同一指标。我们固定问题一优化时使用的 13 域参考权重, 分别对观测损失与预测损失加权, 再比较排序及模型选中配方的观测名次。表35中的名次只针对相应检验集合; 这些集合不等于有界优化的可行域, 不能将名次写成连续优化推荐的实测名次。

表 35 相同加权目标下的检验配方排序与选优

评价集合	线性岭 Spearman	核岭 Spearman	线性选中项 观测名次	核岭选中项 观测名次
五折折外 (512 组)	0.3154	0.9567	142/512	2/512
1M 检验 (256 组)	0.4280	0.9628	58/256	1/256
60M 检验 (256 组)	0.4017	0.9572	72/256	4/256
1B 检验 (64 组)	-0.0795	0.6396	64/64	2/64

核岭在同一加权目标下给出较好的检验集排序，因而用于训练凸包内的正式推荐。四个初值的目标一致且约束成立，为数值求解提供了检查；其在连续可行域中的全局最优性及真实训练收益仍未获得实验验证。旧线性候选的双代理分歧见表10，它说明响应函数选择会改变配方决策。问题二、三进一步迁移该预测差时，仍需检验损失尺度和收益幅度。

为检查对时间窗口的依赖，事后选取三个截点：每次只使用截点及以前的前沿记录拟合，预测其后月份至 2025 年 3 月，并与“保持截点最近累计峰值”的基线比较。表36沿用正式曲线的函数、初值与参数边界；检验对象始终是累计前沿，而非单个新模型的得分。

表 36 累计前沿的时间留出预测误差（分）

拟合截止月份	后续检验月数	logistic RMSE	持平基线 RMSE
2024 年 7 月	8	6.394	12.446
2024 年 8 月	7	8.269	5.653
2024 年 9 月	6	11.140	0.000

7 月截点下曲线误差较小，8、9 月截点下持平基线较小，未显示稳定的预测优势。最后一个检验段恰处于平台，持平误差为零不能推广到增长阶段。三个截点拟合的 K 均触及 100 的上界；仅删去正式样本末尾三个平台点， K 又从 56.61 变为 71.40，对同一未来两时点的预测由 56.33/56.59 变为 69.31/71.09。这说明渐近位置对观察窗口敏感，重拟合分位范围不能代替这种结构与窗口不确定性。

6.3 灵敏度分析

表37比较赋权方式、冲突阈值、岭正则、质量成本、质量基线、上下文长度和预算变化后的结果，同时补充指标方向与跨来源验证的影响。我们通过这些对照识别较稳定的结论，以及依赖特定成本或样本的结论。

表 37 评分规则、成本与预算变化下的结果稳定性

参数/设定变化	输出变化	灵敏度评估
赋权方案：熵权法 → 等权 平均 → 首主成分非负载 荷加权	域级质量排序一致性 0.643 → 0.679；样本级得分相 关 0.875/0.927；得分均值 0.491 → 0.637 → 0.496	中等敏感：整体格局稳 定，但领域名次会变化
指标方向：ad_en 按“索 引 1= 无广告”（正向）→ 若误判为负向	领域质量排序由「arxiv > stackexchange > c4 > commoncrawl > book > wikipedia > github」变为「c4 > arxiv > commoncrawl > stackexchange > book > wikipedia > github」	高度敏感：方向判定 必须用实证证据
冲突阈值 τ : 0.5 → 1.0 → 1.5 → 2.0 → 2.5	冲突率 0.657 → 0.394 → 0.219 → 0.120 → 0.066；与独 立基准之比 0.91 → 0.82 → 0.76 → 0.76 → 0.86	结论稳健：所考察阈 值下冲突率均低于独 立基准

续下页

表 37 评分规则、成本与预算变化下的结果稳定性 (续)

参数/设定变化	输出变化	灵敏度评估
岭正则系数 λ : $10^{-4} \sim 10^{-1}$	训练 R^2 由 0.6291 降至 0.6238, 检验 R^2 由 0.6187 降至 0.6157	拟合精度较稳定; 系数 与候选配比仍会变化
质量成本函数: 指数型 $\rightarrow$ 幂函数型 $\rightarrow$ 对数渐进型 ($C = 10^{19}$)	$Q^* = 0.660 \rightarrow 0.549 \rightarrow 1.000$, L^* 差异 < 0.06 ; 质量份额 $13.1\% \rightarrow 0.0\% \rightarrow 31.7\%$	低预算档较敏感
同上, 考察 $C = 10^{22}$ 、 10^{24}	三种函数下 Q^* 均为 1.000; 在 10^{22} 档, D^* 为 229.4–274.1B tokens, 最大值较最小值高约 19.5%	质量均达到上界, 规 模配置仍有差异
质量基线口径: 配比加权 $Q_0 = 0.5487 \rightarrow$ 样本均值 0.4912	三档预算下 Q^* : $0.549/1.000/1.000 \rightarrow$ $0.507/1.000/1.000$, 低预算质量 份额由 0 变为 2.63%, N^* 最大 变化约 3.24%	规模配置变化较小; 低预算是否启动质量 投入依赖基线口径
上下文长度: $2,048 \rightarrow 131,072$ tokens (C7 可行取值)	注意力/训练倍数 $0.07 \rightarrow 4.37$; N^* 由 6.091B 降至 2.705B; 损 失上升 6.0%	对配置影响较大; $L_{\text{crit}} = 30,000$ 为两项 成本相等的位置
算力预算 C : $10^{19} \rightarrow 10^{24}$ FLOPs	Q^* 由 0.549 升至 1.000; D^*/N^* 由 30.9 升至 76.1; 质量份额 $0 \rightarrow 0.407 \rightarrow 0.016$	本成本与质量曲率下 出现三个投入阶段
半合成数据验证: B2 / B8	B2 的 MAPE 为 33.7%; B8 校 准段翻转质量方向后为 133.4%, 其部分数据损失低于本模型的 参考渐近损失基准	跨来源偏差较大, 两 者均未参与主拟合

表37显示, 不同设定对结果的影响程度并不相同。

表11补充了线性系数及旧候选配比的正则敏感性, 表6比较全量与 A1 校准得到的评分标尺。两类结果提示, 预测精度、配置稳定性与评分标尺一致性应分别评价。

跨间接口的检验见表20和表18: 保持质量排序的映射仍会改变局部投入门槛, 质量指数受限拟合与原模型精度接近, 却给出不同的端点选择。B6 的组内质量收益又随规模变化, 因此当前加性质量项应理解为附件范围内的平均近似。

我们按条件作用的位置解释这些差异。调整权重或指标方向, 会先改变质量评分; 调整质量费用和上下文长度, 会改变预算中各项支出的比例; 扩大总预算, 则是在同一成本形式下改变可配置的资源总量。因而, 我们除比较最终损失外, 也比较中间变量和配置份额。若两种方案的预测损失接近, 但质量份额相差较大, 就说明目标值相近并不要求投入方式相同。

阈值、正则参数和预算扫描均由有限取值构成。相邻取值下的结果可用来判断已检验范围内的变化方向, 首次发生变化的网格点则给出定位边界的线索。我们保留扫描间距和求解状态, 并结合费用关系解释变化。图中连接相邻点仅用于展示趋势, 未检验区间不具有与扫描点相同的数值依据。

(1) 较稳定的结果。在所检验阈值内, 冲突率始终低于独立参照, 比值均 < 0.92 。岭正则变化对拟合精度影响较小; 更换质量基线后, 规模配置变化较小, 但低预算是否启动质量投入随基线口径改变。三类成本在所考察的中高预算档都使质量达到上界, 但参数量、数据量的具体组合仍然不同。

(2) **需要区分的边界。** $L_{\text{crit}} = 30,000$ tokens 只表示注意力费用与基础训练费用相等, 不代表最优解在此处发生突变; 1.58×10^{19} 、 7.08×10^{19} FLOPs 是预算网格中首次出现质量投入和首次达到上界的有效点, 实际边界还受网格间距、求解状态及成本形式影响。

(3) **依赖条件的结果。** $C = 10^{19}$ FLOPs 下更换质量成本时, 最优质量覆盖 $0.549 \sim 1.000$, 说明低预算策略依赖成本设定, 应同时比较各情景; 损失-得分桥接只有 $R^2 = 0.26$, 不能精确换算能力分; 领域排序也会受指标方向和赋权方式影响。

表24将假定配比收益从零逐步保留至全量, 等效预算倍数从1增至约16.9。该倍数随迁移幅度变化, 属于同一损失模型下的条件换算, 不是实测训练加速比。问题四的表29和表31分别检查年份组差及前沿窗口, 时间外推误差则统一列于表36。这些诊断与正式结果分开保存。

6.4 各问结果的衔接与使用条件

完成单项检验后, 我们进一步核对结果在各问之间的传递。质量基线、配比修正和标度律参数对下一问的作用不同, 因此分别检查其影响位置与使用条件, 明确现有检验覆盖的范围。

质量基线 Q_0 一方面确定问题三的质量起点, 另一方面进入增量费用 $[g(Q) - g(Q_0)]_+$ 。所以, 更换基线会同时改变质量可选区间和费用参照, 不能仅把结果表中的质量数值平移后仍视为同一方案。前述两种基线的对照已经给出相应配置; 我们将各自基线、成本形式及配置共同列为使用条件。问题一中缺少直接领域质量观测的部分, 也需要随映射规则一并说明。

配比修正 $M(p)$ 则以损失平移量进入广义模型。配比固定时, 它不参与参数量、数据量和质量的求导, 但仍影响报告的预测损失水平。因而, 讨论规模配置与讨论配方收益时, 需要明确是否沿用同一配比。旧线性候选曾出现响应函数之间的收益方向分歧; 正式核岭候选增加了训练支持范围约束, 但其跨来源绝对收益仍未标定, 因此预算等价保留为条件情景。

标度律参数决定不同投入的边际收益, 成本函数决定实现这些投入要支出多少算力。问题二的拟合误差与问题三的成本敏感性分别考察这两侧的关系。现有结果可以说给定参数和成本下怎样配置, 也可以比较已设定成本情景之间的差别; 对于参数误差与成本误差同时变化后的影响, 本文没有另外给出联合概率区间, 因此不把这些情景差别解释成最优配置的置信范围。

能力预测又使用另一组观测。损失-得分桥接展示了两类指标的关联程度, 历史前沿和合并横截面回归则直接使用评测记录。问题三中的损失下降据此解释为训练目标的改善, 问题四的能力增量保留为相应评测尺度下的结果; 两者均应结合各自的样本范围和桥接假设解读。

七、模型的评价

7.1 模型的优点

- **质量评分有明确依据。** 22个指标的方向、预处理和熵权列于表2。广告字段经过原文核查, 不同列表字段按各自语义压缩, 缺失与异常值处理均可追溯。抽样与扩展集合在可比较领域的差值约为0.002, 提供了记录集合之间的评分对照; 两集合存在重复ID, 不能作为完全独立的验证。

- **指标冲突可以相对比较。** 我们从冲突比例和排序一致性两个层面判断分歧, 同时报告21.9%的冲突率、0.076的Kendall系数和28.9%的独立参照, 并在 $\tau \in [0.5, 2.5]$ 内检查比值均小于0.92。据此, 我们既识别指标的共同倾向, 也保留整体一致程度有限的判断, 并结合字段含义解释分歧。

• **质量项便于解释和比较。**加性质量缺口 $C(1 - Q)^\gamma$ 在 $Q = 1$ 时归零，因而与 Chinchilla 形式相接；相比直接写成 $E - CQ^\gamma$ ，高质量端不会因为相减而出现负损失。选择形式时，我们比较四种候选的 R^2 、残差信息评分及范围外表现。B3 的轨迹指数约为 -0.2805 ，与拟合的 -0.2799 接近，只能算内部一致性检查。

• **配置变化可从成本公式解释。**预算模型保留 $C_Q = D[g(Q) - g(Q_0)]_+$ ，质量费用因数据量而变化；注意力费用与基础训练费用的临界长度为 $L_{\text{crit}} = 6/\eta = 30,000$ 。由此可以推导式(40)，并用与参数量同单位的折算量 $\tau(Q)$ 说明质量处理如何改变参数-数据平衡。质量费用采用题设附录 B 的三类函数参数。

• **不同来源的结果分别解释。**问题四按字段完整性与许可筛选，再按时间和模型类型分组；C8 成功解析 1,860 个目录，其中与主样本交集的 964 个目录汇总 37 项任务；C4 只用于宏观资源与字段覆盖。B2、B8 的异常也保留在检验中，例如 B8 有低于本模型参考渐近损失基准的记录，避免把不同性质的资料当作同一种证据。六维指标另与完整 C1 作名称匹配和记录级秩相关检查，预测时则分别报告重拟合分位范围与情景范围。

7.2 模型的缺点

• **预测增强仍有适用范围。**核岭使 1M 同尺度平均域 R^2 达到 0.9454，但直接跨尺度的绝对损失失配。线性近似保留系数解释与配方对照，核岭负责训练凸包内推荐；支持范围约束减少外推，却不保证凸包内处处有充分观测，也不能保证数值候选全局最优。nih_exporter、enron_emails、europarl、philpapers 没有对应的验证损失列，无法直接评价这些领域自身的预测损失，推荐时固定其参考占比；它们对已有 13 个验证任务的作用仍由配比系数表示。混合质量指数与原配比特征严格线性相关，加入后的 R^2 差异来自岭惩罚的改变，无法据此识别质量的独立作用。

• **部分评分方向仍依赖字段含义。**广告、流畅度等方向已经通过原文和领域对照检查，modernbert 的 6 级指标按有序等级处理；qurater 按四个不同维度处理，其同向等权聚合仍是本文假设，尚无独立任务效果校准。熵权还会受分布形态影响，等权后领域排序一致性为 0.643。第 6.3 节的敏感性结果表明，方向判错会改变排名，因此领域名次需要和评分规则一起报告。

• **质量实验的支持范围有限。**质量项主要由 B6/B7 半合成实验支持。B8 方向相反且出现低于损失下限的记录，不能并入主拟合；B2 的 MAPE 为 33.7%，也提醒我们跨来源估计存在较大偏差。B6/B7 的质量取值覆盖 0.1 至 1.0，包含低于基线的记录。问题三将配置限定在 $[Q_0, 1]$ ，是因为该问讨论从既有基线继续提升质量及其增量费用；主动降低质量的配置不在本模型范围内。不同来源评分仍需标定；单调映射与质量曲率诊断均表明，拟合指标接近时，质量投入门槛及上界选择仍可能变化。

• **能力预测受到时间和评分限制。**损失映射的 $R^2 = 0.26$ 、MAPE 为 67.8%，不能把损失改善精确折成能力分。C1 的可比窗口约 10 个月，却要向前预测其 1.2 ~ 2.4 倍；C3 的历史评分又来自不同来源。长期规模占比 51.4% 与 2024 年 6-9 月的 4.6% 只对应各自窗口，不能互相替代。此外，仅两个年份的回归不足以识别具体技术机制，剩余项混合了样本构成等影响。平台窗口删减和时间留出也表明，曲线重拟合分位范围不能替代对未来观测误差的验证。

八、模型的改进与推广

8.1 模型的改进

结合检验结果，我们认为后续应优先改善跨尺度校准、实际成本估计与数据覆盖。

核岭的同尺度预测较好，跨尺度绝对误差却仍然较大。下一步应在目标规模补充配方实验，用独立留出记录标定损失偏移和响应幅度，并直接比较参考配方与推荐配方的

加权损失。对没有验证损失的四个领域，可补充任务评测或经过校准的代理信息，检验固定其参考份额是否必要。指标冲突则可探索有序 probit 因子模型，但潜在质量及区间解释仍需独立依据。

质量模型可加入 $C(1 - Q)^{\gamma_1} D^{-\gamma_2}$ 等交互形式，检验质量缺口是否随数据量变化。预算规划还可加入重复训练、稀疏化和量化产生的训练与推理费用，以及分阶段课程学习。鉴于低预算配置对成本形式敏感，我们建议记录实际清洗费用，先拟合经验成本曲线，再与题设三类函数比较。

为检验上述改进，我们建议有针对性地补充数据：收集同一批语料的附件 A 型评分与训练损失，标定质量映射，并在 B6/B7 已覆盖的区间内补充独立实验；同时核对发布时间、许可和去重比例等字段能否解释质量标签。C4 与 C1 可先按组织名、参数和时间匹配，经人工核实后再做训练算力与能力的联合分析。若取得 v1、v2 评测的共同校准样本，还可把 2019–2023 年的历史记录换到同一尺度，缓解当前可比窗口只有 10 个月的问题。

8.2 模型的推广

本文的配置分析可用于其他预算受限场景：先估计投入与目标的关系，再计入费用并比较可行方案。迁移时需要重新核对观测数据、损失尺度和成本函数。

在大模型预训练、微调和蒸馏中，可先拟合损失与投入的关系，再计入费用约束求解配置。式(40)和 $\tau(Q)$ 适合比较语料处理与规模投入，但换到新任务时仍应重新估计参数，不能直接套用本文的算力数值。

语料治理时，22 个指标的方向表和冲突率对照可用于筛选、清洗及配比调整；领域系数相似度 s_{ij} 可以帮助寻找作用方式接近或相反的来源。至于两类语料能否等量替换、混合后是否增加收益，仍需配比实验确认。

资源规划可借鉴长期 shift-share 分解与近期同一评测窗口对照，分别观察规模关联项及剩余项对应的增量。这种分解提供的是分析线索，剩余项不能直接当作算法收益。我们据此分别解释较稳定的结果、阶段边界与依赖假设的结果，以限定多来源分析的适用范围。

对于芯片制程、新能源和生物医药等可能逐步饱和的技术指标，也可比较 logistic 曲线与结构情景的预测差异，但前提是先确认该指标具有相应的增长形态。面对半合成或估算资料，本文对评分方向和参考渐近损失基准的检查，也可帮助判断哪些记录不能直接混合。

参考文献

- [1] KAPLAN J, MCCANDLISH S, HENIGHAN T, et al. Scaling laws for neural language models[EB/OL]. (2020-01-23)[2026-09-27]. <https://arxiv.org/abs/2001.08361>.
- [2] HOFFMANN J, BORGEAUD S, MENSCH A, et al. An empirical analysis of compute-optimal large language model training[C]//Advances in Neural Information Processing Systems. Curran Associates, 2022, 35: 30016-30030.
- [3] BESIROGLU T, ERDIL E, BARNETT M, et al. Chinchilla scaling: A replication attempt[EB/OL]. (2024-04-15)[2026-09-27]. <https://arxiv.org/abs/2404.10102>.
- [4] VILLALOBOS P, HO A, SEVILLA J, et al. Position: Will we run out of data? Limits of LLM scaling based on human-generated data[C]//Proceedings of the 41st International Conference on Machine Learning. PMLR, 2024, 235: 49523-49544.
- [5] GUNASEKAR S, ZHANG Y, ANEJA J, et al. Textbooks are all you need[EB/OL]. (2023-06-20)[2026-09-27]. <https://arxiv.org/abs/2306.11644>.

- [6] LI J, FANG A, SMYRNIS G, et al. DataComp-LM: In search of the next generation of training sets for language models[C]//Advances in Neural Information Processing Systems. Curran Associates, 2024, 37: 14200-14282.
- [7] LIU Q, ZHENG X, MUENNIGHOFF N, et al. RegMix: Data mixture as regression for language model pre-training[C/OL]//The Thirteenth International Conference on Learning Representations. 2025[2026-09-27].
<https://openreview.net/forum?id=5BjQOUXq7i>.
- [8] GAO L, BIDERMAN S, BLACK S, et al. The Pile: An 800GB dataset of diverse text for language modeling[EB/OL]. (2020-12-31)[2026-09-27].
<https://arxiv.org/abs/2101.00027>.
- [9] OPENDATALAB. SlimPajama-Meta-rater[DB/OL]. [2026-09-27].
<https://huggingface.co/datasets/pendatalab/SlimPajama-Meta-rater>.
- [10] SHANNON C E. A mathematical theory of communication[J]. Bell System Technical Journal, 1948, 27(3): 379-423. DOI: 10.1002/j.1538-7305.1948.tb01338.x.
- [11] KENDALL M G, SMITH B B. The problem of m rankings[J]. The Annals of Mathematical Statistics, 1939, 10(3): 275-287. DOI: 10.1214/aoms/1177732186.
- [12] HOERL A E, KENNARD R W. Ridge regression: Biased estimation for nonorthogonal problems[J]. Technometrics, 1970, 12(1): 55-67. DOI: 10.1080/00401706.1970.10488634.
- [13] SCIKIT-LEARN DEVELOPERS. Kernel ridge regression[EB/OL]. [2026-09-27].
https://scikit-learn.org/stable/modules/kernel_ridge.html.
- [14] 腾讯. WorkBuddy: AI 办公工作台 [EB/OL]. [2026-09-27].
<https://cloud.tencent.com/product/workbuddy>.
- [15] DEEPSEEK. DeepSeek-V4-Flash (公开测试) [EB/OL]. (2026-07-31)[2026-09-27].
<https://api-docs.deepseek.com/updates>.
- [16] OPENAI. Codex (模型: GPT-5.6-Luna) [EB/OL]. (2026-07-09)[2026-09-27].
<https://developers.openai.com/api/docs/models/gpt-5.6-luna>.
- [17] BIDERMAN S, SCHOELKOPF H, ANTHONY Q G, et al. Pythia: A suite for analyzing large language models across training and scaling[C]//Proceedings of the 40th International Conference on Machine Learning. PMLR, 2023, 202: 2397-2430.
- [18] SCIPY DEVELOPERS. minimize(method=SLSQP)[EB/OL]. [2026-09-27].
<https://docs.scipy.org/doc/scipy/reference/optimize.minimize-slsqp.html>.
- [19] XIE S M, PHAM H, DONG X, et al. DoReMi: Optimizing data mixtures speeds up language model pretraining[C]//Advances in Neural Information Processing Systems. Curran Associates, 2023, 36: 69798-69818.
- [20] MUENNIGHOFF N, RUSH A, BARAK B, et al. Scaling data-constrained language models[C]//Advances in Neural Information Processing Systems. Curran Associates, 2023, 36: 50358-50376.
- [21] SCHAEFFER R, MIRANDA B, KOYEJO S. Are emergent abilities of large language models a mirage?[C]//Advances in Neural Information Processing Systems. Curran Associates, 2023, 36: 55565-55581.
- [22] HUGGING FACE. Open LLM Leaderboard results[DB/OL]. [2026-09-27].
<https://huggingface.co/datasets/open-llm-leaderboard/results>.
- [23] ZHOU J, LU T, MISHRA S, et al. Instruction-following evaluation for large language models[EB/OL]. (2023-11-14)[2026-09-27]. <https://arxiv.org/abs/2311.07911>.

- [24] WANG Y, MA X, ZHANG G, et al. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark[C]//Advances in Neural Information Processing Systems. Curran Associates, 2024, 37: 95266-95290.
- [25] EFRON B. Bootstrap methods: Another look at the jackknife[J]. The Annals of Statistics, 1979, 7(1): 1-26. DOI: 10.1214/aos/1176344552.
- [26] EPOCH AI. Data on AI models[DB/OL]. [2026-09-27]. <https://epoch.ai/data/ai-models>.
- [27] 中国研究生数学建模竞赛组织委员会, 中国研究生数学建模竞赛专家委员会. “华为杯”第二十三届中国研究生数学建模竞赛人工智能工具及输出使用规定[EB/OL]. (2026-09-16)[2026-09-27]. <https://cpipc.acge.org.cn/sysFile/downFile.do?fileId=4fc024f7a1504abb87fc056440c64a28>.

附录

附录 A：数据利用清单

附录 A 按附件编号说明每份数据在本文中的用途，并把真实记录、半合成实验、插值轨迹和外推估算区分开来。各附件对应的计算环节及直接观测、对照或稳健性检查用途见表38。

表 38 附件数据利用清单

编号	文件名	性质	是否使用	使用方式与可信度说明
A1	slimpajama_quality_signal_sample	真实	使用	对全量 51,230 条计算 22 指标熵权评分、领域 Q 与冲突，并抽查原文
A2	slimpajama_quality_extended/arxiv	真实	使用	以全部 17,523 条计算 arxiv 的领域 Q ，与抽样结果比较
A3	slimpajama_quality_extended/github	真实	使用	以全部 203,752 条计算 github 的领域 Q ，与抽样结果比较
A4	train_mixture_1m	真实	使用	512 组训练配方作为线性岭与核岭输入，用于拟合配比与损失
A5	train_pile_loss_1m	真实	使用	提供回归对应的 13 个验证域损失
A6–A9	test_mixture/loss_1m, 60m	真实	使用	分别检验同尺度和跨尺度的预测，每组 256 条
A10–A11	test_mixture/loss_1B	真实	使用	用 64 组记录检验跨尺度的预测
A12–A15	est_mixture/loss_10b, 70b	子集/外推	使用	各 63 组用于比较外推尺度因子的稳定性；属于外推记录，并非观测
A16	domain_mapping_guide	参考	使用	连接 17 域与质量分类：6 域按 direct/near_direct 对应，11 域标为 inferred 并使用全局均值
A17	regmix_domain_summary	真实	未使用	未用于本文分析

表 38 附件数据利用清单 (续)

编号	文件名	性质	是否使用	使用方式与可信度说明
A18	regmix_domain_sample	真实	未使用	未用于本文分析
B1	pythia_training_log_existing	真实	使用	1,176 点用于经典拟合和模型留出检验；这组记录的拟合残差极小
B2	cerebras_training_log	半合成	使用	作模型族外对照，MAPE 为 33.7%；来源差异较大，只作定性参考
B3	training_trajectories	插值	使用	检查轨迹内部的 D 指数： -0.2806 至 -0.2803 ，对照拟合值 -0.2799
B4	scaling_baseline	真实	使用	用不同模型族作检验，相关 $r = 0.975$ ，MAPE 为 8.8%
B5	published_scaling_data	真实	使用	与文献记录比较，相关 $r = 0.910$ ，MAPE 为 8.0%
B6	supplementary_NQ_experiment	半合成	使用	360 点用于估计质量缺口项；说明半合成条件下的质量效应，不作为直接实验观测
B7	supplementary_NQ_experiment_expanded	半合成	使用	新增记录用于开发阶段检查，相关 $r = 0.991$
B8	supplementary_NQ_experiment_large	半合成	部分使用	Q_{score} 与 B6/B7 的方向相反，且含低于本模型参考渐近损失基准的记录；不并入拟合，用于说明来源间的差异
B9	supplementary_large_models	真实	使用	提供百亿参数以上模型的数量、数据量和开源可及性资料
B10	supplementary_large_baseline	估算	使用	比较百亿参数以上的损失估算， $r \approx 1.000$ 、MAPE 为 1.0%；该对照不作为直接观测
B11	open_model_family_metadata	真实	未使用	未用于本文分析
B12	pythia_checkpoint_index	真实	未使用	未用于本文分析
C1	leaderboard_cleaned	真实	使用	按六维得分、许可、类型和提交日期整理样本；筛选后的 2,346 条用于合并横截面回归和前沿分析
C2	leaderboard_enhanced	真实	未使用	作为 C1 的增强替代版本，为避免重复计入而不另用
C3	leaderboard_extended_timeseries	混合	使用	核算 2019–2025 年增量；排除 13 条一致性异常记录及 7 条参数数量为零的记录，保留 4,579 条；长期解释仍受评分来源差异限制

表 38 附件数据利用清单（续）

编号	文件名	性质	是否使用	使用方式与可信度说明
C4	epoch_all_ai_models	真实	使用	观察训练算力趋势，年斜率 +0.589 dex；核对数据量与权重字段，Yes 为 1,325、No 为 1,328
C5	loss_benchmark_bridge	混合	使用	作为损失与得分关系的对照， $n = 43$ 、 $r = -0.502$
C6	loss_benchmark_bridge_expanded	混合	使用	用于主要损失-得分拟合， $n = 75$ ；High、Medium 两层按可比程度分别处理
C7	model_architecture_metadata	真实	使用	将 2048、4096、8192、32768、131072 五档上下文用于问题三灵敏度计算
C8	detailed_results/	真实	使用	1,860 个目录取得有效结果，主样本交集的 964 个用于汇总 37 项任务；六维指标另与完整 C1 逐记录校验
C9	data/（Parquet）	真实	未使用	与 C1 等价，本文不重复计入
C10	pythia_eval_details/	说明	未使用	只有 README 说明，不能作为 C8 的任务记录使用

附录 B：半合成与外推数据的标注

以下资料不全是直接观测，解释相应结果时需分别考虑其生成方式和评分来源。

- **半合成数据：**B2 为 Cerebras 训练日志，B6/B7/B8 为 NQ 质量实验。本文以 B6 估计质量项，以 B7 新增点作开发阶段验证，5.2 节表格标明其半合成性质。检查来源差异后，B2、B8 均不参加主拟合。

- **外推与估算数据：**A12–A15 的 10B/70B 损失由 1M/60M/1B 记录作幂律外推，B10 则为百亿参数以上的损失估算。它们用来比较估算关系的稳定性，不当作真实训练观测。

- **插值数据：**B3 来自 Pythia 检查点之间的插值，只检查轨迹内部幂指数是否一致，不增加独立观测。

- **混合口径数据：**C3 各年的评测标准并不完全相同，异常记录已检查并剔除。C5/C6 的 Loss 来源和验证集也未完全统一，因此按可比程度分层使用。

附录 C：结果文件与核心求解代码

四问的结果分别保存为 CSV，存放在求解/问题一至问题四/结果/ 目录。表39列出主要文件及其作用，表中省略“求解/问题 X/结果/”这一共同前缀，便于阅读。

表 39 主要结果文件说明

文件	说明
质量指标熵权.csv	22 指标的方向判定、语义说明、信息熵与熵权

表 39 主要结果文件说明 (续)

文件	说明
质量信号缺失审计.csv	逐指标字段级缺失计数 (A1–A3 全量)
质量冲突与一致性.csv	抽样集/扩展集/全量的冲突率、独立基准与 Kendall W
冲突样本裁决汇总.csv	按领域、冲突标记和域内中位数判定汇总记录数
非线性代理对照_汇总.csv、逐域.csv	核岭与线性基线的同尺度、跨尺度误差
非线性代理对照_预测明细.csv	各模型的折外及检验预测, 可直接计算图表指标
非线性代理对照_参数搜索.csv、五折.csv、折分配.csv	训练参数选择及折外误差明细
冲突阈值灵敏度.csv	冲突率随阈值 τ 的变化
抽样集与扩展集域级对照.csv	A1 与 A2/A3 的域级质量对照
质量与损失难度一致性.csv	两套质量口径在 6 个映射域上的排序比较
质量项增量检验.csv	冗余质量特征加入后的 R^2 差值
混合系数矩阵.csv、截距.csv	配比—损失岭回归系数与截距
推荐配比调整.csv	核岭在训练凸包约束内得到的推荐配比与调整量
线性有界配比对照.csv	原线性目标下的有界方案, 作为响应函数对照
核岭推荐_训练凸组合.csv、多初值.csv、目标对照.csv	推荐配方的训练支持范围、求解状态及加权损失证据
问题一_拟合汇总.csv	各损失域的拟合与检验指标
岭正则敏感性.csv	正则系数灵敏度
经典标度律参数.csv	经典标度律参数、拟合与留出验证指标
质量字段方向审计.csv	B6/B7/B8 的组内相关与方向结论
广义标度律形式对比.csv	四种质量项形式的 R^2 与残差信息评分比较
广义标度律分层验证.csv	6 组数据来源的验证结果
弹性与替代关系.csv、质量参数替代关系.csv	弹性与替代率
领域替代互补_top.csv	领域效应的相似性与差异
B3 轨迹验证.csv	轨迹内 D 幂指数验证
三档预算最优配置.csv、成本函数对比.csv	三档预算与三型成本函数的优化结果
结构性转移识别.csv	结构性转移门槛
上下文长度敏感性.csv	C7 五档上下文长度下的最优配置
质量基线灵敏度.csv	质量基线口径灵敏度
配比项算力等价.csv	配比优化的算力等价效果
筛选漏斗.csv	字段完整性与许可筛选过程, 以及模型类型构成
C8 逐任务解析结果.csv	C8 成功解析目录的任务级原始指标

表 39 主要结果文件说明 (续)

文件	说明
C8 逐任务聚合.csv、C8 子任务类别 汇总.csv	子任务前沿与剩余空间
C8 与 C1 一致性校验.csv	C8 与 C1 的秩相关校验
Loss_Benchmark 映射.csv	分层桥接拟合结果
规模时间分解.csv、规模时间分解_ 长周期.csv	规模关联项与剩余项分解
C3 口径异常行.csv	C3 自治性审计剔除的记录
前沿预测.csv、前沿预测_结构外推 情景.csv	能力前沿预测与情景
C4 宏观趋势.csv、C4 宏观摘要.csv	C4 宏观算力与开源权重证据
求解/广义标度律参数.csv、问题一_ 关键量.json	跨问共享的接口参数

附录 D: 人工智能工具及输出使用说明

本论文末尾对人工智能工具的实际使用范围作统一说明。腾讯 WorkBuddy 接入 DeepSeek-V4-Flash (2026-07-31 公开测试) 用于文段相关辅助; OpenAI Codex (GPT-5.6-Luna, 2026-07-09) 用于图表展示、排版检查、程序整理、数据整理和结果复核。工具信息见文献 [14, 15, 16]。正文表达与段落衔接另经 OpenAI Codex 辅助润色。

表40集中列示辅助工作的输入类型、涉及内容与核查方式。各问结果附近的辅助分析说明以及各程序开头的工具注释分别保留, 以对应竞赛规定对数据分析和辅助编程的标注要求 [27]。

表 40 人工智能辅助内容与核查方式

辅助环节	输入与涉及内容	核查方式与对应位置
论文表达与排版	已有文段、公式、图表及编译后的 PDF; 涉及表述衔接、题注、交叉引用与分页	对照公式、结果表和图示检查含义及单位; 各问相应结果附近列出辅助说明
图表展示	附件记录或已保存的计算结果; 涉及配色、布局、标记和流程图组织	统计图由附录所列程序读取附件记录或保存结果生成; 附录 E 列出图号、输入与生成程序的对应关系
程序整理与结果复核	已有求解程序、绘图程序、运行输出; 涉及字段解释、数值检查及复现对应关系	程序开头注明辅助范围; 输入、参数及计算过程见附录 E 的完整代码清单与程序依赖表
文献检索与引用整理	论证主题与已有引用; 涉及候选文献、来源核对和引用次序	依据出版页面、原文摘要或官方文档核对作者、题名与所支持的论述

辅助生成的图形属于计算结果的展示, 不能作为新增实验观测; 文献检索得到的条目也不代替原始证据。图表和正文采用的数据范围、计算条件及不确定性均在相应章节说明。

附录 E：运行环境与核心实现

各程序的运行输入、参数和输出关系见附录代码清单及表41；各问程序开头注明辅助工具及使用范围。

程序使用 Python 3，主要依赖 NumPy、pandas、SciPy、scikit-learn、Matplotlib 和 Pillow，数据解压及 JSON 读取调用 Python 标准库。四问共用求解/ `common.py`，附件根目录由环境变量 `MODELING_DATA_DIR` 或程序参数指定，也可放在项目祖先目录的 `real_attachments` 下。压缩 JSONL 文件直接读取，无需修改附件。下面列出四问的完整主程序，涵盖数据读取、模型求解、检验和结果输出；公共模块及正式绘图依赖也在后文完整列出，代码清单直接引用实际源文件。

原保存结果记录的环境为 Windows 11、Python 3.14.0，依赖版本为 NumPy 2.5.3、pandas 3.0.6、SciPy 1.18.1、scikit-learn 1.9.1、Matplotlib 3.11.2 和 Pillow 12.3.0。复现时将下列 Python 代码分别按题注中的相对路径保存为 UTF-8 文件，另保留总体流程图的 TikZ 源码。题目附件保留 `A_data_value`、`B_scaling_laws`、`C_efficiency_evolution` 的目录结构。统计绘图使用宋体（SimSun）和 Times New Roman；第三问另需 Adobe 思源宋体文件 `SourceHanSerifCN-Regular.otf`，放在 `论文/fonts/` 下。该字体可从 Adobe 官方项目 <https://github.com/adobe-fonts/source-han-serif> 获取。字体与依赖版本不同可能改变字形及像素排布，数值核对以计算结果为准。

运行时依次执行问题一、问题二和问题三，问题四读取附件 C、可单独运行。问题一输出质量、配比和关键量，问题二据此估计配比修正及广义标度律参数，问题三读取前两问共享文件。各程序的输入、输出关系见表41。

表 41 问题、程序与数据依赖

问题	核心程序	输入	主要输出
一	求解/问题一/ 问题一.py	附件 A	质量评分、解释系数、核岭支持域推荐
二	求解/问题二/ 问题二.py	附件 B、问题一结果	标度律参数、验证指标、配比修正
三	求解/问题三/ 问题三.py	前两问共享结果、附件 C7	三档配置、预算扫描与敏感性
四	求解/问题四/ 问题四.py	附件 C	任务聚合、关联分解与前沿情景

正式图表与复现顺序

从题目附件首次生成结果时，不需要预先准备结果 CSV。安装上述依赖、保存代码和字体后，在项目根目录依次运行以下命令；其中附件路径替换为实际位置，环境变量设置采用 Windows PowerShell 语法。四问主程序在结果保存后调用对应的正式绘图程序，最后一条命令生成第三、四问的方法流程图。

```
$env:MODELING_DATA_DIR="D:/题目附件"
python 求解/问题一/问题一.py
python 求解/问题二/问题二.py
python 求解/问题三/问题三.py
python 求解/问题四/问题四.py
python 求解/绘制问题三四流程图.py
```

已有结果时，可用 `python 求解/复现论文.py --data-dir < 附件根目录 > --mode figures` 重新绘图；该模式只读取保存结果。`--mode verify-models` 则

在隔离副本依次求解四问，依据 复现预期输出.json 核对 85 项 CSV/JSON，包括跨问质量映射、关键量与标度律参数。每问运行前移除其旧输出，再要求重新生成；缺项、差异或进程错误均记录为失败，不覆盖原结果。

一般数值按相对容差 10^{-6} 、绝对容差 10^{-8} 比较；核岭推荐份额沿用求解精度对应的绝对容差 10^{-6} ，并另验配方约束。训练凸组合可能不唯一，故检查非负性、权重和及配方重建；多初值输出核对可行性、数值保护梯度下的间隙及目标值。平方根核在零份额边界不光滑，该间隙不构成光滑驻点的数学证书。环境版本和运行耗时另列元数据，不视为数值证据。第三问采用无量纲预算约束，微小派生费用的复现差异仍须单列解释。

补充的决策与时间检验使用 python 求解/补充检验.py --data-dir < 附件根目录 >，输出写入独立复现记录目录；其中包括相同加权目标的选优、配方预测分歧、质量结构与收益迁移敏感性、时间留出对照，均不替换四问保存结果。

核岭的独立计算入口为问题一目录中的“非线性代理对照.py”，参数搜索、折分配、预测明细和汇总分别保存。绘图只读取这些输出；核岭支持域推荐程序读取其固定超参数，在训练凸包和既有配比上限内搜索；问题二、三沿用该推荐的核岭损失差，线性系数仅作领域作用解释。总体流程图为原生 TikZ；其余正式图的生成程序见表42，流程图属于方法示意，不代表新增观测数据。

补充诊断位于 求解/审稿补充,与原求解结果分开保存。三个入口分别为“q1_决策与稳健性.py”“q23_尺度与结构敏感性.py”“q4_时间与前沿验证.py”，均接受 --data-dir 和 --output-dir。它们读取原始附件及现有拟合结果，输出候选排序、质量标尺、曲率、收益迁移和预测窗口的诊断表；新增对照不覆盖原拟合参数与配置文件。各诊断目录保存输入哈希、计算设定与检查摘要，表中数值可由对应程序重新计算。复现入口的回归测试位于 求解/tests/test_reproduction.py，使用微型数据检验缺失输出、结果差异与异常报告，不替代四问的实际重算。

表 42 正式图表与生成程序的对应关系

正文图号	生成程序 (Python 路径相对求解 输入或图形内容 目录)
图1	论 文/figures/overall- 总体建模流程；原生 TikZ 源码 architecture.tex
图2	问题一/重绘图表.py 质量评价、配比预测与核岭支持域推荐流程
图3	问题一/重绘图表.py 全量领域质量评分、抽样与扩展域级对照
图4	问题一/重绘图表.py 冲突指标对与熵权结果
图5	问题一/重绘图表.py 附件 A4 的 512 组训练配方
图6	问题一/重绘图表.py 线性拟合汇总、附件 A4–A11
图7	问题一/重绘图表.py 非线性代理对照逐域表及预测明细
图8	问题一/重绘图表.py 17×13 中心化混合系数矩阵
图9	问题一/重绘图表.py 线性系数、截距及附件 A6–A7
图10	问题一/重绘图表.py 损失代理评分、文本评分与附件 A16 映射
图11	问题一/重绘图表.py 附件 A4–A5、A8–A9、A12–A15
图12	问题二/重绘图表.py 问题二保存参数、结果与附件 B
图13	问题二/重绘图表.py 问题二保存参数、结果与附件 B

表 42 正式图表与生成程序的对应关系（续）

正文图号	生成程序（Python 路径相对求解 输入或图形内容 目录）	
图14	问题二/重绘图表.py	问题二保存参数、结果与附件 B
图15	问题二/重绘图表.py	问题二保存参数、结果与附件 B
图16	问题二/重绘图表.py	问题二保存参数、结果与附件 B
图17	问题二/重绘图表.py	问题二保存参数、结果与附件 B
图18	问题二/重绘图表.py	问题二保存参数、结果与附件 B
图19	问题二/重绘图表.py	问题二保存参数、结果与附件 B
图20	绘制问题三四流程图.py	对应小问的方法与输入输出关系
图21	问题三/论文绘图.py	问题三保存的预算、成本与上下文扫描 CSV
图22	问题三/论文绘图.py	问题三保存的预算、成本与上下文扫描 CSV
图23	问题三/论文绘图.py	问题三保存的预算、成本与上下文扫描 CSV
图24	绘制问题三四流程图.py	对应小问的方法与输入输出关系
图25	问题四/重绘图表.py	损失-得分映射结果与附件 C6
图26	问题四/重绘图表.py	合并横截面回归结果与附件 C1
图27	问题四/重绘图表.py	近期与长周期的规模关联项和剩余项
图28	问题四/重绘图表.py	前沿序列、logistic 参数及两类情景预测
图29	问题四/重绘图表.py	经许可筛选的附件 C1 六维得分
图30	问题四/重绘图表.py	附件 C8 的 37 项子任务聚合结果
图31	问题四/重绘图表.py	附件 C4 宏观趋势与字段覆盖结果

问题一：质量评价、冲突统计与配比建模

程序 1: 问题一完整求解程序（求解/问题一/问题一.py）

```

1 # -*- coding: utf-8 -*-
2 # 本程序及代码的整理与核对使用人工智能工具 Codex 辅助。
3 # 模型：GPT-5.6-Luna；开发机构：OpenAI。
4 # 版本发布日期：2026-07-09（型号发布）。
5 """
6 问题一：数据质量评价、质量冲突消解与领域配比建模
7 数据：附件 A（A1-A3 质量信号；A4-A15 配方/Loss；A16 域映射）
8 方法：1) 22 指标方向统一 + 1%-99% 分位数截断归一化 + 熵权法三级综合评分
          （样本/语料/领域）
9         2) 成对 z 分数差冲突定义 + Kendall 协同系数一致性检验 + 域内中位数
          裁决
10        3) 岭正则线性混合回归刻画 17 域配比 -> 13 域损失，并对照冗余质量特
          征的拟合变化

```

```

11 输出: 图片到 图片/, 结果 CSV 到 结果/
12 """
13 import sys, os, json, lzma, glob
14 sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(
    __file__))))
15 from _common import *    # noqa: F401,F403
16
17 DATA = data_path
18 RC_FAILS = []
19
20 def save_csv_safe(df, name_cn):
21     """保存 CSV; 若目标被其他进程占用则回退为 *_new.csv, 保证流程不中
    断。"""
22     p = os.path.join(OUT_DIR, name_cn)
23     try:
24         df.to_csv(p, index=False, encoding='utf-8-sig')
25     except PermissionError:
26         alt = os.path.join(OUT_DIR, name_cn.replace('.csv', '_new.csv'))
27         df.to_csv(alt, index=False, encoding='utf-8-sig')
28         RC_FAILS.append((name_cn, os.path.basename(alt)))
29         print(' ! %s 被占用, 已改存为 %s' % (name_cn, os.path.basename(
    alt)))
30
31 # ===== 0. 22 个质量指标的方向定义 =====
32 # positive = 越大越好; negative = 越小越好; target = 越接近稳健中心越好
33 QUALITY_COLS = [
34     'fineweb_edu', 'fluency_en', 'modernbert_cleanliness',
35     'modernbert_readability', 'modernbert_reasoning',
36     'modernbert_professionalism', 'dsir_books', 'dsir_wiki', 'dsir_math'
37     ,
38     'qurater', 'ad_en', 'rps_doc_word_count', 'rps_doc_num_sentences',
39     'rps_doc_unigram_entropy', 'rps_doc_frac_unique_words',
40     'rps_doc_frac_no_alph_words', 'rps_doc_frac_chars_top_2gram',
41     'rps_doc_frac_chars_top_3gram', 'rps_lines_uppercase_letter_fraction'
42     ,
43     'rps_lines_ending_with_terminal_punctution_mark',
44     'rps_lines_numerical_chars_fraction', 'rps_doc_mean_word_length'
45 ]
46 NEGATIVE_COLS = {
47     'rps_doc_frac_no_alph_words', 'rps_doc_frac_chars_top_2gram',
48     'rps_doc_frac_chars_top_3gram', 'rps_lines_uppercase_letter_fraction'
49     ,
50     'rps_lines_numerical_chars_fraction'
51 }
52 TARGET_COLS = {'rps_doc_word_count', 'rps_doc_num_sentences', '
    rps_doc_mean_word_length'}
53 LIST_COLS = ['fineweb_edu', 'fluency_en', 'modernbert_cleanliness',
54     'modernbert_readability', 'modernbert_reasoning',
55     'modernbert_professionalism', 'qurater', 'ad_en']
56 DIRECTION_NOTE = {
57     'fineweb_edu': '教育价值 (列表型, 1 维 logit) ',
58     'fluency_en': '英语流畅度 (列表型, 2 维, 取第 2 类概率) ',
59     'modernbert_cleanliness': '文本洁净度 (列表型, 6 级, 取归一化期望级
    别) ',
60     'modernbert_readability': '可读性 (列表型, 6 级) ',
61     'modernbert_reasoning': '推理性 (列表型, 6 级) ',

```

```

59     'modernbert_professionalism': '专业性（列表型，6 级）',
60     'dsir_books': 'DSIR 书籍域重要性权重',
61     'dsir_wiki': 'DSIR 维基域重要性权重',
62     'dsir_math': 'DSIR 数学域重要性权重',
63     'qurater': 'QuRating 四维质量（各维稳健归一化后等权聚合）',
64     'ad_en': '广告含量（列表型，2 维；第 1 类=“无广告”，正向。依据：原文抽查显示第 1 类概率最低的样本均为广告文案）',
65     'rps_doc_word_count': '文档词数（过短/过长均不佳）',
66     'rps_doc_num_sentences': '文档句数（过少/过多均不佳）',
67     'rps_doc_unigram_entropy': '一元词熵（越高越多样）',
68     'rps_doc_frac_unique_words': '唯一词占比（越高越多样）',
69     'rps_doc_frac_no_alph_words': '不含字母的词项占比（越低越好）',
70     'rps_doc_frac_chars_top_2gram': '最高频词2-gram覆盖的字符占比（越低越好）',
71     'rps_doc_frac_chars_top_3gram': '最高频词3-gram覆盖的字符占比（越低越好）',
72     'rps_lines_uppercase_letter_fraction': '大写字符占全部字符的比例（越低越好）',
73     'rps_lines_ending_with_terminal_punctution_mark': '以终止标点结尾的行占比（越高越好）',
74     'rps_lines_numerical_chars_fraction': '数字字符占全部字符的比例（越低越好）',
75     'rps_doc_mean_word_length': '平均词长（过短/过长均不佳）',
76 }
77 CONFLICT_TAU = 1.5      # 成对 z 分数差阈值
78
79 def _softmax_expected(v):
80     """把有序类别 logits 压缩为 [0,1] 上的归一化期望级别。"""
81     a = np.asarray(v, dtype=float).ravel()
82     if len(a) == 0:
83         return np.nan
84     if len(a) == 1:
85         return float(a[0])
86     a = a - np.nanmax(a)
87     p = np.exp(a); p = p / np.nansum(p)
88     return float(p @ np.linspace(0.0, 1.0, len(a)))
89
90 def aggregate_qurater(values):
91     """四维含义依次为写作风格、所需专业知识、事实知识、教育价值。
92
93     本文选取四维同向等权；各维缺失以中位数填补，按1%--99%分位缩放。
94     聚合后作为22指标之一，仍进入统一的外层方向与分位归一化。
95     """
96     a = np.asarray(values, dtype=float)
97     if a.ndim != 2 or a.shape[1] != 4:
98         raise ValueError('qurater 应为四个维度，而非类别 logits')
99     a = np.where(np.isfinite(a), a, np.nan)
100     med = np.nanmedian(a, axis=0)
101     if not np.isfinite(med).all():
102         raise ValueError('qurater 某维无有效记录')
103     a = np.where(np.isnan(a), med, a)
104     lo, hi = np.quantile(a, [.01, .99], axis=0)
105     return np.clip((a-lo)/np.maximum(hi-lo, 1e-12), 0, 1).mean(axis=1)
106
107 def _scalarize(name, value):

```

```

108     if name == 'qurater':
109         return np.nan # 读取全量四维后单独归一化聚合
110     if value is None:
111         return np.nan
112     if isinstance(value, (list, tuple)):
113         return _softmax_expected(value)
114     try:
115         return float(value)
116     except Exception:
117         return np.nan
118
119 def _norm_domain(x):
120     s = str(x).lower().replace('-', '_').replace(' ', '_')
121     aliases = {'common_crawl': 'commoncrawl', 'books': 'book', '
wikipedia_en': 'wikipedia'}
122     return aliases.get(s, s)
123
124 def load_quality_records():
125     """流式读取 A1-A3 全量记录，只保留建模字段和 A1 原文摘要。"""
126     sample_path = DATA('slimpajama_quality_signal_sample.jsonl')
127     ext_paths = sorted(glob.glob(os.path.join(DATA_DIR, 'A_data_value',
128
slimpajama_quality_extended', '*.jsonl.xz')))
129     rows, meta, qurater_values = [], [], []
130     miss = {c: 0 for c in QUALITY_COLS}
131
132     def _is_missing(v):
133         """字段级缺失判定：None、空列表、列表内含 None/NaN、标量为非有限
数均计为缺失。"""
134         if v is None:
135             return True
136         if isinstance(v, (list, tuple)):
137             if len(v) == 0:
138                 return True
139             return any((x is None) or (isinstance(x, float) and not np.
isfinite(x)) for x in v)
140         try:
141             return not np.isfinite(float(v))
142         except Exception:
143             return True
144
145     def consume(path, source, domain_hint=None):
146         opener = lzma.open if path.endswith('.xz') else open
147         with opener(path, 'rt', encoding='utf-8') as fh:
148             for line in fh:
149                 try:
150                     obj = json.loads(line)
151                 except Exception:
152                     continue
153                 domain = _norm_domain(obj.get('_source_domain',
domain_hint))
154                 for c in QUALITY_COLS:
155                     if _is_missing(obj.get(c)):
156                         miss[c] += 1
157                 rows.append([_scalarize(c, obj.get(c)) for c in
QUALITY_COLS])
158                 q = obj.get('qurater')

```

```

159         if q is not None and (not isinstance(q, (list, tuple))
or len(q) != 4):
160             raise ValueError('qurater 维数错误')
161             qurater_values.append(q if q is not None else [np.nan]*
4)
162             excerpt = ''
163             if source == 'A1':
164                 excerpt = ' '.join(str(obj.get('content', '')).split
())[:180]
165                 meta.append((obj.get('id', ''), domain, source, excerpt)
)
166
167     consume(sample_path, 'A1')
168     for path in ext_paths:
169         hint = 'arxiv' if os.path.basename(path).lower().startswith('
arxiv') else 'github'
170         consume(path, 'A2' if hint == 'arxiv' else 'A3', hint)
171     X = pd.DataFrame(rows, columns=QUALITY_COLS)
172     X['qurater'] = aggregate_qurater(qurater_values)
173     M = pd.DataFrame(meta, columns=['id', 'domain', 'source', 'excerpt'
])
174     miss_df = pd.DataFrame([{'indicator': k, 'missing_count': v} for k,
v in miss.items()])
175     save_csv_safe(miss_df, '质量信号缺失审计.csv')
176     if miss_df.missing_count.sum() > 0:
177         print('\n质量信号缺失审计 (合计 %d 个指标取值缺失): ' % int(
miss_df.missing_count.sum()))
178         print(miss_df[miss_df.missing_count > 0].to_string(index=False))
179         print('  处理方式: 方向统一前按该指标中位数填充 (不把缺失当作高
质量或低质量)。')
180         return X, M
181
182 def orient_and_normalize(raw):
183     """方向统一 -> 分位数截断的稳健极差归一化 (正向缩放、负向取补、长度
型转中心接近度) 。”"""
184     oriented = raw.copy()
185     for c in TARGET_COLS:
186         z = np.log1p(np.clip(raw[c].values, 0, None))
187         med = np.median(z); mad = np.median(np.abs(z - med)) + 1e-9
188         oriented[c] = np.exp(-np.abs(z - med) / (2.5 * mad))
189     norm = pd.DataFrame(index=raw.index)
190     for c in QUALITY_COLS:
191         lo, hi = np.nanquantile(oriented[c], [0.01, 0.99])
192         v = np.clip((oriented[c] - lo) / max(hi - lo, 1e-12), 0, 1)
193         norm[c] = 1 - v if c in NEGATIVE_COLS else v
194     return oriented, norm
195
196 def kendall_concordance(values):
197     """对象为行、评价指标为列; 平均秩并扣除各列的并列秩修正项。"""
198     values = np.asarray(values, dtype=float)
199     if values.ndim != 2 or not np.isfinite(values).all():
200         raise ValueError('Kendall W 需要有限值二维矩阵')
201     m, n = values.shape
202     if m < 2 or n < 2:
203         raise ValueError('Kendall W 至少需要两个对象和两个评价指标')
204     rank_sum = np.zeros(m)
205     tie_term = 0

```

```

206     for column in values.T:
207         rank_sum += stats.rankdata(column, method='average')
208         _, counts = np.unique(column, return_counts=True)
209         # 转成 Python 整数, 避免大样本分母乘法发生 int64 溢出。
210         tie_term += sum(int(t)**3 - int(t) for t in counts if t > 1)
211     denominator = n**2 * (m**3 - m) - n * tie_term
212     if denominator <= 0:
213         raise ValueError('所有指标均无排序差异, Kendall W 未定义')
214     return float(12 * np.sum((rank_sum - rank_sum.mean())**2) /
215                  denominator)
216
217 def quality_evaluation():
218     raw, meta = load_quality_records()
219     raw = raw.replace([np.inf, -np.inf], np.nan)
220     raw = raw.apply(lambda s: s.fillna(s.median()))
221     _, norm = orient_and_normalize(raw)
222
223     # ---- 熵权法 (主方案) ----
224     P = (norm.values + 1e-12) / (norm.values.sum(axis=0, keepdims=True)
225 + 1e-12)
226     entropy = -(P * np.log(P)).sum(axis=0) / np.log(len(norm))
227     divergence = np.maximum(1 - entropy, 0)
228     w_entropy = divergence / divergence.sum()
229     # ---- 对照方案: 等权 / 首主成分非负载荷加权 ----
230     w_equal = np.full(len(QUALITY_COLS), 1.0 / len(QUALITY_COLS))
231     Z = (norm.values - norm.values.mean(0)) / (norm.values.std(0) + 1e-
232 12)
233     _, _, vt = np.linalg.svd(Z, full_matrices=False)
234     pc1 = vt[0]
235     if pc1.sum() < 0:
236         pc1 = -pc1
237     w_pca = np.clip(pc1, 0, None); w_pca = w_pca / w_pca.sum()
238
239     # CSV 的历史字段名 “PCA第一主成分” 表示非负载荷加权, 并非标准主成分
240     得分。
241     names = ['熵权法', '等权平均', 'PCA第一主成分']
242     wdict = dict(zip(names, [w_entropy, w_equal, w_pca]))
243     sc = {k: norm.values @ wdict[k] for k in names}
244     dom_tmp = meta.assign(**sc).groupby('domain')[names].mean()
245     cmp_df = pd.DataFrame({
246         'scheme': names,
247         'score_mean': [sc[k].mean() for k in names],
248         'score_std': [sc[k].std() for k in names],
249         'corr_with_entropy': [float(np.corrcoef(sc[k], sc['熵权法'])[0,
250 1]) for k in names],
251         'domain_spearman_vs_entropy': [1.0,
252             stats.spearmanr(dom_tmp['熵权法'], dom_tmp['等权平均']).
253             statistic,
254             stats.spearmanr(dom_tmp['熵权法'], dom_tmp['PCA第一主成分'])
255             .statistic]})
256     save_csv_safe(cmp_df, '赋权方案对比.csv')
257
258     # 仅作判向敏感性对照: 将广告方向反转后重新计算熵权, 不替换主评分。
259     flipped = norm.copy()
260     flipped['ad_en'] = 1 - flipped['ad_en']

```

```

254     pf = (flipped.values + 1e-12) / (flipped.values.sum(0, keepdims=True)
255     + 1e-12)
256     ef = -(pf * np.log(pf)).sum(0) / np.log(len(flipped))
257     wf = np.maximum(1-ef, 0); wf /= wf.sum()
258     direction_check = meta[['domain']].assign(
259     main_score=sc['熵权法'], reversed_ad_score=flipped.values @ wf
260     ).groupby('domain').mean().reset_index()
261     save_csv_safe(direction_check, '广告方向敏感性.csv')
262
263     wdf = pd.DataFrame({'indicator': QUALITY_COLS,
264     'list_type': [c in LIST_COLS for c in
265     QUALITY_COLS],
266     'direction': ['negative' if c in NEGATIVE_COLS
267     else
268     ('target' if c in TARGET_COLS else
269     'positive')]
270     for c in QUALITY_COLS],
271     'semantics': [DIRECTION_NOTE[c] for c in
272     QUALITY_COLS],
273     'entropy': entropy, 'weight': w_entropy})
274     save_csv_safe(wdf, '质量指标熵权.csv')
275     meta['quality_Q'] = sc['熵权法']
276
277     # ---- 冲突分析: 成对 z 分数差 > tau ----
278     def agreement(idx, label):
279         Zs = norm.loc[idx]
280         m, n = Zs.shape                # m 条文本 (对象), n=22 指
281         标 (评价者)
282         W = kendall_concordance(Zs.values)
283         Am = (Zs.values - Zs.values.mean(axis=0)) / (Zs.values.std(axis
284         =0) + 1e-12)
285         iu = np.triu_indices(n, 1)
286         pairdiff = np.abs(Am[:, iu[0]] - Am[:, iu[1]])
287         rate = float((pairdiff > CONFLICT_TAU).mean())
288         rate_free = float(2 * stats.norm.sf(CONFLICT_TAU / np.sqrt(2)))
289         chi2 = n * (m - 1) * W
290         pair_rate = pd.Series((pairdiff > CONFLICT_TAU).mean(axis=0),
291         index=['%s|%s' % (QUALITY_COLS[i],
292         QUALITY_COLS[j])
293         for i, j in zip(*iu)])
294         return {'dataset': label, 'n': m, 'kendall_W': W, 'chi2': chi2,
295         'p_value': stats.chi2.sf(chi2, m - 1),
296         'conflict_rate': rate, 'indep_baseline': rate_free,
297         'conflict_ratio_vs_indep': rate / rate_free}, pair_rate
298
299     ag, pair_table = [], {}
300     for idx, label in [(meta.index[meta.source == 'A1'], 'A1抽样集'),
301     (meta.index[meta.source != 'A1'], 'A2+A3扩展集'),
302     (meta.index, '全量')]:
303         a, pf = agreement(idx, label)
304         ag.append(a); pair_table[label] = pf
305     save_csv_safe(pd.DataFrame(ag), '质量冲突与一致性.csv')
306     top_pairs = pair_table['全量'].sort_values(ascending=False).head(20)
307     .rename('conflict_rate')
308     save_csv_safe(top_pairs.reset_index().rename(columns={'index': '
309     indicator_pair'})),

```

```

300         '冲突指标对_top20.csv')
301
302     # ---- 冲突阈值灵敏度: tau 变动对冲突率与结论的影响 ----
303     thr_rows = []
304     Zall = (norm.values - norm.values.mean(axis=0)) / (norm.values.std(
axis=0) + 1e-12)
305     iu_all = np.triu_indices(22, 1)
306     pd_all = np.abs(Zall[:, iu_all[0]] - Zall[:, iu_all[1]])
307     # 样本至少一对指标超过阈值即作标记; 冲突率仍指“样本×指标对”的比
例。
308     # 裁决仅提供域内相对标签, 不删样本、不降权, 也不改变质量评分。
309     flagged = (pd_all > CONFLICT_TAU).any(axis=1)
310     median = meta.groupby('domain')['quality_Q'].transform('median')
311     decision = np.where(meta.quality_Q > median, '高于域中位数',
312                        np.where(meta.quality_Q < median, '低于域中位数'
, '等于域中位数'))
313     diagnostic = meta[['domain']].assign(conflict_flag=flagged, decision
=decision)
314     summary = diagnostic.groupby(['domain', 'conflict_flag', 'decision'
]).size().reset_index(name='n')
315     save_csv_safe(summary, '冲突样本裁决汇总.csv')
316     for tau in [0.5, 1.0, 1.5, 2.0, 2.5]:
317         rc = float((pd_all > tau).mean())
318         r0 = float(2 * stats.norm.sf(tau / np.sqrt(2)))
319         thr_rows.append({'阈值tau': tau, '冲突率': rc, '独立基准': r0,
320                        '比值': rc / r0, '结论': '指标正相关 (冲突率低
于独立基准)'
321                        if rc < r0 else '指标间无正相关'})
322     save_csv_safe(pd.DataFrame(thr_rows), '冲突阈值灵敏度.csv')
323
324     # ---- 域级质量: 抽样集与扩展集对照 ----
325     domain = meta.groupby(['source', 'domain']).agg(
326         n=('quality_Q', 'size'), quality_mean=('quality_Q', 'mean'),
327         quality_std=('quality_Q', 'std')).reset_index()
328     save_csv_safe(domain, '质量域评分_抽样与扩展.csv')
329     comp = domain.pivot(index='domain', columns='source', values='
quality_mean')
330     a1 = comp['A1']
331     ext = comp.drop(columns=[c for c in comp.columns if c == 'A1']).mean
(axis=1)
332     comp_out = pd.DataFrame({'domain': comp.index, 'Q_A1': a1.values, '
Q_扩展集': ext.values})
333     comp_out['差值_A1减扩展'] = comp_out['Q_A1'] - comp_out['Q_扩展集']
334     save_csv_safe(comp_out, '抽样集与扩展集域级对照.csv')
335     save_csv_safe(meta[['id', 'domain', 'source', 'quality_Q']], '样本级
质量评分.csv')
336
337     # ---- 高低分原文抽查 (验证评分方向) ----
338     checks = []
339     s1 = meta[meta.source == 'A1']
340     for d, g in s1.groupby('domain'):
341         checks.append(g.nsmallest(2, 'quality_Q').assign(group='低分'))
342         checks.append(g.nlargest(2, 'quality_Q').assign(group='高分'))
343     save_csv_safe(pd.concat(checks)[['domain', 'group', 'quality_Q', '
excerpt']], '原文抽查.csv')

```

```

344
345 # ---- A16 跨体系映射 ----
346 pooled = meta.groupby('domain')['quality_Q'].mean().to_dict()
347 mapping = pd.read_csv(DATA('domain_mapping_guide.csv'))
348 global_q = float(meta.quality_Q.mean())
349 mapping['quality_Q'] = [pooled.get(str(q), global_q) if str(q) != '(
none)' else global_q
350                                     for q in mapping['quality_domain']]
351 mapping['quality_source'] = np.where(mapping.quality_domain.eq('(
none)'),
352                                     '全局均值推断', 'A1-A3域映射')
353 save_csv_safe(mapping, '17域质量映射.csv')
354 mapping.to_csv(shared_path('17域质量映射.csv'), index=False,
encoding='utf-8-sig')
355 print('质量全量记录 %s 条; A1=%s, A2+A3=%s' % (f'{len(meta):,}',
356         f'{int((meta.source == "A1").sum()):,}', f'{int((meta.source
!= "A1").sum()):,}'))
357 print(pd.DataFrame(ag).to_string(index=False))
358 print('\n赋权方案对比: ')
359 print(cmp_df.to_string(index=False))
360 print('\nad_en 熵权 = %.4f (方向敏感性另列对照) '
361         % w_entropy[QUALITY_COLS.index('ad_en')])
362 print('冲突最集中的 5 组指标对: ')
363 print(top_pairs.head(5).to_string())
364 return meta, domain, mapping, norm, w_entropy, pair_table
365
366 # ===== 1. A1-A3 全量质量评价与冲突检验 =====
367 (quality_meta, quality_compare, quality_mapping,
368  quality_norm, quality_weights, quality_pairs) = quality_evaluation()
369 quality_domain = quality_meta.groupby('domain').agg(
370     n=('quality_Q', 'size'), quality_mean=('quality_Q', 'mean'),
371     quality_std=('quality_Q', 'std')).reset_index().sort_values('
quality_mean')
372 save_csv_safe(quality_domain, '质量域评分_全量.csv')
373
374 # ===== 2. 配比与 Loss 数据加载 =====
375 def load_pair(mix_path, loss_path):
376     m = pd.read_csv(mix_path).sort_values('index').reset_index(drop=True
)
377     l = pd.read_csv(loss_path).sort_values('index').reset_index(drop=
True)
378     mix_cols = [c for c in m.columns if c.startswith('train_the_pile_')]
379     loss_cols = [c for c in l.columns if 'val_loss' in c]
380     return m[mix_cols].values.astype(float), l[loss_cols].values.astype(
float), m['index'].values
381
382 def short(c):
383     return c.replace('train_the_pile_', '').replace('metric/the_pile_',
'').replace('_val_loss', '')
384
385 Xtr, Ytr, itr = load_pair(DATA('train_mixture_1m.csv'), DATA('
train_pile_loss_1m.csv'))
386 Xte1, Yte1, _ = load_pair(DATA('test_mixture_1m.csv'), DATA('
test_pile_loss_1m.csv'))
387 Xte2, Yte2, _ = load_pair(DATA('test_mixture_60m.csv'), DATA('
test_pile_loss_60m.csv'))

```

```

388 Xte3, Yte3, _ = load_pair(DATA('test_mixture_1B.csv'), DATA('
    test_pile_loss_1B.csv'))
389 Xes10, Yes10, es10_idx = load_pair(DATA('est_mixture_10b.csv'), DATA('
    est_pile_loss_10b.csv'))
390 Xes70, Yes70, es70_idx = load_pair(DATA('est_mixture_70b.csv'), DATA('
    est_pile_loss_70b.csv'))
391
392 tm_df = pd.read_csv(DATA('train_mixture_1m.csv'))
393 MIX_COLS = [c for c in tm_df.columns if c.startswith('train_the_pile_')]
394 tl_df = pd.read_csv(DATA('train_pile_loss_1m.csv'))
395 LOSS_COLS = [c for c in tl_df.columns if 'val_loss' in c]
396 DOMAINS = [short(c) for c in MIX_COLS]
397 LOSS_DOMAINS = [short(c) for c in LOSS_COLS]
398 UNMEASURED = ['nih_exporter', 'enron_emails', 'europarl', 'philpapers']
    # 无损失列
399 MEASURED = [d for d in DOMAINS if d not in UNMEASURED]
400
401 # ===== 3. 岭正则线性混合回归 =====
402 def fit_ridge(X, Y, alpha=1e-3):
403     n, K = X.shape
404     Xc = np.hstack([X, np.ones((n, 1))])
405     reg = alpha * np.eye(K + 1); reg[-1, -1] = 0
406     W = np.linalg.solve(Xc.T @ Xc + reg, Xc.T @ Y)
407     return W[:K], W[-1]
408
409 def predict(X, A, b):
410     return X @ A + b
411
412 def r2(y, yhat):
413     ss_res = np.sum((y - yhat) ** 2, axis=0)
414     ss_tot = np.sum((y - y.mean(axis=0)) ** 2, axis=0)
415     return 1 - ss_res / np.maximum(ss_tot, 1e-12)
416
417 def pearson_col(y, yhat):
418     return np.array([np.corrcoef(y[:, j], yhat[:, j])[0, 1] for j in
        range(y.shape[1])])
419
420 A, b = fit_ridge(Xtr, Ytr)
421 r2_tr = r2(Ytr, predict(Xtr, A, b))
422 r2_te1 = r2(Yte1, predict(Xte1, A, b))
423 r_te2 = pearson_col(Yte2, predict(Xte2, A, b))
424 r_te3 = pearson_col(Yte3, predict(Xte3, A, b))
425
426 print('\n=== 岭回归训练 R2 (各损失域) ===')
427 for d, v in zip(LOSS_DOMAINS, r2_tr):
428     print(' %-20s %.4f' % (d, v))
429 print('训练平均 R2 = %.4f; 同尺度 1M 检验平均 R2 = %.4f' % (r2_tr.mean(),
    r2_te1.mean()))
430 print('跨尺度 Pearson: 60M 平均 %.4f, 1B 平均 %.4f' % (r_te2.mean(),
    r_te3.mean()))
431 A_df = pd.DataFrame(A, columns=LOSS_DOMAINS)
432 A_df.insert(0, 'training_domain', DOMAINS)
433 save_csv_safe(A_df, '混合系数矩阵.csv')
434 save_csv_safe(pd.DataFrame({'loss_domain': LOSS_DOMAINS, 'intercept': b
    }), '截距.csv')
435
436 lam_rows = []

```

```

437 for lam in [1e-4, 1e-3, 1e-2, 1e-1]:
438     Al, bl = fit_ridge(Xtr, Ytr, lam)
439     lam_rows.append({'lambda': lam, 'train_mean_R2': r2(Ytr, predict(Xtr
, Al, bl)).mean(),
440                     'testlm_mean_R2': r2(Ytel, predict(Xtel, Al, bl)).
mean(),
441                     'coef_L2': float(np.linalg.norm(Al))})
442 save_csv_safe(pd.DataFrame(lam_rows), '岭正则敏感性.csv')
443
444 # ===== 4. 质量信息引入配比模型的增量检验 =====
445 qmap = quality_mapping.set_index('mixture_domain')['quality_Q']
446 qvec = np.array([qmap.get(d, np.nan) for d in DOMAINS], dtype=float)
447 qvec = np.where(np.isnan(qvec), float(quality_meta.quality_Q.mean()),
qvec)
448 Zq = np.hstack([Xtr, (Xtr @ qvec)[:, None]])
449 Az, bz = fit_ridge(Zq, Ytr, 1e-3)
450 r2_z = r2(Ytr, predict(Zq, Az, bz))
451 save_csv_safe(pd.DataFrame({'loss_domain': LOSS_DOMAINS, 'train_R2_仅配
比': r2_tr,
452                             'train_R2_追加Qbar': r2_z, 'R2增量': r2_z -
r2_tr,
453                             'Qbar系数': Az[-1]}), '质量项增量检验.csv')
454
455 diff_mean = Ytr.mean(axis=0)
456 quality_proxy = 1 - (diff_mean - diff_mean.min()) / (diff_mean.max() -
diff_mean.min())
457 q_df = pd.DataFrame({'domain': LOSS_DOMAINS, 'mean_loss(difficulty)':
diff_mean,
458                     'quality_score_Q(loss代理)': quality_proxy}).
sort_values(
459     'quality_score_Q(loss代理)', ascending=False)
460 save_csv_safe(q_df, 'Loss代理领域难度.csv')
461 save_csv_safe(q_df, '领域质量评分.csv')
462
463 link = pd.read_csv(DATA('domain_mapping_guide.csv'))
464 link = link[link.mapping_type.isin(['direct', 'near_direct'])].copy()
465 qd_map = quality_domain.set_index('domain').quality_mean
466 qp_map = q_df.set_index('domain')['quality_score_Q(loss代理)']
467 link['Q_A1A3'] = [float(qd_map.get(q, np.nan)) for q in link.
quality_domain]
468 link['Q_loss代理'] = [float(qp_map.get(m, np.nan)) for m in link.
mixture_domain]
469 rho_link = float(stats.spearmanr(link.Q_A1A3, link.Q_loss代理).statistic
)
470 save_csv_safe(link[['mixture_domain', 'quality_domain', 'mapping_type',
'Q_A1A3', 'Q_loss代理']],
471                 '质量与损失难度一致性.csv')
472
473 A_cent = A - A.mean(axis=0, keepdims=True)
474 A_cent_df = pd.DataFrame(A_cent, columns=LOSS_DOMAINS)
475 A_cent_df.insert(0, 'training_domain', DOMAINS)
476 save_csv_safe(A_cent_df, '混合系数矩阵_中心化.csv')
477 save_csv_safe(pd.DataFrame({'training_domain': DOMAINS,
478                             'marginal_effect_on_avg_loss': A_cent.mean(
axis=1)}),
479                 '训练域边际效应.csv')
480

```

```

481 # ===== 5. 外推与跨尺度稳健性 =====
482 tr_lookup = {i: Ytr[j] for j, i in enumerate(itr)}
483 ratio10 = Yes10 / np.array([tr_lookup[i] for i in es10_idx])
484 ratio70 = Yes70 / np.array([tr_lookup[i] for i in es70_idx])
485 print('\n=== 外推尺度因子 (est/train 损失比) ===')
486 print(' 10B 均值 %.4f (std %.4f)' % (ratio10.mean(), ratio10.std()))
487 print(' 70B 均值 %.4f (std %.4f)' % (ratio70.mean(), ratio70.std()))
488 save_csv_safe(pd.DataFrame({'loss_domain': LOSS_DOMAINS, '
    scale_factor_10B': ratio10.mean(axis=0),
489     'scale_factor_70B': ratio70.mean(axis=0)}),
    '外推尺度因子.csv')
490 ratio_scale = Yte2 / Yte1
491 save_csv_safe(pd.DataFrame({'loss_domain': LOSS_DOMAINS,
492     'loss_ratio_60m_vs_1m': ratio_scale.mean(
    axis=0),
493     'loss_ratio_std': ratio_scale.std(axis=0)}),
494     '跨尺度损失比_60m_vs_1m.csv')
495
496 # ===== 6. 最优配比 (无约束 + 2.5 倍上界推荐) =====
497 from scipy.optimize import minimize
498 ref = Xtr.mean(axis=0)
499 w_ref = ref[[DOMAINS.index(d) for d in LOSS_DOMAINS]]
500 w_ref = w_ref / w_ref.sum()
501 unmeas_sum = ref[[DOMAINS.index(u) for u in UNMEASURED]].sum()
502 meas_idx = [DOMAINS.index(d) for d in MEASURED]
503
504 def build(pm):
505     p = np.zeros(17); p[meas_idx] = pm
506     for u in UNMEASURED:
507         p[DOMAINS.index(u)] = ref[DOMAINS.index(u)]
508     return p / p.sum()
509
510 def obj(pm):
511     return float(w_ref @ predict(build(pm)[None, :], A, b)[0])
512
513 p0m = ref[meas_idx] / ref[meas_idx].sum()
514 cons = {'type': 'eq', 'fun': lambda pm: pm.sum() - (1 - unmeas_sum)}
515 res = minimize(obj, p0m, method='SLSQP', bounds=[(0, 1)] * len(MEASURED),
    , constraints=cons)
516 p_opt = build(res.x)
517 save_csv_safe(pd.DataFrame({'domain': DOMAINS, 'reference_mixture': ref,
518     'optimal_mixture': p_opt}).sort_values('
    optimal_mixture', ascending=False),
519     '最优配比预测.csv')
520 bnds = [(0, 2.5 * ref[i]) for i in meas_idx]
521 res_b = minimize(obj, p0m, method='SLSQP', bounds=bnds, constraints=cons
    )
522 p_b = build(res_b.x)
523 rec_df = pd.DataFrame({'domain': DOMAINS, 'reference_mixture': ref, '
    recommended_mixture': p_b,
524     'adjustment': p_b - ref}).sort_values('adjustment
    ', ascending=False)
525 save_csv_safe(rec_df, '线性有界配比对照.csv')
526 print('\n线性对照: 无约束最优加权平均损失 %.4f; 有界方案 %.4f; 参考配方
    %.4f'
    % (obj(res.x), obj(res_b.x), float(w_ref @ predict(ref[None, :], A
    , b)[0])))

```

```

528 print('线性有界对照配比（调整量最大增/减各 5 域；正式推荐见后续核岭输
      出）：')
529 print(rec_df.head(5).to_string(index=False))
530 print(rec_df.tail(5).to_string(index=False))
531
532 # 正式图表由文件末尾的独立绘图入口生成。
533
534 # ===== 8. 汇总 =====
535 save_csv_safe(pd.DataFrame({'loss_domain': LOSS_DOMAINS, 'train_R2':
      r2_tr, 'test1m_R2': r2_te1,
536                             'test60m_r': r_te2, 'test1B_r': r_te3, '
      mean_loss': diff_mean,
537                             'quality_Q(loss代理)': quality_proxy}), '问
      题一_拟合汇总.csv')
538
539 conf = pd.read_csv(os.path.join(OUT_DIR, '质量冲突与一致性.csv'))
540 meta_out = {
541     'train_mean_R2': float(r2_tr.mean()), 'test1m_mean_R2': float(r2_te1
      .mean()),
542     'test60m_mean_r': float(r_te2.mean()), 'test1B_mean_r': float(r_te3.
      mean()),
543     'n_quality_records': int(len(quality_meta)),
544     'n_A1': int((quality_meta.source == 'A1').sum()),
545     'n_ext': int((quality_meta.source != 'A1').sum()),
546     'Q0_global_mean': float(quality_meta.quality_Q.mean()),
547     'Q0_mix_weighted': float(ref @ qvec),
548     'kendall_W_full': float(conf.loc[conf.dataset.eq('全量'), 'kendall_W
      '].iloc[0]),
549     'conflict_rate_full': float(conf.loc[conf.dataset.eq('全量'), '
      conflict_rate'].iloc[0]),
550     'conflict_indep_baseline': float(conf.loc[conf.dataset.eq('全量'), '
      indep_baseline'].iloc[0]),
551     'spearman_quality_vs_lossproxy': float(rho_link),
552     'Qbar_R2_gain_mean': float((r2_z - r2_tr).mean()),
553 }
554 with open(shared_path('问题一_关键量.json'), 'w', encoding='utf-8') as
      fh:
555     json.dump(meta_out, fh, ensure_ascii=False, indent=2)
556
557 print('\n===== 问题一求解完成 =====')
558 print('训练平均 R2=%.4f; 同尺度检验 R2=%.4f; 跨尺度相关 60M=%.4f, 1B=%.4
      f'
559       % (r2_tr.mean(), r2_te1.mean(), r_te2.mean(), r_te3.mean()))
560 print('追加混合质量指数 Qbar 的平均 R2 增量 = %.5f' % (r2_z - r2_tr).
      mean())
561 print('全局平均质量 = %.4f; 参考配比加权基线 Q0 = %.4f'
      % (quality_meta.quality_Q.mean(), float(ref @ qvec)))
562
563 if RC_FAILS:
564     print('注意：以下结果文件因被占用而改存：', RC_FAILS)
565
566 # 模型结果全部保存后生成正文采用的正式图。
567 import subprocess as _subprocess
568 _subprocess.run([sys.executable, os.path.join(BASE_DIR, '非线性代理对照.
      py'), '--data-dir', DATA_DIR], check=True)
569 _subprocess.run([sys.executable, os.path.join(BASE_DIR, '核岭支持域推荐.
      py'), '--data-dir', DATA_DIR], check=True)

```

```
570 subprocess.run([sys.executable, os.path.join(BASE_DIR, '重绘图表.py'),
    '--data-dir', DATA_DIR], check=True)
```

问题二：广义标度律、验证与边际分析

程序 2: 问题二完整求解程序（求解/问题二/问题二.py）

```
1 # -*- coding: utf-8 -*-
2 # 本程序及代码的整理与核对使用人工智能工具 Codex 辅助。
3 # 模型: GPT-5.6-Luna; 开发机构: OpenAI。
4 # 版本发布日期: 2026-07-09（型号发布）。
5 """
6 问题二：跨维度数据融合与广义标度律
7 数据: 附件 B（B1 主拟合; B2/B3 族外与轨迹验证; B4/B5 跨族与文献验证;
8       B6/B7 质量 Q; B8 大规模矩阵（方向异常, 仅作可信度边界警示）; B9/
9       B10 百亿参数以上外推）
10 模型: 经典  $L = E + A N^{-a} + B D^{-b}$ 
11        广义  $L = E + A N^{-a} + B D^{-b} + C (1-Q)^g$  ( $Q=1$  退化为经典形式)
12 输出: 图片到 图片/, 结果 CSV 到 结果/, 参数共享至 求解/广义标度律参数.
13        CSV
14 """
15 import sys, os, glob
16 sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(
17     __file__))))
18 from _common import * # noqa: F401,F403
19 from scipy.optimize import least_squares
20
21 DATA = data_path
22
23 def save_csv_safe(df, name_cn):
24     p = os.path.join(OUT_DIR, name_cn)
25     try:
26         df.to_csv(p, index=False, encoding='utf-8-sig')
27     except PermissionError:
28         alt = p.replace('.csv', '_new.csv')
29         df.to_csv(alt, index=False, encoding='utf-8-sig')
30         print(' ! %s 被占用, 已改存为 %s' % (name_cn, os.path.basename(
31             alt)))
32
33 pythia = pd.read_csv(DATA('pythia_training_log_existing.csv'))
34 cerebras = pd.read_csv(DATA('cerebras_training_log.csv'))
35 baseline = pd.read_csv(DATA('scaling_baseline.csv'))
36 published = pd.read_csv(DATA('published_scaling_data.csv'))
37 nq6 = pd.read_csv(DATA('supplementary_NQ_experiment.csv'))
38 nq7 = pd.read_csv(DATA('supplementary_NQ_experiment_expanded.csv'))
39 nq8 = pd.read_csv(DATA('supplementary_NQ_experiment_large.csv'))
40 large_models = pd.read_csv(DATA('supplementary_large_models.csv'))
41 large_loss = pd.read_csv(DATA('supplementary_large_baseline.csv'))
42 traj_files = sorted(glob.glob(os.path.join(DATA_DIR, 'B_scaling_laws',
43     'training_trajectories', '*.
44     csv'))))
45
46 N_p = pythia.N_params_B.to_numpy(float)
47 D_p = pythia.D_tokens_B.to_numpy(float)
```

```

43 L_p = pythia.val_loss.to_numpy(float)
44
45 # ===== 1. 经典标度律 (B1 主拟合 + 模型留出验证) =====
46 def classic(par, N, D):
47     E, A, a, B, b = par
48     return E + A * N ** (-a) + B * D ** (-b)
49
50 r0 = least_squares(lambda p: classic(p, N_p, D_p) - L_p, [1.7, .4, .33,
    1.2, .28],
51                   bounds=([0, 1e-8, .01, 1e-8, .01], [5, 1e3, 2, 1e3,
    2])),
52                   loss='soft_l1', f_scale=.05, max_nfev=40000)
53 E0, A0, a0, B0, b0 = r0.x
54 Lph = classic(r0.x, N_p, D_p)
55 R2_cls = 1 - np.sum((L_p - Lph) ** 2) / np.sum((L_p - L_p.mean()) ** 2)
56 RMSE_cls = float(np.sqrt(np.mean((L_p - Lph) ** 2)))
57
58 Ns = sorted(pythia.N_params_B.unique())
59 tr_m = pythia[pythia.N_params_B.isin(Ns[:4])]
60 te_m = pythia[pythia.N_params_B.isin(Ns[4:])]
61 rh = least_squares(lambda p: classic(p, tr_m.N_params_B.values, tr_m.
    D_tokens_B.values)
62                   - tr_m.val_loss.values, [1.7, .4, .33, 1.2, .28],
63                   bounds=([0, 1e-8, .01, 1e-8, .01], [5, 1e3, 2, 1e3,
    2])),
64                   loss='soft_l1', f_scale=.05, max_nfev=40000)
65 hp = classic(rh.x, te_m.N_params_B.values, te_m.D_tokens_B.values)
66 ym = te_m.val_loss.values
67 hold = {'留出R2': 1 - np.sum((ym - hp) ** 2) / np.sum((ym - ym.mean()) *
    * 2),
68         '留出RMSE': float(np.sqrt(np.mean((ym - hp) ** 2))),
69         '留出MAPE_%': float(np.mean(np.abs(ym - hp) / ym) * 100)}
70
71 print('=== 1. 经典标度律 (B1 Pythia, n=%d) ===' % len(pythia))
72 print('  E=%.4f A=%.4f alpha=%.4f B=%.4f beta=%.4f' % (E0, A0, a0, B0,
    b0))
73 print('  拟合 R2=%.8f RMSE=%.6f MAPE=%.4f%%' % (
74     R2_cls, RMSE_cls, np.mean(np.abs(L_p - Lph) / L_p) * 100))
75 print('  留出验证 (4 小模型 -> 4 大模型): R2=%.8f RMSE=%.6f MAPE=%.4f%%'
    % (
76     hold['留出R2'], hold['留出RMSE'], hold['留出MAPE_%']))
77 print('  提示: B1 拟合残差标准差仅 %.5f (损失标准差 %.4f), 说明该附件近
    乎无观测噪声, '
78     '其上高 R2 只能确认函数形式自治, 不能替代外部验证。' % (
79     float(np.std(L_p - Lph)), float(np.std(L_p))))
80 save_csv_safe(pd.DataFrame([{'E': E0, 'A': A0, 'alpha': a0, 'B': B0, '
    beta': b0,
81                               'R2': R2_cls, 'RMSE': RMSE_cls,
82                               'MAPE_%': float(np.mean(np.abs(L_p - Lph) /
    L_p) * 100),
83                               '残差标准差': float(np.std(L_p - Lph)), **
    hold}]),
84           '经典标度律参数.csv')
85
86 # ===== 2. 质量字段方向审计 =====
87 def group_corr(df, qcol='Q_score'):
88     vals, ng = [], 0

```

```

89     for _, g in df.groupby(['N_params_B', 'D_tokens_B']):
90         if g[qcol].nunique() >= 3:
91             vals.append(g[qcol].corr(g.val_loss)); ng += 1
92     return float(np.nanmean(vals)), ng
93
94 audit = []
95 for nm, sub, note in [('B6', nq6, '质量主数据'),
96                       ('B7', nq7, '质量扩展数据 (含 B6 重复点)'),
97                       ('B8-calibrated', nq8[nq8.data_type.eq('calibrated
    ')], '大规模矩阵校准段'),
98                       ('B8-extrapolated', nq8[nq8.data_type.eq('
    extrapolated']), '大规模矩阵外推段')]:
99     r, ng = group_corr(sub)
100    audit.append({'dataset': nm, 'n': len(sub), 'n_groups': ng, '
    within_ND_corr': r,
101                 'conclusion': ('Q 越大损失越低 -> 质量指标 (越高越好)
    ' if r < -0.5 else
102                               ('Q 越大损失越高 -> 方向与 B6/B7 相反'
    if r > 0.5 else
103                               '组内无显著单调关系'))})
104 save_csv_safe(pd.DataFrame(audit), '质量字段方向审计.csv')
105 corr6 = audit[0]['within_ND_corr']; corr7 = audit[1]['within_ND_corr']
106 corr8 = audit[2]['within_ND_corr']
107 print('\n=== 2. 质量字段方向审计 (控制 N、D 的组内相关) ===')
108 print(pd.DataFrame(audit).to_string(index=False))
109 print('  结论: B6/B7 的 Q_score 与损失负相关 (r=%3f / %3f), 是“质
    量”而非“难度”, '
110        '故取 Q = Q_score (Q=1 最优), 与问题一“越高越好”口径一致。' % (
    corr6, corr7))
111 print('  B8 校准段方向相反 (r=%3f): 两套半合成数据口径不一致, B8 并不
    入拟合。' % corr8)
112
113 # ===== 3. 广义标度律: 形式比较与参数估计 =====
114 N_all = np.r_[N_p, nq6.N_params_B]
115 D_all = np.r_[D_p, nq6.D_tokens_B]
116 Q_all = np.r_[np.ones(len(pythia)), nq6.Q_score]      # B1 无质量信息,
    取参考水平 Q=1
117 L_all = np.r_[L_p, nq6.val_loss]
118
119 def g_v1(p, N, D, Q):
120     E, A, a, B, b, C, g = p
121     return E + A * N ** (-a) + B * D ** (-b) + C * np.clip(1 - Q, 0, 1)
    ** g
122
123 def g_v2(p, N, D, Q):
124     E, A, a, B, b, g = p
125     return E + A * N ** (-a) + B * (D * np.clip(Q, 1e-6, 1) ** g) ** (-b
    )
126
127 def g_v4(p, N, D, Q):
128     E, A, a, B, b, C, g = p
129     return E + A * N ** (-a) + B * D ** (-b) + C * np.clip(Q, 1e-6, 1) *
    * (-g)
130
131 def g_v5(p, N, D, Q):
132     E, A, a, B, b, g1, g2 = p

```

```

133     return (E + A * (N * np.clip(Q, 1e-6, 1) ** g1) ** (-a)
134            + B * (D * np.clip(Q, 1e-6, 1) ** g2) ** (-b))
135
136 FORMS = {
137     '加性质量缺口  $C(1-Q)^g$ ': (g_v1, [E0, A0, a0, B0, b0, .35, 1.],
138                                     [0, 1e-8, .01, 1e-8, .01, 1e-8, .05], [5,
139                                     1e3, 2, 1e3, 2, 50, 10]),
140     '质量作有效数据乘子': (g_v2, [E0, A0, a0, B0, b0, .5],
141                             [0, 1e-8, .01, 1e-8, .01, -10], [5, 1e3, 2, 1
142                             e3, 2, 30]),
143     '加性质量亏损  $C Q^{-g}$ ': (g_v4, [E0, A0, a0, B0, b0, .2, .5],
144                                     [0, 1e-8, .01, 1e-8, .01, 1e-8, .05], [5, 1
145                                     e3, 2, 1e3, 2, 50, 10]),
146     '质量同时乘 N 与 D': (g_v5, [E0, A0, a0, B0, b0, .3, .3],
147                             [0, 1e-8, .01, 1e-8, .01, -10, -10], [5, 1e3,
148                             2, 1e3, 2, 30, 30]),
149 }
150 nq7_new = nq7[~nq7.experiment_id.astype(str).isin(set(nq6.experiment_id.
151                                                         astype(str)))]
152 form_rows, fit_store = [], {}
153 for nm, (fn, p0, lo, hi) in FORMS.items():
154     rr = least_squares(lambda p: fn(p, N_all, D_all, Q_all) - L_all, p0,
155                       bounds=(lo, hi), loss='soft_l1', f_scale=.05,
156                       max_nfev=80000)
157     hh = fn(rr.x, N_all, D_all, Q_all)
158     R2j = 1 - np.sum((L_all - hh) ** 2) / np.sum((L_all - L_all.mean())
159 ** 2)
160     hq = fn(rr.x, nq6.N_params_B.values, nq6.D_tokens_B.values, nq6.
161     Q_score.values)
162     R2q = 1 - np.sum((nq6.val_loss.values - hq) ** 2) / \
163         np.sum((nq6.val_loss.values - nq6.val_loss.mean()) ** 2)
164     h7 = fn(rr.x, nq7_new.N_params_B.values, nq7_new.D_tokens_B.values,
165     nq7_new.Q_score.values)
166     r7 = float(np.corrcoef(nq7_new.val_loss.values, h7)[0, 1])
167     form_rows.append({'形式': nm, '联合R2': R2j, 'B6拟合R2': R2q, 'B7新
168     增点r': r7,
169                      '参数个数': len(rr.x),
170                      'AIC': len(N_all) * np.log(np.mean((L_all - hh) **
171                      2)) + 2 * len(rr.x)})
172     fit_store[nm] = rr.x
173 sv = pd.DataFrame(form_rows).sort_values('联合R2', ascending=False)
174 save_csv_safe(sv, '广义标度律形式对比.csv')
175 print('\n=== 3. 广义标度律形式对比 ===')
176 print(sv.to_string(index=False))
177
178 best_name = sv.iloc[0]['形式']
179 fn_best = FORMS[best_name][0]
180 r1x = fit_store[best_name]
181 E, A, alpha, B, beta, Cq, gamma = r1x
182 Lhat = fn_best(r1x, N_all, D_all, Q_all)
183 R2_gen = 1 - np.sum((L_all - Lhat) ** 2) / np.sum((L_all - L_all.mean())
184 ** 2)
185 RMSE_gen = float(np.sqrt(np.mean((L_all - Lhat) ** 2)))
186 MAPE_gen = float(np.mean(np.abs(L_all - Lhat) / np.abs(L_all)) * 100)
187 print('\n选定形式: %s' % best_name)
188 print(' E=%.4f A=%.4f alpha=%.4f B=%.4f beta=%.4f C=%.4f gamma=%.4f R2
189       =%.4f RMSE=%.4f MAPE=%.2f%%'

```

```

177         % (E, A, alpha, B, beta, Cq, gamma, R2_gen, RMSE_gen, MAPE_gen))
178
179     params = pd.DataFrame({'param': ['E', 'A', 'alpha', 'B', 'beta', 'C', 'gamma'],
180                             'value': [E, A, alpha, B, beta, Cq, gamma]})
181     save_csv_safe(params, '广义标度律参数.csv')
182     params.to_csv(shared_path('广义标度律参数.csv'), index=False, encoding='utf-8-sig')
183
184     # ===== 4. 分层验证 =====
185     def metrics(df, Q=1.0):
186         q = np.full(len(df), Q) if np.isscalar(Q) else np.asarray(Q)
187         ph = fn_best(rlx, df.N_params_B.to_numpy(float), df.D_tokens_B.to_numpy(float), q)
188         y = df.val_loss.to_numpy(float)
189         return ph, float(np.corrcoef(y, ph)[0, 1]), float(np.sqrt(np.mean((y - ph) ** 2))), \
190             float(np.mean(np.abs(y - ph) / np.maximum(np.abs(y), 1e-9)) * 100)
191
192     nq8c = nq8[nq8.data_type.eq('calibrated')]
193     validation, pred_sets = [], []
194     for name, df, q, kind in [('B2_Cerebras(半合成, 族外)', cerebras, 1., '半合成'),
195                               ('B4_跨族收敛点(真实)', baseline, 1., '真实'),
196                               ('B5_已发表标度律(真实)', published, 1., '真实'),
197                               ('B7_质量扩展新增点(半合成)', nq7_new, nq7_new.Q_score, '半合成'),
198                               ('B8_校准段(Q 反向重定向)', nq8c, 1 - nq8c.Q_score, '半合成'),
199                               ('B10_百亿参数以上(估算)', large_loss, 1., '估算')]:
200         ph, r, rm, mp = metrics(df, q)
201         validation.append({'dataset': name, 'n': len(df), '性质': kind, 'r': r,
202                           'RMSE': rm, 'MAPE_%': mp})
203         pred_sets.append((name.split('_')[0], df.val_loss.to_numpy(float), ph))
204     val_df = pd.DataFrame(validation)
205     save_csv_safe(val_df, '广义标度律分层验证.csv')
206     print('\n=== 4. 广义标度律分层验证 ===')
207     print(val_df.to_string(index=False))
208
209     save_csv_safe(large_loss.assign(
210         pred=fn_best(rlx, large_loss.N_params_B.values, large_loss.D_tokens_B.values,
211                     np.ones(len(large_loss)))), '百亿参数以上外推.csv')
212
213     # ===== 5. 弹性、边际效用与替代关系 =====
214     def marg(p, N, D, Q):
215         E_, A_, a_, B_, b_, C_, g_ = p
216         return (-a_ * A_ * N ** (-a_ - 1), -b_ * B_ * D ** (-b_ - 1),
217                -g_ * C_ * np.clip(1 - Q, 1e-9, None) ** (g_ - 1))
218
219     el_rows = []

```

```

220 for N_ref, D_ref, Q_ref in [(1., 100., 0.6), (12., 300., 0.6)]:
221     L_ref = float(fn_best(r1x, N_ref, D_ref, Q_ref))
222     dN, dD, dQ = marg(r1x, N_ref, D_ref, Q_ref)
223     sub = abs(dQ) / abs(dN)
224     el_rows.append({'参考点N(B)': N_ref, 'D(B)': D_ref, 'Q': Q_ref, 'L':
225                    L_ref,
226                    '弹性e_N': dN * N_ref / L_ref, '弹性e_D': dD * D_ref
227                    / L_ref,
228                    '弹性e_Q': dQ * Q_ref / L_ref, '替代率dN/dQ(B)': sub
229                    ,
230                    'dQ=0.1等价dN(B)': sub * 0.1})
231 el_df = pd.DataFrame(el_rows)
232 save_csv_safe(el_df, '弹性与替代关系.csv')
233 print('\n=== 5. 弹性与质量—参数替代关系 ===')
234 print(el_df.round(4).to_string(index=False))
235 N_ref, D_ref, Q_ref = 1., 100., 0.6
236 dN, dD, dQ = marg(r1x, N_ref, D_ref, Q_ref)
237 subst = abs(dQ) / abs(dN)
238 save_csv_safe(pd.DataFrame({'质量提升dQ': [.05, .1, .2, .3],
239                             '局部等价参数增量dN(B)': subst * np.array
240                             ([.05, .1, .2, .3]),
241                             '等价数据增量dD(B)': abs(dQ) / abs(dD) * np.
242                             array([.05, .1, .2, .3])}),
243               '质量参数替代关系.csv')
244 Qs = np.linspace(0.30, 0.99, 80)
245 subs = np.array([abs(marg(r1x, N_ref, D_ref, q)[2]) / abs(marg(r1x,
246                        N_ref, D_ref, q)[0])
247                  for q in Qs])
248 save_csv_safe(pd.DataFrame({'Q': Qs, 'dN_per_dQ(B)': subs,
249                             'dN_for_dQ_0.1(B)': subs * 0.1}), '替代率随
250                             质量变化.csv')
251 save_csv_safe(pd.DataFrame({'Q': Qs, '不可约损失下限': E + Cq * (1 - Qs)
252                             ** gamma}),
253               '质量与损失下限.csv')
254
255 # ===== 6. 领域间的替代/互补关系（基于问题一系数向量）
256 =====
257 p1_dir = os.path.join(SOLVE_DIR, '问题一', '结果')
258 coef = pd.read_csv(os.path.join(p1_dir, '混合系数矩阵.csv')).set_index('
259 training_domain')
260 inter = pd.read_csv(os.path.join(p1_dir, '截距.csv')).set_index('
261 loss_domain').intercept
262 mix = pd.read_csv(DATA('train_mixture_lm.csv'))
263 mix_cols = [c for c in mix if c.startswith('train_the_pile_')]
264 domains = [c.replace('train_the_pile_', '') for c in mix_cols]
265 p_ref = mix[mix_cols].mean().to_numpy(float)
266 loss_domains = list(coef.columns)
267 w_ref = np.array([p_ref[domains.index(d)] for d in loss_domains]); w_ref
268 /= w_ref.sum()
269
270 def mix_obj(p):
271     return float(w_ref @ (np.asarray(p) @ coef.loc[domains].to_numpy()
272                        + inter.loc[loss_domains].to_numpy()))
273

```

```

264 mix_cal = pd.read_csv(os.path.join(p1_dir, '核岭推荐_目标对照.csv'))
265 mix_delta_rec = float(mix_cal.loc[mix_cal.mixture.eq('问题一有界推荐'),'
    M_p'].iloc[0])
266 # 配比修正沿用同一核岭目标, 线性系数仅用于下方领域关联解释。
267 save_csv_safe(mix_cal, '配比项校准.csv')
268
269 Cn = coef.loc[domains].to_numpy()
270 Cc = Cn - Cn.mean(axis=0, keepdims=True)
271 nm_ = np.linalg.norm(Cc, axis=1, keepdims=True)
272 sim = (Cc @ Cc.T) / np.maximum(nm_ @ nm_.T, 1e-12)
273 pd.DataFrame(sim, index=domains, columns=domains).to_csv(
274     os.path.join(OUT_DIR, '领域替代互补矩阵.csv'), encoding='utf-8-sig')
275 tri = np.triu_indices(len(domains), 1)
276 pair_sim = pd.DataFrame({'domain_i': [domains[i] for i in tri[0]],
277     'domain_j': [domains[j] for j in tri[1]],
278     'coefficient_similarity': sim[tri]}).
    sort_values(
279     'coefficient_similarity', ascending=False)
280 save_csv_safe(pd.concat([pair_sim.head(8).assign(关系='可替代'),
281     pair_sim.tail(8).assign(关系='互补或冲突')]), '
    领域替代互补_top.csv')
282 print('\n=== 6. 领域替代与互补 (系数向量余弦相似度) ===')
283 print('最可替代的 4 组: '); print(pair_sim.head(4).to_string(index=False
    ))
284 print('最互补的 4 组: '); print(pair_sim.tail(4).to_string(index=False))
285 print('推荐配比带来的加权平均损失变化 M(p) = %.4f' % mix_delta_rec)
286
287 # ===== 7. B3 插值轨迹验证 =====
288 traj_rows = []
289 for f in traj_files:
290     d = pd.read_csv(f)
291     n_par = float(d.N_params_B.iloc[0])
292     base = E0 + A0 * n_par ** (-a0)
293     y = np.clip(d.val_loss.values - base, 1e-9, None)
294     sl = np.polyfit(np.log(d.D_tokens_B.values), np.log(y), 1)[0]
295     traj_rows.append({'file': os.path.basename(f), 'N(B)': n_par,
296         '轨迹内D幂指数': float(sl), '经典beta': float(-b0)
297     })
298 traj_df = pd.DataFrame(traj_rows)
299 save_csv_safe(traj_df, 'B3轨迹验证.csv')
300 print('\n=== 7. B3 插值轨迹验证 (轨迹内损失对 D 的幂指数 vs 经典 beta)
    ===')
301 print(traj_df.round(4).to_string(index=False))
302
303 # ===== 8. 汇总 =====
304 save_csv_safe(pd.DataFrame({
305     '指标': ['E', 'A', 'alpha', 'B', 'beta', 'C', 'gamma', '经典R2', '经
    典残差标准差',
306     '留出R2', '广义R2', '广义RMSE', '广义MAPE%', 'B6方向相关r',
    'B7方向相关r', 'B8方向相关r',
307     '参考点弹性e_N', '参考点弹性e_D', '参考点弹性e_Q', '替代率
    dN/dQ', '推荐配比M(p)'],
308     '数值': [E, A, alpha, B, beta, Cq, gamma, R2_cls, float(np.std(L_p -
    Lph)),
    hold['留出R2'], R2_gen, RMSE_gen, MAPE_gen, corr6, corr7,
    corr8,

```

```

309         el_rows[0]['弹性e_N'], el_rows[0]['弹性e_D'], el_rows[0]['
        弹性e_Q'],
310         subst, mix_delta_rec})), '问题二_拟合汇总.csv')
311
312 # ===== 9. 绘图 =====
313 from 重绘图表 import render_all
314 render_all(DATA_DIR)
315
316 print('\n===== 问题二 求解完成 =====')
317 print('经典 R2=%.8f, 留出 R2=%.8f; 广义 R2=%.4f; 替代率 dN/dQ=%.3f B/单
        位质量')
318     % (R2_cls, hold['留出R2'], R2_gen, subst))

```

问题三：预算约束优化与敏感性分析

程序 3: 问题三完整求解程序（求解/问题三/问题三.py）

```

1 # -*- coding: utf-8 -*-
2 # 本程序及代码的整理与核对使用人工智能工具 Codex 辅助。
3 # 模型: GPT-5.6-Luna; 开发机构: OpenAI。
4 # 版本发布日期: 2026-07-09 (型号发布)。
5 """
6 问题三: 算力约束下的多维资源联合优化与结构性转移
7 输入: 问题二广义标度律参数  $L(N,D,Q)=E+A/N^a+B/D^b+C(1-Q)^g$ ;
8       问题一的质量基线  $Q_0$  与配比修正  $M(p)$ ; 附件 C7 的上下文长度可行取
        值。
9 成本 (严格按赛题附录 B 与正文公式) :
10      $C_{total} = C_{train} + C_Q + C_{attn}$ 
11      $= 6ND + D * [g(Q) - g(Q_0)]_+ + eta * N * D * L_{ctx}$ ,  $eta$ 
         $= 2e-4$ 
12      $g(Q)$  取附录 B 三型 (指数型  $1e7 * \exp(6Q)$ 、幂函数型  $5e9 * Q^4$ 、对数渐进
        型  $2e9 * \ln(1+10Q)$ )
13 临界上下文长度: 注意力开销 = 基础训练开销  $\Leftrightarrow eta * N * D * L = 6 * N * D \Leftrightarrow$ 
         $L_{crit} = 6/eta = 30000$ 
14 方法: 三档预算联合优化 (SLSQP, 对数参数化); 预算扫略 + 结构性转移识别;
         $L_{ctx}$  敏感性。
15 输出: 图片到 图片/, 结果 CSV 到 结果/。
16 """
17 import sys, os, json
18 sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(
        __file__))))
19 from _common import * # noqa: F401,F403
20 from scipy.optimize import minimize
21
22 DATA = data_path
23
24 def save_csv_safe(df, name_cn):
25     p = os.path.join(OUT_DIR, name_cn)
26     try:
27         df.to_csv(p, index=False, encoding='utf-8-sig')
28     except PermissionError:
29         alt = p.replace('.csv', '_new.csv')

```

```

30         df.to_csv(alt, index=False, encoding='utf-8-sig')
31         print(' ! %s 被占用, 已改存为 %s' % (name_cn, os.path.basename(
alt)))
32
33 # ===== 0. 读取前两问输出 =====
34 params = pd.read_csv(shared_path('广义标度律参数.csv'), index_col='param
')['value']
35 E, A, alpha, B, beta, Cq, gamma = (params['E'], params['A'], params['
alpha'],
36                                     params['B'], params['beta'], params['
C'], params['gamma'])
37
38 with open(shared_path('问题一_关键量.json'), encoding='utf-8') as fh:
39     k1 = json.load(fh)
40 Q0_sample = float(k1['Q0_global_mean'])
41
42 mix_map = pd.read_csv(shared_path('17域质量映射.csv'))
43 ref_mix = pd.read_csv(data_path('train_mixture_lm.csv'))
44 ref_cols = [c for c in ref_mix if c.startswith('train_the_pile_')]
45 ref_p = ref_mix[ref_cols].mean().to_numpy(float)
46 ref_dom = [c.replace('train_the_pile_', '') for c in ref_cols]
47 qmap = mix_map.set_index('mixture_domain')['quality_Q']
48 q_vec = np.array([float(qmap.get(d, Q0_sample)) for d in ref_dom])
49 Q0 = float(ref_p @ q_vec) # 基线质量 = 参考语料配比加权平均质量
50 print('=== 0. 质量基线与模型参数 (来自问题一、二) ===')
51 print(' 参考语料配比加权基线质量 Q0 = %.4f (全量样本均值 %.4f) ' % (Q0,
Q0_sample))
52 print(' 标度律: E=%.4f A=%.4f alpha=%.4f B=%.4f beta=%.4f C=%.4f gamma
=%.4f'
53       % (E, A, alpha, B, beta, Cq, gamma))
54
55 def loss(N_abs, D_abs, Q, M_p=0.0):
56     """验证损失: 参数以 B 为单位代入问题二标度律, M_p 为配比项修正。"""
57     return (E + A * (N_abs / 1e9) ** (-alpha) + B * (D_abs / 1e9) ** (-
beta)
58           + Cq * (1 - Q) ** gamma + M_p)
59
60 # ===== 1. 附录 B 的三型质量成本函数 =====
61 GB = {
62     '指数型': dict(gamma=1e7, lam=6.0),
63     '幂函数型': dict(gamma=5e9, lam=4.0),
64     '对数渐进型': dict(gamma=2e9, lam=10.0),
65 }
66 def g_of(Q, form):
67     par = GB[form]
68     if form == '指数型':
69         return par['gamma'] * np.exp(par['lam'] * Q)
70     if form == '幂函数型':
71         return par['gamma'] * Q ** par['lam']
72     return par['gamma'] * np.log(1 + par['lam'] * Q)
73
74 def c_quality(D_abs, Q, form):
75     return D_abs * max(g_of(Q, form) - g_of(Q0, form), 0.0)
76
77 for form in GB:

```

```

78     print('   g(%) 在 Q0=%.3f 处为 %.4e, 在 Q=1 处为 %.4e (单位算力/
      token) '
79           % (form, Q0, g_of(Q0, form), g_of(1.0, form)))
80
81 # ===== 2. 临界上下文长度 =====
82 ETA = 2e-4
83 L_CRIT = 6.0 / ETA
84 arch = pd.read_csv(DATA('model_architecture_metadata.csv'))
85 L_FEAS = sorted(arch['max_position_embeddings'].dropna().unique().astype
      (int).tolist())
86 print('\n=== 1. 临界上下文长度 (解析解) ===')
87 print('   C_attn = eta*N*D*L_ctx, C_train = 6ND -> L_crit = 6/eta = %.0f
      tokens' % L_CRIT)
88 print('   C7 给出的可行上下文长度 (max_position_embeddings) : %s' %
      L_FEAS)
89 print('   可行取值中跨越临界值的一对: 8192 (%.2f 倍) 与 32768 (%.2f 倍) '
90       % (ETA * 8192 / 6, ETA * 32768 / 6))
91
92 # ===== 3. 总成本与单点最优 =====
93 def total_cost(N_abs, D_abs, Q, L_ctx, form):
94     return 6.0 * N_abs * D_abs + c_quality(D_abs, Q, form) + ETA * N_abs
      * D_abs * L_ctx
95
96 def optimize(C_budget, L_ctx=2048, form='幂函数型', M_p=0.0, n_starts=8,
      Qcap=1.0):
97     """SLSQP 对数参数化联合优化 (N, D, Q)。"""
98     def obj(x):
99         return loss(np.exp(x[0]), np.exp(x[1]), x[2], M_p)
100     def con(x):
101         # 无量纲等价约束, 避免 1e19--1e24 的量级导致有限差分数值失败。
102         return 1.0 - total_cost(np.exp(x[0]), np.exp(x[1]), x[2], L_ctx,
      form) / C_budget
103     best = None
104     for k in range(n_starts):
105         frac = (k + 1) / (n_starts + 1)
106         x0 = [np.log(1e9) + 4 * (frac - .5), np.log(1e11) + 4 * (.5 -
      frac),
107             Q0 + (Qcap - Q0) * frac]
108         try:
109             res = minimize(obj, x0, method='SLSQP',
110                           bounds=[(np.log(1e5), np.log(1e15)),
111                                   (np.log(1e6), np.log(1e16)), (Q0,
      Qcap)],
112                           constraints={'type': 'ineq', 'fun': con},
113                           options={'maxiter': 1500, 'ftol': 1e-14})
114         except Exception:
115             continue
116         if res.success and (best is None or res.fun < best.fun):
117             best = res
118     if best is None:
119         raise RuntimeError('优化失败')
120     N_abs, D_abs, Q = np.exp(best.x[0]), np.exp(best.x[1]), best.x[2]
121     c_base = 6.0 * N_abs * D_abs
122     c_qual = c_quality(D_abs, Q, form)
123     c_attn = ETA * N_abs * D_abs * L_ctx
124     return dict(N_B=N_abs / 1e9, D_B=D_abs / 1e9, D_over_N=D_abs / N_abs
      , Q=Q,

```

```

125         L=loss(N_abs, D_abs, Q, M_p), c_base=c_base, c_qual=
        c_qual,
126         c_attn=c_attn, c_tot=c_base + c_qual + c_attn,
127         f_base=c_base / C_budget, f_qual=c_qual / C_budget,
128         f_attn=c_attn / C_budget, form=form, L_ctx=L_ctx, C=
        C_budget)
129
130 BUDGETS = [1e19, 1e22, 1e24]
131 print('\n=== 2. 三档预算最优配置（幂函数型质量成本，L_ctx=2048）===')
132 rows = []
133 for Cb in BUDGETS:
134     r = optimize(Cb, 2048, '幂函数型')
135     rows.append(r)
136     print(' C=%.0e: N*=%.3fB D*=%.1fB Q*=%.4f L*=%.4f '
137           '[base %.1f%% qual %.1f%% attn %.1f%%]'
138           % (Cb, r['N_B'], r['D_B'], r['Q'], r['L'],
139             r['f_base'] * 100, r['f_qual'] * 100, r['f_attn'] * 100))
140 save_csv_safe(pd.DataFrame(rows), '三档预算最优配置.csv')
141
142 # ===== 4. 三种质量成本函数对比 =====
143 print('\n=== 3. 质量成本函数对最优解的影响 ===')
144 comp = []
145 for Cb in BUDGETS:
146     for form in GB:
147         r = optimize(Cb, 2048, form)
148         comp.append(r)
149 comp_df = pd.DataFrame(comp)
150 save_csv_safe(comp_df, '成本函数对比.csv')
151 for Cb in BUDGETS:
152     sub = comp_df[comp_df.C.eq(Cb)]
153     print(' C=%.0e' % Cb)
154     for _, r in sub.iterrows():
155         print(' %-6s N*=%.3fB D*=%.1fB Q*=%.4f L*=%.4f 质量投入占比
156               %.1f%%'
157               % (r['form'], r['N_B'], r['D_B'], r['Q'], r['L'], r['
158                 f_qual'] * 100))
159
160 # ===== 4b. 质量基线 Q0 口径的灵敏度 =====
161 _Q0_main = Q0
162 Q0 = Q0_sample
163 print('\n=== 3b. 质量基线口径灵敏度（Q0 由配比加权 %.4f 改为样本均值 %.4
164       f）===')
165     % (_Q0_main, Q0_sample))
166 q0_rows = []
167 for Cb in BUDGETS:
168     r = optimize(Cb, 2048, '幂函数型')
169     q0_rows.append(r)
170     print(' C=%.0e: N*=%.3fB D*=%.1fB Q*=%.4f L*=%.4f 质量投入占比 %.1f
171           %%'
172           % (Cb, r['N_B'], r['D_B'], r['Q'], r['L'], r['f_qual'] * 100))
173 save_csv_safe(pd.DataFrame(q0_rows), '质量基线灵敏度.csv')
174 Q0 = _Q0_main
175
176 # ===== 5. 领域配比 M(p) 的算力等价效果 =====
177 mix_cal = pd.read_csv(os.path.join(SOLVE_DIR, '问题二', '结果', '配比项
178     校准.csv'))

```

```

174 M_p = float(mix_cal.loc[mix_cal.mixture.eq('问题一有界推荐'), 'M_p'].
      iloc[0])
175 print('\n=== 4. 领域配比修正的算力等价效果 (M(p)=%.4f) ===' % M_p)
176 r_mix = optimize(1e22, 2048, '幂函数型', M_p=M_p)
177 r_ref = optimize(1e22, 2048, '幂函数型', M_p=0.0)
178 print(' C=1e22: 参考配比 L*=%.4f (N*=%.3fB D*=%.1fB Q*=%.4f) '
179       % (r_ref['L'], r_ref['N_B'], r_ref['D_B'], r_ref['Q']))
180 print('          推荐配比 L*=%.4f (N*=%.3fB D*=%.1fB Q*=%.4f) '
181       % (r_mix['L'], r_mix['N_B'], r_mix['D_B'], r_mix['Q']))
182
183 def budget_for_loss(L_target, L_ctx=2048, form='幂函数型'):
184     lo, hi = 1e17, 1e25
185     for _ in range(60):
186         mid = np.sqrt(lo * hi)
187         try:
188             r = optimize(mid, L_ctx, form, n_starts=3)
189             if r['L'] > L_target:
190                 lo = mid
191             else:
192                 hi = mid
193         except Exception:
194             lo = mid
195     return np.sqrt(lo * hi)
196
197 C_eq = budget_for_loss(r_mix['L'])
198 print(' 推荐配比在 C=1e22 时得到的损失 %.4f, 若改用参考配比实现同一损失
      需要'
199       ' C≈%.3e FLOPs, 即配比优化的效果等价于约 %.1f 倍算力扩张'
200       % (r_mix['L'], C_eq, C_eq / 1e22))
201 print(' (说明: M(p) 是损失水平上的固定平移, 与算力无关, 故换算为算力收
      益会被'
202       ' 损失的算力弹性放大, 此倍数宜作量级参考。) ')
203 save_csv_safe(pd.DataFrame([{'mixture': '参考配比', 'M_p': 0.0, **r_ref,
204                             'equivalent_reference_budget': 1e22,
205                             'budget_multiplier': 1.0},
206                             {'mixture': '问题一有界推荐', 'M_p': M_p, **
207                             r_mix,
208                             'equivalent_reference_budget': C_eq,
209                             'budget_multiplier': C_eq/1e22}]),
210              '配比项算力等价.csv')
211
212 # ===== 6. 预算扫略与结构性转移识别 =====
213 print('\n=== 5. 预算扫略与结构性转移 ===')
214 C_sweep = np.logspace(18, 25, 141)
215 sweep = []
216 for Cb in C_sweep:
217     try:
218         sweep.append(optimize(Cb, 2048, '幂函数型', n_starts=5))
219     except Exception:
220         sweep.append(dict(N_B=np.nan, D_B=np.nan, Q=np.nan, L=np.nan,
221                           c_base=np.nan,
222                           c_qual=np.nan, c_attn=np.nan, c_tot=np.nan,
223                           f_base=np.nan,
224                           f_qual=np.nan, f_attn=np.nan, form='幂函数型',
225                           L_ctx=2048, C=Cb))
226 sw = pd.DataFrame(sweep)
227 save_csv_safe(sw, '预算扫略.csv')

```

```

222
223 Qc = sw['Q'].to_numpy(float)
224 logC = np.log10(C_sweep)
225 ok = np.isfinite(Qc)
226 # 结构性转移的定量定义:
227 # 完成度与费用份额对 log10(C) 的变化率, 用于定位变化最陡的区间; 不称为
    弹性。
228 inQ = (Qc - Q0) / (1 - Q0) # 质量提升完成度
229 d_inQ = np.gradient(inQ[ok], logC[ok])
230 fQ = sw['f_qual'].to_numpy(float)
231 d_fQ = np.gradient(fQ[ok], logC[ok])
232 shift = {'完成度变化率 dTheta/dlog10C 的峰值预算': float(C_sweep[ok][np.
    argmax(d_inQ)]),
233         '质量投入份额变化率 df_Q/dlog10C 的峰值预算': float(C_sweep[ok
    ][np.argmax(d_fQ)]),
234         '质量投入份额峰值预算': float(C_sweep[ok][np.argmax(fQ[ok])]),
235         '质量投入份额峰值': float(np.nanmax(fQ[ok]))}
236 shift['质量投入首次正值预算'] = float(sw.loc[sw.Q > Q0 + 1e-7, 'C'].iloc
    [0])
237 shift['质量首次达到上界预算'] = float(sw.loc[sw.Q >= 1 - 1e-7, 'C'].iloc
    [0])
238 # 以"质量提升完成度"  $\Theta = (Q - Q_0) / (1 - Q_0)$  报告门槛, 避免 Q0 本身已高于固
    定阈值时失去意义
239 for frac in [0.25, 0.5, 0.75, 0.99]:
240     hit = Qc[ok] >= Q0 + frac * (1 - Q0)
241     shift['完成度Theta首次达到%d%%的预算' % int(frac * 100)] = (
242         float(C_sweep[ok][np.argmax(hit)]) if hit.any() else float('nan'
    ))
243 for k, v in shift.items():
244     print(' %s = %.3e FLOPs' % (k, v))
245
246 # N/D 比随预算的演化 (规模结构)
247 sw['D_over_N'] = sw['D_B'] / sw['N_B']
248 print(' D/N 比: C=1e19 时 %.1f, C=1e22 时 %.1f, C=1e24 时 %.1f'
249       % tuple(sw.loc[(sw.C - c).abs().idxmin(), 'D_over_N'] for c in [1
    e19, 1e22, 1e24]))
250 save_csv_safe(pd.DataFrame([shift]), '结构性转移识别.csv')
251
252 # ===== 7. 上下文长度敏感性 (C7 可行取值 + 临界值) =====
253 print('\n=== 6. 上下文长度敏感性 (C=1e22, 幂函数型) ===')
254 sens = []
255 for L in L_FEAS:
256     r = optimize(1e22, L, '幂函数型')
257     r['attn_over_train'] = ETA * L / 6.0
258     r['is_above_crit'] = L > L_CRIT
259     sens.append(r)
260     print(' L=%6d (%.2f 倍训练开销) : N*=%.3fB D*=%.1fB Q*=%.4f L*=%.4f
        注意力占比 %.1f%%s'
261           % (L, r['attn_over_train'], r['N_B'], r['D_B'], r['Q'], r['L'
    ],
262             r['f_attn'] * 100, ' [超临界]' if L > L_CRIT else ''))
263 sens_df = pd.DataFrame(sens)
264 save_csv_safe(sens_df, '上下文长度敏感性.csv')
265
266 # 连续 L 扫描 (含临界点)

```

```

267 L_grid = np.logspace(np.log10(1024), np.log10(200000), 40)
268 cont = []
269 for L in L_grid:
270     try:
271         r = optimize(1e22, float(L), '幂函数型', n_starts=3)
272         r['attn_over_train'] = ETA * L / 6.0
273         cont.append(r)
274     except Exception:
275         pass
276 save_csv_safe(pd.DataFrame(cont), '上下文长度连续扫描.csv')
277
278 # ===== 8. 绘图 =====
279 plt.rcParams['font.size'] = 12
280 C1_, C2_, C3_ = '#3b6ea5', '#e07a3f', '#5aa05a'
281
282 opt_df = pd.DataFrame(rows)
283 fig, axes = plt.subplots(1, 2, figsize=(11.5, 4.6))
284 x = np.arange(len(BUDGETS))
285 axes[0].plot(x, opt_df['N_B'], 'o-', lw=2, ms=8, color=C1_, label='参数量 N* (B)')
286 axes[0].plot(x, opt_df['D_B'], 's--', lw=2, ms=8, color=C2_, label='数据量 D* (B tokens)')
287 for xi, (nb, db) in enumerate(zip(opt_df['N_B'], opt_df['D_B'])):
288     axes[0].annotate('%0.3g' % nb, (xi, nb), fontsize=8, xytext=(0, 8),
289                     textcoords='offset points', ha='center', color=C1_)
290     axes[0].annotate('%0.4g' % db, (xi, db), fontsize=8, xytext=(0, -15),
291                     textcoords='offset points', ha='center', color=C2_)
292 axes[0].set_yscale('log'); axes[0].set_xticks(x)
293 axes[0].set_xticklabels(['1019', '1022', '1024'])
294 axes[0].set_xlabel('算力预算 C (FLOPs)'); axes[0].set_ylabel('最优规模 (对数尺度)')
295 axes[0].set_title('(a) 最优参数量与数据量'); axes[0].legend(fontsize=9);
296     despine(axes[0])
297 axes[1].plot(x, opt_df['Q'], 'o-', lw=2, ms=8, color=C3_, label='最优质量 Q*')
298 axes[1].axhline(Q0, color='gray', ls=':', lw=1.2, label='基线质量 Q0=%.3f' % Q0)
299 for xi, qv in enumerate(opt_df['Q']):
300     axes[1].annotate('%0.3f' % qv, (xi, qv), fontsize=9, xytext=(0, 8),
301                     textcoords='offset points', ha='center', color=C3_)
302 axes[1].set_xticks(x); axes[1].set_xticklabels(['1019', '1022', '1024'])
303 axes[1].set_ylim(0, 1.12); axes[1].set_xlabel('算力预算 C (FLOPs)')
304 axes[1].set_ylabel('最优质量 Q*'); axes[1].set_title('(b) 最优质量随预算的变化')
305 axes[1].legend(fontsize=9); despine(axes[1])
306 fig.tight_layout(); save_fig(fig, '图1_三档预算最优配置')
307
308 fig, axes = plt.subplots(1, 2, figsize=(11.5, 4.5))
309 axes[0].plot(C_sweep, Qc, lw=2, color=C3_)
310 axes[0].axhline(Q0, color='gray', ls=':', lw=1.2, label='基线质量 Q0=%.3f' % Q0)
311 axes[0].axvline(shift['质量投入份额变化率 df_Q/dlog10C 的峰值预算'],
312                 color='#d62728', ls='--', lw=1.2,
313                 label='投入份额拐点')
314 axes[0].set_xscale('log'); axes[0].set_xlabel('算力预算 C (FLOPs)')
315 axes[0].set_ylabel('最优质量 Q*'); axes[0].set_ylim(0, 1.05)

```

```

314 axes[0].set_title('(a) 最优质量随预算的演化'); axes[0].legend(fontsize
    =8); despine(axes[0])
315 axes[1].plot(C_sweep, sw['f_base'] * 100, lw=2, color=C1_, label='基础训
    练 6ND')
316 axes[1].plot(C_sweep, sw['f_qual'] * 100, lw=2, color=C2_, label='质量提
    升 D[g(Q)-g(Q₀)]')
317 axes[1].plot(C_sweep, sw['f_attn'] * 100, lw=2, color='#d62728', label='
    长文本注意力')
318 axes[1].set_xscale('log'); axes[1].set_xlabel('算力预算 C (FLOPs)')
319 axes[1].set_ylabel('预算份额 (%)'); axes[1].set_title('(b) 预算份额的结
    构性演化')
320 axes[1].legend(fontsize=8); despine(axes[1])
321 fig.tight_layout(); save_fig(fig, '图2_预算份额结构性转移')
322
323 fig, axes = plt.subplots(1, 2, figsize=(11.5, 4.5))
324 axes[0].plot([r['attn_over_train'] for r in cont], [r['N_B'] for r in
    cont], 'o-', ms=3, color=C1_)
325 axes[0].set_xscale('log'); axes[0].set_yscale('log')
326 axes[0].axvline(1.0, color='#d62728', ls='--', lw=1.2,
327     label='临界  $L^* = \eta f$  (注意力=训练)' % L_CRIT)
328 axes[0].set_xlabel('注意力开销 / 训练开销 =  $\eta L/6$ '); axes[0].set_ylabel('
    N* (B)')
329 axes[0].set_title('(a) 上下文长度对最优规模的影响'); axes[0].legend(
    fontsize=8); despine(axes[0])
330 # 右图: 不同预算档下  $Q^*$  随注意力开销倍数的变化 (低预算档才有内点解, 故三
    档并列)
331 L_small = np.logspace(np.log10(1024), np.log10(150000), 14)
332 _qL_rows = []
333 for Cb, col, lbl in [(1e19, C3_, '1019'), (1e20, C2_, '1020'), (1e21,
    C1_, '1021)]:
334     xs, ys = [], []
335     for L in L_small:
336         try:
337             rr = optimize(Cb, float(L), '幂函数型', n_starts=3)
338             xs.append(ETA * L / 6.0); ys.append(rr['Q'])
339             _qL_rows.append({'C': Cb, 'L_ctx': float(L), '
    attn_over_train': ETA * L / 6.0,
340                 'N_B': rr['N_B'], 'D_B': rr['D_B'], 'Q': rr
    ['Q'], 'L': rr['L']})
341         except Exception:
342             pass
343     axes[1].plot(xs, ys, 'o-', ms=3, color=col, label='预算 %s FLOPs' %
    lbl)
344 save_csv_safe(pd.DataFrame(_qL_rows), '上下文长度与最优质量.csv')
345 axes[1].axvline(1.0, color='#d62728', ls='--', lw=1.2, label='临界  $L^*$ ')
346 axes[1].set_xscale('log'); axes[1].set_xlabel('注意力开销 / 训练开销 =
     $\eta L/6$ ')
347 axes[1].set_ylabel('Q*'); axes[1].set_ylim(0, 1.05)
348 axes[1].yaxis.get_major_formatter().set_useOffset(False)
349 axes[1].set_title('(b) 上下文长度对质量投入的影响')
350 axes[1].legend(fontsize=8); despine(axes[1])
351 fig.tight_layout(); save_fig(fig, '图3_上下文长度敏感性')
352
353 fig, ax = plt.subplots(figsize=(7, 5))
354 Q_g = np.linspace(Q0, 1.0, 300)
355 for form, lbl in [('幂函数型', '幂函数型  $5 \times 10^9 Q^4$ '),

```

```

356         ('指数型', '指数型  $10^7 e^{(6Q)}$ '),
357         ('对数渐进型', '对数渐进型  $2 \times 10^9 \ln(1+10Q)$ '))]:
358     ax.plot(Q_g, [g_of(q, form) - g_of(Q0, form) for q in Q_g], lw=2,
               label=lbl)
359 ax.set_xlabel('数据质量 Q'); ax.set_ylabel('单位 token 质量成本  $g(Q)-g(Q_0)$ ')
360 ax.set_title('附录 B 三种质量成本函数 ( $Q_0=0.3f$ ) ' % Q0)
361 ax.legend(fontsize=9); despine(ax); fig.tight_layout(); save_fig(fig, '
    图4_质量成本函数对比')
362
363 # ===== 9. 汇总 =====
364 print('\n===== 问题三求解完成 =====')
365 print('    临界上下文长度  $L_{crit} = 6/\eta = 0.0f$  tokens; C7 可行取值 %s' %
        (L_CRIT, L_FEAS))
366 print('    三档预算 (幂函数型): ' +
367       '; '.join('C=%0e -> N*=%0.3fB D*=%0.1fB Q*=%0.4f L*=%0.4f'
368                % (r['C'], r['N_B'], r['D_B'], r['Q'], r['L']) for r in
                    rows))
369
370 # 以保存的扫描结果生成正文正式图; 旧绘图辅助计算不作为最终图版本。
371 import subprocess as _subprocess
372 _subprocess.run([sys.executable, os.path.join(BASE_DIR, '论文绘图.py')],
                  check=True)

```

问题四：任务解析、能力分解与前沿预测

程序 4: 问题四完整求解程序 (求解/问题四/问题四.py)

```

1 # -*- coding: utf-8 -*-
2 # 本程序及代码的整理与核对使用人工智能工具 Codex 辅助。
3 # 模型: GPT-5.6-Luna; 开发机构: OpenAI。
4 # 版本发布日期: 2026-07-09 (型号发布)。
5 """
6 问题四: 技术演进分析与前沿预测
7 数据: C1/C2 排行榜、C3 时序、C4 Epoch 宏观元数据、C5/C6 Loss-Benchmark
    桥接、C7 架构、C8 逐任务 JSON
8 流程: 1) 明确口径: 时间轴(提交日期)、开放权重样本口径(许可白名单)、模型
    类型(pretrained/chat)
9       2) C8 逐任务聚合(必做), 并与 C1 六维交叉校验
10      3) C6 桥接(按可比性分层) + C5 对照, 建立 Loss -> Benchmark 映射
11      4) 合并横截面回归(含年份控制项)核算规模关联项与未解释剩余项
12      5) 情景设定下 logistic 饱和外推 12/24 个月能力前沿 + 残差
    bootstrap 不确定性
13      6) C4 宏观算力/数据量/开放权重参照, 不直接代入能力回归
14 输出: 图片到 图片/, 结果 CSV 到 结果/
15 """
16 import sys, os, json, glob, re, collections
17 sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(
    __file__))))
18 from _common import * # noqa: F401,F403
19 from scipy.optimize import curve_fit
20

```

```

21 DATA = data_path
22 C_DIR = os.path.join(DATA_DIR, 'C_efficiency_evolution')
23
24 def save_csv_safe(df, name_cn):
25     p = os.path.join(OUT_DIR, name_cn)
26     try:
27         df.to_csv(p, index=False, encoding='utf-8-sig')
28     except PermissionError:
29         alt = p.replace('.csv', '_new.csv')
30         df.to_csv(alt, index=False, encoding='utf-8-sig')
31         print(' ! %s 被占用, 已改存为 %s' % (name_cn, os.path.basename(
alt)))
32
33 TASKS = ['IFEval', 'BBH', 'MATH Lvl 5', 'GPQA', 'MUSR', 'MMLU-PRO']
34
35 # ===== 1. C1 排行榜: 口径定义与筛选 =====
36 lb = pd.read_csv(os.path.join(C_DIR, 'leaderboard_cleaned.csv'))
37 lb['P'] = pd.to_numeric(lb['#Params (B)'], errors='coerce')
38 lb['date'] = pd.to_datetime(lb['Submission Date'], errors='coerce')
39 lb['Year'] = lb['date'].dt.year
40 lb['avg'] = pd.to_numeric(lb['Average \u2b06\u2013'], errors='coerce')
41 for t in TASKS:
42     lb[t] = pd.to_numeric(lb[t], errors='coerce')
43
44 # 字段中的特殊符号使用 Unicode 转义, 保证附录打印后仍能准确复制。
45 TYPE_PRE = ['\U0001f7e2 pretrained', '\U0001f7e9 continuously pretrained']
46 TYPE_CHAT = ['\U0001f4ac chat models (RLHF, DPO, IFT, ...)',
47 '\U0001f536 fine-tuned on domain-specific datasets']
48 TYPE_EXCL = ['\U0001f91d base merges and moerges', '\U0001f338
multimodal', '\u2753 other']
49
50 PERMISSIVE = {'apache-2.0', 'mit', 'gpl-3.0', 'cc-by-4.0', 'mpl-2.0', '
bsd-3-clause',
51 'llama2', 'llama3', 'llama3.1', 'llama3.2', 'llama3.3', '
gemma',
52 'cc-by-sa-4.0', 'openrail'}
53
54 def license_ok(x):
55     """按本文列明的许可白名单筛选; 不将各许可的具体条款视为相同。"""
56     return str(x).strip().lower() in PERMISSIVE
57
58 lb['license_ok'] = lb['Hub License'].map(license_ok)
59 lb['type_group'] = np.where(lb['Type'].isin(TYPE_PRE), 'pretrained',
60 np.where(lb['Type'].isin(TYPE_CHAT), 'chat/
finetuned', '其他'))
61 lb['six_complete'] = lb[TASKS].notna().all(axis=1)
62
63 funnel = [{'层级': 'C1 全量记录', 'n': len(lb)},
64 {'层级': '六维得分完整', 'n': int(lb.six_complete.sum())}]
65 core = lb[lb.six_complete & lb.date.notna() & lb.P.notna()]
66 funnel.append({'层级': '时间/参数/六维齐备', 'n': len(core)})
67 core_o = core[core.license_ok]
68 funnel.append({'层级': '许可允许研究复现 (开源口径)', 'n': len(core_o)
})
69 core_pt = core_o[core_o.type_group.eq('pretrained')]
70 core_ch = core_o[core_o.type_group.eq('chat/finetuned')]

```

```

71 funnel.append({'层级': '  其中 pretrained', 'n': len(core_pt)})
72 funnel.append({'层级': '  其中 chat/finetuned', 'n': len(core_ch)})
73 save_csv_safe(pd.DataFrame(funnel), '筛选漏斗.csv')
74 print('=== 1. 数据口径与筛选漏斗 ===')
75 print(pd.DataFrame(funnel).to_string(index=False))
76 print('  时间轴口径: 以 C1 的 Submission Date (提交日期) 为基准。')
77 print('  主样本口径: 时间、参数量与六维得分完整, 且 Hub License 在本文白
    名单中。')
78 print('  模型类型口径: 主样本保留全部类型; pretrained 与 chat/finetuned
    另作分组分析。')
79 print('  主样本中的其他类型: base 合并模型 %d 条, 多模态 %d 条。'
80       % (int(core_o['Type'].eq(TYPE_EXCL[0]).sum()),
81          int(core_o['Type'].eq(TYPE_EXCL[1]).sum())))
82 print('  提交日期范围: %s ~ %s' % (str(lb.date.min())[10], str(lb.date.
    max())[10]))
83
84 # ===== 2. C8 逐任务聚合 (必用) =====
85 def norm_name(s):
86     return re.sub(r'^a-z0-9', '', str(s).lower())
87
88 C8_DIR = os.path.join(C_DIR, 'detailed_results')
89 dirs = sorted(d for d in os.listdir(C8_DIR) if os.path.isdir(os.path.
    join(C8_DIR, d)))
90 METRIC_PREF = ['acc_norm,none', 'acc,none', 'prompt_level_loose_acc,none',
    'inst_level_loose_acc,none', 'exact_match,none']
91
92 recs, broken = [], 0
93 for d in dirs:
94     fl = sorted(glob.glob(os.path.join(C8_DIR, d, '*.json')))
95     if not fl:
96         continue
97     ok = False
98     for f in reversed(fl):
99         try:
100             o = json.load(open(f, encoding='utf-8'))
101             res = o.get('results')
102             if not isinstance(res, dict) or not res:
103                 continue
104         except Exception:
105             broken += 1
106             continue
107         rec = {'dir': d, 'json_model_name': o.get('model_name', ''),
108              'file': os.path.basename(f)}
109         for k, v in res.items():
110             if not isinstance(v, dict):
111                 continue
112             for mk in METRIC_PREF:
113                 if isinstance(v.get(mk), (int, float)):
114                     rec[k] = float(v[mk]) * 100.0
115                     break
116         recs.append(rec)
117         ok = True
118         break
119     if not ok:
120         broken += 1
121
122 c8 = pd.DataFrame(recs)

```

```

123 print('\n=== 2. C8 逐任务聚合（必做） ===')
124 print(' 模型目录 %d 个；成功解析 %d 个；跳过损坏/无有效记录 %d 个' % (
    len(dirs), len(c8), broken))
125 save_csv_safe(c8.fillna(''), 'C8逐任务解析结果.csv')
126
127 C8_MAP = {'IFEval': 'leaderboard_ifeval', 'BBH': 'leaderboard_bbh',
128           'MATH Lvl 5': 'leaderboard_math_hard', 'GPQA': '
    leaderboard_gpqa',
129           'MUSR': 'leaderboard_musr', 'MMLU-PRO': 'leaderboard_mmlu_pro'
    }
130 for t, k in C8_MAP.items():
131     c8[t] = c8[k] if k in c8.columns else np.nan
132 c8['key'] = c8['dir'].map(norm_name)
133
134 lb['key'] = lb['Model'].map(norm_name)
135 core_o = core_o.assign(key=core_o['Model'].map(norm_name))
136 mrg = lb[['key'] + TASKS].merge(c8[['key'] + TASKS], on='key', suffixes
    =('_C1', '_C8'))
137 chk_rows = []
138 for t in TASKS:
139     a = pd.to_numeric(mrg[t + '_C1'], errors='coerce')
140     bq = pd.to_numeric(mrg[t + '_C8'], errors='coerce')
141     okm = a.notna() & bq.notna()
142     chk_rows.append({'任务': t, '匹配模型数': int(okm.sum()),
143                    'Pearson_r': float(np.corrcoef(a[okm], bq[okm])[0,
144                    1]) if okm.sum() > 2 else np.nan,
145                    'Spearman_rho': float(stats.spearmanr(a[okm], bq[
146                    okm])).statistic)
147                    if okm.sum() > 2 else np.nan,
148                    '平均绝对偏差': float(np.mean(np.abs(a[okm] - bq[
149                    okm]))) if okm.sum() else np.nan})
150 chk = pd.DataFrame(chk_rows)
151 save_csv_safe(chk, 'C8与C1一致性校验.csv')
152 print('  C8 与 C1 逐维一致性（规范化模型名匹配，得到 %d 条配对记录）： '
    % len(mrg))
153 print(chk.to_string(index=False))
154 print(' 口径说明：C8 存的是各任务的原始指标（acc/acc_norm/exact_match
    ），
155     'C1 存的是官方按随机基线归一化后的六维得分（例如 GPQA=(raw-0.25)
    /0.75、
156     'MMLU-PRO=(raw-0.10)/0.90、IFEval=(prompt_strict+inst_strict)/2
    ），
157     '故两者存在系统性水平差但秩一致性很高。因此：跨模型得分分析用 C1,
    '
158     '逐任务结构分析在 C8 的原始指标空间内进行。')
159
160 # 逐任务（子任务）聚合：BBH 24 项、MATH 7 项、GPQA 3 项、MUSR 3 项
161 SUB_PREFIX = {'BBH子任务': 'leaderboard_bbh_', 'MATH子任务': '
    leaderboard_math_',
162               'GPQA子集': 'leaderboard_gpqa_', 'MUSR子任务': '
    leaderboard_musr_'
    }
163 sub_cols = {}
164 for grp, pre in SUB_PREFIX.items():
165     cols = [c for c in c8.columns if c.startswith(pre) and c.count('_')
    >= 2]
166     cols = [c for c in cols if c not in ('leaderboard_bbh', '
    leaderboard_musr',

```

```

164                                     'leaderboard_gpqa', '
        leaderboard_math_hard'))]
165     sub_cols[grp] = cols
166 print('    C8 子任务维度数: ' + ', '.join('%s %d 项' % (k, len(v)) for k,
        v in sub_cols.items()))
167
168 # 子任务前沿与剩余空间 (限定在开源筛选后的模型集合上)
169 screened_keys = set(core_o['key'])
170 c8s = c8[c8.key.isin(screened_keys)]
171 sub_rows = []
172 for grp, cols in sub_cols.items():
173     for c in cols:
174         s = pd.to_numeric(c8s[c], errors='coerce').dropna()
175         if len(s) < 10:
176             continue
177         sub_rows.append({'类别': grp, '子任务': c.replace('leaderboard_',
        , ''), '样本数': len(s),
178                             '最优得分': float(s.max()),
179                             '前5%均值': float(s.nlargest(max(1, int(len(s)
        * 0.05))).mean()),
180                             '中位数': float(s.median()),
181                             '剩余空间_%': float((100 - s.max()) / 100 *
        100)})
182 sub_df = pd.DataFrame(sub_rows)
183 save_csv_safe(sub_df, 'C8逐任务聚合.csv')
184 print('\n    子任务层面最弱 6 项 (剩余空间最大): ')
185 print(sub_df.nlargest(6, '剩余空间_%')[['类别', '子任务', '样本数', '最
        优得分', '前5%均值',
186                                     '剩余空间_%']].to_string(index=
        False))
187
188 CAT_ROWS = []
189 for grp, cols in sub_cols.items():
190     s = pd.to_numeric(c8s[cols].stack(), errors='coerce').dropna()
191     CAT_ROWS.append({'类别': grp, '维度数': len(cols), '平均最优得分':
        float(
192         np.mean([pd.to_numeric(c8s[c], errors='coerce').max() for c in
        cols])),
193         '整体前5%均值': float(s.quantile(0.95)), '整体中位数': float(s.
        median())})
194 save_csv_safe(pd.DataFrame(CAT_ROWS), 'C8子任务类别汇总.csv')
195 print('\n    子任务类别汇总: ')
196 print(pd.DataFrame(CAT_ROWS).to_string(index=False))
197
198 # ===== 3. Loss - Benchmark 桥接 (C6 主用, 按可比性分层; C5 对
        照) =====
199 br = pd.read_csv(os.path.join(C_DIR, 'loss_benchmark_bridge_expanded.csv
        '))
200 br['cmp'] = np.where(br['Loss_Comparability'].str.startswith('High'), '
        高可比',
201                     '中可比')
202 print('\n=== 3. Loss-Benchmark 桥接 (C6 扩展桥接) ===')
203 bridge_rows, fits = [], {}
204 for lvl in ['高可比', '中可比', '全样本']:
205     d = br if lvl == '全样本' else br[br.cmp.eq(lvl)]
206     sl, ic = np.polyfit(d.Val_Loss, d.LB_Average, 1)

```

```

207     pred = ic + sl * d.Val_Loss
208     fits[lvl] = (sl, ic)
209     bridge_rows.append({'可比性': lvl, 'n': len(d),
210                        'Pearson_r': float(np.corrcoef(d.Val_Loss, d.
211 LB_Average)[0, 1]),
212                        'Spearman_rho': float(stats.spearmanr(d.Val_Loss
213 , d.LB_Average).statistic),
214                        '斜率': float(sl), '截距': float(ic),
215                        'MAPE_%': float(np.mean(np.abs(pred - d.
216 LB_Average) / d.LB_Average) * 100),
217                        'R2': float(1 - np.sum((d.LB_Average - pred) **
218 2)
219 / np.sum((d.LB_Average - d.
220 LB_Average.mean()) ** 2))})
221 bridge_df = pd.DataFrame(bridge_rows)
222 save_csv_safe(bridge_df, 'Loss_Benchmark映射.csv')
223 print(bridge_df.to_string(index=False))
224 br['pred'] = fits['全样本'][1] + fits['全样本'][0] * br.Val_Loss
225 save_csv_safe(br[['Model', 'N_params_B', 'D_tokens_B', 'Val_Loss', '
226 LB_Average', 'pred',
227 'Loss_Comparability']], 'Loss_Benchmark映射明细.csv')
228
229 try:
230     b5 = pd.read_csv(os.path.join(C_DIR, 'loss_benchmark_bridge.csv'))
231     s5, i5 = np.polyfit(b5.Val_Loss, b5.LB_Average, 1)
232     print(' C5 (原始桥接, n=%d) 对照: r=%.3f, 斜率=%.3f'
233           % (len(b5), float(np.corrcoef(b5.Val_Loss, b5.LB_Average)[0,
234 1]), s5))
235 except Exception as e:
236     print(' C5 对照读取失败: ', e)
237 print(' 说明: 全样本 |r|=%.2f (R2=%.2f), 映射误差较大; 高可比子集仅 %d
238 条且落在 '
239 'Pythia 小模型区间, 得分变化幅度极小 (%.2f-%.2f), 故桥接误差对能
240 力结论的影响 '
241 '必须显式讨论。 '
242 % (abs(bridge_df.iloc[2]['Pearson_r']), bridge_df.iloc[2]['R2'],
243 int(bridge_df.iloc[0]['n']),
244 br[br.cmp.eq('高可比')].LB_Average.min(), br[br.cmp.eq('高可比'
245 )].LB_Average.max()))
246
247 # ===== 4. 合并横截面回归与增量核算 =====
248 # 思路: 规模系数 bP 在“同一年内的横截面”上识别最稳健 (年份效应吸收了当
249 年的技术水平),
250 # 再由 bP 与配对前沿规模的  $\Delta \log_{10} P$  核算规模关联项, 其余为未解释剩
251 余项。
252 # 历史 CSV 文件名“面板回归数据”及贡献/分年列名为接口兼容保留, 不
253 表示纵向或因果估计。
254 panel = core_o.copy()
255 lb_o = core_o.copy()
256 panel['logP'] = np.log10(panel.P)
257 panel['t'] = panel.Year - 2019
258 panel = panel[panel.t.between(0, 12)]
259
260 years = sorted(panel.t.unique())
261 def fit_model(sub, label, fe=True):
262     if fe:

```

```

250     Dm = np.column_stack([(sub.t.values == yy).astype(float) for yy
in years])
251     X = np.column_stack([Dm, sub.logP.values])
252     else:
253         X = np.column_stack([np.ones(len(sub)), sub.logP.values, sub.t.
values])
254     b, *_ = np.linalg.lstsq(X, sub.avg.values, rcond=None)
255     pred = X @ b
256     R2 = 1 - np.sum((sub.avg.values - pred) ** 2) / \
257         np.sum((sub.avg.values - sub.avg.values.mean()) ** 2)
258     return (b[-1], b[:-1], R2) if fe else (b[1], b[2], R2)
259
260 bP_fe, alpha_t, R2_fe = fit_model(panel, 'FE', True)
261 bP_ols, bt_ols, R2_ols = fit_model(panel, 'OLS', False)
262 panel['alpha_t'] = panel.t.map(dict(zip(years, alpha_t)))
263 save_csv_safe(pd.DataFrame({'Year': [y + 2019 for y in years], '年效应
alpha_t': alpha_t}),
264               '年份效应.csv')
265 save_csv_safe(panel[['Model', 'P', 'logP', 't', 'avg', 'type_group', '
Year']],
266               '面板回归数据.csv')
267 print('\n=== 4. 规模系数估计 (C1 开源筛选样本, %d 条, %d-%d 年) ==='
268       % (len(panel), int(panel.Year.min()), int(panel.Year.max())))
269 print('  年份固定效应模型: bP=%0.3f 分/10 倍参数, R2=%0.3f' % (bP_fe,
R2_fe))
270 print('  合并横截面 OLS: bP=%0.3f 分/10 倍参数, 年份组差=%0.3f 分, R2=%0.3f
,
271       % (bP_ols, bt_ols, R2_ols))
272 for tg in ['pretrained', 'chat/finetuned']:
273     sub = panel[panel.type_group.eq(tg)]
274     if len(sub) > 30:
275         bP_t, bt_t, R2_t = fit_model(sub, tg, False)
276         print('    %s 子样本 (n=%d): bP=%0.3f 分/10 倍参数, 年份组差=%0.3f
分, R2=%0.3f'
277               % (tg, len(sub), bP_t, bt_t, R2_t))
278
279 # 口径分层: C3 全期序列 (2019-2025, 跨年标准混合) 用于长周期分解;
280 #           C1 开源筛选样本 (2024-2025, 标准一致) 用于近端稳健性检验。
281 ts = pd.read_csv(os.path.join(C_DIR, 'leaderboard_extended_timeseries.
csv'))
282 ts['avg'] = pd.to_numeric(ts['Average'], errors='coerce')
283 ts['P'] = pd.to_numeric(ts['Params_B'], errors='coerce')
284 six_ts = ['IFEval', 'BBH', 'MATH_Lvl5', 'GPQA', 'MUSR', 'MMLU_PRO']
285 for c in six_ts:
286     ts[c] = pd.to_numeric(ts[c], errors='coerce')
287 ts['mean6'] = ts[six_ts].mean(axis=1)
288 ts['gap'] = (ts.avg - ts.mean6).abs()
289 bad = ts[ts.gap > 5]
290 ts_ok = ts[(ts.gap <= 5) & ts.P.notna() & ts.avg.notna() & (ts.P > 0)].
copy()
291 ts_ok['logP'] = np.log10(ts_ok.P)
292 ts_ok = ts_ok[np.isfinite(ts_ok.logP) & np.isfinite(ts_ok.avg)].copy()
293 ts_ok['t'] = ts_ok.Year - 2019
294 save_csv_safe(bad[['Model', 'Year', 'Average', 'mean6', 'gap']], 'C3口径
异常行.csv')
295 print('\n C3 自治性审计: |Average - 六维均值| > 5 的记录 %d 条 (跨口径
混入, 已剔除), '

```

```

296         '最大差值 %.1f (%s)' % (len(bad), bad.gap.max(),
297                                   bad.loc[bad.gap.idxmax(), 'Model'])
298
299 X1 = np.column_stack([np.ones(len(ts_ok)), ts_ok.logP.values, ts_ok.t.
300                       values])
301 bl, *_ = np.linalg.lstsq(X1, ts_ok.avg.values, rcond=None)
302 b0_l, bP_l, bt_l = bl
303 pred_l = X1 @ bl
304 R2_l = 1 - np.sum((ts_ok.avg.values - pred_l) ** 2) / \
305         np.sum((ts_ok.avg.values - ts_ok.avg.values.mean()) ** 2)
306
307 def matched_annual_frontier(records):
308     """累计最高分与其模型规模配对；同分时保留先达到的前沿记录。"""
309     rows=[];incumbent=None
310     for year,group in records.groupby('Year',sort=True):
311         candidate=group.sort_values(['avg','P','Model'],ascending=[False
312 ,True,True]).iloc[0]
313         if incumbent is None or candidate.avg>incumbent.avg:
314             incumbent=candidate
315         rows.append(dict(Year=int(year),avg=float(candidate.avg),P=float
316 (incumbent.P),
317                               cum_avg=float(incumbent.avg),frontier_model=
318 incumbent.Model,
319                               frontier_record_year=int(incumbent.Year)))
320     return pd.DataFrame(rows)
321
322 front_year = matched_annual_frontier(ts_ok)
323 front_year['t'] = front_year.Year - 2019
324 front_year['logP'] = np.log10(front_year.P)
325 save_csv_safe(front_year, '前沿序列.csv')
326
327 f1 = lb_o.groupby('Year').agg(avg=('avg', 'max'), P=('P', 'max')).
328     reset_index()
329 dF = front_year.cum_avg.iloc[-1] - front_year.cum_avg.iloc[0]
330 dLP = front_year.logP.iloc[-1] - front_year.logP.iloc[0]
331 dec = pd.DataFrame([
332     {'口径': 'C3 长周期（跨年标准混合）', '前沿区间': '%d-%d'
333     % (front_year.Year.iloc[0], front_year.Year.iloc[-1]),
334     '前沿提升分': dF, 'Δlog10P': dLP, 'bP': bP_l, 'bt(分/年)': bt_l, '
335     R2': R2_l,
336     '规模贡献分': bP_l * dLP, '非规模贡献分': dF - bP_l * dLP,
337     '规模占比%': bP_l * dLP / dF * 100, '非规模占比%': (dF - bP_l * dLP
338     ) / dF * 100}])
339 save_csv_safe(dec, '规模时间分解_长周期.csv')
340 print('\n 前沿序列（年度最高分 + 累计最大, %d-%d）: '
341       % (front_year.Year.iloc[0], front_year.Year.iloc[-1]))
342 print(front_year.round(2).to_string(index=False))
343 print(' 规模关联项与未解释剩余项核算（CSV 沿用历史列名）: ')
344 print(dec.round(2).to_string(index=False))
345 print(' 提示: C3 早期年份为公开报告值（每 1-7 条），跨年评测标准不完全
346     一致（C3 标注'
347     '为“混合”口径），故长周期占比视为量级参考；C1 近端口径标准一致但
348     窗口仅约 1 年。')
349
350 # ===== 5. 能力前沿的 logistic 外推 =====

```

```

342 # 前沿定义：每个时间窗内开源筛选模型平均分的上侧分位（月度取前 1%，历史
    年度取最高分）
343 lb_o['ym'] = lb_o.date.dt.to_period('M')
344 mon_top1 = lb_o.groupby('ym').apply(
345     lambda g: g.avg.nlargest(max(1, int(len(g) * 0.01))).mean(),
    include_groups=False)
346 mon_top1 = mon_top1[mon_top1.index >= pd.Period('2024-01')]
347 t_mon = np.array([(pd.Timestamp(str(p)).year - 2019) + (pd.Timestamp(str
    (p)).month - .5) / 12
    for p in mon_top1.index])
348
349
350 # 前沿边界单调不减：历史段取 C3 累计最大，近端取月度前 1% 的累计最大
351 hist_rows = front_year[front_year.Year <= 2023]
352 t_h = hist_rows.t.values + .5
353 s_h = np.maximum.accumulate(hist_rows.cum_avg.values)
354 t_front = np.r_[t_h, t_mon]
355 s_front = np.r_[s_h, np.maximum.accumulate(mon_top1.values)]
356 idx = np.argsort(t_front)
357 t_front, s_front = t_front[idx], np.maximum.accumulate(s_front[idx])
358
359 # 近端口径（一致窗口）内的前沿分解：从首月到峰值月
360 def month_stats(period):
361     g = lb_o[lb_o.date.dt.to_period('M').eq(period)]
362     g = g.nlargest(max(1, int(len(g) * 0.01)), 'avg')
363     # 同一组模型的均分与平均对数规模配对；返回几何均值用于原对数比公式。
364     return float(g.avg.mean()), float(10**np.log10(g.P).mean())
365
366 p0, pk = mon_top1.index[0], mon_top1.idxmax()
367 F0_, P0_ = month_stats(p0)
368 Fk_, Pk_ = month_stats(pk)
369 dF1 = Fk_ - F0_
370 dLP1 = float(np.log10(Pk_ / P0_))
371 dec1 = pd.DataFrame([
372     '口径': 'C1 近端（一致窗口 %s->%s, 前沿模型 %sB->%sB) '
373     % (str(p0), str(pk), ('%.1f' % P0_), ('%.1f' % Pk_)),
374     '前沿区间': '%s->%s' % (str(p0), str(pk)),
375     '前沿提升分': dF1, 'Δlog10P': dLP1, 'bP': bP_fe, 'bt(分/年)': bt_ols
    , 'R2': R2_fe,
376     '规模贡献分': bP_fe * dLP1, '非规模贡献分': dF1 - bP_fe * dLP1,
377     '规模占比%': bP_fe * dLP1 / dF1 * 100 if dF1 != 0 else np.nan,
378     '非规模占比%': (dF1 - bP_fe * dLP1) / dF1 * 100 if dF1 != 0 else np.
    nan]])
379 dec_all = pd.concat([dec, dec1], ignore_index=True)
380 save_csv_safe(dec_all, '规模时间分解.csv')
381 print('\n 近端口径（一致窗口）分解：前沿 %s->%s 由 %.2f 升至 %.2f (+%.2
    f 分)，'
382     '前列组参数几何均值 %sB->%sB (Δlog10P=%.3f) '
383     % (str(p0), str(pk), F0_, Fk_, dF1, ('%.1f' % P0_), ('%.1f' % Pk_)
    , dLP1))
384 print(' -> 规模关联项 %.2f 分，未解释剩余项 %.2f 分；剩余项不能识别技术
    进步的因果份额。'
385     % (bP_fe * dLP1, dF1 - bP_fe * dLP1))
386
387 def logistic(t, K, r, t0):
388     return K / (1 + np.exp(-r * (t - t0)))
389

```

```

390 t_last = float(np.max(t_mon))
391 t_pred = np.array([t_last + 1.0, t_last + 2.0])      # 严格对应未来 12 /
    24 个月
392 try:
393     popt, _ = curve_fit(logistic, t_front, s_front, p0=[55, .8, 3.5],
394                         bounds=([20, .01, 0.5], [100, 5, 8]), maxfev
    =40000)
395     K_fit, r_fit, t0_fit = popt
396     pred_fit = logistic(t_front, *popt)
397     R2_log = 1 - np.sum((s_front - pred_fit) ** 2) / np.sum((s_front -
    s_front.mean()) ** 2)
398 except Exception as e:
399     K_fit, r_fit, t0_fit, R2_log = 52., .3, 3., 0.
400     print('logistic 拟合失败: ', e)
401 s_pred = logistic(t_pred, K_fit, r_fit, t0_fit)
402
403 resid = s_front - logistic(t_front, K_fit, r_fit, t0_fit)
404 rng = np.random.default_rng(42)
405 boot = []
406 for _ in range(600):
407     s_b = logistic(t_front, K_fit, r_fit, t0_fit) + rng.choice(resid,
    size=len(resid), replace=True)
408     try:
409         pb, _ = curve_fit(logistic, t_front, s_b, p0=[K_fit, r_fit,
    t0_fit], maxfev=40000)
410         boot.append(logistic(t_pred, *pb))
411     except Exception:
412         pass
413 boot = np.array(boot)
414 lo, hi = np.percentile(boot, 5, axis=0), np.percentile(boot, 95, axis=0)
415
416 # 条件情景：以 logistic 增长参数减半表征放缓，不估计算力到增长率的传导。
417 def logistic_slow(t, K, r, t0, halve_from):
418     """减速前沿用原曲线，减速后增长参数为 r/2，连接点连续。"""
419     t = np.asarray(t, dtype=float)
420     effective_t = np.minimum(t, halve_from) + 0.5 * np.maximum(t -
    halve_from, 0.0)
421     return logistic(effective_t, K, r, t0)
422 s_slow = logistic_slow(t_pred, K_fit, r_fit, t0_fit, t_last)
423
424 fut = pd.DataFrame({'t(2019起算年)': t_pred, '对应日历时点':
425                     ['%d-%02d' % (2019 + int(np.floor(tt)), int(round((
    tt % 1) * 12)) + 1)
426                     for tt in t_pred],
427                     'frontier_pred': s_pred,
428                     'CI_low_5%': lo, 'CI_high_95%': hi, '情景_技术增速减
    半': s_slow})
429 save_csv_safe(fut, '前沿预测.csv')
430
431 # 结构外推：F(t+Δ) = F(t) + [bP·Δlog10P + bt]·Δ（把分解式直接用作预测方
    程）
432 F_last = float(np.max(s_front))
433 scen = []
434 for yrs in [1, 2]:
435     # 情景名和列名沿用历史 CSV 接口；实际假设为把年份组差逐年累加或减
    半，不估计算力传导。

```

```

436     for dlogP, bt_use, lbl in [(0.0, bt_ols, '基准（前沿模型规模持平，非
    规模增速不变）'),
437                               (0.0, bt_ols / 2, '算力增长放缓（非规模增
    速减半）'),
438                               (0.3, bt_ols, '规模继续扩张（前沿 +0.3
    dex/年）')]:
439         scen.append({'情景': lbl, '前瞻年数': yrs,
440                     'Δlog10P假设': dlogP, 'bt假设(分/年)': bt_use,
441                     '结构外推前沿': F_last + (bP_fe * dlogP + bt_use) *
                        yrs})
442     scen_df = pd.DataFrame(scen)
443     save_csv_safe(scen_df, '前沿预测_结构外推情景.csv')
444     save_csv_safe(pd.DataFrame([{'K': K_fit, 'r': r_fit, 't0(2019起)':
    t0_fit, 'R2': R2_log,
445                                 '数据截止': str(max(lb_o.date)[:10], 'n前
    沿点': len(t_front),
446                                 't_last': t_last}])),
447                    'logistic参数.csv')
448     print('\n=== 5. 能力前沿 logistic 外推 ===')
449     print('    F(t)=K/(1+exp(-r(t-t0))): K=%.2f, r=%.3f, t0=%.2f (2019 起
    算), R2=%.3f'
450           % (K_fit, r_fit, t0_fit, R2_log))
451     print(fut.round(2).to_string(index=False))
452
453     # ===== 6. C4 宏观证据（算力、数据量、开源权重） =====
454     ep = pd.read_csv(os.path.join(C_DIR, 'epoch_all_ai_models.csv'),
    low_memory=False)
455     ep['compute'] = pd.to_numeric(ep['Training compute (FLOP)'].astype(str).
    str.replace(',', '' ),
456                                 errors='coerce')
457     ep['dsize'] = pd.to_numeric(ep['Training dataset size (total)'].astype(
    str).str.replace(',', '' ),
458                                 errors='coerce')
459     ep['P'] = pd.to_numeric(ep['Parameters'].astype(str).str.replace(',', ''
    ), errors='coerce')
460     ep['year'] = pd.to_datetime(ep['Publication date'], errors='coerce').dt.
    year
461     ep['open_w'] = ep['Open model weights?'].astype(str).str.strip().eq('Yes
    ')
462
463     c4 = ep[(ep.Domain.astype(str).str.contains('Language', na=False)) &
464             ep.year.between(2018, 2026)].copy()
465     g = c4.groupby('year').agg(n=('Model', 'size'),
466                               n_compute=('compute', 'count'),
467                               前沿算力_max=('compute', 'max'),
468                               前沿算力_中位=('compute', 'median'),
469                               最大参数量_B=('P', 'max'),
470                               最大数据量_tokens=('dsize', 'max'),
471                               开源权重占比=('open_w', 'mean')).reset_index
    ()
472     save_csv_safe(g, 'C4宏观趋势.csv')
473     print('\n=== 6. C4 宏观证据（Epoch AI 全模型元数据, Language 域）===')
474     print(g.round(3).to_string(index=False))
475     gc = g.dropna(subset=['前沿算力_max'])
476     gc = gc[gc.前沿算力_max > 0]
477     sl_c, ic_c = np.polyfit(gc.year, np.log10(gc.前沿算力_max), 1)

```

```

478 print('  前沿训练算力的年增长率: log10(C) 每年 +%.3f 个数量级 (即约 %.2f
      倍/年) '
479       % (sl_c, 10 ** sl_c))
480 ov = ep['Open model weights?'].astype(str).str.strip()
481 print('  C4 开源权重字段分布: ' + ';' .join('%s=%d' % (k, v)
482       for k, v in ov.value_counts().head(4).items()))
483 save_csv_safe(pd.DataFrame([{'log10前沿算力年增速': sl_c, '年倍数': 10 *
      * sl_c,
484                               '开源权重_Yes': int(ov.eq('Yes').sum()),
485                               '开源权重_No': int(ov.eq('No').sum()),
486                               '有算力记录行数': int(ep.compute.notna().
      sum()),
487                               '有数据量记录行数': int(ep.dsize.notna().
      sum())}]),
488          'C4宏观摘要.csv')
489
490 # ===== 7. 六维 Benchmark 前沿与剩余空间 (开源筛选集)
      =====
491 task_rows = []
492 for tk in TASKS:
493     s = lb_o[tk].dropna()
494     task_rows.append({'task': tk, 'n': len(s), 'max': float(s.max()),
495                      'top1_mean': float(s.nlargest(max(1, int(len(s) *
      0.01))).mean()),
496                      'median': float(s.median()),
497                      'headroom_%': float((100 - s.max()))})
498 task_df = pd.DataFrame(task_rows).sort_values('max')
499 save_csv_safe(task_df, '逐任务聚合.csv')
500 print('\n=== 7. 六维 Benchmark 前沿与剩余空间 (开源筛选集, n=%d) ===' %
      len(lb_o))
501 print(task_df.to_string(index=False))
502
503 # 正式图表由文件末尾的独立绘图入口生成。
504
505 # ===== 9. 汇总 =====
506 d0 = dec.iloc[0]
507 print('\n===== 问题四求解完成 =====')
508 print('  桥接: 全样本 r=%.3f; 核算 (%s): 规模关联 %.1f%% / 未解释剩余
      %.1f%%; '
509       '近端: 规模关联 %.1f%% / 未解释剩余 %.1f%%'
510       % (bridge_df.iloc[2]['Pearson_r'], d0['前沿区间'], d0['规模占比%']
      ], d0['非规模占比%'],
511          dec_all.iloc[1]['规模占比%'], dec_all.iloc[1]['非规模占比%']))
512 print('  前沿预测 (+12/+24 个月): %s %.2f (90%%CI %.2f-%.2f); %s %.2f
      (90%%CI %.2f-%.2f); '
513       'logistic 增长参数减半情景 %.2f / %.2f'
514       % (fut['对应日历时点'].iloc[0], fut.frontier_pred.iloc[0], fut['
      CI_low_5%'].iloc[0],
515          fut['CI_high_95%'].iloc[0], fut['对应日历时点'].iloc[1], fut.
      frontier_pred.iloc[1],
516          fut['CI_low_5%'].iloc[1], fut['CI_high_95%'].iloc[1],
517          fut['情景_技术增速减半'].iloc[0], fut['情景_技术增速减半'].iloc
      [1]))
518 print('\n  结构外推情景 (基准前沿 F=%.2f): ' % F_last)
519 print(scen_df.round(2).to_string(index=False))

```

```

520 print('  两条路线用于比较预测假设，不构成独立验证。logistic 的重抽样区间
      见上表； '
521       '结构外推给出的是情景点值，其范围不能作为置信区间。')
522
523 # 模型结果全部保存后生成正文采用的正式图。
524 import subprocess as _subprocess
525 _subprocess.run([sys.executable, os.path.join(BASE_DIR, '重绘图表.py'),
      '--data', DATA_DIR], check=True)

```

公共模块、非线性预测与完整绘图程序

程序 5: 统一复现入口（求解/复现论文.py）

```

1 # 本程序的整理与核对使用 Codex 辅助；模型：GPT-5.6-Luna；机构：OpenAI；
  发布日期：2026-07-09。
2 # 本程序由 OpenAI Codex（GPT-5.6-Luna）辅助实现，用于统一执行正式绘图与
  隔离复核。
3 """在仓库根目录执行：
4 python 求解/复现论文.py --data-dir <附件根目录> --mode figures
5 python 求解/复现论文.py --data-dir <附件根目录> --mode verify-models
6
7 figures: 读取已存结果、重绘正文全部图。verify-models: 在临时副本运行完整
  四问，
8 按复现预期输出.json逐一比较重新生成的CSV/JSON；不会回写原结果。
9 """
10 from pathlib import Path
11 import argparse
12 import hashlib
13 import json
14 import os
15 import re
16 import shutil
17 import subprocess
18 import sys
19 import tempfile
20 import numpy as np
21 import pandas as pd
22
23 ROOT = Path(__file__).resolve().parents[1]
24
25 def sha(path):
26     return hashlib.sha256(path.read_bytes()).hexdigest()
27
28 def snapshots(root):
29     return {p.relative_to(root).as_posix(): sha(p) for p in (root/'求解'
30     ).rglob('*')
31     if p.is_file() and p.suffix in ('.csv', '.json') and '图片'
32     not in p.parts
33     and '__pycache__' not in p.parts}
34
35 def run(root, script, args, data):
36     env = dict(os.environ, MODELING_DATA_DIR=str(data), PYTHONIOENCODING
37     ='utf-8')
38     print(f'执行: {script}', flush=True)

```

```

36     subprocess.run([sys.executable, str(root/script), *map(str,args)],
37                     cwd=root, env=env, check=True)
38 def figure_map(root):
39     # 从真实主入口递归读取；旧拆分稿和注释不参与清单。
40     visited = set(); rows = []
41     def walk(p):
42         p=p.resolve()
43         if p in visited: return
44         visited.add(p)
45         text='\n'.join(re.split(r'(?<!\\\)%',line)[0] for line in p.
read_text(encoding='utf-8').splitlines())
46         for name in re.findall(r'\\(?:input|include)\{([^\}]+\)\}',text):
47             target=root/'论文'/name
48             if not target.suffix: target=target.with_suffix('.tex')
49             if target.exists(): walk(target)
50         for name in re.findall(r'\\includegraphics(?:\\[[^\]]*\])
?\\{([^\}]+\)\}',text):
51             target=(root/'论文'/name).resolve()
52             if not target.is_file(): raise FileNotFoundError(target)
53             rel=target.relative_to(root).as_posix()
54             if '/问题一/' in rel: source='求解/问题一/重绘图表.py'
55             elif '/问题二/' in rel: source='求解/问题二/重绘图表.py'
56             elif '/问题三/' in rel: source='求解/问题三/论文绘图.py'
57             elif '/问题四/' in rel: source='求解/问题四/重绘图表.py'
58             elif 'workflow' in rel: source='求解/绘制问题三四流程图.py'
59             else: source='模板静态资源'
60             rows.append(dict(tex=p.relative_to(root).as_posix(), figure=
rel, source=source, sha256=sha(target)))
61         walk(root/'论文/论文.tex')
62         rows.append(dict(tex='论文/2. 总体分析.tex',figure='论文/figures/
overall-architecture.tex',
63                         source='原生TikZ, 随论文编译',sha256=sha(root/'论文
/figures/overall-architecture.tex'))
64         return rows
65
66 def figures(root, data):
67     before=snapshots(root)
68     for script,args in [
69         ('求解/问题一/重绘图表.py',['--data-dir',data]),
70         ('求解/问题二/重绘图表.py',['--data-dir',data]),
71         ('求解/问题三/论文绘图.py',[]),
72         ('求解/问题四/重绘图表.py',['--data',data]),
73         ('求解/绘制问题三四流程图.py',[]),
74     ]: run(root,Path(script),args,data)
75     if before != snapshots(root): raise RuntimeError('绘图修改了结果或跨
问接口')
76     return figure_map(root)
77
78 def compare_csv(a,b,atol=1e-8):
79     x,y=pd.read_csv(a),pd.read_csv(b)
80     if x.shape != y.shape or list(x.columns) != list(y.columns):
81         return dict(equal=False,reason='shape/columns differ',
saved_shape=list(x.shape),new_shape=list(y.shape))
82     # 数值相同的零份额可能有不同稳定排序；按明确唯一的业务键对齐。

```

```

83     for key in ['index', 'domain', 'training_domain', 'loss_domain', 'param'
84 ]:
85         if key in x and not x[key].duplicated().any() and not y[key].
duplicated().any():
86             x=x.sort_values(key).reset_index(drop=True)
87             y=y.sort_values(key).reset_index(drop=True)
88             break
89     bad=[]
90     for c in x:
91         if pd.api.types.is_numeric_dtype(x[c]) and pd.api.types.
is_numeric_dtype(y[c]):
92             if not np.allclose(x[c],y[c],rtol=1e-6,atol=atol,equal_nan=
True): bad.append(c)
93             elif not x[c].fillna('<NA>').astype(str).equals(y[c].fillna('<NA
>').astype(str)):
94                 bad.append(c)
95     return dict(equal=not bad,columns_differ=bad)
96
97 def compare_json(a, b, ignored_keys=()):
98     """数值容差与 CSV 一致；仅忽略清单中明示的顶层运行环境字段。"""
99     x, y = [json.loads(p.read_text(encoding='utf-8-sig')) for p in (a, b
)]
100     ignored = {}
101     for key in ignored_keys:
102         if key not in x or key not in y:
103             return dict(equal=False, reason=f'ignored metadata field
missing: {key}')
104         ignored[key] = dict(saved=x.pop(key), regenerated=y.pop(key))
105     bad = []
106     def walk(left, right, location='$'):
107         if isinstance(left, dict) and isinstance(right, dict):
108             # 输入路径分隔符随操作系统变化，但文件及其哈希必须相同。
109             if location == '$.inputs':
110                 left = {k.replace('\\', '/'): v for k, v in left.items()}
111                 right = {k.replace('\\', '/'): v for k, v in right.items
()}
112             if left.keys() != right.keys():
113                 bad.append(location + ': keys differ')
114             for key in left.keys() & right.keys():
115                 walk(left[key], right[key], location + '.' + key)
116             elif isinstance(left, list) and isinstance(right, list):
117                 if len(left) != len(right): bad.append(location + ': length
differs')
118                 for i, (u, v) in enumerate(zip(left, right)): walk(u, v, f'{
location} [{i}]')
119             elif isinstance(left, (int, float)) and not isinstance(left,
bool) and isinstance(right, (int, float)) and not isinstance(right,
bool):
120                 if not np.isclose(left, right, rtol=1e-6, atol=1e-8,
equal_nan=True): bad.append(location)
121                 elif type(left) is not type(right) or left != right:
122                     bad.append(location)
123                 walk(x, y)
124             return dict(equal=not bad, fields_differ=bad, metadata_not_compared=
ignored)
125
126 def check_kernel_certificate(root, data):

```

```

126     """凸组合表示不唯一：核对其数学含义，不要求权重逐项重现。"""
127     folder=root/'求解/问题一/结果'
128     train=pd.read_csv(data/'A_data_value/regmix_tables/train_mixture_1m.
csv').set_index('index').sort_index()
129     columns=[c for c in train if c.startswith('train_the_pile_')]
130     domains=[c.removeprefix('train_the_pile_') for c in columns]
131     raw=train[columns].to_numpy(float)
132     if (not columns or not len(raw) or train.index.has_duplicates or
train.index.isna().any()
133         or not np.isfinite(raw).all() or (raw<0).any() or (raw.sum(
axis=1)<=0).any()):
134         return dict(equal=False,reason='训练配方存在重复索引、非有限数或
无效份额')
135     closed=raw/raw.sum(axis=1,keepdims=True)
136     ref=raw.mean(axis=0);ref/=ref.sum()
137     certificate=pd.read_csv(folder/'核岭推荐_训练凸组合.csv').set_index(
'index')
138     if certificate.index.has_duplicates or set(certificate.index)!=set(
train.index):
139         return dict(equal=False,reason='训练凸组合 index 缺失或重复')
140     z=certificate.loc[train.index,'convex_weight'].to_numpy(float)
141     recipe=pd.read_csv(folder/'推荐配比调整.csv').set_index('domain').
loc[domains]
142     p=recipe.recommended_mixture.to_numpy(float)
143     # 先检查所有输入，再做 max: Python max 可能忽略后续位置的 NaN。
144     reference=recipe.reference_mixture.to_numpy(float)
145     if recipe.index.has_duplicates or not all(np.isfinite(x).all() for x
in [closed,ref,z,p,reference]):
146         return dict(equal=False,reason='凸组合或配方字段包含非有限数、重
复领域')
147     loss_columns=pd.read_csv(data/'A_data_value/regmix_tables/
train_pile_loss_1m.csv',nrows=0).columns
148     measured=[c.removeprefix('metric/the_pile_').removesuffix('_val_loss
') for c in loss_columns if c.endswith('_val_loss')]
149     fixed=[i for i,d in enumerate(domains) if d not in measured]
150     residual=max(abs(z.sum()-1),max(0.,-z.min()),float(np.max(abs(
z@closed-p))),
151                 abs(p.sum()-1),float(np.max(abs(p[fixed]-ref[fixed]))),
152                 float(np.maximum(p-2.5*ref,0).max()),float(np.max(abs(
reference-ref))))
153     return dict(equal=bool(np.isfinite(residual) and np.isfinite(z).all
() and np.isfinite(p).all() and residual<1e-7),
154                 validation='nonnegative convex reconstruction and recipe
constraints',max_residual=float(residual))
155
156 def check_kernel_runs(root):
157     """检查四个初值的记录；保护梯度间隙不构成边界光滑驻点证明。"""
158     folder=root/'求解/问题一/结果'
159     runs=pd.read_csv(folder/'核岭推荐_多初值.csv')
160     required={'start','success','objective','constraint_residual','
iterations','stationarity_gap'}
161     if not required.issubset(runs.columns) or len(runs)!=4:
162         return dict(equal=False,reason='多初值记录必须包含完整字段及四行
数据')
163     if not pd.api.types.is_bool_dtype(runs.success) or runs.success.isna
().any():
164         return dict(equal=False,reason='success 必须是无缺失的布尔值')

```

```

165     numeric=['start','objective','constraint_residual','iterations','
stationarity_gap']
166     if any(not pd.api.types.is_numeric_dtype(runs[c]) or pd.api.types.
is_bool_dtype(runs[c]) for c in numeric):
167         return dict(equal=False,reason='多初值数值字段类型错误')
168     if not np.isfinite(runs[numeric].to_numpy(float)).all():
169         return dict(equal=False,reason='多初值数值字段包含非有限数')
170     if set(runs.start)!={0,1,2,3} or runs.start.duplicated().any():
171         return dict(equal=False,reason='start 必须为互异的 0、1、2、3')
172     if ((runs.iterations<1)|(runs.iterations>150)|(runs.iterations!=np.
floor(runs.iterations))).any():
173         return dict(equal=False,reason='外层迭代次数必须是 1 至 150 的整
数')
174     targets=pd.read_csv(folder/'核岭推荐_目标对照.csv')
175     if not {'mixture','predicted_weighted_loss'}.issubset(targets.
columns):
176         return dict(equal=False,reason='核岭目标对照缺少字段')
177     target_rows=targets.loc[targets.mixture.eq('问题一有界推荐'),'
predicted_weighted_loss']
178     if len(target_rows)!=1 or not pd.api.types.is_numeric_dtype(
target_rows):
179         return dict(equal=False,reason='核岭推荐目标必须为唯一数值')
180     target=float(target_rows.iloc[0])
181     if not np.isfinite(target):
182         return dict(equal=False,reason='核岭推荐目标不是有限数')
183     good=runs[runs.success.eq(True)]
184     # 精确线性子问题的间隙非负。仅容许 1e-12 的负舍入记录,
185     # 比原 1e-6 停止阈值小六个数量级;这不是梯度有效性或驻点证明。
186     negative_gap_roundoff=1e-12
187     ok=(len(good)>0 and (runs.constraint_residual>=0).all()
and (runs.constraint_residual<1e-7).all()
188         and (runs.stationarity_gap>=-negative_gap_roundoff).all()
189         and (good.stationarity_gap<1e-6).all()
190         and abs(good.objective.min()-target)<1e-7 and runs.objective.max
()-runs.objective.min())<1e-6)
191     return dict(equal=bool(ok),validation='feasibility, recorded
protected-gradient gap threshold and objective agreement',
192         successful_starts=len(good),negative_gap_roundoff=
negative_gap_roundoff,
193         limitation='recorded protected-gradient diagnostic only;
no smooth-stationarity or global-optimality certificate')
194
195 def compare_expected_output(source, work, item, data):
196     """一般输出按数值比较;核岭非唯一证书沿用主线的可行性核验。"""
197     rel = item['path']
198     policy = item.get('comparison', 'numeric')
199     checks = {'kernel_certificate': check_kernel_certificate, '
kernel_runs': check_kernel_runs}
200     if policy in checks:
201         saved = checks[policy](source, data) if policy == '
kernel_certificate' else checks[policy](source)
202         regenerated = checks[policy](work, data) if policy == '
kernel_certificate' else checks[policy](work)
203         return dict(equal=bool(saved['equal'] and regenerated['equal']),
comparison=policy,
204             saved_validation=saved, regenerated_validation=
regenerated)

```

```

206     if item['kind'] == 'csv':
207         atol = item.get('atol', 1e-8)
208         result = compare_csv(source/rel, work/rel, atol=atol)
209         result.update(comparison='numeric', rtol=1e-6, atol=atol)
210         return result
211     return compare_json(source/rel, work/rel, item.get('ignore_top_level', []))
212
213 def checked_output(root, relative):
214     """只接受隔离副本求解目录内的相对文件；不允许越界或符号链接。"""
215     rel = Path(relative)
216     if rel.is_absolute() or '..' in rel.parts or rel.parts[:1] != ('求解',):
217         raise ValueError(f'非法预期输出路径: {relative}')
218     target = root / rel
219     if not target.resolve().is_relative_to((root / '求解').resolve()):
220         raise ValueError(f'预期输出路径越界: {relative}')
221     if any(p.is_symlink() for p in [target, *target.parents] if p != root.parent):
222         raise ValueError(f'预期输出不得经过符号链接: {relative}')
223     return target
224
225 def verify_models(root, data, report_dir, *, full_scope=True):
226     """旧结果先从副本移除，要求每项重新生成并与基准比较；失败也保存报告。"""
227     report_dir.mkdir(parents=True, exist_ok=True)
228     before = snapshots(root)
229     report = dict(passed=False, expected_count=0, checked_count=0, outputs={}, commands=[])
230     try:
231         manifest = json.loads((root/'求解/复现预期输出.json').read_text(encoding='utf-8'))
232         if not isinstance(manifest, dict) or not manifest:
233             raise ValueError('预期输出清单不能为空')
234         if full_scope and set(manifest) != {'问题一', '问题二', '问题三', '问题四'}:
235             raise ValueError('正式复现清单必须覆盖全部四问')
236         seen = set()
237         for name, entries in manifest.items():
238             if name not in ['问题一', '问题二', '问题三', '问题四'] or not entries:
239                 raise ValueError(f'无效的问题或空清单: {name}')
240             for item in entries:
241                 rel = item['path']
242                 source = checked_output(root, rel)
243                 if rel in seen or item['kind'] not in ('csv', 'json'):
244                     raise ValueError(f'重复路径或无效格式: {rel}')
245                 if not source.is_file(): raise FileNotFoundError(f'缺少比较基准: {rel}')
246                 policy = item.get('comparison', 'numeric')
247                 special = {
248                     'kernel_certificate': '求解/问题一/结果/核岭推荐_训练凸组合.csv',
249                     'kernel_runs': '求解/问题一/结果/核岭推荐_多初值.csv',
250                 }

```

```

251         if policy != 'numeric' and special.get(policy) != rel:
252             raise ValueError(f'无效的输出生比较规则: {rel}: {
policy}')
253         if 'atol' in item and (rel != '求解/问题一/结果/推荐配比
调整.csv' or item['atol'] != 1e-6):
254             raise ValueError(f'未约定的数值容差: {rel}')
255             seen.add(rel)
256         if full_scope:
257             # CLI 自身检查覆盖, 不能依赖使用者先运行单元测试。
258             required = {p.relative_to(root).as_posix() for name in
manifest
259                 for p in (root/'求解'/name/'结果').iterdir()
260                 if p.is_file() and p.suffix in ('.csv', '.json')}
261             required.update({'求解/17域质量映射.csv', '求解/问题一_关键
量.json', '求解/广义标度律参数.csv'})
262             if seen != required:
263                 raise ValueError(f'预期输出清单覆盖不完整: missing={
sorted(required-seen)}, extra={sorted(seen-required)}')
264             report['expected_count'] = len(seen)
265             work = Path(tempfile.mkdtemp(prefix='model-check-', dir=
report_dir))
266             report['workspace'] = str(work)
267             shutil.copytree(root/'求解', work/'求解', ignore=shutil.
ignore_patterns('__pycache__', '图片', '*.ipynb'))
268             if (root/'论文/fonts').exists(): shutil.copytree(root/'论文/
fonts', work/'论文/fonts')
269             for name, entries in manifest.items():
270                 # 删除只发生在隔离副本, 且逐项验证其绝对路径仍在副本求解目录
内。
271                 for item in entries:
272                     target = checked_output(work, item['path'])
273                     target.unlink(missing_ok=True)
274                     script = Path('求解')/name/(name+'.py')
275                     command = dict(script=script.as_posix(), status='running')
276                     report['commands'].append(command)
277                     try:
278                         run(work, script, [], data)
279                         command['status'] = 'completed'
280                     except Exception as exc:
281                         command.update(status='failed', error=f'{type(exc)}.
__name__: {exc}')
282                     raise
283                 for item in entries:
284                     rel = item['path']; target = checked_output(work, rel)
285                     if not target.is_file():
286                         result = dict(equal=False, regenerated=False, reason
='预期输出未生成或缺失')
287                     else:
288                         result = compare_expected_output(root, work, item,
data)
289                         result.update(regenerated=True, saved_sha256=sha(
root/rel), new_sha256=sha(target))
290                         report['checked_count'] += 1
291                         report['outputs'][rel] = result
292                 # 数值差异先记录, 让可运行的后续问也得到检查; 最终仍整体失败。

```

```

293     # 进程异常（例如上游缺文件）由上面的异常分支及时报告。
294     failed = [rel for rel, result in report['outputs'].items() if
not result['equal']]
295     if failed: raise RuntimeError('复现输出缺失或结果不一致: ' + ',
'.join(failed))
296     if report['checked_count'] != report['expected_count'] or report
['checked_count'] == 0:
297         raise RuntimeError('未完整核对预期输出')
298     report['passed'] = True
299 except Exception as exc:
300     report['error'] = f'{type(exc).__name__}: {exc}'
301     raise
302 finally:
303     report['source_results_unchanged'] = before == snapshots(root)
304     if not report['source_results_unchanged']: report['passed'] =
False
305     (report_dir/'模型复现比较.json').write_text(json.dumps(report,
ensure_ascii=False, indent=2), encoding='utf-8')
306     if not report['source_results_unchanged']: raise RuntimeError('源结
果发生变化')
307     print(f'已完整核对 {report["checked_count"]} 项重新生成的 CSV/JSON;
原结果未改动。')
308     return report
309
310 def main():
311     ap=argparse.ArgumentParser(description=__doc__)
312     ap.add_argument('--data-dir', type=Path, required=True)
313     ap.add_argument('--mode', choices=['figures', 'verify-models'], default
='figures')
314     ap.add_argument('--report-dir', type=Path, default=R00T/'复现记录')
315     args=ap.parse_args(); data=args.data_dir.resolve()
316     if not (data/'A_data_value').is_dir(): raise FileNotFoundError(data)
317     args.report_dir.mkdir(parents=True, exist_ok=True)
318     if args.mode=='figures':
319         rows=figures(R00T, data)
320         (args.report_dir/'图表来源.json').write_text(json.dumps(rows,
ensure_ascii=False, indent=2), encoding='utf-8')
321         print(f'{len(rows)} 项图形资源已对应到源码; 结果文件未改动。')
322         return
323     verify_models(R00T, data, args.report_dir)
324
325 if __name__=='__main__':
326     if hasattr(sys.stdout, 'reconfigure'): sys.stdout.reconfigure(
encoding='utf-8')
327     main()

```

程序 6: 决策、质量结构与时间外推补充检验（求解/补充检验.py）

```

1 # 本补充检验程序使用 OpenAI Codex 辅助编写与核对。
2 # 用途：检验保存结果的决策含义、跨问假设和时间外推；不改写正式求解结果。
3 """运行：python 求解/补充检验.py --data-dir <附件根目录>
4 输出写入复现记录/补充检验；核岭超参数沿用保存值，不重新搜索。
5 两组检验依次执行，后一组读取前一组重建的前沿序列。
6 """
7 from pathlib import Path
8 import argparse, hashlib, json
9 import numpy as np

```

```

10 import pandas as pd
11 from scipy.linalg import solve
12 from scipy.optimize import linprog, curve_fit, least_squares, brentq,
    minimize_scalar, minimize
13 from scipy.spatial.distance import cdist
14 from scipy.stats import spearmanr
15
16 def check_q1_q4(REPO, DATA, OUT):
17     report = {}
18     def digest(p): return hashlib.sha256(p.read_bytes()).hexdigest()
19     before = {str(p): digest(p) for p in (REPO/'求解').rglob('*') if p.
        is_file() and p.suffix in ('.csv', '.json')}
20
21     # Q1: evaluate the exact saved recommendation objective, without
    hyperparameter search.
22     a_dir = DATA/'A_data_value/regmix_tables'
23     mix = pd.read_csv(a_dir/'train_mixture_lm.csv').sort_values('index')
24     loss = pd.read_csv(a_dir/'train_pile_loss_lm.csv').sort_values('
        index')
25     assert np.array_equal(mix['index'], loss['index'])
26     mc = [c for c in mix if c.startswith('train_the_pile_')]
27     lc = [c for c in loss if c.endswith('_val_loss')]
28     domains = [c.removeprefix('train_the_pile_') for c in mc]
29     ld = [c.removeprefix('metric/the_pile_').removesuffix('_val_loss')
        for c in lc]
30     xraw = mix[mc].to_numpy(float); x = xraw/xraw.sum(1,keepdims=True);
    y = loss[lc].to_numpy(float)
31     rec = pd.read_csv(REPO/'求解/问题一/结果/线性有界配对比照.csv').
    set_index('domain').loc[domains]
32     ref = rec.reference_mixture.to_numpy(); proposed = rec.
    recommended_mixture.to_numpy()
33     assert np.allclose(ref,xraw.mean(0),atol=1e-14)
34     weights = ref[[domains.index(d) for d in ld]]; weights /= weights.
    sum()
35     saved = pd.read_csv(REPO/'求解/问题一/结果/非线性代理对照_预测明细.
    csv')
36     ranking = []
37     for (scope,model), rows in saved.groupby(['scope','model']):
38         obs = rows.pivot(index='index',columns='loss_domain',values='
            observed')[ld]
39         pred = rows.pivot(index='index',columns='loss_domain',values='
            predicted')[ld]
40         ov, pv = obs.to_numpy()@weights,pred.to_numpy()@weights
41         pick = int(np.argmax(pv)); ranks=pd.Series(ov).rank(method='min'
            ).to_numpy()
42         ranking.append(dict(scope=scope,model=model,n=len(ov),spearman=
            float(spearmanr(ov,pv).statistic),
43                             selected_index=int(obs.index[pick]),
44                             selected_observed_rank=int(ranks[pick]),
45                             selected_observed=float(ov[pick]),
46                             best_observed=float(ov.min()),
47                             selection_regret=float(ov[pick]-ov.min()))
48     pd.DataFrame(ranking).to_csv(OUT/'q1_same_objective_ranking.csv',
    index=False,encoding='utf-8-sig')
49     manifest=json.loads((REPO/'求解/问题一/结果/非线性代理对照_复现清单.
    json').read_text('utf-8-sig'))
50     g,l=manifest['selected_gamma'],manifest['selected_lambda']

```

```

49     kernel=lambda u,v:np.exp(-g*cdist(np.sqrt(u),np.sqrt(v),'sqeuclidean')
50     k=kernel(x,x);means=k.mean(0);grand=k.mean();kc=k-means[None,:]-
means[:,None]+grand
51     coef=solve(kc+l*np.eye(len(x)),y-y.mean(0),assume_a='pos')
52     def kpredict(v):
53         v=v/v.sum(1,keepdims=True); kv=kernel(v,x)
54         return (kv-kv.mean(1,keepdims=True)-means[None,:]+grand)@coef+y.
mean(0)
55     tm=pd.read_csv(a_dir/'test_mixture_lm.csv').sort_values('index')
56     check=kpredict(tm[mc].to_numpy(float))
57     saved_k=saved[(saved.scope=='lm')&(saved.model=='sqrt_krr')].pivot(
index='index',columns='loss_domain',values='predicted')[ld]
58     assert np.array_equal(tm['index'],saved_k.index)
59     rebuild_error=float(np.max(np.abs(check-saved_k.to_numpy())))
60     A=pd.read_csv(REP0/'求解/问题一/结果/混合系数矩阵.csv').set_index('
training_domain').loc[domains,ld].to_numpy()
61     b=pd.read_csv(REP0/'求解/问题一/结果/截距.csv').set_index('
loss_domain').loc[ld,'intercept'].to_numpy()
62     targets=np.stack([ref,proposed]); linear=(targets@A+b)@weights;
nonlinear=kpredict(targets)@weights
63     # Convex combination feasibility, then nearest Linf distance as a
numerical certificate.
64     hull={}
65     for name,target in [('reference_closed',ref/ref.sum()),('recommended
',proposed)]:
66         eq=np.vstack([x.T,np.ones(len(x))]);rhs=np.r_[target,1]
67         fit=linprog(np.zeros(len(x)),A_eq=eq,b_eq=rhs,bounds=(0,None),
method='highs')
68         upper=np.vstack([np.column_stack([x.T,-np.ones(len(domains))]),
np.column_stack([-x.T,-np.ones(len(domains))])])
69         dist=linprog(np.r_[np.zeros(len(x)),1.],A_ub=upper,b_ub=np.r_[
target,-target],
70                     A_eq=np.array([np.r_[np.ones(len(x)),0.]]),b_eq
=[1.],bounds=(0,None),method='highs')
71         hull[name]=dict(feasible=bool(fit.success),status=int(fit.status
),minimum_linf_distance=float(dist.fun))
72         report['q1']=dict(weights=dict(zip(ld,weights)),same_objective=
ranking,krr_rebuild_max_abs_error=rebuild_error,
73                          reference_and_recommended=dict(linear=linear.
tolist(),sqrt_krr=nonlinear.tolist(),
74                  linear_change=float(linear[1]-linear[0]),
krr_change=float(nonlinear[1]-nonlinear[0]),hull=hull)
75     # 同一上限与固定未观测领域下, 检查正则改变对系数和配方的影响。
76     measured = [domains.index(d) for d in ld]
77     fixed_idx = [i for i in range(len(domains)) if i not in measured]
78     def fit_linear(alpha):
79         xc=xraw-xraw.mean(axis=0);yc=y-y.mean(axis=0)
80         co=solve(xc.T@xc+alpha*np.eye(xraw.shape[1]),xc.T@yc,assume_a='
pos')
81         return co,y.mean(axis=0)-xraw.mean(axis=0)@co
82     def recommend(co, intercept):
83         def build(z):
84             v=ref.copy();v[measured]=z
85             return v/v.sum()
86         # 等式约束使配方和为 1, 目标为线性; 用 LP 避免近乎平坦目标的提前
停止。

```

```

87         sol=linprog((co@weights)[measured],A_eq=np.ones((1,len(measured)
88         )),
89         b_eq=[1-ref[fixed_idx].sum()],bounds=[(0,2.5*ref[i])
90         for i in measured],method='highs')
91         if not sol.success:raise RuntimeError(sol.message)
92         mixture=build(sol.x)
93         return mixture,float((mixture@co+intercept)@weights)
94         co0,b0=fit_linear(.001);co1,b1=fit_linear(.1)
95         pp0,v0=recommend(co0,b0);pp1,v1=recommend(co1,b1)
96         cc0=co0-co0.mean(axis=0);cc1=co1-co1.mean(axis=0)
97         report['q1']['regularization']=dict(centers_relative_change=float(
98         np.linalg.norm(cc1-cc0)/np.linalg.norm(cc0)),
99         sign_flips=int(np.sum(np.sign(cc1)!=np.sign(cc0))),domains=
100         domains,
101         mixture_lambda_001=pp0.tolist(),mixture_lambda_01=pp1.tolist(),
102         objective_lambda_001=v0,objective_lambda_01=v1,
103         both_recipes_under_original_model=[float((v@co0+b0)@weights) for
104         v in [pp0,pp1]])
105         full=pd.read_csv(REP0/'求解/问题一/结果/质量域评分_全量.csv')
106         sample=pd.read_csv(REP0/'求解/问题一/结果/质量域评分_抽样与扩展.csv'
107         ).query("source == 'A1'")
108         report['q1']['github_shares']=dict(full=float(full.loc[full.domain.
109         eq('github'),'n'].sum())/full.n.sum()),
110         A1=float(sample.loc[sample.domain.
111         eq('github'),'n'].sum())/sample.n.sum())
112
113     # Q4: reconstruct filters, time encoding and frontier from raw
114     tables, no solver imports.
115     cdir=DATA/'C_efficiency_evolution';lb=pd.read_csv(cdir/'
116     leaderboard_cleaned.csv')
117     lb['P']=pd.to_numeric(lb['#Params (B)'],errors='coerce');lb['date']=
118     pd.to_datetime(lb['Submission Date'],errors='coerce')
119     lb['Year']=lb.date.dt.year;lb['avg']=pd.to_numeric(lb['Average \
120     u2b06\ufe0f'],errors='coerce')
121     tasks=['IFEval','BBH','MATH Lvl 5','GPQA','MUSR','MMLU-PRO']
122     lb[tasks]=lb[tasks].apply(pd.to_numeric,errors='coerce')
123     licenses={'apache-2.0','mit','gpl-3.0','cc-by-4.0','mpl-2.0','bsd-3-
124     clause','llama2','llama3','llama3.1','llama3.2','llama3.3','gemma','
125     cc-by-sa-4.0','openrail'}
126     lb=lb[lb[tasks].notna().all(axis=1)&lb.date.notna()&lb.P.notna()&lb[
127     'Hub License'].astype(str).str.strip().str.lower().isin(licenses)].
128     copy()
129     lb['logP']=np.log10(lb.P);lb['t']=lb.Year-2019;lb=lb[lb.t.between
130     (0,12)].copy()
131     sample=lb.groupby('Model').Year.nunique();mean_dates=lb.groupby('
132     Year').date.mean()
133     def regress(t):
134         X=np.column_stack([np.ones(len(lb)),lb.logP,t]);coef=np.linalg.
135         lstsq(X,lb.avg,rcond=None)[0]
136         return dict(intercept=float(coef[0]),scale=float(coef[1]),time=
137         float(coef[2]),r2=float(1-((lb.avg-X@coef)**2).sum()/((lb.avg-lb.avg.
138         mean())**2).sum()))
139     annual=regress(lb.t);continuous=regress((lb.date-pd.Timestamp('2019-
140     01-01')).dt.total_seconds()/(365.25*86400))
141     ts=pd.read_csv(cdir/'leaderboard_extended_timeseries.csv')
142     ts['avg']=pd.to_numeric(ts.Average,errors='coerce');ts['P']=pd.
143     to_numeric(ts.Params_B,errors='coerce')

```

```

121     six=['IFEval','BBH','MATH_Lvl5','GPQA','MUSR','MMLU_PRO'];ts[six]=ts
122     [six].apply(pd.to_numeric,errors='coerce')
123     ts=ts[((ts.avg-ts[six].mean(axis=1)).abs())<=5]&ts.P.notna()&ts.avg.
notna()&(ts.P>0)].copy()
124     fy=ts.groupby('Year').agg(avg=('avg','max'),P=('P','max')).
reset_index();fy['cum_avg']=fy.avg.cummax()
125     monthly=lb.groupby(lb.date.dt.to_period('M')).avg.apply(lambda z:z.
nlargest(max(1,int(len(z)*.01))).mean())
126     monthly=monthly[monthly.index>=pd.Period('2024-01')]
127     mt=np.array([(p.year-2019)+(p.month-.5)/12 for p in monthly.index]);
hist=fy[fy.Year<=2023]
128     tf=np.r_[hist.Year.to_numpy()-2019+.5,mt];sf=np.r_[hist.cum_avg.
to_numpy(),np.maximum.accumulate(monthly.to_numpy())]
129     idx=np.argsort(tf);tf=tf[idx];sf=np.maximum.accumulate(sf[idx]);
future=np.array([mt.max()+1,mt.max()+2])
130     def logistic(t,K,r,t0):return K/(1+np.exp(-r*(np.asarray(t)-t0)))
131     def lfit(t,s):return curve_fit(logistic,t,s,p0=[55,.8,3.5],bounds
=([20,.01,.5],[100,5,8]),maxfev=40000)[0]
132     full=lfit(tf,sf);short=lfit(tf[:-3],sf[:-3])
133     split=2024-2019+(9-.5)/12;train=tf<=split;test=~train;hold=lfit(tf[
train],sf[train])
134     rmse=lambda a,b:float(np.sqrt(np.mean((a-b)**2)))
135     pd.DataFrame(dict(t=tf,score=sf,hold_out=test)).to_csv(OUT/'
q4_reconstructed_frontier.csv',index=False,encoding='utf-8-sig')
136     annual_models=[]
137     for year,group in ts.groupby('Year'):
138         peak=group.loc[group.avg.idxmax()];big=group.loc[group.P.idxmax
()])
139         annual_models.append(dict(year=int(year),score_peak_model=peak.
Model,score_peak=float(peak.avg),score_peak_parameters=float(peak.P),
max_size_model=big.Model,max_size=float(big.P),max_size_model_score=
float(big.avg)))
140         report['q4']=dict(records=len(lb),unique_model_names=int(lb.Model.
nunique()),names_across_years=int((sample>1).sum()),
141         year_mean_dates={str(k):str(v) for k,v in mean_dates.items()},
mean_date_gap_years=float((mean_dates.iloc[-1]-mean_dates.iloc[0]).
total_seconds()/(365.25*86400)),
142         annual_label_regression=annual,continuous_date_regression=
continuous,annual_frontier_subjects=annual_models,
143         frontier=dict(n=len(tf),historical_n=len(hist),monthly_n=len(mt)
,peak_repeated_points=int((sf==sf.max()).sum()),parameters=full.
tolist(),prediction=logistic(future,*full).tolist(),
144         truncated_last_3_parameters=short.tolist(),
truncated_last_3_prediction=logistic(future,*short).tolist(),
145         holdout_cutoff='2024-09',holdout_n=int(test.sum()),
holdout_logistic_parameters=hold.tolist(),
146         holdout_logistic_rmse=rmse(logistic(tf[test],*hold),sf[test]),
holdout_persistence_rmse=rmse(np.full(test.sum(),sf[train][-1]),sf[
test])))
147         report['official_results_unchanged']=all(Path(p).exists() and digest
(Path(p))==v for p,v in before.items())
148         assert report['official_results_unchanged']
149         (OUT/'independent_q1_q4_results.json').write_text(json.dumps(report,
ensure_ascii=False,indent=2),encoding='utf-8')
150         return report
151 def check_q2_q3(REPO, DATA, OUT):
152     sha=lambda p:hashlib.sha256(p.read_bytes()).hexdigest()

```

```

153     before={str(p):sha(p) for p in (REP0/'求解').rglob('*') if p.is_file
154     () and p.suffix in ('.csv','.json')}}
154     bdir=DATA/'B_scaling_laws';b1=pd.read_csv(bdir/'
pythia_training_log_existing.csv');b6=pd.read_csv(bdir/'
supplementary_NQ_experiment.csv')
155     p0=pd.read_csv(REP0/'求解/问题二/结果/经典标度律参数.csv').iloc[0][[
'E','A','alpha','B','beta']].to_numpy(float)
156     p=pd.read_csv(REP0/'求解/广义标度律参数.csv').set_index('param').loc
[ ['E','A','alpha','B','beta','C','gamma'],'value'].to_numpy(float)
157     classic=lambda p,N,D:p[0]+p[1]*N**(-p[2])+p[3]*D**(-p[4])
158     general=lambda p,N,D,Q:classic(p[:5],N,D)+p[5]*np.maximum(1-Q,0)**p
[6]
159     def metrics(y,yh):
160         e=np.asarray(yh)-np.asarray(y)
161         return dict(n=len(e),R2=float(1-(e*e).sum()/((y-np.mean(y))**2).
sum()),RMSE=float(np.sqrt(np.mean(e*e))),MAE=float(np.mean(abs(e))),
MAPE_percent=float(np.mean(abs(e)/abs(y))*100))
162     N=np.r_[b1.N_params_B,b6.N_params_B];D=np.r_[b1.D_tokens_B,b6.
D_tokens_B];Q=np.r_[np.ones(len(b1)),b6.Q_score];Y=np.r_[b1.val_loss,
b6.val_loss]
163     errors=dict(classic_B1=metrics(b1.val_loss.to_numpy(),classic(p0,b1.
N_params_B.to_numpy(),b1.D_tokens_B.to_numpy())),general_B1_B6=
metrics(Y,general(p,N,D,Q)))
164     groups=[]
165     for (n,d),g in b6.groupby(['N_params_B','D_tokens_B']):
166         low=g[np.isclose(g.Q_score,.1)];high=g[np.isclose(g.Q_score,1)]
167         assert len(low)==len(high)==1
168         groups.append(dict(N=n,D=d,delta=float(low.val_loss.iloc[0]-high
.val_loss.iloc[0])))
169     groups=pd.DataFrame(groups);groups.to_csv(OUT/'
q2_fixed_ND_quality_gain.csv',index=False,encoding='utf-8-sig')
170     gain=dict(groups=len(groups),min=float(groups.delta.min()),max=float
(groups.delta.max()),spearman_with_N=float(spearmanr(groups.N,groups.
delta).statistic),additive_model_gain=float(p[5]*(.9**p[6])))
171     # Fixed gamma=1 retains identical data, loss, bounds, and six
remaining parameters.
172     fixed=least_squares(lambda v:general(np.r_[v,1.],N,D,Q)-Y,p[:6],
bounds=([0,1e-8,.01,1e-8,.01,1e-8],[5,1e3,2,1e3,2,50]),loss='soft_l1'
,f_scale=.05,max_nfev=80000)
173     pg=np.r_[fixed.x,1.]
174     soft=lambda e:float(.05**2*np.sum(np.sqrt(1+(e/.05)**2)-1))
175     base_cost=soft(general(p,N,D,Q)-Y);fixed_cost=soft(general(pg,N,D,Q)
-Y)
176     Q0=json.loads((REP0/'求解/问题一_关键量.json').read_text('utf-8-sig'
))['Q0_mix_weighted'];k=6+2e-4*2048
177     def optimum(budget,par):
178         E,A,a,B,b,C,g=par;bc=budget/1e18
179         def atq(q):
180             h=5*(q**4-Q0**4)
181             def deriv(logn):
182                 n=np.exp(logn)
183                 return -a*A*n**(-a-1)+b*B*bc**(-b)*k*(k*n+h)**(b-1)
184             ln=brentq(deriv,np.log(1e-4),np.log(1e6),xtol=1e-13)
185             n=np.exp(ln);d=bc/(k*n+h)
186             return dict(Q=float(q),N_B=float(n),D_B=float(d),loss=float(
E+A*n**(-a)+B*d**(-b)+C*(1-q)**g))

```

```

187     grid=np.linspace(Q0,1,61);vals=[atq(q) for q in grid];cand=[vals
188     [0],vals[-1]]
189     for i in range(1,len(vals)-1):
190         if vals[i]['loss']<=vals[i-1]['loss'] and vals[i]['loss']<=
vals[i+1]['loss']:
191             sol=minimize_scalar(lambda q:atq(q)['loss'],bounds=(grid
192             [i-1],grid[i+1]),method='bounded',options={'xatol':1e-13})
193             cand.append(atq(sol.x))
194         return min(cand,key=lambda x:x['loss'])
195     sweep=pd.read_csv(REP0/'求解/问题三/结果/预算扫略.csv');sat=sweep[
196     sweep.Q>=1-1e-8].iloc[0];budget=float(sat.C)
197     base_sat=optimum(budget,p);fixed_sat=optimum(budget,p)
198     base22=optimum(1e22,p)
199     mix=pd.read_csv(REP0/'求解/问题三/结果/配比项算力等价.csv');M=float(
200     mix.loc[mix.mixture=='问题一有界推荐','M_p'].iloc[0])
201     transfer=[]
202     for fraction in [0,.25,.5,.75,1]:
203         target=base22['loss']+fraction*M
204         root=brentq(lambda logc:optimum(np.exp(logc),p)['loss']-target,
np.log(1e21),np.log(1e25),xtol=1e-11)
205         transfer.append(dict(retention=fraction,target_loss=target,
equivalent_budget=float(np.exp(root)),multiplier=float(np.exp(root)/1
e22)))
206     # C3 sample count uses the exact documented six-score consistency
filter.
207     ts=pd.read_csv(DATA/'C_efficiency_evolution/
208     leaderboard_extended_timeseries.csv');cols=['IFEval','BBH','MATH_Lvl5
','GPQA','MUSR','MMLU_PRO']
209     ts['avg']=pd.to_numeric(ts.Average,errors='coerce');ts['P']=pd.
to_numeric(ts.Params_B,errors='coerce');ts[cols]=ts[cols].apply(pd.
to_numeric,errors='coerce')
210     mask=((ts.avg-ts[cols].mean(axis=1)).abs())<=5&ts.P.notna()&ts.avg.
notna()&(ts.P>0)
211     c3=ts[mask].copy();X=np.column_stack([np.ones(len(c3)),np.log10(c3.P
),c3.Year-2019]);c=np.linalg.lstsq(X,c3.avg,rcond=None)[0]
212     c3_summary=dict(raw_n=len(ts),retained_n=len(c3),excluded_n=int((~
mask).sum()),R2=float(1-((c3.avg-X@c)**2).sum()/((c3.avg-c3.avg.mean
())**2).sum()))
213     # Three cutoffs all evaluate their subsequent months through the
same last month.
214     front=pd.read_csv(OUT/'q4_reconstructed_frontier.csv');t=front.t.
to_numpy();s=front.score.to_numpy()
215     logistic=lambda t,K,r,t0:K/(1+np.exp(-r*(np.asarray(t)-t0)))
216     cuts=[]
217     for month in [7,8,9]:
218         cutoff=5+(month-.5)/12;tr=t<=cutoff+1e-12;te=~tr
219         theta=curve_fit(logistic,t[tr],s[tr],p0=[55,.8,3.5],bounds
=([20,.01,.5],[100,5,8]),maxfev=40000)[0]
220         pred=logistic(t[te],*theta);baseline=np.full(te.sum(),s[tr][-1])
;rmse=lambda p:float(np.sqrt(np.mean((p-s[te])**2)))
221         cuts.append(dict(cutoff=f'2024-{month:02}',train_n=int(tr.sum())
,test_n=int(te.sum()),test_end='2025-03',parameters=theta.tolist(),
logistic_RMSE=rmse(pred),persistence_RMSE=rmse(
baseline),test_time=t[te].tolist(),test_observed=s[te].tolist(),
logistic_prediction=pred.tolist(),persistence_prediction=baseline.
tolist()))

```

```

218     result=dict(error_metrics=errors,B6_quality_gain=gain,
gamma_fixed_one=dict(parameters=pg.tolist(),original_cost=base_cost,
fixed_cost=fixed_cost,
219     cost_increase_percent=(fixed_cost/base_cost-1)*100,
original_first_saturation_budget=budget,original_saved_Q=float(sat.Q)
,original_recomputed=base_sat,fixed_gamma_recomputed=fixed_sat),
220     transfer_sensitivity=transfer,C3=c3_summary,Q4_time_holdouts=
cuts,official_results_unchanged=all(Path(f).exists() and sha(Path(f))
==v for f,v in before.items()))
221     # 保持 A 侧费用不变, 只改变未知的 A -> B 质量映射; 不是纯坐标替换。
222     E,A,a,B,b,C,g=p
223     mapping=[]
224     for power in [.5,1.,2.]:
225         def local_derivative(logbudget):
226             bc=np.exp(logbudget)/1e18
227             n=(a*A/(b*B)*(bc/k)**b)**(1/(a+b));d=bc/(k*n)
228             return b*B*d**(-b)*20*Q0**3/(k*n)-C*g*(1-Q0**power)**(g-1)*
power*Q0**(power-1)
229         boundary=np.exp(brentq(local_derivative,np.log(1e17),np.log(1e22
)))
230         mapping.append(dict(power=power,local_start_budget=float(
boundary)))
231     result['quality_mapping_local_sensitivity']=mapping
232     assert result['official_results_unchanged']
233     (OUT/'independent_q23_and_timecuts_results.json').write_text(json.
dumps(result,ensure_ascii=False,indent=2),encoding='utf-8')
234     return result
235
236 def main():
237     repo = Path(__file__).resolve().parents[1]
238     ap = argparse.ArgumentParser(description=__doc__)
239     ap.add_argument('--data-dir', type=Path, required=True)
240     ap.add_argument('--out', type=Path, default=repo/'复现记录/补充检验 '
)
241     args = ap.parse_args()
242     data = args.data_dir.resolve()
243     out = args.out.resolve()
244     if out.is_relative_to(repo/'求解') or out.is_relative_to(data):
245         raise ValueError('诊断输出目录不得位于正式求解或附件目录内')
246     out.mkdir(parents=True, exist_ok=True)
247     first = check_q1_q4(repo, data, out)
248     second = check_q2_q3(repo, data, out)
249     print(json.dumps({'output': str(out), 'q1_krr_rebuild_error': first[
'q1']['krr_rebuild_max_abs_error'],
250                     'C3_retained_n': second['C3']['retained_n'], '
official_results_unchanged':
251                     first['official_results_unchanged'] and second['
official_results_unchanged']},
252                     ensure_ascii=False, indent=2))
253
254 if __name__ == '__main__':
255     main()

```

程序 7: 公共数据与绘图模块 (求解/_common.py)

1 # 本程序的整理与核对使用 Codex 辅助; 模型: GPT-5.6-Luna; 机构: OpenAI;
版本发布日期: 2026-07-09。

```

2 # 本程序的整理、复现核对或绘图实现使用 OpenAI Codex (GPT-5.6-Luna) 辅助。
3 # 公共代码模板：求解/_common.py (系统自动播种，每个任务只写一次，不要覆盖覆盖函数)
4 import sys, os
5 try:
6     sys.stdout.reconfigure(encoding='utf-8', errors='replace')
7     sys.stderr.reconfigure(encoding='utf-8', errors='replace')
8 except Exception:
9     pass
10 # 问题目录 = 运行脚本所在目录 (兼容 cd 求解/问题X 与 求解/ 根运行两种方式)
11 _script_dir = os.path.dirname(os.path.abspath(sys.argv[0]))
12 # 在 Jupyter / ipynb 下 sys.argv[0] 指向内核启动脚本，须改用工作目录 (notebook 所在目录)
13 if 'ipykernel' in sys.modules or 'ipykernel_launcher' in os.path.basename(sys.argv[0]):
14     _script_dir = os.getcwd()
15 if not _script_dir or not os.path.isdir(_script_dir):
16     _script_dir = os.getcwd()
17 BASE_DIR = _script_dir
18
19 import pandas as pd
20 import numpy as np
21 import matplotlib
22 matplotlib.use('Agg')
23 import matplotlib.pyplot as plt
24 import matplotlib.font_manager as _fm
25 import warnings, logging
26 warnings.filterwarnings('ignore')
27 logging.getLogger('matplotlib').setLevel(logging.ERROR) # 抑制findfont警告
28 logging.getLogger('matplotlib.font_manager').setLevel(logging.ERROR) # 抑制字体管理器警告
29
30 # ★ 统一绘图类导入中枢：一次性提供常用 matplotlib 类/子模块，
31 # 使脚本只写 `from _common import *` 即可直接用，无需各自重复 import (防止遗漏导致 NameError)。
32 import matplotlib.ticker as mticker
33 import scipy.stats as stats
34 from matplotlib.patches import (Rectangle, FancyBboxPatch,
35                                 FancyArrowPatch, FancyArrow,
36                                 Circle, RegularPolygon, Ellipse, Wedge,
37                                 Polygon, Arrow, Arc)
38 from matplotlib.collections import PatchCollection, LineCollection, PolyCollection
39 from matplotlib.path import Path
40 from matplotlib.spines import Spine
41 try:
42     from matplotlib.projections.polar import PolarAxes
43     from matplotlib.projections import register_projection
44     from mpl_toolkits.axes_grid1.inset_locator import inset_axes, mark_inset
45 except Exception:
46     pass

```

```

45
46 # 字体: 仅首次运行时重建缓存 (marker 防止每次都全量扫描字体拖慢)
47 def _find_font_dir():
48     starts = [BASE_DIR, os.getcwd()]
49     for start in starts:
50         d = start
51         for _ in range(6):
52             candidate = os.path.join(d, 'fonts')
53             if os.path.isdir(candidate) and os.path.isfile(os.path.join(
candidate, 'SourceHanSerifCN-Regular.otf')):
54                 return candidate
55                 parent = os.path.dirname(d)
56                 if parent == d: break
57                 d = parent
58     return None
59 _SOLVE_DIR = os.path.dirname(BASE_DIR)
60 # 求解/ 根目录 (脚本所在 问题X 的上一级), 跨问题共享中间文件统一放这里
61 SOLVE_DIR = _SOLVE_DIR
62 _FONT_MARKER = os.path.join(_SOLVE_DIR, '_fonts_ready')
63 if not os.path.exists(_FONT_MARKER):
64     try:
65         import glob as _glb
66         for _fc in _glb.glob(os.path.join(matplotlib.get_cachedir(), '
font*')):
67             try: os.remove(_fc)
68             except: pass
69         _fm._load_fontmanager(try_read_cache=False)
70         open(_FONT_MARKER, 'w').close()
71     except Exception:
72         pass
73 _FONT_DIR = _find_font_dir()
74 if _FONT_DIR:
75     for _f in ['SourceHanSerifCN-Regular.otf', 'SourceHanSerifCN-Bold.
otf']:
76         _p = os.path.join(_FONT_DIR, _f)
77         if os.path.exists(_p): _fm.fontManager.addfont(_p)
78 _SYS_FONT = os.path.join(os.environ.get('WINDIR', 'C:\\Windows'), 'Fonts
')
79 for _f in ['times.ttf', 'timesbd.ttf', 'timesi.ttf', 'simsum.ttc']:
80     _p = os.path.join(_SYS_FONT, _f)
81     if os.path.exists(_p): _fm.fontManager.addfont(_p)
82 plt.rcParams['font.family'] = ['Times New Roman', 'Source Han Serif CN',
'SimSun', 'serif']
83 plt.rcParams['font.weight'] = 'normal'
84 plt.rcParams['axes.unicode_minus'] = False
85 plt.rcParams['figure.dpi'] = 150
86 plt.rcParams['savefig.dpi'] = 220
87 plt.rcParams['savefig.bbox'] = 'tight'
88 plt.rcParams['pdf.fonttype'] = 42
89
90 # ★ 兼容层: monkey-patch 旧 API, 防止 agent 用 plt.cm.get_cmap() 报错
91 if not hasattr(matplotlib.cm, 'get_cmap'):
92     matplotlib.cm.get_cmap = lambda name, n=None: plt.colormaps[name].
resampled(n) if n else plt.colormaps[name]
93 try:
94     from mpl_toolkits.mplot3d import Axes3D as _Axes3D
95 except ImportError:

```

```

96     pass
97
98     FIG_DIR = os.path.join(BASE_DIR, '图片')
99     OUT_DIR = os.path.join(BASE_DIR, '结果')
100     os.makedirs(FIG_DIR, exist_ok=True)
101     os.makedirs(OUT_DIR, exist_ok=True)
102
103     # ★ 数据目录：兼容工程内“数据/”与赛题目录中的 real_attachments/。
104     _project_root = os.path.normpath(os.path.join(BASE_DIR, '..', '..'))
105     _data_candidates = ([os.environ['MODELING_DATA_DIR']] if os.environ.get(
106         'MODELING_DATA_DIR') else []) + [
107         os.path.join(_project_root, '数据'),
108         os.path.join(_project_root, 'real_attachments'),
109     ]
110     from pathlib import Path as _Path
111     _data_candidates += [str(p / 'real_attachments') for p in _Path(BASE_DIR)
112         .parents]
113     DATA_DIR = next((p for p in _data_candidates if os.path.isdir(p)),
114         _data_candidates[0])
115     _DATA_INDEX = None
116
117     def data_path(name):
118         """按文件名定位附件；既支持平铺数据目录，也支持 A/B/C 分目录。"""
119         direct = os.path.join(DATA_DIR, name)
120         if os.path.exists(direct):
121             return direct
122         global _DATA_INDEX
123         if _DATA_INDEX is None:
124             _DATA_INDEX = {}
125             for root, _, files in os.walk(DATA_DIR):
126                 for fn in files:
127                     _DATA_INDEX.setdefault(fn.lower(), os.path.join(root, fn))
128         key = os.path.basename(name).lower()
129         if key not in _DATA_INDEX and key.endswith('.jsonl') and key + '.xz'
130             in _DATA_INDEX:
131                 key += '.xz'
132         if key not in _DATA_INDEX:
133             raise FileNotFoundError(f'附件中未找到数据文件：{name}（根目录 {
134                 DATA_DIR}）')
135         return _DATA_INDEX[key]
136
137     def despine(ax):
138         """只保留左侧和底侧边框，移除上侧和右侧"""
139         ax.spines['top'].set_visible(False)
140         ax.spines['right'].set_visible(False)
141
142     def save_fig(fig, name_cn, dpi=300):
143         """保存单张图片，自动加.png后缀（默认 300dpi 高清）"""
144         path = os.path.join(FIG_DIR, name_cn + '.png')
145         fig.savefig(path, dpi=dpi, bbox_inches='tight', facecolor='white')
146         plt.close(fig)
147         print(f' 已保存：{name_cn}.png')
148
149     def save_dual(fig, name_cn, dpi=300, bw=False):

```

```

145     """保存图片（默认 300dpi 高清）。绘图规则只输出全彩，默认不再生成黑白版（bw=False，仅保存彩色）。
146     如需临时验证黑白效果可手动传 bw=True，但最终产出图一律不得黑白。"""
147     from PIL import Image, ImageOps
148     color_path = os.path.join(FIG_DIR, name_cn + '.png')
149     fig.savefig(color_path, dpi=dpi, bbox_inches='tight', facecolor='white')
150     print(f' 已保存: {name_cn}.png')
151     if bw:
152         bw_path = os.path.join(FIG_DIR, name_cn + '_bw.png')
153         try:
154             img = Image.open(color_path)
155             img_gray = ImageOps.grayscale(img)
156             img_gray.save(bw_path)
157             img.close()
158         except Exception:
159             fig.savefig(bw_path, dpi=dpi, bbox_inches='tight', facecolor='white')
160     print(f' 已保存: {name_cn}_bw.png')
161     print(f'  ★ 双版本完成: {name_cn}')
162     plt.close(fig)
163
164 def save_csv(df, name_cn):
165     df.to_csv(OUT_DIR + '/' + name_cn, index=False, encoding='utf-8-sig')
166
167 def shared_path(name):
168     """跨问题共享中间文件的统一路径（存到 求解/ 根，供各问题脚本互相读取）。
169     例：问题一存 shared_path('预处理数据.csv')，问题二用 pd.read_csv(shared_path('预处理数据.csv')) 读取。"""
170     return os.path.join(SOLVE_DIR, name)
171
172 def is_ax_empty(ax):
173     """检测坐标轴是否无数据（空图）"""
174     return (len(ax.lines) == 0 and len(ax.collections) == 0
175            and len(ax.patches) == 0 and len(ax.images) == 0)
176
177 def is_flat_line(x, y):
178     """折线是否为直线（完全无变化→不画）：所有y相同或所有x相同"""
179     if x is None or y is None or len(x) < 2 or len(y) < 2:
180         return False
181     y_arr = np.asarray(y, dtype=float)
182     x_arr = np.asarray(x, dtype=float)
183     y_flat = np.allclose(y_arr, y_arr[0])
184     x_flat = np.allclose(x_arr, x_arr[0])
185     return y_flat or x_flat
186
187 def validate_subplots(nrows, ncols):
188     """
189     子图布局校验，不合法返回错误原因字符串，合法返回None。
190     规则（硬性）：单图1×1；一行多列1×N(N≥2)；两行2×N且N≥4；其余禁止。
191     禁止：三行及以上；两行列数≤3；2×2四宫格。
192     """
193     if nrows >= 3:

```

```

194         return f"禁止超过两行（{nrows}行），行列最多两行"
195     if nrows == 2 and ncols <= 3:
196         return f"两行列数必须>3（当前{ncols}列），换2×{max(4, ncols)}或
    改单行布局"
197     return None
198
199 def get_colors(cmap_name, n):
200     """安全获取colormap的n个颜色（替代plt.cm.get_cmap，已废弃会报错）。
    """
201     cmap = plt.colormaps[cmap_name].resampled(n)
202     return [cmap(i) for i in range(n)]
203
204 def safe_cmap(cmap_name):
205     """安全获取colormap对象（不带resampled）。"""
206     return plt.colormaps[cmap_name]
207
208 def safe_boxplot(ax, data, labels, **kwargs):
209     """安全的boxplot（替代ax.boxplot(labels=...），labels参数已废弃会报
    错）。
210     内部强制 patch_artist=True，使 boxes 为 Patch 对象，AI 可直接用 bp['
    boxes'][i].set_facecolor(...) 填色；
211     否则 boxes 是 Line2D，set_facecolor 会抛 AttributeError。"""
212     kwargs.setdefault('patch_artist', True)
213     bp = ax.boxplot(data, **kwargs)
214     ax.set_xticklabels(labels, fontsize=11)
215     return bp
216
217 def safe_heatmap(ax, data, row_labels=None, col_labels=None, cmap='
    RdBu_r',
218                 linewidth=0.3, edgecolor='black', fontsize=11, **kwargs
    ):
219     """
220     强制带黑色细边框的热力图（替代裸 imshow/heatmap 的纯色块）。
221     用 pcolormesh 绘制，每个格子边线统一为黑色细框（edgecolor=black,
    linewidth=linewidth），
222     避免"色块连成一片"的观感。NaN 单元格填白色。默认 vmin/vmax 取数据范
    围。
223     用法：
224     safe_heatmap(ax, corr.values, row_labels=corr.index, col_labels=
    corr.columns, cmap='RdBu_r')
225     返回 pcolormesh 的 QuadMesh（可用其 .set_clim 调色标范围）。
226     """
227     data = np.asarray(data, dtype=float)
228     rows, cols = data.shape
229     cmap_obj = plt.colormaps[cmap]
230     cmap_obj.set_bad(color='#FFFFFF') # NaN 为白，避免黑洞
231     mesh = ax.pcolormesh(np.ma.masked_invalid(data), cmap=cmap_obj,
232                          edgecolors=edgecolor, linewidth=linewidth, **
    kwargs)
233     if row_labels is not None:
234         ax.set_yticks(np.arange(rows) + 0.5)
235         ax.set_yticklabels(list(row_labels), fontsize=fontsize)
236     if col_labels is not None:
237         ax.set_xticks(np.arange(cols) + 0.5)

```

```

238         ax.set_xticklabels(list(col_labels), rotation=45, ha='right',
239                             rotation_mode='anchor', fontsize=fontsize)
240     ax.set_xlim(0, cols)
241     ax.set_ylim(0, rows)
242     ax.set_aspect('auto')
243     return mesh
244
245 def png_wh(png_path):
246     """读取PNG实际宽高（字节，读16-24字节IHDR）"""
247     import struct
248     with open(png_path, 'rb') as f:
249         f.read(16)
250         w, h = struct.unpack('>II', f.read(8))
251     return w, h
252
253 def latex_includegraphics(png_path, caption, label, pair=False,
254                             float_pos='ht'):
255     r"""
256     按高宽比生成匹配档位的 \includegraphics LaTeX 片段（直接粘论文）。
257     pair=True: 同一张figure里两张并排minipage，给半宽。float_pos默认'ht'
258     ，禁止'H'锁死。
259     """
260     if float_pos.upper() == 'H':
261         raise ValueError("float_pos禁止'H'强制定位，默认用'ht'允许浮动")
262     w, h = png_wh(png_path)
263     aspect = h / w
264     if pair:
265         width = '0.48\\textwidth'
266     elif aspect >= 0.9:
267         width = '0.60\\textwidth'
268     elif aspect >= 0.6:
269         width = '0.65\\textwidth'
270     else:
271         width = '0.70\\textwidth'
272     rel = png_path.replace('\\', '/')
273     return (
274         f"\\begin{{{figure}}}[{float_pos}]\n"
275         f"  \\centering\n"
276         f"  \\includegraphics[width={width}][{{{rel}}}]\n"
277         f"  \\caption{{{caption}}}\n"
278         f"  \\label{{{fig:{label}}}}\n"
279         f"\\end{{{figure}}}"
280     )
281
282 # ★ 数据清洗辅助（避免 "--" 等非数值导致 min/max/idxmax 报错）
283 def numeric_clean(s, default=None):
284     """将列安全转为数值：非数值元素→NaN；default 不为 None 时用 default
285     填充NaN。
286     用法：df['评分'] = numeric_clean(df['评分']) # 先转数值（NaN保留）
287     df['评分'] = numeric_clean(df['评分'], default=0.0) # 转数值
288     并填0"""
289     s = pd.to_numeric(s, errors='coerce')
290     if default is not None:
291         s = s.fillna(default)
292     return s

```

```

290 def safe_idxmax(s, default_idx=None):
291     """安全的 idxmax: 空序列/全NA 返回 default_idx, 否则返回最大值索引。
    """
292     s = pd.to_numeric(s, errors='coerce')
293     s = s.dropna()
294     if s.empty:
295         return default_idx
296     return s.idxmax()
297
298 def safe_idxmin(s, default_idx=None):
299     """安全的 idxmin: 空序列/全NA 返回 default_idx, 否则返回最小值索引。
    """
300     s = pd.to_numeric(s, errors='coerce')
301     s = s.dropna()
302     if s.empty:
303         return default_idx
304     return s.idxmin()
305
306 def has_na(df):
307     """检测DataFrame是否含NaN/空值（用于空值防线）。"""
308     return bool(df.isna().any().any())

```

程序 8: 问题一核岭预测与交叉验证（求解/问题一/非线性代理对照.py）

```

1 # 本程序的整理与核对使用 Codex 辅助；模型：GPT-5.6-Luna；机构：OpenAI；
  版本发布日期：2026-07-09。
2 # -*- coding: utf-8 -*-
3 # 本程序的实现、验证与说明由 OpenAI Codex（GPT-5.6-Luna）辅助完成。
4 """同尺度配比预测：既有线性岭基线与平方根嵌入核岭。
5
6 python 求解/问题一/非线性代理对照.py --data-dir <附件根目录>
7 只写“非线性代理对照_*”结果，不写任何跨问接口。绘图另由重绘图表.py执
  行。
8 核岭采用闭合配方、训练核中心化、未惩罚截距；全部超参数只用训练五折选择。
9 CV最小值是调参得分，不是嵌套CV泛化估计；既有检验集不是新的盲测集。
10 """
11 from pathlib import Path
12 import argparse
13 import hashlib
14 import json
15 import platform
16 import sys
17 import time
18 import importlib.metadata
19
20 import numpy as np
21 import pandas as pd
22 from scipy.linalg import eigh, solve
23 from scipy.spatial.distance import cdist
24 from scipy.stats import spearmanr
25 from sklearn.model_selection import KFold
26
27 HERE = Path(__file__).resolve().parent
28 SEED = 20260926
29 LAMBDA = 10.0 ** np.arange(-6, 3)

```

```

30 GAMMAS = 2.0 ** np.arange(-6, 3)
31
32 def find_data(path=None):
33     candidates = [Path(path)] if path else [p / 'real_attachments' for p
34     in HERE.parents]
35     for p in candidates:
36         if (p / 'A_data_value/regmix_tables').is_dir():
37             return p.resolve()
38     raise FileNotFoundError('请用 --data-dir 指定包含 A_data_value 的附
39     件根目录')
40
41 def load_pair(root, split, scale):
42     folder = root / 'A_data_value/regmix_tables'
43     paths = [folder / f'{split}_mixture_{scale}.csv',
44             folder / f'{split}_pile_loss_{scale}.csv']
45     mix, loss = [pd.read_csv(p).sort_values('index').reset_index(drop=
46     True) for p in paths]
47     if mix['index'].duplicated().any() or loss['index'].duplicated().any
48     ():
49         raise ValueError('配方或损失存在重复 index')
50     if not np.array_equal(mix['index'].values, loss['index'].values):
51         raise ValueError('配方与损失 index 不匹配')
52     mc = [c for c in mix if c.startswith('train_the_pile_')]
53     lc = [c for c in loss if c.endswith('_val_loss')]
54     x, y = mix[mc].to_numpy(float), loss[lc].to_numpy(float)
55     if not (np.isfinite(x).all() and np.isfinite(y).all() and (x >= 0).
56     all()
57     and (x.sum(1) > 0).all()):
58         raise ValueError('非有限值、负配比或零行和')
59     domains = [c.replace('metric/the_pile_', '').replace('_val_loss', '')
60     ] for c in lc]
61     return x, y, mix['index'].values, mc, domains, paths
62
63 def closed(x):
64     return x / x.sum(axis=1, keepdims=True)
65
66 def ridge_predict(x, y, v, alpha=1e-3):
67     # 与问题一.py完全相同: 17个原始份额, 截距不受惩罚。
68     a = np.column_stack([x, np.ones(len(x))])
69     penalty = alpha * np.eye(a.shape[1])
70     penalty[-1, -1] = 0
71     w = np.linalg.solve(a.T @ a + penalty, a.T @ y)
72     return np.column_stack([v, np.ones(len(v))]) @ w
73
74 def kernel(x, v, gamma):
75     return np.exp(-gamma * cdist(np.sqrt(x), np.sqrt(v), 'sqeuclidean'))
76
77 def centered_kernel(k):
78     means, grand = k.mean(axis=0), float(k.mean())
79     return k - means[None, :] - means[:, None] + grand, means, grand
80
81 def cross_center(k, means, grand):
82     return k - k.mean(axis=1, keepdims=True) - means[None, :] + grand
83
84 def krr_predict(x, y, v, gamma, lam):
85     kc, means, grand = centered_kernel(kernel(x, x, gamma))
86     ym = y.mean(0)

```

```

81     coef = solve(kc + lam * np.eye(len(x)), y - ym, assume_a='pos')
82     return cross_center(kernel(v, x, gamma), means, grand) @ coef + ym
83
84 def metrics(y, pred):
85     err = y - pred
86     r2 = 1 - (err**2).sum(0) / np.maximum(((y - y.mean(0))**2).sum(0), 1
87     e-12)
88     rho = np.array([spearmanr(y[:, j], pred[:, j]).statistic for j in
89     range(y.shape[1])])
90     rmse = np.sqrt((err**2).mean(0))
91     mae = np.abs(err).mean(0)
92     overall = dict(R2_mean=float(r2.mean()), Spearman_domain_mean=float(
93     rho.mean()),
94     RMSE_pooled=float(np.sqrt((err**2).mean(0))),
95     RMSE_domain_mean=float(rmse.mean()), MAE=float(mae.
96     mean()),
97     RMSE_recipe_mean=float(np.sqrt(((y.mean(1) - pred.
98     mean(1))**2).mean(0))),
99     Spearman_recipe_mean=float(spearmanr(y.mean(1), pred.
100     mean(1)).statistic))
101     return overall, dict(R2=r2, Spearman=rho, RMSE=rmse, MAE=mae)
102
103 def cv_search(xraw, y, folds, gammas=GAMMAS, lambdas=LAMBDA):
104     x = closed(xraw)
105     oof_ridge = np.empty_like(y)
106     squared = np.zeros((len(gammas), len(lambdas)))
107     fold_rows = []
108     for fold, (tr, va) in enumerate(folds, 1):
109         oof_ridge[va] = ridge_predict(xraw[tr], y[tr], xraw[va])
110         for g, gamma in enumerate(gammas):
111             kc, means, grand = centered_kernel(kernel(x[tr], x[tr],
112             gamma))
113             eig, vectors = eigh(kc)
114             if eig.min() < -1e-9:
115                 raise ValueError('中心化核出现显著负特征值')
116             eig = np.maximum(eig, 0)
117             ym = y[tr].mean(0)
118             rotated = vectors.T @ (y[tr] - ym)
119             projection = cross_center(kernel(x[va], x[tr], gamma), means
120             , grand) @ vectors
121             for a, lam in enumerate(lambdas):
122                 pred = projection @ (rotated / (eig[:, None] + lam)) +
123                 ym
124                 sse = float(((y[va] - pred)**2).sum())
125                 squared[g, a] += sse
126                 fold_rows.append(dict(fold=fold, n_valid=len(va), gamma=
127                 gamma, lambda_=lam,
128                 SSE=sse, RMSE=np.sqrt(sse / y[va].
129                 size)))
130     print(f'五折选择: {fold}/{len(folds)}', flush=True)
131     scores = np.sqrt(squared / y.size)
132     g, a = np.unravel_index(scores.argmin(), scores.shape)
133     gamma, lam = float(gammas[g]), float(lambdas[a])
134     oof_krr = np.empty_like(y)
135     for tr, va in folds:
136         oof_krr[va] = krr_predict(x[tr], y[tr], x[va], gamma, lam)
137     grid = [dict(gamma=float(gammas[g]), lambda_=float(lambdas[a]),
138     CV_RMSE=float(scores[g, a]))]

```

```

127         for g in range(len(gammas)) for a in range(len(lambdas))]
128     return gamma, lam, oof_ridge, oof_krr, pd.DataFrame(grid), pd.
DataFrame(fold_rows)
129
130 def self_check(data_root):
131     # 与增广核方程独立比较, 检验训练/预测中心化及不惩罚截距。
132     raw, y, _, _, _ = load_pair(data_root, 'train', 'lm')
133     x = closed(raw[:32]); v = closed(raw[32:40]); y = y[:32]
134     gamma, lam = .125, .01
135     k = kernel(x, x, gamma)
136     augmented = np.block([[k + lam*np.eye(len(x)), np.ones((len(x), 1))
],
137                             [np.ones((1, len(x))), np.zeros((1, 1))]])
138     rhs = np.vstack([y, np.zeros((1, y.shape[1]))])
139     solution = np.linalg.solve(augmented, rhs)
140     reference = kernel(v, x, gamma) @ solution[:-1] + solution[-1]
141     actual = krr_predict(x, y, v, gamma, lam)
142     np.testing.assert_allclose(actual, reference, atol=1e-9, rtol=1e-9)
143     np.testing.assert_allclose(krr_predict(x, y+7, v, gamma, lam),
actual+7, atol=1e-9)
144     folds = list(KFold(2, shuffle=True, random_state=SEED).split(raw
[:40]))
145     cv_search(raw[:40], load_pair(data_root, 'train', 'lm')[1][:40],
folds,
146                 gammas=np.array([.125]), lambdas=np.array([.01]))
147     print(f'最小核验证通过; 增广方程最大偏差 {abs(actual-reference).max()
:.3g}')
148
149 def main():
150     ap = argparse.ArgumentParser(description=__doc__)
151     ap.add_argument('--data-dir', type=Path)
152     ap.add_argument('--output-dir', type=Path, default=HERE/'结果')
153     ap.add_argument('--self-check', action='store_true')
154     args = ap.parse_args(); root = find_data(args.data_dir)
155     if args.self_check:
156         self_check(root); return
157     start = time.monotonic()
158     raw, y, indices, columns, domains, inputs = load_pair(root, 'train',
'lm')
159     x = closed(raw)
160     if len(np.unique(x, axis=0)) != len(x):
161         raise ValueError('重复配方需采用分组交叉验证')
162     folds = list(KFold(5, shuffle=True, random_state=SEED).split(x))
163     gamma, lam, cv_r, cv_k, grid, foldscores = cv_search(raw, y, folds)
164     fold_ids = np.empty(len(y), int)
165     for fold, (_, va) in enumerate(folds, 1): fold_ids[va] = fold
166     summary, detail, predictions = [], [], []
167
168     def record(scope, idx, obs, preds):
169         for model, pred in preds.items():
170             overall, per_domain = metrics(obs, pred)
171             summary.append(dict(scope=scope, model=model, n=len(obs), **
overall))
172         for j, domain in enumerate(domains):
173             detail.append(dict(scope=scope, model=model, loss_domain
=domain,
174                             **{key: float(val[j]) for key, val in
per_domain.items()}))

```

```

175         predictions.append(pd.DataFrame(dict(scope=scope, model=
model, index=idx,
176         loss_domain=domain, observed=obs[:, j], predicted=
pred[:, j])))
177     record('train_oof', indices, y, dict(ridge=cv_r, sqrt_krr=cv_k))
178     profiles = [dict(scope='train', n=len(raw), min_share=float(raw.min
()),
179         max_row_sum_error=float(abs(raw.sum(1)-1).max()))]
180     for scale in ['1m', '60m', '1B']:
181         test, obs, idx, mc, ds, paths = load_pair(root, 'test', scale)
182         if mc != columns or ds != domains: raise ValueError('跨文件字段
顺序不一致')
183         inputs += paths
184         record(scale, idx, obs, dict(ridge=ridge_predict(raw, y, test),
185             sqrt_krr=krr_predict(x, y, closed(
test), gamma, lam)))
186         profiles.append(dict(scope=scale, n=len(test), min_share=float(
test.min()),
187             max_row_sum_error=float(abs(test.sum(1)-1).
max()))))
188     out = args.output_dir; out.mkdir(parents=True, exist_ok=True)
189     tables = dict(汇总=pd.DataFrame(summary), 逐域=pd.DataFrame(detail),
190         预测明细=pd.concat(predictions, ignore_index=True), 参
数搜索=grid,
191         五折=foldscores, 折分配=pd.DataFrame(dict(index=
indices, fold=fold_ids)))
192     for name, table in tables.items():
193         table.to_csv(out/f'非线性代理对照_{name}.csv', index=False,
encoding='utf-8-sig')
194     manifest = dict(seed=SEED, n_train=len(y), n_domains=len(domains),
selected_gamma=gamma,
195         selected_lambda=lam, baseline_lambda=.001, profiles=
profiles,
196         lambda_grid=LAMBDA_S.tolist(), gamma_grid=GAMMA_S.
tolist(),
197         selection='minimum pooled out-of-fold RMSE; not
nested CV',
198         preprocessing='KRR: row closure, sqrt, fold-local
kernel/output centering; ridge: original shares',
199         test_status='previously inspected existing test sets
; not a fresh blind holdout',
200         python=platform.python_version(),
201         packages={p: importlib.metadata.version(p) for p in
['numpy', 'pandas', 'scipy', 'scikit-learn']},
202         inputs={str(p.relative_to(root)): hashlib.sha256(p.
read_bytes()).hexdigest() for p in inputs},
203         script_sha256=hashlib.sha256(Path(__file__).
read_bytes()).hexdigest(),
204         command='python 求解/问题一/非线性代理对照.py --data
-dir <附件根目录>',
205         elapsed_seconds=time.monotonic()-start,
206         shared_outputs_modified=False)
207     (out/'非线性代理对照_复现清单.json').write_text(json.dumps(manifest,
ensure_ascii=False, indent=2), encoding='utf-8')
208     print(f'选择 gamma={gamma}, lambda={lam}')
209     print(tables['汇总'].to_string(index=False))
210

```

```

211 if __name__ == '__main__':
212     if hasattr(sys.stdout, 'reconfigure'): sys.stdout.reconfigure(
        encoding='utf-8')
213     main()

```

程序 9: 核岭训练支持范围内的配比推荐 (求解/问题一/核岭支持域推荐.py)

```

1 # 本程序使用 OpenAI Codex 辅助实现与数值核对。
2 # 用途: 沿用已选核岭, 以训练配方凸组合限制推荐范围; 不重新搜索超参数。
3 """python 求解/问题一/核岭支持域推荐.py --data-dir <附件根目录>
4 推荐写入原结果目录; --output-dir 可用于隔离核验。
5
6 输出字段 stationarity_gap 沿用原名称, 实际记录的是“数值保护梯度间隙”:
7 零份额处以 1e-30 保护平方根导数; success 仅表示该停止量达阈值。
8 它与约束残差共同用于数值筛选, 不构成边界光滑驻点或全局最优证书。
9 """
10 from pathlib import Path
11 import argparse
12 import json
13 import numpy as np
14 import pandas as pd
15 from scipy.linalg import solve
16 from scipy.optimize import linprog, minimize
17 from 非线性代理对照 import find_data, load_pair, closed, kernel,
    krr_predict
18
19 HERE=Path(__file__).resolve().parent
20
21 def recommend(data, result_dir, output_dir):
22     raw,y,indices,columns,loss_domains,_=load_pair(data,'train','lm')
23     domains=[s.removeprefix('train_the_pile_') for s in columns]
24     x=closed(raw);ref=raw.mean(axis=0);ref=ref/ref.sum()
25     measured=[domains.index(d) for d in loss_domains]
26     fixed=[i for i in range(len(domains)) if i not in measured]
27     weights=ref[measured];weights=weights/weights.sum()
28     config=json.loads((result_dir/'非线性代理对照_复现清单.json').
        read_text('utf-8-sig'))
29     gamma,lam=config['selected_gamma'],config['selected_lambda']
30     k=kernel(x,x,gamma);means=k.mean(axis=0);grand=k.mean()
31     kc=k-means[None,:]-means[:,None]+grand
32     beta=solve(kc+lam*np.eye(len(x)),y-y.mean(axis=0),assume_a='pos')
    @weights
33     offset=float(y.mean(axis=0)@weights-means@beta+grand*beta.sum())
34     beta=beta-beta.mean()
35     sx=np.sqrt(x)
36     def value_gradient(z):
37         # 仅在正份额内为通常的光滑梯度; 零份额用数值保护, 不据此证明驻
        点。
38         p=z@x;sp=np.sqrt(np.maximum(p,1e-30))
39         kval=np.exp(-gamma*np.sum((sx-sp)**2,axis=1))
40         v=kval*beta
41         gp=-gamma*(v.sum()-(v@sx)/sp)
42         return float(v.sum()+offset),x@gp
43     # 全部约束在凸组合权重上为线性; 四个无验证域固定闭合参考占比。
44     eq=np.vstack([np.ones(len(x)),x[:,fixed].T]);rhs=np.r_[1.,ref[fixed]
    ]

```

```

45     cap=2.5*ref[measured];upper=x[:,measured].T
46     init=linprog(np.zeros(len(x)),A_eq=eq,b_eq=rhs,A_ub=upper,b_ub=cap,
47     bounds=(0,None),method='highs')
48     if not init.success:raise RuntimeError('训练支持域与配比约束无共同可行点')
49     starts=[init.x]
50     rng=np.random.default_rng(20260927)
51     for _ in range(3):
52         vertex=linprog(rng.normal(size=len(x)),A_eq=eq,b_eq=rhs,A_ub=
53         upper,b_ub=cap,
54         bounds=(0,None),method='highs')
55         if vertex.success:starts.append(vertex.x)
56     fits=[]
57     for j,start in enumerate(starts):
58         z=start.copy();vertices=[z.copy()];combination=np.ones(1)
59         # 全校正条件梯度: 在少量可行顶点的凸组合上求解, 约束始终可行。
60         for iteration in range(150):
61             value,gradient=value_gradient(z)
62             vertex=linprog(gradient,A_eq=eq,b_eq=rhs,A_ub=upper,b_ub=cap
63             ,
64             bounds=(0,None),method='highs')
65             if not vertex.success:raise RuntimeError(vertex.message)
66             direction=vertex.x-z;gap=float(-gradient@direction) # 数值
67             保护梯度间隙
68             if gap<1e-6:break
69             vertices.append(vertex.x);basis=np.stack(vertices)
70             def reduced(c):
71                 val,grad=value_gradient(c@basis)
72                 return val,basis@grad
73             fit=minimize(reduced,np.r_[combination,0.],jac=True,method='
74             SLSQP',
75             bounds=[(0,1)]*len(vertices),constraints={'type
76             ':'eq','fun':lambda c:c.sum()-1.,'jac':lambda c:np.ones_like(c)},
77             options={'maxiter':1000,'ftol':1e-13})
78             combination=np.maximum(fit.x,0.);combination/=combination.
79             sum()
80             keep=combination>1e-10;combination=combination[keep];
81             combination/=combination.sum()
82             vertices=[v for v,yes in zip(vertices,keep) if yes];z=
83             combination@np.stack(vertices)
84             residual=max(float(np.max(abs(eq@z-rhs))),float(np.maximum(
85             upper@z-cap,0)).max()),float(max(0,-z.min())))
86             fits.append(dict(start=j,success=bool(gap<1e-6),objective=
87             value_gradient(z)[0],
88             constraint_residual=residual,iterations=
89             iteration+1,stationarity_gap=gap,weights=z))
90             print({k:v for k,v in fits[-1].items() if k!='weights'},flush=
91             True)
92             valid=[f for f in fits if f['success'] and f['constraint_residual'
93             ]<1e-7]
94             if not valid:raise RuntimeError('多初值推荐求解未通过可行性检查')
95             best=min(valid,key=lambda f:f['objective']);z=best['weights'];p=z@x
96             # 与正式核岭预测函数交叉核验目标值, 并保存训练凸组合证书。
97             scores=krr_predict(x,y,np.stack([ref/ref.sum(),p]),gamma,lam)
98             @weights
99             assert abs(scores[1]-best['objective'])<1e-7
100             output_dir.mkdir(parents=True,exist_ok=True)

```

```

86     pd.DataFrame(dict(domain=domains,reference_mixture=ref,
recommended_mixture=p,adjustment=p-ref)).sort_values('adjustment',
ascending=False).to_csv(output_dir/'推荐配比调整.csv',index=False,
encoding='utf-8-sig')
87     pd.DataFrame(dict(index=indices,convex_weight=z)).to_csv(output_dir/
'核岭推荐_训练凸组合.csv',index=False,encoding='utf-8-sig')
88     pd.DataFrame([{k:v for k,v in f.items() if k!='weights'} for f in
fits]).to_csv(output_dir/'核岭推荐_多初值.csv',index=False,encoding='
utf-8-sig')
89     pd.DataFrame([dict(mixture=name,predicted_weighted_loss=float(score)
,M_p=float(score-scores[0])) for name,score in zip(['参考配比','问题
一有界推荐'],scores)]).to_csv(output_dir/'核岭推荐_目标对照.csv',
index=False,encoding='utf-8-sig')
90     print(json.dumps(dict(reference=float(scores[0]),recommended=float(
scores[1]),change=float(scores[1]-scores[0]),active_training_rows=int
(np.sum(z>1e-8)),runs=[{k:v for k,v in f.items() if k!='weights'} for
f in fits]),ensure_ascii=False,indent=2))
91     return p,scores
92
93 def main():
94     ap=argparse.ArgumentParser(description=__doc__)
95     ap.add_argument('--data-dir',type=Path)
96     ap.add_argument('--output-dir',type=Path,default=HERE/'结果')
97     args=ap.parse_args()
98     recommend(find_data(args.data_dir),HERE/'结果',args.output_dir)
99
100 if __name__=='__main__':main()

```

程序 10: 问题一正式绘图入口 (求解/问题一/重绘图表.py)

```

1 # 本程序的整理与核对使用 Codex 辅助; 模型: GPT-5.6-Luna; 机构: OpenAI;
  版本发布日期: 2026-07-09。
2 """问题一十幅图的展示版入口: 只读既有结果, 不执行求解程序。
3
4 运行: python 重绘图表.py --data-dir <附件根目录>
5 输出: 同名400 dpi PNG、矢量SVG、数据及画布边界核验记录。
6 开发辅助工具: OpenAI Codex; 采用 academic-research-suite 图形 workflow。
7 """
8 from pathlib import Path
9 import argparse
10 import hashlib
11 import json
12 import sys
13 from 展示风格公共 import configure
14 from 绘图数据 import HERE, DOMAIN_NAMES, load_plot_data,
    verify_inputs_unchanged
15 from 展示风格概览 import plot_quality, plot_mixture, plot_fit
16 from 展示风格诊断 import plot_prediction, plot_coefficients,
    plot_quality_comparison, plot_extrapolation
17 from 展示风格冲突 import plot_conflicts
18 from 展示风格流程 import plot_workflow
19 from 展示风格非线性 import plot_nonlinear
20
21 NAMES = ['图0_A1-A3全量质量评分', '图1_训练配比结构', '图2_线性模型R2',
22          '图3_预测vs实测', '图4_混合系数热力图', '图5_领域质量评分',

```

```

23         '图6_冲突与赋权诊断', '图7_外推与跨尺度', '图8_问题一求解流程',
        '图9_非线性代理对照']
24
25 def main():
26     if hasattr(sys.stdout, 'reconfigure'):
27         sys.stdout.reconfigure(encoding='utf-8')
28     parser = argparse.ArgumentParser(description=__doc__)
29     parser.add_argument('--data-dir', type=Path)
30     parser.add_argument('--output-dir', type=Path, default=HERE/'图片')
31     parser.add_argument('--only', type=int, nargs='+', choices=range(10)
    , help='只重绘指定图文件编号')
32     args = parser.parse_args()
33     configure()
34     data = load_plot_data(args.data_dir)
35     args.output_dir.mkdir(parents=True, exist_ok=True)
36     functions = [plot_quality, plot_mixture, plot_fit, plot_prediction,
    plot_coefficients,
37                 plot_quality_comparison, plot_conflicts,
    plot_extrapolation, plot_workflow, plot_nonlinear]
38     selected = args.only if args.only is not None else list(range(10))
39     for i in selected:
40         functions[i](data, args.output_dir)
41         print(f'已绘制 {NAMES[i]}', flush=True)
42     verify_inputs_unchanged(data)
43     manifest = {'style': '建模展示风',
44                'skill': 'academic-research-suite 3.22.0 /
    visualization_agent',
45                'scripts_sha256': {p.name: hashlib.sha256(p.read_bytes()).
    hexdigest() for p in
46                [Path(__file__), HERE/'绘图数据.py', HERE/'出版导出.py',
    '*sorted(HERE.glob('展示风格*.py'))]}, 'input_checks': data.checks,
47                'inputs': {k: {field: value for field, value in v.items() if
    field != '_path'} for k, v in data.inputs.items()},
48                'output_figures': [NAMES[i] for i in selected],
49                'indicator_key': data.weights[['code', 'indicator', '
    display_name']].to_dict(orient='records'),
50                'domain_key': DOMAIN_NAMES, 'prediction_points': int(data.
    observed.size),
51                'coefficient_entries': int(data.centered.size), '
    whole_paper_compiled': False}
52     (args.output_dir/'绘图数据核验.json').write_text(json.dumps(manifest
    , ensure_ascii=False, indent=2), encoding='utf-8')
53     provenance = {
54         'scope': '本次生成的图形、实际读取输入和绘图程序；不替代模型假设
    与论文分页审查。',
55         'scripts_sha256': manifest['scripts_sha256'], 'inputs': manifest
    ['inputs'],
56         'figures': [{ 'file': NAMES[i]+ext,
57                        'sha256': hashlib.sha256((args.output_dir/(NAMES[i]
    )+ext)).read_bytes()).hexdigest()}
58                     for i in selected for ext in ('.png', '.svg')],
59     }
60     (args.output_dir/'图形溯源.json').write_text(json.dumps(provenance,
    ensure_ascii=False, indent=2), encoding='utf-8')
61     print(f'{len(data.checks)} 项数值核对通过；{len(data.inputs)} 个输入
    文件哈希未变。', flush=True)
62

```

```

63 if __name__ == '__main__':
64     main()

```

程序 11: 问题一绘图数据读取 (求解/问题一/绘图数据.py)

```

1 # 本程序的整理与核对使用 Codex 辅助; 模型: GPT-5.6-Luna; 机构: OpenAI;
  版本发布日期: 2026-07-09。
2 # 本程序的整理、复现核对或绘图实现使用 OpenAI Codex (GPT-5.6-Luna) 辅
  助。
3 """读取问题一既有结果并核对绘图数值; 不拟合、不优化、不写结果表。"""
4 from pathlib import Path
5 from types import SimpleNamespace
6 import hashlib
7 import json
8
9 import numpy as np
10 import pandas as pd
11
12 HERE = Path(__file__).resolve().parent
13 ROOT = HERE.parents[1]
14
15 DOMAIN_NAMES = {
16     'arxiv': '学术预印本', 'freelaw': '法律判例', 'nih_exporter': '科研
    项目',
17     'pubmed_central': '医学全文', 'wikipedia_en': '百科', '
    dm_mathematics': '数学',
18     'github': '代码', 'philpapers': '哲学论文', 'stackexchange': '专业问
    答',
19     'enron_emails': '邮件', 'gutenberg_pg_19': '图书', 'pile_cc': '网页'
    ,
20     'ubuntu_irc': '技术聊天', 'europarl': '议会记录', 'hackernews': '科
    技社区',
21     'pubmed_abstracts': '医学摘要', 'uspto_backgrounds': '专利背景',
22     'wikipedia': '百科', 'book': '图书', 'commoncrawl': 'Common Crawl',
    'c4': 'C4',
23 }
24
25 INDICATOR_NAMES = {
26     'fineweb_edu': '教育价值', 'fluency_en': '英语流畅度',
27     'modernbert_cleanliness': '文本洁净度', 'modernbert_readability': '
    可读性',
28     'modernbert_reasoning': '推理性', 'modernbert_professionalism': '专
    业性',
29     'dsir_books': '书籍域权重', 'dsir_wiki': '维基域权重', 'dsir_math':
    '数学域权重',
30     'qurater': '四维质量聚合', 'ad_en': '广告信号', 'rps_doc_word_count'
    : '文档词数',
31     'rps_doc_num_sentences': '文档句数', 'rps_doc_unigram_entropy': '一
    元词熵',
32     'rps_doc_frac_unique_words': '不同词占比', '
    rps_doc_frac_no_alph_words': '无字母词项占比',
33     'rps_doc_frac_chars_top_2gram': '高频词2元字符比',
34     'rps_doc_frac_chars_top_3gram': '高频词3元字符比',
35     'rps_lines_uppercase_letter_fraction': '大写字母占比',
36     'rps_lines_ending_with_terminal_punctution_mark': '终止标点行占比',
37     'rps_lines_numerical_chars_fraction': '数字字符占比',

```

```

38     'rps_doc_mean_word_length': '平均词长',
39 }
40
41 def sha256(path):
42     return hashlib.sha256(Path(path).read_bytes()).hexdigest()
43
44 def short_name(name):
45     return name.replace('train_the_pile_', '').replace('metric/the_pile_',
46     '').replace('_val_loss', '')
47
48 def load_plot_data(data_dir=None):
49     candidates = [Path(data_dir)] if data_dir else [ROOT/'
50     real_attachments', ROOT.parent/'real_attachments']
51     data_root = next((p for p in candidates if p.is_dir()), None)
52     if data_root is None:
53         raise FileNotFoundError('请用 --data-dir 指定 real_attachments
54         根目录。')
55     regmix = data_root/'A_data_value/regmix_tables'
56     if not regmix.is_dir():
57         raise FileNotFoundError(f'缺少配比附件目录: {regmix}')
58     inputs, checks = {}, {}
59
60     def read(path):
61         path = path.resolve()
62         df = pd.read_csv(path)
63         try:
64             label = path.relative_to(ROOT).as_posix()
65             except ValueError:
66                 label = '附件/' + path.relative_to(data_root).as_posix()
67             inputs[label] = {'sha256': sha256(path), 'rows': len(df), '
68             columns': len(df.columns),
69             '_path': path}
70         return df
71
72     def result(name, index=None):
73         frame = read(HERE/'结果'/name)
74         return frame.set_index(index, verify_integrity=True) if index
75         else frame
76
77     def attachment(name, prefix):
78         frame = read(regmix/name).set_index('index', verify_integrity=
79         True).sort_index()
80         columns = [c for c in frame.columns if c.startswith(prefix)]
81         frame = frame.loc[:, columns].rename(columns=short_name)
82         if frame.empty or not np.isfinite(frame.to_numpy()).all():
83             raise ValueError(f'附件 {name} 无有效数值或含缺失。')
84         return frame
85
86     def same(name, actual, expected, tol=1e-10):
87         actual, expected = np.asarray(actual, dtype=float), np.asarray(
88         expected, dtype=float)
89         if actual.shape != expected.shape or not np.allclose(actual,
90         expected, atol=tol, rtol=tol):
91             raise ValueError(f'绘图数值与保存结果不一致: {name}')
92         checks[name] = {'passed': True, 'max_abs_diff': float(np.max(np.
93         abs(actual-expected)))}
94
95     coef = result('混合系数矩阵.csv', 'training_domain')

```

```

87     centered = result('混合系数矩阵_中心化.csv', 'training_domain').loc[
coef.index, coef.columns]
88     intercept = result('截距.csv', 'loss_domain').loc[coef.columns, '
intercept']
89     metrics = result('问题一_拟合汇总.csv', 'loss_domain').loc[coef.
columns]
90     recommended = result('推荐配比调整.csv', 'domain').loc[coef.index]
91     scale = result('外推尺度因子.csv', 'loss_domain').loc[coef.columns]
92     cross = result('跨尺度损失比_60m_vs_1m.csv', 'loss_domain').loc[coef
.columns]
93     quality = result('质量域评分_全量.csv').sort_values('quality_mean',
ascending=False).reset_index(drop=True)
94     weights = result('质量指标熵权.csv')
95     conflicts = result('冲突指标对_top20.csv').sort_values('
conflict_rate', ascending=False)
96     proxy = result('Loss代理领域难度.csv').sort_values('quality_score_Q(
loss代理)', ascending=False)
97     link = result('质量与损失难度一致性.csv')
98     nonlinear_domains = result('非线性代理对照_逐域.csv')
99     nonlinear_predictions = result('非线性代理对照_预测明细.csv')
100
101     same('中心化系数', coef.to_numpy()-coef.to_numpy().mean(axis=0),
centered)
102     same('熵权之和', weights.weight.sum(), 1.)
103     assert len(weights) == 22 and len(quality) == 7 and coef.shape ==
(17, 13)
104     assert quality.n.sum() == 272505
105     assert set(weights.indicator) == set(INDICATOR_NAMES)
106     weights['code'] = [f'M{i:02d}' for i in range(1, 23)]
107     weights['display_name'] = weights.indicator.map(INDICATOR_NAMES)
108
109     train_x = attachment('train_mixture_1m.csv', 'train_the_pile').loc
[:, coef.index]
110     train_y = attachment('train_pile_loss_1m.csv', 'metric/the_pile').
loc[:, coef.columns]
111     if not train_x.index.equals(train_y.index):
112         raise ValueError('训练配方与损失的 index 不一致。')
113     mean_mixture=train_x.mean(axis=0)
114     same('闭合参考配比', mean_mixture/mean_mixture.sum(), recommended.
reference_mixture)
115     same('训练领域平均损失', train_y.mean(axis=0), metrics.mean_loss)
116
117     def r_squared(y, pred):
118         return 1-((y-pred)**2).sum(axis=0)/((y-y.mean(axis=0))**2).sum(
axis=0)
119
120     pred_train = train_x.to_numpy() @ coef.to_numpy() + intercept.
to_numpy()
121     same('训练 R2', r_squared(train_y.to_numpy(), pred_train), metrics.
train_R2)
122     test_x = attachment('test_mixture_1m.csv', 'train_the_pile').loc[:,
coef.index]
123     test_y = attachment('test_pile_loss_1m.csv', 'metric/the_pile').loc
[:, coef.columns]
124     if not test_x.index.equals(test_y.index):
125         raise ValueError('1M 检验配方与损失的 index 不一致。')

```

```

126     prediction = test_x.to_numpy() @ coef.to_numpy() + intercept.
to_numpy()
127     same('1M 检验 R2', r_squared(test_y.to_numpy(), prediction), metrics
.test1m_R2)
128
129     ratios = {}
130     for size, column in [('10b', 'scale_factor_10B'), ('70b', '
scale_factor_70B')]:
131         est_y = attachment(f'est_pile_loss_{size}.csv', 'metric/
the_pile_').loc[:, coef.columns]
132         if not est_y.index.isin(train_y.index).all():
133             raise ValueError(f'{size} 外推损失无法按 index 对应训练记
录。')
134         values = est_y.to_numpy()/train_y.loc[est_y.index].to_numpy()
135         same(f'{size} 外推均值', values.mean(axis=0), scale[column])
136         ratios[size] = values
137
138     test60 = attachment('test_pile_loss_60m.csv', 'metric/the_pile_').
loc[:, coef.columns]
139     if set(test60.index) != set(test_y.index):
140         raise ValueError('60M、1M 检验记录的 index 集合不同。')
141     cross_values = test60.loc[test_y.index].to_numpy()/test_y.to_numpy()
142     same('60M/1M 均值', cross_values.mean(axis=0), cross.
loss_ratio_60m_vs_1m)
143     same('60M/1M 标准差', cross_values.std(axis=0), cross.loss_ratio_std
)
144     rho = float(link.Q_A1A3.corr(link['Q_loss代理'], method='spearman'))
145     shared_path = HERE.parent/'问题一_关键量.json'
146     shared = json.loads(shared_path.read_text(encoding='utf-8'))
147     inputs[shared_path.relative_to(R00T).as_posix()] = {'sha256': sha256
(shared_path), '_path': shared_path}
148     same('质量口径 Spearman', rho, shared['spearman_quality_vs_lossproxy
'])
149     return SimpleNamespace(quality=quality, weights=weights, conflicts=
conflicts, metrics=metrics,
150                             coef=coef, centered=centered, train_x=train_x
, reference=train_x.mean(),
151                             observed=test_y.to_numpy(), prediction=
prediction, proxy=proxy, link=link,
152                             rho=rho, ratios=ratios, scale=scale, cross=
cross, inputs=inputs, checks=checks,
153                             nonlinear_domains=nonlinear_domains,
nonlinear_predictions=nonlinear_predictions)
154
155 def verify_inputs_unchanged(data):
156     for label, info in data.inputs.items():
157         if sha256(info['_path']) != info['sha256']:
158             raise RuntimeError(f'绘图期间输入文件发生变化: {label}')

```

程序 12: 问题一图形样式与导出 (求解/问题一/展示风格公共.py)

- 1 # 本程序的整理与核对使用 Codex 辅助; 模型: GPT-5.6-Luna; 机构: OpenAI; 版本发布日期: 2026-07-09。
- 2 # 本程序的整理、复现核对或绘图实现使用 OpenAI Codex (GPT-5.6-Luna) 辅助。
- 3 """Q1 展示版的字体、分组标题和导出约定。"""
- 4 from pathlib import Path

```

5 import json
6 import os
7
8 # Keep matplotlib's cache in a writable, task-local directory.
9 os.environ.setdefault('MPLCONFIGDIR', str(Path(__file__).resolve().
    parent/'__pycache__'/'matplotlib'))
10 import matplotlib
11 matplotlib.use('Agg')
12 import matplotlib.pyplot as plt
13 from matplotlib import font_manager as fm
14 from matplotlib.patches import Rectangle
15 from matplotlib.text import Text
16
17 PURPLE, TEAL, GOLD, BLUE = '#665080', '#287C78', '#BC8030', '#365E8D'
18 INK, GRAY, GRID = '#243348', '#637186', '#E6EBF1'
19
20 def configure():
21     for name in ['simsun.ttc', 'times.ttf', 'timesi.ttf', 'timesbd.ttf']:
22         path=Path('C:/Windows/Fonts')/name
23         if path.exists():
24             fm.fontManager.addfont(str(path))
25     for family in ['SimSun', 'Times New Roman']:
26         fm.findfont(family, fallback_to_default=False)
27     plt.rcParams.update({
28         'font.family':['Times New Roman', 'SimSun'], 'font.size':10,
29         'axes.labelsize':10, 'axes.titlesize':11, 'xtick.labelsize':9,
30         'ytick.labelsize':9, 'legend.fontsize':9, 'mathtext.fontset':
31         'stix',
32         'axes.unicode_minus':False, 'axes.spines.top':False, 'axes.
33         spines.right':False,
34         'axes.edgecolor': '#9AA7B7', 'axes.linewidth':.6,
35         'text.color':INK, 'axes.labelcolor':INK, 'xtick.color':GRAY, 'ytick
36         .color':INK,
37         'grid.color':GRID, 'grid.linewidth':.55, 'legend.frameon':False,
38         'axes.axisbelow':True, 'figure.facecolor': 'white', 'axes.
39         facecolor': 'white',
40         'savefig.facecolor': 'white', 'savefig.dpi':400, 'svg.fonttype': '
41         path',
42         'svg.hashsalt': 'Q1-ARS-display-v2',
43     })
44
45 def panel(ax, letter, title, color=BLUE):
46     """A compact group header; keep enough top margin for its 11 pt
47     title."""
48     ax.add_patch(Rectangle((0,1.035),1,.095,transform=ax.transAxes,
49         facecolor=color,alpha=.075,edgecolor='none',
50         clip_on=False))
51     ax.text(.015,1.082,letter,transform=ax.transAxes,va='center',ha='
52     left',
53         fontsize=11,fontweight='bold',color=color)
54     ax.text(.105,1.082,title,transform=ax.transAxes,va='center',ha='left
55     ',
56         fontsize=10.5,color=INK)
57
58 def clean(ax, axis='x', bands=False, n=None):
59     ax.grid(axis=axis)
60     ax.tick_params(length=0,pad=5)
61     if bands:

```

```

53         for y in range(0,n,2):
54             ax.axhspan(y-.5,y+.5,color='#F4F6FA',zorder=-2)
55
56 def save(fig, stem, out):
57     """Export fixed-size, 400 dpi PNG/SVG and a canvas-boundary audit.
58     """
59     out=Path(out)
60     out.mkdir(parents=True,exist_ok=True)
61     fig.canvas.draw()
62     renderer=fig.canvas.get_renderer()
63     canvas=fig.bbox
64     issues=[]
65     # Tick labels outside the current axis interval may be invisible
66     # after clipping.
67     for text in fig.findobj(Text):
68         if not text.get_visible() or not text.get_text().strip():
69             continue
70         if text.axes is not None and text.get_clip_on():
71             continue
72         b=text.get_window_extent(renderer)
73         if b.width and b.height and (b.x0<canvas.x0-1 or b.x1>canvas.x1
74 +1 or
75                                     b.y0<canvas.y0-1 or b.y1>canvas.y1
76 +1):
77             issues.append({'text':text.get_text(),'bbox':[round(v,2) for
78 v in b.bounds]})
79 fig.savefig(out/f'{stem}.png',dpi=400)
80 svg=out/f'{stem}.svg'
81 fig.savefig(svg,metadata={'Date':None,'Creator':'Q1 ARS
82 visualization workflow'})
83 svg.write_text('\n'.join(x.rstrip() for x in svg.read_text(encoding=
84 'utf-8').splitlines())+'\n',encoding='utf-8')
85 # This only checks canvas bounds, not semantic correctness or
86 internal overlaps.
87 (out/f'{stem}_布局核验.json').write_text(json.dumps({
88     'canvas_inches':list(fig.get_size_inches()),'dpi':400,
89     'boundary_issues':issues,'scope':'canvas text bounds only;
90 visual review still required'
91 },ensure_ascii=False,indent=2),encoding='utf-8')
92 plt.close(fig)
93 if issues:
94     print(f'{stem}: {len(issues)} canvas-boundary item(s) need
95 review',flush=True)

```

程序 13: 问题一概览图 (求解/问题一/展示风格概览.py)

- 1 # 本程序的整理与核对使用 Codex 辅助; 模型: GPT-5.6-Luna; 机构: OpenAI; 版本发布日期: 2026-07-09。
- 2 # 本程序的整理、复现核对或绘图实现使用 OpenAI Codex (GPT-5.6-Luna) 辅助。
- 3 """问题一质量、配方结构与拟合概览的展示图。
- 4
- 5 仅读取绘图数据模块提供的既有结果; 图中均值、标准差、配方占比及
- 6 拟合统计均按原值展示。学术图表规范来源与项目字体约定同公共模块。
- 7 """
- 8 from pathlib import Path

```

9
10 import numpy as np
11 import matplotlib.pyplot as plt
12 from matplotlib.colors import LinearSegmentedColormap
13 from matplotlib.ticker import PercentFormatter
14
15 from 绘图数据 import DOMAIN_NAMES
16 from 展示风格公共 import (
17     PURPLE, TEAL, GOLD, BLUE, INK, GRAY, GRID, configure, panel, clean,
18     save,
19 )
20 STEMS = ['图0_A1-A3全量质量评分', '图1_训练配比结构', '图2_线性模型R2']
21
22 def _names(domains):
23     return [DOMAIN_NAMES[name] for name in domains]
24
25 def plot_quality(data, out):
26     q = data.quality.sort_values('quality_mean', ascending=False).
27     reset_index(drop=True)
28     short_labels = {
29         'arxiv': '学术\n预印本', 'stackexchange': '专业\n问答',
30         'commoncrawl': 'Common\nCrawl', 'github': '代码',
31         'wikipedia': '百科', 'book': '图书', 'c4': 'C4',
32     }
33     x = np.arange(len(q))
34     fig, ax = plt.subplots(figsize=(6.5, 3.6))
35     colors = [PURPLE] + [TEAL] * (len(q) - 1)
36     ax.bar(x, q.quality_mean, width=.66, color=colors, edgecolor='white',
37           linewidth=.6, zorder=3)
38     ax.errorbar(x, q.quality_mean, yerr=q.quality_std, fmt='none',
39                ecolor=INK, elinewidth=.9, capsize=3, capthick=.9,
40                zorder=4)
41     for pos, row in q.iterrows():
42         ax.text(pos, row.quality_mean + row.quality_std + .025,
43                f'{row.quality_mean:.3f}', fontsize=9.5,
44                ha='center', va='bottom', color=colors[pos])
45         ax.text(pos, .055, f'n = \n{int(row.n):,}', fontsize=9,
46                ha='center', va='bottom', linespacing=1.2, color='white',
47                zorder=5)
48     ax.set(xticks=x, xticklabels=[short_labels[name] for name in q.
49                                domain],
50           ylabel='质量评分 Q', ylim=(0, .87), yticks=[0, .2, .4, .6,
51           .8],
52           xlim=(-.65, len(q)-.35))
53     ax.text(.99, .985, '误差线: 标准差', transform=ax.transAxes,
54            fontsize=9, color=GRAY, ha='right', va='top')
55     ax.spines['bottom'].set_color(GRID)
56     clean(ax, axis='y')
57     panel(ax, 'A', '语料领域的质量评分', PURPLE)
58     fig.subplots_adjust(left=.11, right=.975, bottom=.18, top=.82)
59     save(fig, STEMS[0], out)
60
61 def plot_mixture(data, out):
62     order = data.reference.sort_values(ascending=False).index
63     assert len(order) == 17

```

```

59 matrix = data.train_x.loc[:, order].iloc[:8].to_numpy().T
60 assert matrix.shape == (17, 8)
61 values = data.reference.loc[order].to_numpy()
62 fig = plt.figure(figsize=(5.0, 3.65))
63 grid = fig.add_gridspec(1, 2, width_ratios=[1.30, 1],
64                             left=.20, right=.98, bottom=.21, top=.86,
wspace=.18)
65 ax = fig.add_subplot(grid[0])
66 right = fig.add_subplot(grid[1], sharey=ax)
67 cmap = LinearSegmentedColormap.from_list(
68     'q1_mixture_share', ['#F4F7FA', '#D4E8E6', '#68A5A0', TEAL,
PURPLE])
69 image = ax.imshow(matrix, vmin=0, vmax=1, cmap=cmap,
70                    interpolation='nearest', aspect='auto')
71 ax.set(xticks=np.arange(8),
72        xticklabels=[str(i) for i in data.train_x.index[:8]],
73        yticks=np.arange(17), yticklabels=_names(order), xlabel='配方
编号',
74        ylim=(16.5, -.5))
75 ax.set_xticks(np.arange(-.5, 8, 1), minor=True)
76 ax.set_yticks(np.arange(-.5, 17, 1), minor=True)
77 ax.grid(which='minor', color='white', linewidth=.65)
78 ax.tick_params(which='both', length=0, pad=5)
79 for row, col in zip(*np.where(matrix >= .10)):
80     value = matrix[row, col]
81     ax.text(col, row, f'{value:.0%}', ha='center', va='center',
fontsize=9,
82             color='white' if value >= .45 else INK)
83 for spine in ax.spines.values():
84     spine.set_visible(False)
85 panel(ax, 'A', '前 8 组训练配方', PURPLE)
86
87 y = np.arange(17)
88 right.barh(y, values, color=TEAL, height=.62, edgecolor='white',
linewidth=.5,
89            zorder=3)
90 for pos, value in enumerate(values):
91     right.text(value + .005, pos, f'{value:.1%}',
92              ha='left', va='center', fontsize=9, color=TEAL)
93 right.set(xlim=(0, .17), xticks=[0, .05, .10, .15],
94          xlabel='平均训练占比')
95 right.tick_params(axis='y', labelleft=False)
96 right.xaxis.set_major_formatter(PercentFormatter(1, decimals=0))
97 right.spines['left'].set_visible(False)
98 clean(right, bands=True, n=17)
99 right.tick_params(axis='y', which='both', length=0)
100 panel(right, 'B', '平均领域占比', TEAL)
101 cax = fig.add_axes([.205, .055, .38, .020])
102 bar = fig.colorbar(image, cax=cax, orientation='horizontal',
103                    ticks=[0, .25, .5, .75, 1], format=
PercentFormatter(1, decimals=0))
104 bar.ax.tick_params(length=2, labelsize=9, pad=2)
105 bar.outline.set_linewidth(.4)
106 fig.text(.64, .065, '配方实际占比', ha='left', va='center', fontsize
=9, color=GRAY)
107 save(fig, STEMS[1], out)
108
109 def plot_fit(data, out):

```

```

110 m = data.metrics.sort_values('test1m_R2', ascending=False)
111 fig, axes = plt.subplots(1, 2, figsize=(5.0, 3.45), sharey=True)
112 y = np.arange(len(m))
113 specs = [
114     (axes[0], 'train_R2', 'test1m_R2', '训练 1M', '检验 1M',
115      PURPLE, TEAL, 'o', 'o', '决定系数 R2'),
116     (axes[1], 'test60m_r', 'test1B_r', '检验 60M', '检验 1B',
117      BLUE, GOLD, '^', 's', 'Pearson 相关系数 r'),
118 ]
119 for ax, col1, col2, lab1, lab2, color1, color2, marker1, marker2,
xlabel in specs:
120     ax.hlines(y, np.minimum(m[col1], m[col2]), np.maximum(m[col1], m
[col2])),
121              color='#B9C5D3', linewidth=1.35, zorder=2)
122     ax.scatter(m[col1], y, s=29, marker=marker1,
123               color=color1, edgecolor='white', linewidth=.45, label
=lab1, zorder=4)
124     ax.scatter(m[col2], y, s=33, marker=marker2,
125               facecolor='none', edgecolor=color2, linewidth=1.05,
label=lab2, zorder=5)
126     ax.set(xlim=(0, 1), xticks=[0, .25, .5, .75, 1],
127            ylim=(len(m)-.6, -.6), xlabel=xlabel)
128     ax.legend(loc='upper center', bbox_to_anchor=(.5, -.27), ncol=2,
129              fontsize=9, handletextpad=.28, columnspacing=.8,
borderaxespad=0)
130     clean(ax, bands=True, n=len(m))
131 axes[0].set(yticks=y, yticklabels=_names(m.index))
132 axes[1].tick_params(axis='y', labelleft=False)
133 axes[1].spines['left'].set_visible(False)
134 panel(axes[0], 'A', '同尺度拟合与检验', PURPLE)
135 panel(axes[1], 'B', '跨尺度相关性', BLUE)
136 fig.subplots_adjust(left=.20, right=.96, bottom=.25, top=.85, wspace
=.18)
137 save(fig, STEMS[2], out)
138
139 if __name__ == '__main__':
140     import argparse
141     import sys
142     from 绘图数据 import load_plot_data, verify_inputs_unchanged
143     if hasattr(sys.stdout, 'reconfigure'):
144         sys.stdout.reconfigure(encoding='utf-8')
145     parser = argparse.ArgumentParser(description=__doc__)
146     parser.add_argument('--data-dir', type=Path, required=True)
147     parser.add_argument('--output-dir', type=Path, required=True)
148     args = parser.parse_args()
149     configure()
150     data = load_plot_data(args.data_dir)
151     for draw in [plot_quality, plot_mixture, plot_fit]:
152         draw(data, args.output_dir)
153     verify_inputs_unchanged(data)
154     print(f'{len(data.checks)} 项数据核对通过; 已绘制 3 张概览图。')

```

程序 14: 问题一诊断图 (求解/问题一/展示风格诊断.py)

- 1 # 本程序的整理与核对使用 Codex 辅助; 模型: GPT-5.6-Luna; 机构: OpenAI;
版本发布日期: 2026-07-09。

```

2 # 本程序的整理、复现核对或绘图实现使用 OpenAI Codex (GPT-5.6-Luna) 辅助。
3 """问题一预测、系数、质量口径及尺度诊断的展示图；不重新求解。"""
4 from 展示风格公共 import panel, clean, save, PURPLE, TEAL, GOLD, BLUE,
    INK, GRAY
5 import numpy as np
6 import matplotlib.pyplot as plt
7 from matplotlib.colors import LinearSegmentedColormap, SymLogNorm,
    to_rgb
8 from matplotlib.patches import Patch
9 from matplotlib.ticker import MaxNLocator
10 from 绘图数据 import DOMAIN_NAMES
11
12 SEQ = LinearSegmentedColormap.from_list('q1_density', ['#EEF4F5', '#8
    DC1BE', TEAL, '#24426C'])
13 DIVERGING = LinearSegmentedColormap.from_list('q1_signed', [TEAL, '#
    ACD2CD', '#F9FAFC', '#C2B5D1', PURPLE])
14
15 def labels(domains):
16     return [DOMAIN_NAMES[d] for d in domains]
17
18 def annotation_color(background):
19     """Select the higher-contrast text colour for the actual rendered
    cell."""
20     def luminance(rgb):
21         c=np.array(rgb[:3])
22         linear=np.where(c<=.04045,c/12.92,((c+.055)/1.055)**2.4)
23         return float(linear @ np.array([.2126,.7152,.0722]))
24     bg=luminance(background)
25     dark=luminance(to_rgb(INK))
26     white_contrast=1.05/(bg+.05)
27     dark_contrast=(max(bg,dark)+.05)/(min(bg,dark)+.05)
28     return 'white' if white_contrast>dark_contrast else INK
29
30 def plot_prediction(data, out):
31     x, y = data.observed.ravel(), data.prediction.ravel()
32     lo, hi = min(x.min(),y.min()), max(x.max(),y.max())
33     lo, hi = np.floor(lo*2)/2-.1, np.ceil(hi*2)/2+.1
34     fig = plt.figure(figsize=(4.3,3.7))
35     # Equal axis lengths and identical numerical ranges preserve the 45-
    degree reference.
36     square_height = .57 * 4.3 / 3.7
37     ax = fig.add_axes([.15,.17,.57,square_height])
38     top = fig.add_axes([.15,.855,.57,.085], sharex=ax)
39     right = fig.add_axes([.735,.17,.075,square_height], sharey=ax)
40     cax = fig.add_axes([.865,.29,.025,.39])
41     h = ax.hexbin(x,y,gridsize=43,mincnt=1,cmap=SEQ,linewidths=0,
42         extent=(lo,hi,lo,hi),rasterized=False)
43     ax.plot([lo,hi],[lo,hi],color=GOLD,lw=1.15,ls=(0,(4,3)),label='预测
    值 = 实测值')
44     ax.set(xlim=(lo,hi),ylim=(lo,hi),xlabel='实测验验证损失',ylabel='模型
    预测损失',aspect='equal')
45     ax.legend(loc='lower right',fontsize=8)
46     ax.text(.035,.963,f'N = {len(x):,}',transform=ax.transAxes,ha='left'
    ,va='top',
47         fontsize=10,color=BLUE,bbox={'facecolor':'white','edgecolor'
    : 'none','alpha':.9,'pad':3})

```

```

48     bins=np.linspace(lo,hi,37)
49     top.hist(x,bins=bins,color=TEAL,alpha=.65,edgecolor='white',
linewidth=.2)
50     right.hist(y,bins=bins,orientation='horizontal',color=PURPLE,alpha
=.65,edgecolor='white',linewidth=.2)
51     top.axis('off');right.axis('off')
52     cb=fig.colorbar(h,cax=cax)
53     cb.ax.set_title('记录数',fontsize=9,pad=8)
54     cb.locator=MaxNLocator(5,integer=True);cb.update_ticks()
55     cb.outline.set_visible(False)
56     save(fig,'图3_预测vs实测',out)
57
58 def plot_coefficients(data,out):
59     matrix=data.centered.to_numpy()
60     vmax=float(np.abs(matrix).max())
61     fig=plt.figure(figsize=(5.3,4.3))
62     ax=fig.add_axes([.18,.29,.79,.57])
63     # Only colour mapping changes: all 221 original coefficients are
retained.
64     norm=SymLogNorm(linthresh=1,linscale=.5,vmin=-vmax,vmax=vmax,base
=10)
65     mesh=ax.imshow(matrix,aspect='auto',interpolation='nearest',cmap=
DIVERGING,norm=norm)
66     wrapped={'学术预印本':'学术\n预印本','法律判例':'法律\n判例','医学全
文':'医学\n全文',
67             '专业问答':'专业\n问答','技术聊天':'技术\n聊天','科技社区':
'科技\n社区',
68             '医学摘要':'医学\n摘要','专利背景':'专利\n背景'}
69     ax.set(yticks=np.arange(17),yticklabels=labels(data.centered.index),
70           xticks=np.arange(13),xticklabels=[wrapped.get(t,t) for t in
labels(data.centered.columns)])
71     ax.tick_params(length=0,pad=5)
72     ax.set_xticks(np.arange(-.5,13,1),minor=True)
73     ax.set_yticks(np.arange(-.5,17,1),minor=True)
74     ax.grid(which='minor',color='white',linewidth=.5)
75     ax.tick_params(which='minor',length=0)
76     for j in range(13):
77         i=int(np.argmax(np.abs(matrix[:,j])))
78         v=matrix[i,j]
79         ax.text(j,i,f'{v:+.1f}',ha='center',va='center',fontsize=9,
80               color=annotation_color(DIVERGING(norm(v))))
81     panel(ax,'A','17 个训练域 × 13 个验证域',PURPLE)
82     ax.set_xlabel('验证领域',labelpad=9)
83     cax=fig.add_axes([.29,.12,.60,.025])
84     cb=fig.colorbar(mesh,cax=cax,orientation='horizontal',ticks=[-15,-5,
-1,0,1,5,15])
85     cb.ax.set_xticklabels(['-15','-5','-1','0','1','5','15'])
86     cb.set_label('中心化系数 (对称对数色标)',labelpad=4)
87     cb.outline.set_visible(False)
88     save(fig,'图4_混合系数热力图',out)
89
90 def plot_quality_comparison(data,out):
91     p=data.proxy
92     fig=plt.figure(figsize=(5.3,3.65))
93     ax=fig.add_axes([.16,.17,.315,.66])
94     ar=fig.add_axes([.56,.17,.40,.66])
95     y=np.arange(len(p))

```

```

96     vals=p['quality_score_Q(loss代理)'].to_numpy()
97     ax.barh(y,vals,height=.57,color=BLUE,alpha=.85)
98     for i,v in enumerate(vals):
99         ax.text(v+.025,i,f'{v:.2f}',ha='left',va='center',fontsize=9,
color=BLUE)
100     ax.set(yticks=y,yticklabels=labels(p.domain),ylim=(12.6,-.6),xlim
=(0,1.2),
101           xticks=[0,.5,1],xlabel='损失代理评分')
102     clean(ax,bands=True,n=13)
103     panel(ax,'A','13 域代理评分',BLUE)
104     link=data.link.set_index('mixture_domain')
105     rank_a=link.Q_A1A3.rank(ascending=False).astype(int)
106     rank_b=link['Q_loss代理'].rank(ascending=False).astype(int)
107     # Lines encode identity across two methods, not a time trend or
score change.
108     colors=[BLUE,TEAL,GOLD,PURPLE,'#708398','#9D7465']
109     for color,d in zip(colors,rank_a.sort_values().index):
110         a,b=rank_a[d],rank_b[d]
111         ar.plot([0,1],[a,b],color=color,lw=1.8,alpha=.85,zorder=2)
112         ar.scatter([0,1],[a,b],s=110,c=color,edgecolors='white',
linewidth=1,zorder=3)
113         for xpos,r in [(0,a),(1,b)]:
114             ar.text(xpos,r,str(r),color='white',fontsize=9,ha='center',
va='center',zorder=4)
115             ar.text(-.11,a,DOMAIN_NAMES[d],ha='right',va='center',fontsize
=9,color=color)
116             ar.text(1.11,b,DOMAIN_NAMES[d],ha='left',va='center',fontsize=9,
color=color)
117         ar.set(xlim=(-.82,1.82),ylim=(6.65,.2))
118         for xpos,name in [(0,'指标质量'),(1,'损失代理')]:
119             ar.text(xpos,.48,name,ha='center',va='center',fontsize=9,color=
INK)
120         ar.text(.5,6.5,rf'Spearman $\rho$={data.rho:.2f}$',ha='center',va='
center',fontsize=10,color=GRAY)
121         ar.axis('off')
122     panel(ar,'B','6 个公共领域的排名',TEAL)
123     save(fig,'图5_领域质量评分',out)
124
125 def plot_extrapolation(data,out):
126     domains=list(data.coef.columns)
127     order=np.argsort(data.ratios['10b'].mean(axis=0))
128     names=[domains[i] for i in order]
129     cross=data.cross.loc[names]
130     fig=plt.figure(figsize=(6.5,4.8))
131     ax=fig.add_axes([.15,.15,.40,.64])
132     ar=fig.add_axes([.64,.15,.325,.64],sharey=ax)
133     y=np.arange(13)
134     for size,offset,color in [('10b',-.17,TEAL),('70b',.17,PURPLE)]:
135         samples=[data.ratios[size][:,i] for i in order]
136         ax.boxplot(samples,positions=y+offset,vert=False,widths=.24,
patch_artist=True,
137                   whis=(0,100),showliers=False,manage_ticks=False,
138                   boxprops={'facecolor':color,'edgecolor':color,'alpha
':.42,'linewidth':.7},
139                   medianprops={'color':color,'linewidth':1.3},
140                   whiskerprops={'color':color,'linewidth':.75},
141                   capprops={'color':color,'linewidth':.75})

```

```

142     ax.set(yticks=y,yticklabels=labels(names),ylim=(12.6,-.6),xlabel='外
    推损失 / 1M 训练损失')
143     ax.xaxis.set_major_locator(MaxNLocator(4))
144     ar.errorbar(cross.loss_ratio_60m_vs_1m,y,xerr=cross.loss_ratio_std,
    fmt='o',
145                 color=GOLD,ecolor=GOLD,ms=4,capsize=2.5,elinewidth=1.1)
146     ar.axvline(1,color=GRAY,lw=.8,ls=(0,(3,3)))
147     ar.tick_params(labelleft=False)
148     ar.set(xlim=(.42,1.035),xticks=[.5,.7,.9,1],xlabel='60M / 1M 检验损
    失')
149     for a in [ax,ar]:
150         clean(a,bands=True,n=13)
151     panel(ax,'A','10B / 70B 外推分布',TEAL)
152     panel(ar,'B','同配方跨尺度比值',GOLD)
153     fig.legend(handles=[Patch(facecolor=TEAL,alpha=.55,label='10B'),
154                          Patch(facecolor=PURPLE,alpha=.55,label='70B')],
155               loc='upper left',bbox_to_anchor=(.15,.962),ncol=2,
    handlelength=1.3,columnspacing=1.8)
156     save(fig,'图7_外推与跨尺度',out)

```

程序 15: 问题一冲突图 (求解/问题一/展示风格冲突.py)

```

1 # 本程序的整理与核对使用 Codex 辅助; 模型: GPT-5.6-Luna; 机构: OpenAI;
    发布日期: 2026-07-09。
2 # 本程序的整理、复现核对或绘图实现使用 OpenAI Codex (GPT-5.6-Luna) 辅
    助。
3 """用原有前 12 对冲突关系和 22 项熵权绘制网络与权重面板。"""
4 from pathlib import Path
5
6 import numpy as np
7 import matplotlib.pyplot as plt
8 from matplotlib.lines import Line2D
9 from matplotlib.ticker import PercentFormatter
10
11 from 展示风格公共 import configure, save, panel, PURPLE, TEAL, GOLD,
    BLUE, INK, GRAY, GRID
12
13 def plot_conflicts(data, out):
14     """节点等大、连线宽度从零按冲突率线性编码, 不推断其他连接。"""
15     weights = data.weights.sort_values('weight', ascending=False)
16     key = data.weights.set_index('indicator')
17     pairs = data.conflicts.sort_values('conflict_rate', ascending=False)
18     .head(12)
19     edges = [(key.loc[a, 'code'], key.loc[b, 'code'], float(row.
    conflict_rate))
20              for row in pairs.itertuples()
21              for a, b in [row.indicator_pair.split('|')]]
22     nodes = {node for edge in edges for node in edge[:2]}
23     # A fixed tree layout keeps every observed edge visible and avoids
    crossings.
24     positions = {
25         'M15': (.39, .64), 'M14': (.05, .92), 'M06': (.54, .92),
26         'M02': (.93, .69), 'M04': (.04, .63),
27         'M05': (.05, .38), 'M19': (.37, .30), 'M01': (.62, .40),
28         'M22': (.95, .38), 'M16': (.74, .14), 'M03': (1.04, .13),
29         'M12': (.38, .08), 'M18': (.06, .08),

```

```

30     if nodes != set(positions):
31         raise ValueError('冲突指标集合改变, 需要重新检查固定网络布局。')
32     kind = data.weights.set_index('code').list_type.to_dict()
33
34     fig = plt.figure(figsize=(6.5, 5.8))
35     network = fig.add_axes([.035, .14, .415, .71])
36     bars = fig.add_axes([.73, .14, .245, .71])
37     # A separate header spans both the bar labels and plotting area.
38     header = fig.add_axes([.48, .14, .495, .71], frameon=False)
39     header.set_axis_off()
40     panel(network, 'A', '高冲突指标网络', color=GOLD)
41     panel(header, 'B', '22 项质量指标的熵权', color=PURPLE)
42     network.set(xlim=(-.055, 1.135), ylim=(-.015, 1.06))
43     network.set_axis_off()
44
45     for a, b, value in edges:
46         x1, y1 = positions[a]
47         x2, y2 = positions[b]
48         network.plot([x1, x2], [y1, y2], color=GOLD, alpha=.65,
49                     linewidth=6 * value, zorder=1, solid_capstyle='
round')
50     # Offset perpendicular to each edge by a small screen-space
distance.
51     dx = (x2-x1) * .415 * 6.5
52     dy = (y2-y1) * .71 * 5.8
53     length = np.hypot(dx, dy)
54     offset = (-dy/length*5.8, dx/length*5.8)
55     network.annotate(f'{value:.1%}', ((x1+x2)/2, (y1+y2)/2),
56                   xytext=offset, textcoords='offset points',
57                   ha='center', va='center', fontsize=9, color=
GRAY,
58                   bbox={'facecolor':'white', 'edgecolor':'none',
'pad':.35}, zorder=3)
59     for node in sorted(nodes):
60         color = PURPLE if kind[node] else TEAL
61         network.scatter(*positions[node], s=475, color=color, edgecolor=
'white',
62                       linewidth=1.2, zorder=4)
63         network.text(*positions[node], node, color='white', fontsize=9,
64                   ha='center', va='center', zorder=5)
65
66     y = np.arange(len(weights))
67     colors = [PURPLE if is_list else TEAL for is_list in weights.
list_type]
68     for i in range(0, len(weights), 2):
69         bars.axhspan(i-.5, i+.5, color='#F4F6FA', zorder=0)
70         bars.barh(y, weights.weight, color=colors, height=.58, edgecolor='
none', zorder=2)
71         for i, value in enumerate(weights.weight):
72             bars.text(value+.006, i, f'{value:.1%}', color=INK, fontsize=9,
73                   ha='left', va='center')
74         bars.set(yticks=y,
75               yticklabels=[f'{row.code} {row.display_name}' for row in
weights.itertuples()],
76               ylim=(21.7, -.7), xlim=(0, .19), xticks=[0,.05,.10,.15],
77               xlabel='熵权占比')
78         bars.tick_params(axis='y', length=0, pad=5, labelsize=9)

```

```

79     bars.tick_params(axis='x', length=0, labelsz=9)
80     bars.spines['left'].set_visible(False)
81     bars.spines['bottom'].set_visible(False)
82     bars.xaxis.set_major_formatter(PercentFormatter(1, decimals=0))
83     bars.grid(axis='x', color=GRID, linewidth=.55)
84
85     handles = [Line2D([], [], marker='o', color='none', markerfacecolor=
color,
86                     markeredgecolor='none', markersize=6, label=label)
87               for color, label in [(PURPLE, '列表型指标'), (TEAL, '标量
型指标')]]
88     fig.legend(handles=handles, loc='lower center', bbox_to_anchor=(.50,
.025),
89               ncol=2, columnspacing=2.5, handletextpad=.5, fontsize=9)
90     network.text(.5, -.065, '前 12 对冲突关系; 线宽正比于冲突率',
91                 transform=network.transAxes, ha='center', va='top',
fontsize=9, color=GRAY)
92     save(fig, '图6_冲突与赋权诊断', Path(out))
93
94 if __name__ == '__main__':
95     import argparse
96     from 绘图数据 import load_plot_data, verify_inputs_unchanged
97     parser = argparse.ArgumentParser(description=__doc__)
98     parser.add_argument('--data-dir', type=Path, required=True)
99     parser.add_argument('--output-dir', type=Path, required=True)
100     args = parser.parse_args()
101     configure()
102     source = load_plot_data(args.data_dir)
103     plot_conflicts(source, args.output_dir)
104     verify_inputs_unchanged(source)

```

程序 16: 问题一流程图 (求解/问题一/展示风格流程.py)

```

1 # 本程序的整理与核对使用 Codex 辅助; 模型: GPT-5.6-Luna; 机构: OpenAI;
  版本发布日期: 2026-07-09。
2 # 本程序的整理、复现核对或绘图实现使用 OpenAI Codex (GPT-5.6-Luna) 辅
  助。
3 """问题一双泳道流程图; 仅调整展示, 不运行模型或读取原始附件。
4
5 使用 academic-research-suite 的 visualization_agent 绘图检查规范;
6 中文宋体、英文 Times New Roman 与竞赛图注格式遵循本项目要求。
7 """
8 from pathlib import Path
9
10 import matplotlib.pyplot as plt
11 from matplotlib.patches import FancyArrowPatch, FancyBboxPatch,
  Rectangle
12 from matplotlib.colors import to_rgb
13
14 from 展示风格公共 import PURPLE, TEAL, GOLD, BLUE, INK, GRAY, GRID, save
15
16 def _tint(color, strength=.075):
17     """Blend a group color into the white paper background."""
18     return tuple(1 - strength * (1 - value) for value in to_rgb(color))
19
20 def plot_workflow(data, out):

```

```

21     """Render the existing Q1 procedure as two clearly connected
    swimlanes.
22
23     ``data`` is accepted for compatibility with the numeric figure
    generators.
24     There are no computed quantities in this conceptual figure.
25     """
26     out = Path(out)
27     out.mkdir(parents=True, exist_ok=True)
28     fig = plt.figure(figsize=(6.5, 4.5))
29     ax = fig.add_axes([0, 0, 1, 1])
30     ax.set(xlim=(0, 6.5), ylim=(0, 4.5))
31     ax.axis('off')
32
33     def lane(y, h, label, color):
34         ax.add_patch(FancyBboxPatch(
35             (.08, y), 6.34, h,
36             boxstyle='round,pad=0,rounding_size=.06',
37             facecolor=_tint(color, .045), edgecolor=_tint(color, .22),
38             linewidth=.7, zorder=0))
39         ax.add_patch(Rectangle((.08, y + h - .33), .045, .22,
40                                facecolor=color, edgecolor='none', zorder
41                                =1))
42         ax.text(.23, y + h - .22, label, fontsize=10.5, color=color,
43                ha='left', va='center')
44
45     def node(x, y, w, h, heading, body, color, body_size=9.2):
46         ax.add_patch(FancyBboxPatch(
47             (x, y), w, h, boxstyle='round,pad=0,rounding_size=.035',
48             facecolor='white', edgecolor=_tint(color, .55),
49             linewidth=.85, zorder=3))
50         ax.add_patch(Rectangle((x + .012, y + h - .27), w - .024, .258,
51                                facecolor=_tint(color, .105), edgecolor='
52                                none',
53                                zorder=4))
54         ax.text(x + w/2, y + h - .139, heading, fontsize=9.4,
55                color=color, ha='center', va='center', zorder=5)
56         ax.text(x + w/2, y + (h - .27)/2 - .006, body, fontsize=
57                body_size,
58                color=INK, ha='center', va='center', linespacing=1.33,
59                zorder=5)
60
61     def arrow(points, color=GRAY, dashed=False):
62         linestyle = (0, (3.2, 2.1)) if dashed else '-'
63         if len(points) > 2:
64             xx, yy = zip(*points[:-1])
65             ax.plot(xx, yy, color=color, linewidth=.95,
66                    linestyle=linestyle, solid_capstyle='round', zorder
67                    =2)
68         ax.add_patch(FancyArrowPatch(
69             points[-2], points[-1], arrowstyle='->', mutation_scale=8,
70             linewidth=.95, color=color, linestyle=linestyle,
71             shrinkA=0, shrinkB=0, zorder=2))
72
73     lane(2.14, 2.21, '质量评价', PURPLE)
74     lane(.44, 1.57, '配比回归', TEAL)

```

```

71 # Upper lane: normalized quality signals drive both scoring and
    diagnostics.
72 node(.20, 2.95, 1.88, 1.01, 'A1—A3 质量信号',
73      '22 项指标的缺失处理\n列表标量化、方向统一\n归一化并保留领域标
    签', PURPLE)
74 node(2.55, 3.06, 1.50, .90, '熵权综合评价',
75      '样本与语料评分\n领域质量评分', PURPLE)
76 node(4.54, 3.06, 1.72, .90, 'A16 领域映射',
77      '构造配比加权\n质量指数', PURPLE)
78 node(.20, 2.18, 1.88, .64, '指标冲突诊断',
79      '成对标准化差异\nKendall 并列秩校正', GOLD, body_size=9)
80
81 arrow([(2.11, 3.50), (2.50, 3.50)], PURPLE)
82 arrow([(4.08, 3.50), (4.49, 3.50)], PURPLE)
83 arrow([(1.14, 2.92), (1.14, 2.85)], GOLD)
84
85 # Lower lane: exactly the same four stages as the existing
    manuscript.
86 y, w, h = .59, 1.40, 1.04
87 node(.18, y, w, h, 'A4—A5 配方与损失',
88      '按 index 对齐\n17 域训练配比\n13 域验证损失', TEAL, body_size
    =9)
89 node(1.76, y, w, h, '岭回归与核岭',
90      '线性系数解释\n核岭同尺度预测', TEAL)
91 node(3.34, y, w, h, '检验与跨尺度',
92      '1M / 60M / 1B 检验\n10B / 70B 外推表', BLUE, body_size=9)
93 node(4.92, y, w, h, '核岭支持域推荐',
94      '训练配方凸组合\n固定未测量域\n保留配比上限', GOLD, body_size
    =9)
95 for right, left in [(1.58, 1.76), (3.16, 3.34), (4.74, 4.92)]:
96     arrow([(right + .025, 1.105), (left - .025, 1.105)], TEAL)
97
98 # The quality term is an incremental diagnostic, not the sole
    regression input.
99 arrow([(5.40, 3.02), (5.40, 2.20), (2.46, 2.20), (2.46, 1.66)],
100      color=PURPLE, dashed=True)
101 ax.text(3.88, 2.31, '追加质量项的冗余特征对照', fontsize=9,
102      color=PURPLE, ha='center', va='center',
103      bbox={'boxstyle': 'square,pad=.15',
104      'facecolor': _tint(PURPLE, .045), 'edgecolor': 'none'
    }, zorder=5)
105
106 # Compact line-style key; figure number and caption remain in the
    LaTeX file.
107 arrow([(0.22, .21), (.62, .21)], GRAY)
108 ax.text(.72, .21, '主要处理流程', fontsize=9, va='center', color=
    GRAY)
109 arrow([(3.08, .21), (3.48, .21)], PURPLE, dashed=True)
110 ax.text(3.58, .21, '冗余质量特征对照', fontsize=9, va='center',
    color=GRAY)
111
112 save(fig, '图8_问题一求解流程', out)
113
114 if __name__ == '__main__':
115     import argparse

```

```

116 from 展示风格公共 import configure
117 parser = argparse.ArgumentParser(description=__doc__)
118 parser.add_argument('--output-dir', type=Path, required=True)
119 args = parser.parse_args()
120 configure()
121 plot_workflow(None, args.output_dir)

```

程序 17: 问题一核岭验证图 (求解/问题一/展示风格非线性.py)

```

1 # 本程序的整理与核对使用 Codex 辅助; 模型: GPT-5.6-Luna; 机构: OpenAI;
  版本发布日期: 2026-07-09。
2 # 本程序由 OpenAI Codex (GPT-5.6-Luna) 辅助实现; 仅从保存的验证结果绘
  图。
3 """核岭增强的证据图: 逐域精度及逐配方平均损失; 不重新拟合。"""
4 from pathlib import Path
5 import numpy as np
6 import pandas as pd
7 import matplotlib.pyplot as plt
8 from 展示风格公共 import PURPLE, TEAL, configure
9 from 绘图数据 import DOMAIN_NAMES
10 from 出版导出 import export_figure
11
12 def plot_nonlinear(data, out):
13     table = data.nonlinear_domains
14     table = table[table.scope.eq('lm')]
15     a = table[table.model.eq('ridge')].set_index('loss_domain')
16     b = table[table.model.eq('sqrt_krr')].set_index('loss_domain').loc[a
        .index]
17     pred = data.nonlinear_predictions
18     pred = pred[pred.scope.eq('lm') & pred.model.eq('sqrt_krr')]
19     p = pred.groupby('index')[['observed', 'predicted']].mean()
20     assert len(p) == 256 and len(a) == 13 and len(pred) == 3328
21     configure()
22     fig, axes = plt.subplots(1, 2, figsize=(5.92, 3.65), gridspec_kw={'
        width_ratios':[1.1,1]})
23     ax = axes[0]; y = np.arange(len(a))
24     for i in y:
25         ax.plot([a.R2.iloc[i], b.R2.iloc[i]], [i,i], color='#CAD1D9', lw
            =1.4)
26     ax.scatter(a.R2,y,s=24,marker='o',color=PURPLE,label='线性岭')
27     ax.scatter(b.R2,y,s=29,marker='D',color=TEAL,label='平方根核岭')
28     ax.set(yticks=y, yticklabels=[DOMAIN_NAMES[d] for d in a.index],
        xlim=(0,1.025),
29         xlabel='逐域 R2', title='(a) 同尺度预测精度')
30     ax.invert_yaxis(); ax.grid(axis='x'); ax.tick_params(labelsize=8)
31     ax.legend(loc='lower left', bbox_to_anchor=(-.04,-.34), ncol=2,
        fontsize=8,
32         handletextpad=.3, columnspacing=.9)
33     ax=axes[1]
34     ax.scatter(p.observed,p.predicted,s=9,alpha=.6,color=TEAL,edgecolors
        ='none')
35     lim=[min(p.min())-.08,max(p.max())+.08]
36     ax.plot(lim,lim,color='#79818C',ls='--',lw=.9)
37     ax.set(xlim=lim,ylim=lim,xlabel='观测平均损失',ylabel='预测平均损失'
        ,
38         title='(b) 256 组检验配方')
39     ax.set_aspect('equal', adjustable='box');ax.grid()

```

```

40     rmse=np.sqrt(np.mean((p.observed-p.predicted)**2))
41     ax.text(.04,.95,f'配方均值 RMSE = {rmse:.4f}',transform=ax.transAxes
,va='top',fontsize=8)
42     for ax in axes:
43         ax.title.set_fontsize(9); ax.xaxis.label.set_fontsize(9);ax.
yaxis.label.set_fontsize(9)
44     fig.subplots_adjust(left=.14,right=.99,bottom=.22,top=.91,wspace
=.42)
45     export_figure(fig,str(Path(out)+'/图9_非线性代理对照'),formats=['png'
,'svg'],
46                     dpi=400,size_inches=(5.92,3.65),tight=False)
47     # 统一矢量文本行尾并去除导出时钟，保持版本对照稳定。
48     svg = Path(out)+'/图9_非线性代理对照.svg'
49     svg.write_text('\n'.join(line.rstrip() for line in svg.read_text(
encoding='utf-8').splitlines()
50                     if '<dc:date>' not in line)+'\n', encoding=
'utf-8')
51     plt.close(fig)

```

程序 18: 出版图形导出 (求解/问题一/出版导出.py)

```

1 # 本程序的整理与核对使用 Codex 辅助；模型：GPT-5.6-Luna；机构：OpenAI；
   版本发布日期：2026-07-09。
2 # 本程序的整理、复现核对或绘图实现使用 OpenAI Codex (GPT-5.6-Luna) 辅
   助。
3 """
4 scipilot-figure-skill :: export_figure.py
5 =====
6 Unified figure export to multiple formats at exact final size.
7
8 - Vector preferred: PDF / SVG / EPS for line/bar/scatter (lossless,
   journal-friendly).
9 - Raster for photos / micrographs: PNG / TIFF at >= 300 DPI; never JPEG
   for data figures.
10 - Embeds TrueType fonts (fonttype 42) so journals don't reject Type-3
   PDFs.
11 - Optional grayscale preview to sanity-check colorblind safety.
12
13 Usage
14 -----
15     from export_figure import export_figure
16     import matplotlib.pyplot as plt
17
18     fig, ax = plt.subplots()
19     ax.plot([0,1,2],[3,1,4])
20
21     paths = export_figure(
22         fig,
23         basename="figs/fig1_main",
24         formats=["pdf", "svg", "png"],
25         size_inches=(3.5, 2.625), # 强制成 Nature 单栏尺寸
26         dpi=600,
27         grayscale_preview=True,
28     )

```

```

29     # -> ['figs/fig1_main.pdf', 'figs/fig1_main.svg',
30     #     'figs/fig1_main.png', 'figs/fig1_main_grayscale.png']
31
32 CLI: ``python export_figure.py demo`` 生成一张演示图并多格式导出。
33 """
34 from __future__ import annotations
35
36 import argparse
37 import os
38 import sys
39 from typing import Iterable
40
41 import matplotlib.pyplot as plt
42
43 VECTOR_FORMATS = {"pdf", "svg", "eps"}
44 RASTER_FORMATS = {"png", "tiff", "tif", "jpg", "jpeg"}
45 SUPPORTED_FORMATS = VECTOR_FORMATS | RASTER_FORMATS
46
47 def _ensure_parent(path: str) -> None:
48     parent = os.path.dirname(os.path.abspath(path))
49     if parent and not os.path.exists(parent):
50         os.makedirs(parent, exist_ok=True)
51
52 def export_figure(
53     fig,
54     basename: str,
55     formats: Iterable[str] | None = None,
56     dpi: int = 300,
57     size_inches: tuple[float, float] | None = None,
58     grayscale_preview: bool = False,
59     tight: bool = True,
60     pad_inches: float = 0.05,
61     transparent: bool = False,
62 ) -> list[str]:
63     """
64     Export a matplotlib Figure to one or more formats at exact final
65     size.
66
67     Args:
68         fig: matplotlib Figure object.
69         basename: 输出路径前缀（不含扩展名）；可包含子目录，自动创建。
70         formats: list/tuple of extensions, e.g. ['pdf', 'svg', 'png'].
71             Default: ['pdf', 'svg', 'png'].
72         dpi: raster 格式分辨率；建议 300（普通） / 600（IEEE 等）。
73         size_inches: (width, height) 英寸；指定后会 fig.set_size_inches
74         ( ) 强制
75             最终尺寸。强烈建议传入——保证导出后不必在 Word/LaTeX 里二次
76             缩放。
77         grayscale_preview: 额外生成一张 _grayscale.png 供色盲安全检查。
78         tight: 是否走 bbox_inches='tight'（裁掉留白）。
79         pad_inches: tight 模式下保留的边距（英寸）。
80         transparent: 透明背景（PPT/海报可能需要）。
81
82     Returns:

```

```

80     实际写出的文件路径列表。
81     """
82     if formats is None:
83         formats = ("pdf", "svg", "png")
84     formats = [f.lower().lstrip(".") for f in formats]
85     unknown = [f for f in formats if f not in SUPPORTED_FORMATS]
86     if unknown:
87         raise ValueError(f"Unsupported formats: {unknown}. "
88                           f"Supported: {sorted(SUPPORTED_FORMATS)}")
89
90     if size_inches is not None:
91         if len(size_inches) != 2:
92             raise ValueError("size_inches must be (width, height)")
93         fig.set_size_inches(*size_inches)
94
95     # 强制嵌入 TrueType 字体 (fonttype 42) ; 多家期刊明确拒绝 Type-3 PDF
96     plt.rcParams["pdf.fonttype"] = 42
97     plt.rcParams["ps.fonttype"] = 42
98     plt.rcParams["svg.fonttype"] = "none" # 文本仍可编辑, 期刊通常更欢迎
99
100    saved: list[str] = []
101    for fmt in formats:
102        if fmt in {"jpg", "jpeg"}:
103            print(f"[scipilot-figure-skill] WARNING: skipping {fmt} — "
104                  "JPEG is lossy and unsuitable for line/text figures.",
105                  file=sys.stderr)
106            continue
107        path = f"{basename}.{fmt}"
108        _ensure_parent(path)
109        kwargs: dict = {
110            "bbox_inches": "tight" if tight else None,
111            "pad_inches": pad_inches,
112            "transparent": transparent,
113        }
114        if fmt in RASTER_FORMATS:
115            kwargs["dpi"] = dpi
116        fig.savefig(path, **kwargs)
117        saved.append(path)
118        print(f"[scipilot-figure-skill] wrote {path}")
119
120    if grayscale_preview:
121        gray_path = _grayscale_from(fig, basename, dpi=dpi)
122        if gray_path:
123            saved.append(gray_path)
124    return saved
125
126    def _grayscale_from(fig, basename: str, dpi: int) -> str | None:
127        """
128        导出灰度预览版用于色盲安全检查。
129        优先用 PIL 转灰度; 找不到 PIL 时退化为重新画图 (关闭颜色)。
130        """
131        try:
132            from PIL import Image
133        except ImportError:
134            print("[scipilot-figure-skill] Pillow not available; ")

```

```

135         "grayscale preview skipped.", file=sys.stderr)
136     return None
137
138     png_path = f"{basename}.png"
139     _ensure_parent(png_path)
140     fig.savefig(png_path, dpi=dpi, bbox_inches="tight")
141
142     gray_path = f"{basename}_grayscale.png"
143     Image.open(png_path).convert("L").save(gray_path)
144     print(f"[scipilot-figure-skill] wrote {gray_path} (grayscale preview
145 )")
146     return gray_path
147
148 def _demo(out_basename: str) -> None:
149     """Generate a small demo figure for quick smoke testing."""
150     import numpy as np
151     rng = np.random.default_rng(7)
152     x = np.linspace(0, 10, 50)
153     y1 = np.sin(x) + rng.normal(0, 0.1, x.size)
154     y2 = np.cos(x) + rng.normal(0, 0.1, x.size)
155
156     fig, ax = plt.subplots(figsize=(3.5, 2.625))
157     ax.plot(x, y1, label="sin", marker="o", markersize=3)
158     ax.plot(x, y2, label="cos", marker="s", markersize=3, linestyle="--"
159 )
160     ax.set_xlabel("x")
161     ax.set_ylabel("y")
162     ax.legend(frameon=False)
163
164     paths = export_figure(
165         fig, out_basename,
166         formats=["pdf", "svg", "png"],
167         size_inches=(3.5, 2.625),
168         dpi=300,
169         grayscale_preview=True,
170     )
171     print("\nDemo done. Files:")
172     for p in paths:
173         print(f" {p}")
174
175 def _cli() -> int:
176     p = argparse.ArgumentParser(description="scipilot-figure-skill
177 figure exporter")
178     p.add_argument("cmd", choices=["demo"], help="\`demo`: 跑一张演示图导
179 出 4 种格式")
180     p.add_argument("--out", default="./scipilot_demo",
181                     help="输出 basename (默认 ./scipilot_demo)")
182     args = p.parse_args()
183     if args.cmd == "demo":
184         _demo(args.out)
185     return 0
186
187 if __name__ == "__main__":
188     sys.exit(_cli())

```

程序 19: 问题二正式绘图 (求解/问题二/重绘图表.py)

1 # 本绘图程序及代码在人工智能工具 Codex 辅助下完成。

```

2 # 开发机构: OpenAI; 模型: GPT-5.6-Luna (2026-07-09) 。
3 # 仅读取既有附件、参数和结果绘图, 不重新拟合或修改结果数据。
4 """问题二图表: 读取既有参数及附件, 不重新拟合或覆盖跨问共享结果。
5
6 运行: python 重绘图表.py
7 依赖: numpy pandas scipy matplotlib; 字体: Windows 宋体、Times New Roman
8
9 输出: 原图片目录中的同名 PNG; 不写入任何结果或参数文件。
10 """
11 from pathlib import Path
12 import argparse
13 import json
14 import numpy as np
15 import pandas as pd
16 import matplotlib
17 matplotlib.use('Agg')
18 import matplotlib.pyplot as plt
19 from matplotlib import font_manager as fm
20 from matplotlib.colors import LinearSegmentedColormap, Normalize
21 from matplotlib.patches import FancyBboxPatch, FancyArrowPatch
22 from matplotlib.ticker import ScalarFormatter, NullLocator,
23     NullFormatter
24 from scipy.cluster.hierarchy import linkage, leaves_list
25 from scipy.spatial.distance import squareform
26
27 HERE = Path(__file__).resolve().parent
28 ROOT = HERE.parents[1]
29 BLUE, TEAL, RED, GRAY = '#665080', '#367E73', '#CB8B38', '#45414C'
30 PALE, INK = '#FAF8F4', '#302F3D'
31 CMAP = LinearSegmentedColormap.from_list('effect', ['#FAF3D8', '#D8B979',
32     , '#AD8792', '#6D527D', '#382F58'])
33
34 def render_all(data_root=None):
35     data_root = Path(data_root) if data_root else ROOT.parent / '
36     real_attachments'
37     if not data_root.is_dir():
38         data_root = ROOT / 'real_attachments'
39     if not data_root.is_dir():
40         raise FileNotFoundError('未找到 real_attachments, 请使用 --data-
41     dir 指定附件根目录。')
42     figdir = HERE / '图片'
43     figdir.mkdir(exist_ok=True)
44     fontdir = Path('C:/Windows/Fonts')
45     for name in ['simsun.ttc', 'times.ttf', 'timesi.ttf', 'timesbd.ttf'
46     ]:
47         if (fontdir / name).exists():
48             fm.fontManager.addfont(str(fontdir / name))
49     for family in ['SimSun', 'Times New Roman']:
50         fm.findfont(family, fallback_to_default=False)
51     plt.rcParams.update({
52         'font.family': ['Times New Roman', 'SimSun'], 'font.size': 11,
53         'axes.titlesize': 11.5, 'axes.labelsize': 11, 'xtick.labelsize':
54         10,
55         'ytick.labelsize': 10, 'legend.fontsize': 9.5,
56         'mathtext.fontset': 'stix', 'axes.unicode_minus': False,
57         'axes.edgecolor': '#9AA8B3', 'axes.linewidth': .7,
58         'axes.labelcolor': INK, 'text.color': INK,

```

```

52         'xtick.color': INK, 'ytick.color': INK,
53         'axes.spines.top': False, 'axes.spines.right': False,
54         'grid.color': '#E0E7ED', 'grid.linewidth': .6,
55         'legend.frameon': False, 'pdf.fonttype': 42,
56         'savefig.dpi': 400, 'figure.facecolor': 'white', 'axes.facecolor
': 'white',
57     })
58     bdir = data_root / 'B_scaling_laws'
59     read = lambda f: pd.read_csv(bdir / f)
60     py = read('pythia_training_log_existing.csv')
61     b6 = read('supplementary_NQ_experiment.csv')
62     b7 = read('supplementary_NQ_experiment_expanded.csv')
63     b7n = b7[~b7.experiment_id.isin(b6.experiment_id)]
64     b8 = read('supplementary_NQ_experiment_large.csv')
65     b8c = b8[b8.data_type.eq('calibrated')]
66     par = pd.read_csv(HERE/'结果/广义标度律参数.csv').set_index('param')
        .value
67     E,A,a,B,b,C,g = [float(par[k]) for k in ['E','A','alpha','B','beta',
        'C','gamma']]
68     cl = pd.read_csv(HERE/'结果/经典标度律参数.csv').iloc[0]
69     def classic(N,D): return cl.E+cl.A*np.asarray(N)**(-cl.alpha)+cl.B*
np.asarray(D)**(-cl.beta)
70     def model(N,D,Q): return E+A*np.asarray(N)**(-a)+B*np.asarray(D)**(-
b)+C*(1-np.asarray(Q))**g
71     def save(fig, name):
72         from matplotlib.text import Text
73         # Dense automatic log minor ticks form a serrated edge at
manuscript scale.
74         # Retain the scale, limits, major ticks and all plotted values.
75         for ax in fig.axes:
76             for axis, scale in [(ax.xaxis, ax.get_xscale()), (ax.yaxis,
ax.get_yscale())]:
77                 if scale in ('log', 'symlog'):
78                     axis.set_minor_locator(NullLocator())
79                     axis.set_minor_formatter(NullFormatter())
80             for text in fig.findobj(Text):
81                 if text.get_text().strip() and text.get_fontsize() < 11:
82                     text.set_fontsize(11)
83             fig.savefig(figdir/f'{name}.png', bbox_inches='tight',
pad_inches=.08)
84         plt.close(fig)
85     def axes_style(ax, axis='both'):
86         ax.set_axisbelow(True); ax.grid(True, axis=axis)
87     def note(ax, text, xy=(.04,.95)):
88         ax.text(*xy, text, transform=ax.transAxes, ha='left', va='top',
fontsize=9.5,
89                 bbox=dict(facecolor='white',edgecolor='none',alpha=.88,
pad=2.5))
90
91     redesign(py,b6,b7,b7n,b8,b8c,read,model,classic,E,A,a,B,b,C,g,save)
92     summarize_large_baseline(read, model)
93     print('已更新原图片目录中的八张 PNG; 模型参数未更改。')
94
95 def summarize_large_baseline(read, model):
96     """只读复现正文 B9/B10 对照表, 不改变模型或已有结果文件。"""
97     metadata = read('supplementary_large_models.csv')
98     losses = read('supplementary_large_baseline.csv')

```

```

99     matched = losses.merge(metadata, left_on='family', right_on='
model_name',
100                               how='left', validate='one_to_one',
101                               suffixes=('', '_meta'), indicator=True)
102     assert matched['_merge'].eq('both').all()
103     for field in ['N_params_B', 'D_tokens_B']:
104         assert np.allclose(matched[field], matched[field + '_meta'])
105     excluded = metadata[~metadata.model_name.isin(losses.family)]
106     assert len(metadata) == 132 and len(matched) == 128
107     assert len(excluded) == 4 and excluded.D_tokens_B.eq(0).all()
108     prediction = model(matched.N_params_B, matched.D_tokens_B, 1.)
109     matched['absolute_relative_error'] = 100 * abs(prediction - matched.
val_loss) / matched.val_loss
110     rows = []
111     for low, high, label in [(100, 300, '[100,300)'), (300, 1000, '
[300,1000)'),
112                               (1000, np.inf, '[1000,10000]')]:
113         group = matched[matched.N_params_B.ge(low) & matched.N_params_B.
lt(high)]
114         rows.append({'N / B': label, '记录数': len(group),
115                     'D最小值': group.D_tokens_B.min(), 'D最大值': group
.D_tokens_B.max(),
116                     'MAPE_%': group.absolute_relative_error.mean()})
117     assert sum(row['记录数'] for row in rows) == len(matched)
118     assert matched.N_params_B.max() == 10000
119     print('B9/B10 一对一匹配; MAPE 对照 B10 估算值, 统一 Q=1: ')
120     print(pd.DataFrame(rows).round(2).to_string(index=False))
121     return rows
122
123 def redesign(py,b6,b7,b7n,b8,b8c,read,model,classic,E,A,a,B,b,C,g,save):
124     """改变视觉编码方式, 以附件记录和只读模型输出生成八张图。"""
125     from scipy.cluster.hierarchy import dendrogram
126     from matplotlib.lines import Line2D
127     def title(ax,s): ax.set_title(s,loc='left',pad=12,fontsize=12)
128     def style(ax): ax.set_axisbelow(True);ax.grid(axis='x',color='#
E5E1E8');ax.tick_params(length=3)
129     def foot(fig,s): fig.text(.02,.012,s,fontsize=11,color=INK,ha='left'
)
130
131     # 0. Evidence lanes keep the conditional p transfer outside the
fitted N/D/Q equation.
132     fig,ax=plt.subplots(figsize=(7.8,3.5));ax.axis('off');ax.set(xlim
=(0,10),ylim=(0,4.1))
133     for x,num,label,col in [(.1,'01','数据依据',BLUE),(3.55,'02','关系识
别',TEAL),(7,'03','结果解释',RED)]:
134         ax.text(x,3.82,num,color=col,fontsize=23,va='center');ax.text(x
+.65,3.82,label,fontsize=13,va='center',color=col)
135     def box(x,y,w,h,head,body,col):
136         ax.add_patch(FancyBboxPatch((x,y),w,h,boxstyle='round,pad=.025,
rounding_size=.06',fc=PALE,ec='#E5E1E8',lw=.8))
137         ax.plot([x+.12,x+.12],[y+.15,y+h-.15],color=col,lw=3,
solid_capstyle='round')
138         ax.text(x+.27,y+h-.23,head,fontsize=11.5,color=col,va='center')
139         ax.text(x+.27,y+.22,body,fontsize=10,va='center')
140     def arrow(x,y,u,v,dash=False):
141         ax.add_patch(FancyArrowPatch((x,y),(u,v),arrowstyle='->',
mutation_scale=11,color=GRAY,lw=1.1,linestyle='--' if dash else '-'))

```

```

142     for y,h,t,col in [(2.4,'B1 · 规模轨迹','1,176 个检查点',BLUE),(1.25,
    'B6 · 质量实验','360 条半合成记录',TEAL),(0.1,'问题一 · 配比输出','线性
    系数 / 核岭损失差',RED)]:box(0.1,y,2.8,.86,h,t,col)
143     box(3.55,1.85,2.9,1.4,'规模—质量标度律',r'$E+AN^{\alpha}+BD^{-\beta}$'+'\n'+r'$+\ C(1-Q)^{\gamma}$',BLUE)
144     box(3.55,.1,2.9,1.05,'配比影响分析','领域相似性 / 收益校准',RED)
145     box(7,2.4,2.85,.86,'跨来源检验','真实 / 半合成 / 估算',BLUE)
146     box(7,1.25,2.85,.86,'规模与质量取舍','弹性 / 等损失条件',TEAL)
147     box(7,.1,2.85,.86,'配比情景修正','在损失尺度可迁移时',RED)
148     arrow(2.95,2.8,3.5,2.8);arrow(2.95,1.68,3.5,2.05);arrow
    (2.95,.53,3.5,.53)
149     arrow(6.5,2.8,6.95,2.8);arrow(6.5,2.1,6.95,1.68);arrow
    (6.5,.53,6.95,.53,True)
150     fig.subplots_adjust(left=.01,right=.99,bottom=.01,top=.99);save(fig,
    '图0_问题二分析框架')
151
152     # 1. All B1 losses in a scale map, alongside per-model residual
    distributions.
153     ns=np.sort(py.N_params_B.unique());ds=np.sort(py.D_tokens_B.unique()
    )
154     z=py.pivot(index='N_params_B',columns='D_tokens_B',values='val_loss'
    ).loc[ns,ds].to_numpy()
155     fig=plt.figure(figsize=(7.8,3.7));gs=fig.add_gridspec(1,2,
    width_ratios=[1.15,1],wspace=.3)
156     ax=fig.add_subplot(gs[0]);ar=fig.add_subplot(gs[1])
157     edges=np.r_[ds[0]/np.sqrt(ds[1]/ds[0]),np.sqrt(ds[-1]*ds[1:]),ds[-
    1]*np.sqrt(ds[-1]/ds[-2])]
158     im=ax.pcolormesh(edges,np.arange(9)-.5,z,cmap=CMAP,shading='flat')
159     ax.set(xscale='log',yticks=np.arange(8),yticklabels=[f'{n:.2f}' for
    n in ns],xlabel='训练数据量 D / B tokens',ylabel='参数量 N / B',ylim
    =(-.5,7.5))
160     ax.set_xticks([.2,1,10,100]);ax.set_xticklabels(['0.2','1','10','100
    ']);title(ax,'(a) 附件记录的损失地图')
161     cb=fig.colorbar(im,cax=ax.inset_axes([0,1.16,1,.045]),orientation='
    horizontal');cb.ax.tick_params(labelsize=11,length=2)
162     ax.set_title('(a) 附件记录的损失地图',loc='left',pad=52,fontsize=12)
163     residuals=[]
164     for n in ns:
165         sub=py[py.N_params_B.eq(n)];residuals.append((classic(n,sub.
    D_tokens_B)-sub.val_loss).to_numpy()*1e4)
166         v=ar.violinplot(residuals,positions=np.arange(8),vert=False,widths
    =.8,showextrema=False)
167         for body in v['bodies']:body.set(facecolor=BLUE,edgecolor=BLUE,alpha
    =.24,linewidth=.7)
168         for j,err in enumerate(residuals):
169             ar.scatter(err,np.full(len(err),j),s=3,color=BLUE,alpha=.2,
    edgecolors='none')
170             q25,med,q75=np.quantile(err,[.25,.5,.75]);ar.plot([q25,q75],[j,j
    ],color=BLUE,lw=3);ar.scatter(med,j,s=18,color=RED,zorder=4)
171             ar.axvline(0,color=GRAY,ls='--',lw=.8);ar.set(yticks=np.arange(8),
    yticklabels=[f'{n:.2f}' for n in ns],ylim=(-.5,7.5),xlabel='预测残差
    / 10-4')
172     ar.set_title('(b) 各参数规模的拟合残差',loc='left',pad=52,fontsize
    =12);style(ar)
173     fig.subplots_adjust(left=.09,right=.985,bottom=.23,top=.75)
174     foot(fig,'B1: 8 个模型 × 147 个检查点; 左图颜色表示验证损失。 \n右
    图: 记录点、四分位区间及金色中位点; 轮廓表示平滑密度。')

```

```

175     save(fig, '图1_经典标度律拟合')
176
177     # 2. Signed error distributions complement the summary-metric table.
178     sets=[('B4 跨族收敛点', '真实', read('scaling_baseline.csv'), 1., BLUE),
179           ('B5 文献标度数据', '真实', read('published_scaling_data.csv'), 1.,
180            BLUE),
181           ('B7 新增质量点', '半合成', b7n, b7n.Q_score, TEAL), ('B2 Cerebras '
182            '半合成', read('cerebras_training_log.csv'), 1., TEAL),
183           ('B8 校准段', '半合成', b8c, 1-b8c.Q_score, TEAL), ('B10 大规模损失 '
184            '估算', read('supplementary_large_baseline.csv'), 1., RED)]
185     fig, ax=plt.subplots(figsize=(6.2, 3.8)); rng=np.random.default_rng
186     (20260925)
187     saved=pd.read_csv(HERE/'结果/广义标度律分层验证.csv'); labels=[]
188     for j, (name, kind, df, q, col) in enumerate(sets):
189         y=df.val_loss.to_numpy(); pred=model(df.N_params_B, df.D_tokens_B,
190         q); err=100*(pred-y)/y; mape=np.mean(abs(err)); r=np.corrcoef(y, pred)
191         [0, 1]
192         old=saved[saved.dataset.str.startswith(name.split()[0])].iloc[0]
193         assert np.isclose(mape, old['MAPE_%'], atol=1e-10) and np.isclose(
194         r, old.r, atol=1e-10)
195         labels.append(f'{name}\n{kind} · n={len(df):,}')
196         ax.axhspan(j-.45, j+.45, color=PALE, zorder=0)
197         v=ax.violinplot([err], positions=[j], vert=False, widths=.62,
198         showextrema=False); v['bodies'][0].set(facecolor=col, edgecolor=col,
199         alpha=.2)
200         ax.scatter(err, j+rng.uniform(-.1, .1, len(err)), s=5, alpha=.26,
201         color=col, lw=0)
202         q25, med, q75=np.quantile(err, [.25, .5, .75]); ax.plot([q25, q75], [j, j],
203         lw=3, color=col); ax.scatter(med, j, s=20, fc='white', ec=col, zorder=4)
204         ax.axvline(0, color=INK, lw=.8, ls='--'); ax.set(yticks=np.arange(6),
205         yticklabels=labels, ylim=(5.6, -.65), xlabel='有符号相对误差 / %'); style
206         (ax)
207         ax.set_xscale('symlog', linthresh=10); ax.set_xticks([-50, -
208         10, 0, 10, 100, 500]); ax.set_xticklabels(['-50', '-10', '0', '10', '100', '500
209         '], fontsize=11)
210         fig.subplots_adjust(left=.26, right=.98, bottom=.25, top=.96)
211         foot(fig, '相对误差 = 100 × (预测值 - 给定值) / 给定值; 横轴在 ±10%
212         内线性。 \nB8 使用 1-Q; 空心点: 中位数; 粗线: 四分位区间; 轮廓: 平滑密
213         度。')
214     save(fig, '图2_广义标度律验证')
215
216     # 3. N-Q contour map and asymptotic loss increments; raw design
217     sites are explicit.
218     fig=plt.figure(figsize=(7.8, 3.7)); gs=fig.add_gridspec(1, 2,
219     width_ratios=[1.25, 1], wspace=.42)
220     ax=fig.add_subplot(gs[0]); ar=fig.add_subplot(gs[1]); q=np.linspace
221     (.1, 1, 240); n=np.geomspace(.07, 12, 240); Q, N=np.meshgrid(q, n); Z=model(N
222     , 300, Q)
223     cs=ax.contourf(Q, N, Z, levels=np.linspace(2.08, 3.15, 20), cmap=CMAP,
224     extend='both')
225     cont=ax.contour(Q, N, Z, levels=[2.2, 2.4, 2.6, 2.8, 3.0], colors='white',
226     linewidths=.8); ax.clabel(cont, fmt='%.1f', fontsize=9, inline_spacing=5,
227     manual=[(.7, 6), (.55, 1.6), (.45, .45), (.3, .14), (.3, .075)])
228     design=b6[b6.D_tokens_B.eq(300)]; ax.scatter(design.Q_score, design.
229     N_params_B, s=9, facecolors='none', edgecolors=INK, linewidths=.35, alpha
230     =.5)

```

```

203     ax.set(yscale='log',xlabel='数据质量 Q',ylabel='参数量 N / B',xlim
= (.1,1),ylim=(.07,12));title(ax,'(a) 规模—质量响应面')
204     cb=fig.colorbar(cs,ax=ax,pad=.03,fraction=.06,ticks=[2.2,2.6,3.0]);
cb.set_label('预测损失',fontsize=10)
205     for j,(q0,col) in enumerate(zip([1.,.6,.3],[TEAL,RED,BLUE])):
206         floor=E+C*(1-q0)**g;ar.barh(j,floor-E,left=E,height=.4,color=col
,alpha=.85);ar.scatter(floor,j,s=50,color=col,zorder=3);ar.text(floor
+.012,j,f'{floor:.3f}',va='center',color=col,fontsize=12)
207     ar.axvline(E,color=GRAY,ls='--',lw=1);ar.set(yticks=[0,1,2],
yticklabels=['Q = 1.0','Q = 0.6','Q = 0.3'],ylim=(2.7,-.7),xlim
=(1.62,2.015),xlabel='模型渐近损失')
208     ar.text(E,2.57,f'E = {E:.3f}',ha='center',fontsize=10,color=GRAY);
title(ar,'(b) 质量缺口对下限的抬升');style(ar)
209     fig.subplots_adjust(left=.085,right=.98,bottom=.23,top=.85);foot(fig
,'D=300 B tokens; 空心点: B6 实验位置; 底色与等值线: 模型预测。');
save(fig,'图3_质量边际影响')

210
211     # 4. Marginal response locations and a dumbbell comparison of saved
elasticities.
212     fig,axs=plt.subplots(1,3,figsize=(7.8,3.5),gridspec_kw={'
width_ratios':[1,1,1.35]})
213     nvals=np.array([.07,.3,1,3,12]);qvals=np.array([.1,.3,.6,.9,.99])
214     for ax,effect,col,labels,xlabel,heading in [
215         (axs[0],a*A*nvals**(-a-1),BLUE,[f'{n:g}' for n in nvals],r'$\backslash
partial L/\partial N|$\ / B$^{-1}$','(a) 参数边际效应'),
216         (axs[1],C*g*(1-qvals)**(g-1),RED,[f'{q:g}' for q in qvals],r'$\backslash
partial L/\partial Q|$', '(b) 质量边际效应')]:
217         ypos=np.arange(5)
218         ax.scatter(effect,ypos,color=col,s=42,zorder=3)
219         ax.set(yticks=ypos,yticklabels=labels,ylim=(4.5,-.6),xlabel=
xlabel);title(ax,heading);style(ax)
220     axs[0].set_xscale('log');axs[0].set_ylabel('参考参数量 N / B')
221     axs[1].set(xlim=(.34,.391),ylabel='参考质量 Q',xticks=[.35,.37,.39])
222     el=pd.read_csv(HERE/'结果/弹性与替代关系.csv');v=-el[['弹性e_N','弹
性e_D','弹性e_Q']].to_numpy();ar=axs[2]
223     for j in range(3):
224         ar.plot(v[:,j],[j,j],color='#E5E1E8',lw=5,zorder=1)
225         for i,col in enumerate([BLUE,TEAL]):
226             ar.scatter(v[i,j],j,s=48,color=col,zorder=3,marker=['o','D'
][i],label=['N=1, D=100','N=12, D=300'][i] if j==0 else None)
227             ar.annotate(f'{v[i,j]:.3f}',(v[i,j],j),xytext=(0,10 if i==0
else -16),textcoords='offset points',ha='center',fontsize=9,color=col
)
228     ar.set(yticks=[0,1,2],yticklabels=['参数 N','数据 D','质量 Q'],ylim
=(2.55,-.75),xlim=(.01,.112),xlabel='弹性绝对值')
229     title(ar,'(c) 两个参考点的敏感度');style(ar);ar.legend(loc='upper
center',bbox_to_anchor=(.5,-.2),ncol=1,fontsize=9)
230     fig.subplots_adjust(left=.075,right=.98,bottom=.34,top=.87,wspace
=.62)
231     foot(fig,'左、中图: 边际导数, 各用自身量纲; 右图: 无量纲弹性。\\n仅右
图两个参考点均取 Q=0.6; N、D 以 B 为单位。');save(fig,'图4_边际效用对
比')

232
233     # 5. Three original loss targets, separate panels, explicit scale
extrapolation.

```

```

234     fig,axs=plt.subplots(1,3,figsize=(7.8,3.25),sharex=True);ng=np.
geomspace(.07,1e4,1200);base=E+B*300**(-b)
235     for ax,target,col in zip(axs,[2.30,2.10,1.95],[BLUE,TEAL,RED]):
236         remaining=target-base-A*ng**(-a);qreq=np.full_like(ng,np.nan);
valid=remaining>=0
237         qreq[valid]=1-(remaining[valid]/C)**(1/g);qreq[(qreq<.1)|(qreq
>1)]=np.nan
238         ax.axvspan(.07,12,color=TEAL,alpha=.065);ax.axvspan(12,1e4,color
=RED,alpha=.075);ax.axvline(12,color=GRAY,ls=':',lw=1)
239         ax.plot(ng,np.where(ng<=12,qreq,np.nan),color=col,lw=2.5);ax.
plot(ng,np.where(ng>12,qreq,np.nan),color=col,lw=2,ls='--')
240         ax.set(xscale='log',xlim=(.07,1e4),ylim=(.08,1.035),xlabel='参数
量 N / B',xticks=[.1,10,1e3],ylabel='所需质量 Q');title(ax,f'目标损失
L = {target:.2f}');ax.grid(axis='y')
241         min_n=(A/(target-base))**(1/a)
242         if .07<min_n<1e4:
243             ax.scatter(min_n,1,s=32,color=col,zorder=3)
244             pos=(.74,.83) if target==2.30 else ((.27,.48) if target
==2.10 else (.43,.73))
245             ax.annotate(f'Q=1\nN={min_n:.2f} B' if min_n<10 else f'Q=1\
nN={min_n:.1f} B',(min_n,1),xytext=pos,textcoords='axes fraction',
fontsize=11,color=col,ha='center',va='top',bbox=dict(facecolor=PALE,
edgecolor='none',pad=1.5),arrowprops=dict(arrowstyle='-',lw=.7,color=
col))
246         if target==1.95:
247             ax.set_ylim(.95,1.008);ax.set_yticks([.96,.98,1]);ax.text
(.06,.1,'高质量区间放大',transform=ax.transAxes,fontsize=10,color=col
)
248     fig.subplots_adjust(left=.08,right=.985,bottom=.28,top=.84,wspace
=.42)
249     foot(fig,'D=300 B tokens; 实线: N≤12 B; 虚线: 规模外推.\n右图纵轴单
独放大; 阴影区分拟合规模与外推规模。');save(fig,'图5_质量参数替代')
250
251     # 6. All 17 domains via A-derived question-one coefficients, not raw
A refitting.
252     names={'arxiv':'学术预印本','freelaw':'法律判例','nih_exporter':'科
研项目','pubmed_central':'医学全文',
253           'wikipedia_en':'百科','dm_mathematics':'数学','github':'代码','
philpapers':'哲学论文','stackexchange':'专业问答',
254           'enron_emails':'邮件','gutenberg_pg_19':'图书','pile_cc':'网页',
'ubuntu_irc':'技术聊天','europarl':'议会记录',
255           'hackernews':'科技社区','pubmed_abstracts':'医学摘要','
uspto_backgrounds':'专利背景'}
256     coef=pd.read_csv(ROOT/'求解/问题一/结果/混合系数矩阵.csv').set_index
('training_domain')
257     saved_sim=pd.read_csv(HERE/'结果/领域替代互补矩阵.csv',index_col=0);
coef=coef.loc[saved_sim.index]
258     center=coef.to_numpy()-coef.to_numpy().mean(axis=0);unit=center/np.
linalg.norm(center,axis=1,keepdims=True);sim=unit@unit.T
259     assert np.allclose(sim,saved_sim.to_numpy(),atol=1e-12)
260     dist=np.sqrt(np.maximum(0,2*(1-sim)));np.fill_diagonal(dist,0);tree=
linkage(squareform(dist,checks=False),method='average',
optimal_ordering=True)
261     pairs=sorted([(sim[i,j],i,j) for i in range(17) for j in range(i)],
key=lambda t:t[0]);selected=list(reversed(pairs[-4:]))+pairs[:4]
262     fig=plt.figure(figsize=(7.8,5.15));gs=fig.add_gridspec(1,2,
width_ratios=[1,1.48],wspace=.88)

```

```

263     ax=fig.add_subplot(gs[0]);ar=fig.add_subplot(gs[1])
264     dendrogram(tree,orientation='left',labels=[names[x] for x in coef.
index],ax=ax,color_threshold=0,above_threshold_color=BLUE,
leaf_font_size=10)
265     ax.set_xlabel('平均连接距离');title(ax,'(a) 17 域效应模式聚类');ax.
grid(axis='x');ax.spines[['top','right','left']].set_visible(False)
266     labels=[]
267     for j,(value,i,k) in enumerate(selected):
268         col=TEAL if value>=0 else RED;labels.append(names[coef.index[i
]]+' / '+names[coef.index[k]])
269         ar.barh(j,value,height=.55,color=col,alpha=.85);ar.text(value*
.5,j,f'{value:+.3f}',ha='center',va='center',fontsize=10,color='white
')
270     ar.axvline(0,color=GRAY,lw=.8);ar.axhline(3.5,color='#E5E1E8',lw
=1.5)
271     ar.set(yticks=np.arange(8),yticklabels=labels,ylim=(7.7,-.8),xlim=(-
1.05,1.05),xlabel='中心化系数的余弦相似度');title(ar,'(b) 最相似与最
相反的领域对');style(ar)
272     fig.subplots_adjust(left=.04,right=.98,top=.87,bottom=.17)
273     foot(fig,'聚类距离  $\sqrt{2-2s}$ ; 绿色: 效应趋同; 金色: 效应相反。右图列
示 s 最大与最小的各 4 对; 相似性不代表互补增益。');save(fig,'图6_领域
替代互补')
274
275     # 7. Group correlation distributions rather than point clouds mixing
all groups.
276     fig,ax=plt.subplots(figsize=(7.8,3.1));rng=np.random.default_rng(19)
277     datasets=[('B6',b6,BLUE),('B7',b7,TEAL),('B8 校准段',b8c,RED),('B8
外推段',b8[b8.data_type.eq('extrapolated')],RED)];labels=[]
278     for j,(name,df,col) in enumerate(datasets):
279         cors=np.array([group.Q_score.corr(group.val_loss) for _,group in
df.groupby(['N_params_B','D_tokens_B']) if group.Q_score.nunique()
>=3]);assert len(cors)>0
280         labels.append(f'{name}\n{len(df):,} 条记录 / {len(cors)} 组');ax
.axhspan(j-.44,j+.44,color=PALE)
281         ax.scatter(cors,j+rng.uniform(-.19,.19,len(cors)),s=22,color=col
,alpha=.43,lw=0);ax.scatter(cors.mean(),j,s=75,marker='D',color=col,
ec='white',lw=.9,zorder=4)
282         ax.text(1.13,j,f'{cors.mean():+.3f}',va='center',ha='left',color
=col,fontsize=12)
283         ax.axvline(0,color=GRAY,ls='--',lw=.9);ax.set(xlim=(-1.08,1.42),ylim
=(3.6,-.65),yticks=np.arange(4),yticklabels=labels,xticks=[-1,-
.5,0,.5,1],xlabel='固定 (N,D) 组内的 Pearson 相关系数')
284         ax.text(1.13,-.6,'组间均值',fontsize=10,ha='left');title(ax,'质量字
段与损失: 组内方向及组间差异');style(ax)
285         fig.subplots_adjust(left=.21,right=.98,top=.84,bottom=.32)
286         foot(fig,'圆点: 一个 (N,D) 组; 菱形: 组间均值。均为半合成数据; B7
含 B6, 不构成独立证据。');save(fig,'图7_质量方向对照')
287
288 if __name__=='__main__':
289     parser=argparse.ArgumentParser();parser.add_argument('--data-dir');
args=parser.parse_args()
290     render_all(args.data_dir)

```

程序 20: 问题三正式绘图 (求解/问题三/论文绘图.py)

1 # 本程序的整理与核对使用 Codex 辅助; 模型: GPT-5.6-Luna; 机构: OpenAI;
版本发布日期: 2026-07-09。

```

2 # 本程序的整理、复现核对或绘图实现使用 OpenAI Codex (GPT-5.6-Luna) 辅助。
3 """从已保存的 CSV 重绘论文图；不运行优化器，也不修改原始结果。
4
5 用法: python 求解/问题三/论文绘图.py
6 依赖: numpy、pandas、matplotlib。输出同名 PNG (300 dpi) 和矢量 PDF。
7 圆点对应 CSV 有效记录；缺失区间用虚线连接两端，仅引导视线，不补算或插值。
8 """
9 from pathlib import Path
10 import hashlib
11 import json
12 import numpy as np
13 import pandas as pd
14 import matplotlib
15 matplotlib.use("Agg")
16 import matplotlib.pyplot as plt
17 from matplotlib import font_manager as fm, ticker
18
19 ROOT = Path(__file__).resolve().parent
20 DATA, OUT = ROOT / "结果", ROOT / "图片"
21 FONT = ROOT.parents[1] / "论文" / "fonts" / "SourceHanSerifCN-Regular.
    otf"
22 fm.fontManager.addfont(str(FONT))
23 plt.rcParams.update({
24     "font.family": [fm.FontProperties(fname=str(FONT)).get_name(), "
    DejaVu Sans"],
25     "font.size": 10, "axes.labelsize": 10, "legend.fontsize": 8.5,
26     "xtick.labelsize": 9, "ytick.labelsize": 9, "axes.unicode_minus":
    False,
27     "pdf.fonttype": 42, "savefig.dpi": 300, "axes.linewidth": .7,
28 })
29 BLUE, ORANGE, CYAN, PURPLE = "#356B9A", "#C87530", "#238A8D", "#8064A2"
30
31 AUDIT = {}
32
33 def read(name):
34     path = DATA / (name + ".csv")
35     df = pd.read_csv(path)
36     AUDIT[name] = {"sha256": hashlib.sha256(path.read_bytes()).hexdigest
    (), "rows": len(df)}
37     return df
38
39 def finish(fig, name):
40     for ax in fig.axes:
41         ax.spines[["top", "right"]].set_visible(False)
42         ax.grid(axis="y", color="#E4E7EA", lw=.55)
43         ax.set_axisbelow(True)
44         ax.tick_params(which="both", direction="out", length=3)
45     for ext in ("png", "pdf"):
46         fig.savefig(OUT / (name + "." + ext), bbox_inches="tight",
    pad_inches=.07)
47     plt.close(fig)
48
49 def panel(ax, text):

```

```

50     ax.text(0, 1.045, text, transform=ax.transAxes, va="bottom",
    fontsize=10)
51
52 def recover_grid(df, grid, key):
53     """将已输出的有效点放回原脚本定义的网格，显式暴露被跳过的失败点。"""
54     output = pd.DataFrame({key: grid})
55     df = df.select_dtypes(include="number")
56     for col in df.columns:
57         if col != key:
58             output[col] = np.nan
59     for _, row in df.iterrows():
60         index = np.argmin(abs(np.log(grid / row[key])))
61         if not np.isclose(grid[index], row[key], rtol=1e-9):
62             raise ValueError(f"{key}={row[key]} 不属于原始扫描网格")
63         for col in df.columns:
64             if col != key:
65                 output.loc[index, col] = row[col]
66     return output
67
68 def measured_line(ax, x, y, *, color, lw=1.3, ls="-", label=None, marker
    ="o", ms=2.5):
69     """实心标记只画有效记录；跨缺测的连接单独用点线，不生成任何中间数
    据。"""
70     x, y = np.asarray(x, float), np.asarray(y, float)
71     good = np.isfinite(x) & np.isfinite(y)
72     ax.plot(x, y, color=color, lw=lw, ls=ls, label=label)
73     ax.plot(x[good], y[good], ls="none", marker=marker, ms=ms, color=
    color)
74     ids = np.flatnonzero(good)
75     for left, right in zip(ids[:-1], ids[1:]):
76         if right-left > 1:
77             ax.plot(x[[left,right]], y[[left,right]], color=color, lw=lw
    , ls=":")
78
79 opt = read("三档预算最优配置").sort_values("C")
80 q0 = float(opt.iloc[0].Q)
81 fig, ax = plt.subplots(1, 2, figsize=(7.0, 2.65), layout="constrained")
82 for col, color, marker, label in [(("N_B", BLUE, "o", "参数量 (B)"),
83                                     ("D_B", CYAN, "s", "数据量 (B tokens
    ) ")]):
84     ax[0].plot(opt.C, opt[col], marker=marker, ls="--", color=color, lw
    =1.4,
85                 ms=5, label=label)
86     for x, y in zip(opt.C, opt[col]):
87         label_value = f"{y:.1f}" if col == "D_B" else f"{y:.3f}"
88         ax[0].annotate(label_value, (x, y), xytext=(0, 8 if col == "D_B"
    else -13),
89                        textcoords="offset points", ha="center", fontsize
    =8, color=color)
90 ax[0].set_yscale("log")
91 ax[0].set_ylim(.045, 10000)
92 ax[0].set_ylabel("规模 (对数刻度)")
93 ax[0].legend(loc="upper left")
94 panel(ax[0], "(a) 参数量与数据量")
95 ax[1].plot(opt.C, opt.Q, "o--", color=ORANGE, lw=1.4, ms=5)
96 ax[1].axhline(q0, color="#777777", ls=":", lw=1)
97 for x, y in zip(opt.C, opt.Q):

```

```

98     ax[1].annotate(f"{y:.3f}", (x, y), xytext=(0, 8), textcoords="offset
99         points",
100         ha="center", fontsize=8)
101 ax[1].text(1.4e20, q0-.065, rf"基线 $Q_0={q0:.3f}$", fontsize=9, color="
102     #666666")
103 ax[1].set_ylim(.45, 1.12)
104 ax[1].set_ylabel(r"最优质量 $Q^*$")
105 panel(ax[1], "(b) 质量水平")
106 for a in ax:
107     a.set_xscale("log")
108     a.set_xlim(3.5e18, 3e24)
109     a.set_xticks(opt.C)
110     a.xaxis.set_minor_locator(ticker.NullLocator())
111     a.set_xlabel(r"算力预算 $C_{\mathrm{b}}$ (FLOPs) ")
112 finish(fig, "图1_三档预算最优配置")
113
114 sw = read("预算扫描").sort_values("C")
115 theta = (sw.Q-q0)/(1-q0)
116 valid = sw.Q.notna()
117 start = float(sw.loc[valid & (sw.Q > q0+1e-7), "C"].iloc[0])
118 sat = float(sw.loc[valid & (sw.Q >= 1-1e-7), "C"].iloc[0])
119 peak = sw.loc[sw.f_qual.idxmax()]
120 fig, axs = plt.subplots(1, 2, figsize=(7.2, 3.35), layout="constrained")
121 a, b = axs
122 measured_line(a, sw.C, theta*100, color=ORANGE)
123 for col, color, ls, label in [("f_base", BLUE, "-", "基础训练"),
124     ("f_qual", ORANGE, "--", "质量提升"),
125     ("f_attn", PURPLE, "-.", "注意力计算")]:
126     measured_line(b, sw.C, sw[col]*100, color=color, ls=ls, label=label,
127         ms=2)
128 for aa in axs:
129     aa.set_xscale("log")
130     aa.set_xlim(1e18, 1e25)
131     aa.set_ylim(-3, 108)
132     aa.set_xticks([1e18, 1e20, 1e22, 1e24])
133     aa.xaxis.set_minor_locator(ticker.NullLocator())
134     aa.axvline(start, color="#777777", lw=.8, ls="--")
135     aa.axvline(sat, color="#777777", lw=.8, ls=":")
136     aa.axvspan(start, sat, color=ORANGE, alpha=.07)
137     aa.set_xlabel(r"算力预算 $C_{\mathrm{b}}$ (FLOPs) ")
138 a.set_ylabel(r"质量完成度 $\Theta$ (%)")
139 b.set_ylabel("预算份额 (%)")
140 panel(a, "(a) 质量由基线升至上界")
141 panel(b, "(b) 质量份额先升后降")
142 b.legend(loc="upper right", bbox_to_anchor=(1,.78), fontsize=8,
143     framealpha=.95)
144 b.annotate(f"峰值 {peak.f_qual*100:.1f}%", (peak.C, peak.f_qual*100),
145     xytext=(2e21, 42),
146     fontsize=8, arrowprops={"arrowstyle":"-", "color":ORANGE, "lw
147     ":",.8})
148 zoom = a.inset_axes([.40, .14, .57, .50])
149 measured_line(zoom, sw.C, theta*100, color=ORANGE, ms=3)
150 zoom.set_xscale("log")
151 zoom.set_xlim(1e19, 1e20)
152 zoom.set_ylim(-4,104)
153 zoom.set_xticks([1e19,1e20])
154 zoom.set_xticklabels([r"$10^{19}$",r"$10^{20}$"],fontsize=8)

```

```

149 zoom.xaxis.set_minor_locator(ticker.NullLocator())
150 zoom.set_yticks([0,50,100])
151 zoom.tick_params(labelsize=8)
152 zoom.axvline(start,color="#777777",ls="--",lw=.8)
153 zoom.axvline(sat,color="#777777",ls=":",lw=.8)
154 zoom.text(.04,.9,"转移区间放大",transform=zoom.transAxes,fontsize=8)
155 zoom.grid(axis="y",color="#e4e7ea",lw=.5)
156 finish(fig, "图2_预算份额结构性转移")
157
158 ctx = read("上下文长度敏感性")
159 cont = recover_grid(read("上下文长度连续扫描"), np.geomspace(1024,
    200000, 40), "L_ctx")
160 ql = read("上下文长度与最优质量")
161 fig, ax = plt.subplots(1, 2, figsize=(7.0, 2.85), layout="constrained")
162 measured_line(ax[0], cont.L_ctx, cont.N_B, color=BLUE, label="扫描有效记
    录")
163 ax[0].plot(ctx.L_ctx, ctx.N_B, "s", color=BLUE, ms=4, label="C7 五档配置
    ")
164 ax[0].set_ylabel(r"最优参数量  $N^*(B)$ ")
165 ax[0].set_ylim(2, 6.6)
166 ax[0].legend(loc="lower left", fontsize=8)
167 panel(ax[0], r"(a) 规模配置:  $10^{22}$  FLOPs")
168 for budget, color, ls, marker, label in [(1e19, ORANGE, "--", "o", r" $10^{19}$ 
    FLOPs"),
169                                           (1e20, BLUE, "--", "s", r" $10^{20}$ 
    FLOPs"),
170                                           (1e21, CYAN, "--", "^", r" $10^{21}$ 
    FLOPs")]:
171     d = recover_grid(ql.loc[ql.C.eq(budget)], np.geomspace(1024, 150000,
    14), "L_ctx")
172     measured_line(ax[1], d.L_ctx, d.Q, color=color, ls=ls, marker=marker
    , ms=3, label=label)
173 ax[1].set_ylim(.49, 1.085)
174 ax[1].set_ylabel(r"最优质量  $Q^*(B)$ ")
175 ax[1].text(1600, .925, r" $10^{20}$  FLOPs 与  $10^{21}$  FLOPs 曲线重合", fontsize=8)
176 ax[1].legend(loc="center left", fontsize=8)
177 panel(ax[1], "(b) 不同预算下的质量响应")
178 for a in ax:
179     a.set_xscale("log")
180     a.set_xlim(850, 220000)
181     a.axvline(30000, color=PURPLE, ls=":", lw=1.1)
182     a.set_xticks([1e3, 1e4, 3e4, 1e5])
183     a.set_xticklabels([r" $10^3$ ", r" $10^4$ ", r" $3 \times 10^4$ ", r" $10^5$ "])
184     a.xaxis.set_minor_locator(ticker.NullLocator())
185     a.set_xlabel(r"上下文长度  $L_{\text{ctx}}$  (tokens)")
186 finish(fig, "图3_上下文长度敏感性")
187
188 # 图4只使用成本函数对比.csv的九组求解记录, 不再绘制解析函数采样线。
189 comp = read("成本函数对比")
190 fig, ax = plt.subplots(1, 2, figsize=(7.0, 2.85), layout="constrained")
191 budgets = sorted(comp.C.unique())
192 x = np.arange(len(budgets))
193 for i, (form, color) in enumerate([("幂函数型", BLUE), ("指数型", ORANGE)
    ], ("对数渐进型", CYAN))]:
194     d = comp.loc[comp.form.eq(form)].sort_values("C")
195     assert d.C.tolist() == budgets and len(d) == 3
196     for a, column, factor in [(ax[0], "Q", 1), (ax[1], "f_qual", 100)]:

```

```

197     bars = a.bar(x+(i-1)*.24, d[column]*factor, width=.22, color=
    color, label=form)
198     a.bar_label(bars, labels=[f"{v:.3f}" if column == "Q" else f"{v
    :.1f}" for v in d[column]*factor], fontsize=8, padding=3, rotation
    =90)
199 for a in ax:
200     a.set_xticks(x, [rf"$10^{{{int(np.log10(c))}}}" for c in budgets])
201     a.set_xlabel(r"算力预算 $C_{\mathrm{b}}$ (FLOPs) ")
202 ax[0].set_ylim(0, 1.3)
203 ax[1].set_ylim(0, 40)
204 ax[0].set_ylabel(r"最优质量 $Q^*$")
205 ax[1].set_ylabel("质量投入份额 (%) ")
206 panel(ax[0], "(a) 质量水平")
207 panel(ax[1], "(b) 质量投入份额")
208 fig.legend(*ax[0].get_legend_handles_labels(), loc="outside lower center
    ", ncol=3, frameon=False)
209 finish(fig, "图4_质量成本函数对比")
210
211 AUDIT["预算扫描"]["valid_Q"] = int(valid.sum())
212 AUDIT["预算扫描"]["missing_C"] = sw.loc[~valid, "C"].tolist()
213 AUDIT["上下文长度连续扫描"]["expected_grid_rows"] = 40
214 AUDIT["上下文长度连续扫描"]["missing_L_ctx"] = cont.loc[cont.N_B.isna(),
    "L_ctx"].tolist()
215 AUDIT["上下文长度与最优质量"]["valid_rows_by_budget"] = {str(c): len(d)
    for c,d in ql.groupby("C")}
216 (OUT / "论文绘图数据核验.json").write_text(json.dumps(AUDIT,
    ensure_ascii=False, indent=2), encoding="utf-8")
217
218 print(f"已生成四组 PNG/PDF; 扫描点 {len(sw)}, 有效点 {valid.sum()}。")
219 print(f"首次正投入 {start:.6g}; 首次饱和 {sat:.6g}; 份额峰值 {peak.
    f_qual:.6f}。")

```

程序 21: 问题四正式绘图 (求解/问题四/重绘图表.py)

```

1 # 本绘图程序在人工智能工具 Codex 辅助下完成。
2 # 开发机构: OpenAI; 模型: GPT-5.6-Luna (2026-07-09) 。
3 # 仅读取附件和已有计算结果, 不拟合模型, 不写入结果目录。
4 """重绘问题四的七幅图。运行时以 --data 指定 real_attachments 目录。
5
6 例: python 重绘图表.py --data ../../../../real_attachments
7 可选 --audit 指定内部审查目录, 输出可编辑 SVG、数据摘要和布局检查。
8 """
9 from pathlib import Path
10 import argparse
11 import hashlib
12 import json
13 import sys
14 import numpy as np
15 import pandas as pd
16 import matplotlib
17 matplotlib.use('Agg')
18 import matplotlib.pyplot as plt
19 from matplotlib import font_manager
20 from matplotlib.lines import Line2D
21
22 HERE = Path(__file__).resolve().parent

```

```

23 RESULT = HERE / '结果'
24 PURPLE, TEAL, GOLD, INK = '#665080', '#367E73', '#CB8B38', '#302F3D'
25 TASKS = ['IFEval', 'BBH', 'MATH Lvl 5', 'GPQA', 'MUSR', 'MMLU-PRO']
26 LICENSES = {'apache-2.0', 'mit', 'gpl-3.0', 'cc-by-4.0', 'mpl-2.0',
27             'bsd-3-clause', 'llama2', 'llama3', 'llama3.1', 'llama3.2',
28             'llama3.3', 'gemma', 'cc-by-sa-4.0', 'openrail'}
29 NAMES = {
30     'bbh_boolean_expressions': '布尔表达式', 'bbh_causal_judgement': '因果
    判断',
31     'bbh_date_understanding': '日期理解', 'bbh_disambiguation_qa': '歧义消
    解',
32     'bbh_formal_fallacies': '形式谬误', 'bbh_geometric_shapes': '几何形状'
    ,
33     'bbh_hyperbaton': '形容词排序', 'bbh_logical_deduction_five_objects':
    '逻辑推断·5对象',
34     'bbh_logical_deduction_seven_objects': '逻辑推断·7对象',
35     'bbh_logical_deduction_three_objects': '逻辑推断·3对象',
36     'bbh_movie_recommendation': '电影推荐', 'bbh_navigate': '路径导航',
37     'bbh_object_counting': '对象计数', 'bbh_penguins_in_a_table': '表格中
    的企鹅',
38     'bbh_reasoning_about_colored_objects': '彩色对象推理', '
    bbh_ruin_names': '名称变换',
39     'bbh_salient_translation_error_detection': '翻译错误识别', '
    bbh_snarks': '讽刺识别',
40     'bbh_sports_understanding': '体育知识理解', 'bbh_temporal_sequences':
    '时间序列推理',
41     'bbh_tracking_shuffled_objects_five_objects': '位置跟踪·5对象',
42     'bbh_tracking_shuffled_objects_seven_objects': '位置跟踪·7对象',
43     'bbh_tracking_shuffled_objects_three_objects': '位置跟踪·3对象',
44     'bbh_web_of_lies': '真假陈述推理',
45     'math_algebra_hard': '代数', 'math_counting_and_prob_hard': '计数与概
    率',
46     'math_geometry_hard': '几何', 'math_intermediate_algebra_hard': '中级
    代数',
47     'math_num_theory_hard': '数论', 'math_prealgebra_hard': '初等代数',
48     'math_precalculus_hard': '预备微积分',
49     'gpqa_diamond': 'Diamond', 'gpqa_extended': 'Extended', 'gpqa_main': '
    Main',
50     'musr_murder_mysteries': '谋杀谜题', 'musr_object_placements': '物体摆
    放',
51     'musr_team_allocation': '团队分配',
52 }
53
54 def read(name):
55     return pd.read_csv(RESET / (name + '.csv'))
56
57 def hashes():
58     return {str(p.relative_to(RESET)): hashlib.sha256(p.read_bytes()).
    hexdigest()
59             for p in RESET.rglob('*') if p.is_file()}
60
61 def style():
62     # 固定原导出样式, 避免依赖绘图者个人目录中的可选技能包。
63     plt.rcParams.update({'figure.dpi': 150, 'figure.figsize': [5, 3.5],
64                          'font.sans-serif': ['Arial', 'Helvetica', 'DejaVu Sans'],

```

```

65     'font.serif':['Noto Serif SC','Times New Roman','Times','DejaVu
    Serif'],
66     'lines.linewidth':1.2, 'lines.markersize':5, 'pdf.fonttype':42,
67     'ps.fonttype':42, 'savefig.dpi':300})
68     for name in ['simsum.ttc', 'times.ttf']:
69         p = Path('C:/Windows/Fonts') / name
70         if p.exists():
71             font_manager.fontManager.addfont(str(p))
72     plt.rcParams.update({'font.family':['Times New Roman', 'SimSun'], '
    font.size':10,
73     'axes.labelsize':10, 'axes.titlesize':11, 'xtick.labelsize':9, '
    ytick.labelsize':9,
74     'text.color':INK, 'axes.labelcolor':INK, 'axes.edgecolor':'
    #88838D',
75     'axes.spines.top':False, 'axes.spines.right':False, 'axes.
    unicode_minus':False,
76     'axes.linewidth':.6, 'legend.fontsize':9, 'legend.frameon':False
    ,
77     'svg.fonttype':'none', 'savefig.facecolor':'white',
78     'figure.constrained_layout.use':False, 'figure.autolayout':False
    })
79
80 def grid(ax, axis='y'):
81     ax.grid(axis=axis, color='#E8E4E9', lw=.6)
82     ax.set_axisbelow(True)
83
84 def title(ax, label):
85     ax.set_title(label, loc='left', pad=9)
86
87 def save(fig, name, audit):
88     fig.canvas.draw()
89     issues = []
90     try:
91         from visual_qa import audit_layout
92         issues = audit_layout(fig)
93     except ImportError:
94         pass
95     if any(level == 'FAIL' for level, _ in issues):
96         raise RuntimeError(issues)
97     out = HERE / '图片' / (name + '.png')
98     fig.savefig(out, dpi=400)
99     if audit:
100         fig.savefig(audit / (name + '.svg'))
101         (audit / (name + '_layout.json')).write_text(json.dumps(issues,
    ensure_ascii=False, indent=2), encoding='utf-8')
102     plt.close(fig)
103     print(name, issues)
104
105 def bridge(audit):
106     df, fits = read('Loss_Benchmark映射明细'), read('Loss_Benchmark映射'
    ).set_index('可比性')
107     fig, axes = plt.subplots(1, 2, figsize=(6.2, 3.05))
108     for ax, level, prefix, color, marker in zip(axes, ['高可比', '中可比'
    ], ['High', 'Medium'], [TEAL, PURPLE], ['D', 'o']):
109         d = df[df.Loss_Comparability.str.startswith(prefix)]
110         f = fits.loc[level]
111         assert len(d) == f['n']

```

```

112     ax.scatter(d.Val_Loss, d.LB_Average, s=25, color=color, marker=
marker, alpha=.75, edgecolors='white', linewidths=.4)
113     x = np.linspace(d.Val_Loss.min(), d.Val_Loss.max(), 80)
114     ax.plot(x, f['截距'] + f['斜率']*x, color=color, lw=1.7)
115     title(ax, f'{"(a)" if level == "高可比" else "(b)"} {level} · n
={len(d)}')
116     ax.set_xlabel('验证交叉熵损失')
117     grid(ax)
118     ax.text(.97, .93, f'$R^2$ = {f.R2:.2f}\nMAPE = {f["MAPE_%"]:.1f
}% ', ha='right', va='top', transform=ax.transAxes, fontsize=9)
119     axes[0].set(ylabel='Leaderboard 均分', ylim=(4.7,6.6))
120     axes[1].set(ylim=(0,65))
121     fig.subplots_adjust(left=.09, right=.98, bottom=.20, top=.85, wspace
=.30)
122     save(fig, '图1_Loss_Benchmark映射', audit)
123
124 def decomposition(audit):
125     df = read('规模时间分解')
126     fig, axes = plt.subplots(1,2,figsize=(6.2,3.45),sharey=True)
127     for ax, (_, r), label in zip(axes, df.iterrows(), ['(a) C3 · 2019—
2025', '(b) C1 · 2024年6—9月']):
128         v = [r['规模贡献分'], r['非规模贡献分'], r['前沿提升分']]
129         ax.bar([0,1,2], v, bottom=[0,v[0],0], color=[TEAL,GOLD,PURPLE],
width=.62)
130         ax.plot([.31,.69],[v[0],v[0]],lw=.7,color=INK)
131         ax.plot([1.31,1.69],[v[2],v[2]],lw=.7,color=INK)
132         for i, (value, y) in enumerate(zip(v,[v[0],v[2],v[2]])):
133             ax.text(i, y+1.2, f'{value:.2f}',ha='center',fontsize=10)
134         ax.set(xticks=[0,1,2],xticklabels=['规模\n关联项', '未解释\n剩余
项', '总增量'],ylim=(0,67))
135         title(ax,label)
136         grid(ax)
137         ax.text(.03,.93,f'关联项 {r["规模占比%"]:.1f}%\n剩余项 {r["非规
模占比%"]:.1f}%', transform=ax.transAxes,va='top',fontsize=9)
138         axes[0].set_ylabel('前沿提升 / 分')
139         fig.subplots_adjust(left=.10,right=.98,bottom=.21,top=.85,wspace
=.20)
140         save(fig,'图2_规模时间分解',audit)
141
142 def forecast(lb, audit):
143     par=read('logistic参数').iloc[0]
144     pred=read('前沿预测')
145     hist=read('前沿序列').query('Year <= 2023')
146     mon=lb.groupby(lb.date.dt.to_period('M'))['avg'].apply(lambda s:s.
nlargest(max(1,int(len(s)*.01))).mean())
147     times=np.array([p.year+(p.month-.5)/12 for p in mon.index])
148     x=np.r_[hist.Year.to_numpy()+.5,times]
149     y=np.maximum.accumulate(np.r_[hist.cum_avg.to_numpy(),mon.cummax().
to_numpy()])
150     assert len(x)==par['n前沿点']
151     log=lambda xx:par.K/(1+np.exp(-par.r*(np.asarray(xx)-2019-par['t0
(2019起)'])))
152     assert np.allclose(log(pred.iloc[:,0]+2019),pred.frontier_pred)
153     fig=plt.figure(figsize=(6.2,4.65))
154     gs=fig.add_gridspec(2,1,height_ratios=[1.35,1],hspace=.54)
155     ax=fig.add_subplot(gs[0]); last=2019+par.t_last
156     xx=np.linspace(x.min(),2027.3,250)

```

```

157 ax.axvspan(last,2027.4,color='#F3EFF6',zorder=0)
158 ax.plot(xx,log(xx),color=PURPLE,lw=1.6,label='logistic 条件曲线')
159 slow_x=np.linspace(last,2027.3,100)
160 slow_y=log(last+0.5*(slow_x-last))
161 assert np.allclose(log(last+0.5*(pred.iloc[:,0]+2019-last)),pred['情景_技术增速减半'])
162 ax.plot(slow_x,slow_y,color=GOLD,lw=1.4,ls='--',label='logistic 增长参数减半')
163 ax.plot(x,y,ls=':',lw=.8,color='#8E8994')
164 ax.scatter(x[:len(hist)],y[:len(hist)],color=GOLD,marker='s',s=25,label='C3 年度锚点',zorder=3)
165 ax.scatter(x[len(hist):],y[len(hist):],color=TEAL,marker='o',s=20,label='C1 月度前沿',zorder=3)
166 xp=pred.iloc[:,0].to_numpy()+2019
167 ax.scatter(xp,pred['情景_技术增速减半'],marker='v',s=23,color=GOLD,zorder=4)
168 ax.errorbar(xp,pred.frontier_pred,yerr=[pred.frontier_pred-pred['CI_low_5%'],pred['CI_high_95%']-pred.frontier_pred],fmt='D',color=PURPLE,capsize=4,ms=4,label='重拟合 5% - 95% 分位')
169 ax.set(xlim=(2019,2027.5),ylim=(0,75),xticks=[2019,2021,2023,2025,2027],ylabel='前沿得分 / 分')
170 ax.axvline(last,color='#ACA5B2',ls='--',lw=.8)
171 ax.text(last+.15,7,'预测期',fontsize=9,color=PURPLE)
172 title(ax,'(a) 历史前沿与曲线外推')
173 ax.legend(loc='upper left',ncol=2,fontsize=8,handlelength=1.4,columnsspacing=.8)
174 grid(ax)
175 ax=fig.add_subplot(gs[1]); scen=read('前沿预测_结构外推情景')
176 styles=[('年增量减半',1,GOLD,'v'),('年增量延续',0,TEAL,'o'),('规模扩容',2,PURPLE,'s')]
177 for name,i,color,mark in styles:
178     vals=[scen[scen['前瞻年数'].eq(yr)].iloc[i]['结构外推前沿'] for yr in [1,2]]
179     yy=np.array([1,0])+{'年增量减半':-.19,'年增量延续':0,'规模扩容':.19}[name]
180     ax.scatter(vals,yy,color=color,marker=mark,s=32,label=name)
181     for v,z in zip(vals,yy):
182         ax.text(v+.4,z,f'{v:.2f}',va='center',fontsize=9)
183 ax.set(yticks=[1,0],yticklabels=['2026-03','2027-03'],ylim=(-.5,1.5),xlim=(50,73),xlabel='结构外推得分 / 分')
184 title(ax,'(b) 三种结构情景的分歧')
185 ax.legend(loc='upper right',ncol=3,fontsize=8,handletextpad=.3,columnsspacing=.8)
186 grid(ax,'x')
187 fig.subplots_adjust(left=.13,right=.98,bottom=.12,top=.92)
188 save(fig,'图3_能力前沿预测',audit)
189
190 def subtasks(audit):
191     df=read('C8逐任务聚合')
192     assert len(df)==37 and set(df['子任务'])==set(NAMES)
193     fig=plt.figure(figsize=(6.2,6.45))
194     gs=fig.add_gridspec(3,2,width_ratios=[1.12,1],height_ratios=[7,3,3],wspace=.85,hspace=.76)
195     blocks=[(fig.add_subplot(gs[:,0]),'BBH子任务',PURPLE,'(a) BBH · 24项'),

```

```

196         (fig.add_subplot(gs[0,1]), 'MATH子任务', GOLD, '(b) MATH · 7项 '
197     ),
198         (fig.add_subplot(gs[1,1]), 'GPQA子集', TEAL, '(c) GPQA · 3项 '),
199         (fig.add_subplot(gs[2,1]), 'MUSR子任务', '#38678E', '(d) MUSR ·
200     3项')])
201     for ax, cat, color, label in blocks:
202         d=df[df['类别'].eq(cat)].sort_values('最优得分')
203         yy=np.arange(len(d))
204         assert (d['中位数']<=d['前5%均值']).all() and (d['前5%均值']<=d[
205             '最优得分']).all()
206         ax.hlines(yy, d['中位数'], d['最优得分'], color=color, alpha=.40, lw
207             =2.5)
208         for col, mark, fill in [('中位数', 'o', 'white'), ('前5%均值', '|',
209             color), ('最优得分', 'D', color)]:
210             if mark == '|':
211                 ax.scatter(d[col], yy, s=55, marker=mark, color=color,
212                     linewidths=.9, zorder=3)
213             else:
214                 ax.scatter(d[col], yy, s=20, marker=mark, facecolors=fill,
215                     edgecolors=color, linewidths=.9, zorder=3)
216         ax.set(yticks=yy, yticklabels=[NAMES[k] for k in d['子任务']],
217             xlim=(-2, 103), xticks=[0, 50, 100], ylim=(-.65, len(d)-.35))
218         ax.invert_yaxis(); ax.tick_params(axis='y', length=0, labelsize
219             =8.5)
220         title(ax, label); grid(ax, 'x')
221         ax.set_xlabel('原始指标 / 分', fontsize=9)
222         handles=[Line2D([], [], marker='o', mfc='white', mec=INK, ls='', label='中
223             位数'), Line2D([], [], marker='|', color=INK, ls='', label='前5%均值'),
224             Line2D([], [], marker='D', color=INK, ls='', label='最优得分')]
225         fig.legend(handles=handles, loc='lower center', bbox_to_anchor
226             =(0.52, .008), ncol=3, fontsize=9)
227         fig.subplots_adjust(left=.205, right=.98, bottom=.115, top=.93)
228         save(fig, '图4_逐任务聚合', audit)
229
230 def macro(audit):
231     d=read('C4宏观趋势'); summary=read('C4宏观摘要').iloc[0]
232     fig=plt.figure(figsize=(6.2, 4.75))
233     gs=fig.add_gridspec(2, 2, height_ratios=[1.15, 1], hspace=.72, wspace
234         =.32)
235     ax=fig.add_subplot(gs[0,:])
236     ax.vlines(d.year, d['前沿算力_中位'], d['前沿算力_max'], color=TEAL,
237         alpha=.5, lw=3)
238     ax.scatter(d.year, d['前沿算力_中位'], s=26, facecolors='white',
239         edgecolors=TEAL, label='年度中位数', zorder=3)
240     ax.scatter(d.year, d['前沿算力_max'], marker='D', s=25, color=PURPLE,
241         label='年度最大值', zorder=3)
242     ax.axvspan(2025.6, 2026.4, color='#F5F0E6')
243     ax.text(2026, 2e18, '5条算力记录', ha='center', fontsize=8, rotation=90)
244     ax.set(yscale='log', ylim=(1e16, 5e27), xlim=(2017.6, 2026.4), xticks=d.
245         year, ylabel='训练算力 / FLOP')
246     ax.set_yticks([1e17, 1e20, 1e23, 1e26]); title(ax, '(a) Language 领域的算
247         力跨度'); grid(ax)
248     ax.legend(loc='upper left', ncol=2, fontsize=9)
249     ax=fig.add_subplot(gs[1,0])
250     ax.plot(d.year, d['开源权重占比']*100, color=TEAL, marker='o', ms=4, lw
251         =1.2)

```

```

233     ax.axvspan(2025.6,2026.4,color='#F5F0E6')
234     ax.set(ylim=(0,100),yticks=[0,25,50,75,100],xticks
=[2018,2020,2022,2024,2026],xlabel='年份',ylabel='Yes 标记比例 / %')
235     title(ax,'(b) 开放权重的收录比例');grid(ax)
236     ax=fig.add_subplot(gs[1,1])
237     counts=[int(summary['有算力记录行数']),int(summary['有数据量记录行数
'])],int(summary['开源权重_Yes']+summary['开源权重_No']))
238     ax.barh([2,1,0],[3523]*3,color='#EFECE1',height=.5)
239     ax.barh([2,1,0],counts,color=[PURPLE,GOLD,TEAL],height=.5)
240     for yy,nn in zip([2,1,0],counts):ax.text(nn+65,yy,f'{nn:,}',va='
center',fontsize=9)
241     ax.set(yticks=[2,1,0],yticklabels=['算力','数据量','权重状态'],xlim
=(0,3800),xticks=[0,1500,3000],xlabel='有效记录数 / 条',ylim=(-
.6,2.6))
242     title(ax,'(c) C4 全量字段覆盖')
243     fig.subplots_adjust(left=.105,right=.98,bottom=.12,top=.92)
244     save(fig,'图5_C4宏观趋势',audit)
245
246 def distributions(lb,audit):
247     fig,ax=plt.subplots(figsize=(6.2,3.15))
248     order=['GPQA','MUSR','MATH Lvl 5','BBH','MMLU-PRO','IFEval']
249     vals=[lb[t].to_numpy() for t in order]
250     # Full range is explicit; no data points are suppressed as outliers.
251     bp=ax.boxplot(vals,orientation='horizontal',tick_labels=order,whis
=(0,100),showfliers=False,patch_artist=True,widths=.52,
252                     medianprops={'color':INK,'lw':1.4},boxprops={'
edgecolor':PURPLE},whiskerprops={'color':PURPLE},capprops={'color':
PURPLE})
253     for patch in bp['boxes']:patch.set_facecolor('#E5DEED')
254     ax.invert_yaxis();ax.set(xlim=(0,100),xticks=np.arange(0,101,20),
xlabel='C1 基准得分 / 分')
255     grid(ax,'x');ax.tick_params(axis='y',length=0)
256     fig.subplots_adjust(left=.16,right=.98,bottom=.20,top=.94)
257     save(fig,'图6_六维能力分布',audit)
258
259 def scale(audit):
260     d=read('面板回归数据');alpha=read('年份效应').set_index('Year')['年
效应alpha_t'];bp=read('规模时间分解').iloc[1]['bP']
261     fig,axes=plt.subplots(1,2,figsize=(6.2,3.15),sharex=True,sharey=True
)
262     for ax,year,color,mark in zip(axes,[2024,2025],[TEAL,PURPLE],['o','^
']):
263         sub=d[d.Year.eq(year)]
264         ax.scatter(sub.P,sub.avg,c=color,s=6,alpha=.25,marker=mark,
edgecolors='none')
265         xx=np.geomspace(sub.P.min(),sub.P.max(),150)
266         ax.plot(xx,alpha[year]+bp*np.log10(xx),color=INK,lw=1.4)
267         ax.set(xscale='log',xlim=(.001,220),ylim=(-35,60),xticks
=[.001,.01,.1,1,10,100],xticklabels=['0.001','0.01','0.1','1','10','
100'],xlabel='参数量 / B (对数轴)')
268         assert sub.P.between(.001,220).all() and sub.avg.between(-35,60)
.all()
269         ax.axhline(0,color='#B6AFBA',lw=.6,ls=':')
270         title(ax,f'{"(a)" if year==2024 else "(b)"} {year}年 · n={len(
sub):,}')
271         grid(ax)
272         axes[0].set_ylabel('Leaderboard 均分')

```

```

273     fig.subplots_adjust(left=.10,right=.98,bottom=.22,top=.85,wspace
    =.12)
274     save(fig,'图7_年度规模效应',audit)
275
276 def main():
277     ap=argparse.ArgumentParser(description=__doc__)
278     ap.add_argument('--data',type=Path,required=True)
279     ap.add_argument('--audit',type=Path)
280     ap.add_argument('--figures',nargs='+',choices=['bridge','
    decomposition','forecast','subtasks','macro','distributions','scale'
    ],
281                     help='只重绘指定图；省略时重绘全部图。')
282     args=ap.parse_args(); before=hashes(); style()
283     if args.audit:args.audit.mkdir(parents=True,exist_ok=True)
284     src=args.data/'C_efficiency_evolution'
285     lb=pd.read_csv(src/'leaderboard_cleaned.csv')
286     lb['P']=pd.to_numeric(lb['#Params (B)'],errors='coerce')
287     lb['date']=pd.to_datetime(lb['Submission Date'],errors='coerce')
288     lb['avg']=pd.to_numeric(lb['Average \u2b06\u2b06\u2b06'],errors='coerce')
289     lb[TASKS]=lb[TASKS].apply(pd.to_numeric,errors='coerce')
290     lb=lb[lb[TASKS].notna().all(axis=1)&lb.date.notna()&lb.P.notna()&lb[
    'Hub License'].astype(str).str.strip().str.lower().isin(LICENSES)].
    copy()
291     assert len(lb)==2346
292     td=read('逐任务聚合').set_index('task')
293     for task in TASKS:
294         s=lb[task]
295         assert np.allclose([s.max(),s.nlargest(max(1,int(len(s)*.01))).
    mean(),s.median()],td.loc[task,['max','top1_mean','median']].astype(
    float))
296     actual=pd.read_csv(src/'loss_benchmark_bridge_expanded.csv')
297     saved=read('Loss_Benchmark映射明细')
298     assert actual.Model.tolist()==saved.Model.tolist()
299     assert np.allclose(actual[['Val_Loss','LB_Average']],saved[['
    Val_Loss','LB_Average']])
300     renderers={'bridge':lambda:bridge(args.audit),'decomposition':lambda
    :decomposition(args.audit),
301               'forecast':lambda:forecast(lb,args.audit),'subtasks':
    lambda:subtasks(args.audit),
302               'macro':lambda:macro(args.audit),'distributions':lambda:
    distributions(lb,args.audit),
303               'scale':lambda:scale(args.audit)}
304     selected=args.figures or list(renderers)
305     for name in selected:
306         renderers[name]()
307     assert before==hashes(),'绘图期间结果文件发生改变'
308     if args.audit:
309         report={'rendered_figures':selected,'records':len(lb),'years':lb
    .date.dt.year.value_counts().to_dict(),
310               'task_summary':lb[TASKS].describe().to_dict(),'
    result_files_unchanged':True,
311               'chart_contracts':{
312                   '图1':'分层散点与已存拟合；仅组内损失范围；两面板纵
    轴不同。',
313                   '图2':'两个窗口的规模关联项和未解释剩余项；同一纵
    轴；占比不是技术增长的因果份额。',

```

```

314         '图3': '历史汇总点、条件曲线、仅两时点曲线重拟合5% -
95%分位范围；非未来观测预测区间；结构情景独立面板。',
315         '图4': '37子任务中位数/前5%均值/最大值；原始指标；按
类别分面。',
316         '图5': '年度算力跨度、Yes记录比例、全量字段覆盖；2026
覆盖不完整。',
317         '图6': '2346条实际附件记录的四分位区间与完整极差；展
示总体分布。',
318         '图7': '合并横截面的年度分面规模散点及既有年份控制直
线；未重拟合。'}}
319     (args.audit/'plot_checks.json').write_text(json.dumps(report,
ensure_ascii=False, indent=2), encoding='utf-8')
320
321 if __name__ == '__main__': main()

```

程序 22: 问题三四流程图（求解/绘制问题三四流程图.py）

```

1 # 本程序的整理与核对使用 Codex 辅助；模型：GPT-5.6-Luna；机构：OpenAI；
   版本发布日期：2026-07-09。
2 # 本程序的整理、复现核对或绘图实现使用 OpenAI Codex（GPT-5.6-Luna）辅
   助。
3 """按论文已有分析绘制问题三、四流程图，不读取数据、不运行求解模型。
4
5 运行：python 求解/绘制问题三四流程图.py
6 输出：论文/figures 下的 PDF、SVG、400 dpi PNG 及文字边界核验记录。
7 """
8 from pathlib import Path
9 import argparse
10 import json
11 import os
12 import tempfile
13
14 os.environ.setdefault('MPLCONFIGDIR', str(Path(tempfile.gettempdir()) /
'huawei-expand-mpl'))
15 import matplotlib
16 matplotlib.use('Agg')
17 import matplotlib.pyplot as plt
18 from matplotlib import font_manager
19 from matplotlib.colors import to_rgb
20 from matplotlib.patches import FancyArrowPatch, FancyBboxPatch,
   Rectangle
21
22 PURPLE, TEAL, GOLD, BLUE = '#665080', '#287C78', '#BC8030', '#365E8D'
23 INK, GRAY = '#243348', '#637186'
24
25 def configure():
26     for name in ('simsun.ttc', 'times.ttf', 'timesi.ttf', 'timesbd.ttf')
       :
27         path = Path('C:/Windows/Fonts') / name
28         if path.exists():
29             font_manager.fontManager.addfont(str(path))
30     for family in ('SimSun', 'Times New Roman'):
31         font_manager.findfont(family, fallback_to_default=False)
32     plt.rcParams.update({
33         'font.family': ['Times New Roman', 'SimSun'], 'font.size': 10,
34         'mathtext.fontset': 'stix', 'axes.unicode_minus': False,

```

```

35         'text.color': INK, 'figure.facecolor': 'white',
36         'savefig.facecolor': 'white', 'pdf.fonttype': 42,
37         'svg.fonttype': 'path', 'svg.hashsalt': 'huawei-expand-workflows
',
38     })
39
40 def tint(color, strength):
41     return tuple(1 - strength * (1 - value) for value in to_rgb(color))
42
43 class Workflow:
44     def __init__(self, height):
45         self.fig = plt.figure(figsize=(6.5, height))
46         self.ax = self.fig.add_axes([0, 0, 1, 1])
47         self.ax.set(xlim=(0, 6.5), ylim=(0, height))
48         self.ax.axis('off')
49         self.labels = []
50
51     def text(self, x, y, text, size=10, color=INK, bounds=None, **kwargs
):
52         if '$' in text:
53             # Mathtext needs a Chinese-capable default font for its non-
math runs.
54             kwargs['fontfamily'] = 'SimSun'
55             artist = self.ax.text(x, y, text, fontsize=size, color=color,
56                                 ha='center', va='center', linespacing=1.4,
57                                 zorder=5, **kwargs)
58             self.labels.append((artist, bounds))
59
60     def box(self, x, y, w, h, title, content, color, size=10):
61         self.ax.add_patch(FancyBboxPatch(
62             (x, y), w, h, boxstyle='round,pad=0,rounding_size=.045',
63             facecolor='white', edgecolor=tint(color, .55), linewidth
=.85,
64             zorder=3))
65         self.ax.add_patch(Rectangle(
66             (x+.012, y+h-.29), w-.024, .278, facecolor=tint(color, .11),
67             edgecolor='none', zorder=4))
68         self.text(x+w/2, y+h-.145, title, 10.4, color,
69                 bounds=(x+.03, y+h-.28, w-.06, .26))
70         self.text(x+w/2, y+(h-.29)/2, content, size,
71                 bounds=(x+.04, y+.025, w-.08, h-.33))
72
73     def arrow(self, points, color=GRAY, dashed=False):
74         style = (0, (3, 2)) if dashed else '-'
75         if len(points) > 2:
76             xx, yy = zip(*points[:-1])
77             self.ax.plot(xx, yy, color=color, lw=.95, linestyle=style,
zorder=2)
78             self.ax.add_patch(FancyArrowPatch(
79                 points[-2], points[-1], arrowstyle='->', mutation_scale=8,
80                 color=color, lw=.95, linestyle=style, shrinkA=0, shrinkB=0,
81                 zorder=2))
82
83     def lane(self, y, title, color):
84         self.ax.add_patch(FancyBboxPatch(
85             (.07, y), 6.36, 1.19,
86             boxstyle='round,pad=0,rounding_size=.055',
87             facecolor=tint(color, .04), edgecolor=tint(color, .19),

```

```

88         linewidth=.65, zorder=0))
89     self.ax.add_patch(Rectangle((.17, y+.97), .04, .14,
90                               facecolor=color, edgecolor='none'))
91     self.text(.29, y+1.04, title, 10.1, color)
92     self.labels[-1][0].set_ha('left')
93
94     def save(self, out, stem):
95         out.mkdir(parents=True, exist_ok=True)
96         self.fig.canvas.draw()
97         renderer = self.fig.canvas.get_renderer()
98         errors = []
99         for artist, bounds in self.labels:
100             box = artist.get_window_extent(renderer)
101             if bounds is None:
102                 outer = self.fig.bbox
103             else:
104                 x, y, w, h = bounds
105                 from matplotlib.transforms import Bbox
106                 outer = Bbox.from_bounds(x, y, w, h).transformed(self.ax
.transData)
107                 if (box.x0 < outer.x0-1 or box.x1 > outer.x1+1 or
108                     box.y0 < outer.y0-1 or box.y1 > outer.y1+1):
109                     errors.append(artist.get_text())
110             if errors:
111                 raise ValueError(f'{stem}: text exceeds bounds: {errors}')
112             self.fig.savefig(out / f'{stem}.pdf', metadata={
113                 'Title': stem, 'CreationDate': None, 'ModDate': None})
114             svg = out / f'{stem}.svg'
115             self.fig.savefig(svg, metadata={'Date': None})
116             svg.write_text('\n'.join(line.rstrip() for line in
117                                     svg.read_text(encoding='utf-8').
splitlines()))+'\n',
118                               encoding='utf-8')
119             self.fig.savefig(out / f'{stem}.png', dpi=400)
120             (out / f'{stem}-layout.json').write_text(json.dumps({
121                 'canvas_inches': list(self.fig.get_size_inches()),
122                 'text_count': len(self.labels), 'text_bounds_issues': errors
123             },
124             'scope': 'Text within nodes and canvas; visual review also
required.',
125             }, ensure_ascii=False, indent=2)+'\n', encoding='utf-8')
126             plt.close(self.fig)
127             print(f'{stem}: exported PDF/SVG/PNG, text bounds passed')
128
129     def q3(out):
130         f = Workflow(5.18)
131         f.box(.15, 4.02, 1.96, 1.00, '问题一：质量与配比',
132              '配比加权质量  $Q_0$ \n配比修正  $M$ ', PURPLE)
133         f.box(2.27, 4.02, 1.96, 1.00, '问题二：损失关系',
134              '广义标度律参数\n构建  $L(N,D,Q)$ ', TEAL)
135         f.box(4.39, 4.02, 1.96, 1.00, '题设：预算与费用',
136              '训练、质量、注意力\n预算与上下文条件', BLUE)
137         for x in (1.13, 3.25, 5.37):
138             f.arrow([(x, 3.995), (x, 3.79), (3.25, 3.79), (3.25, 3.72)])
139         f.box(.15, 2.77, 6.2, .91, '构建预算约束下的资源配置模型',
140              '以预测验证损失最小为目标，合计三类算力费用\n'
141               $r \cdot C_{\mathrm{tot}} \leq C_{\mathrm{b}}, \quad Q_0 \leq Q \leq 1$ )

```

```

141         '; 固定配比与上下文, 统一规模计量口径', PURPLE)
142     f.arrow([(3.25, 2.74), (3.25, 2.55)], PURPLE)
143     f.box(.15, 1.60, 6.2, .91, '多初值 SLSQP 求解',
144         r'采用 $\ln N, \ln D$ 与 $Q$; 三档预算设 8 组初值, 预算扫描设
145         5 组'
146         '\n比较求解成功的结果, 取其中损失最低者, 并处理质量上界', TEAL
147     )
148     for x in (1.13, 3.25, 5.37):
149         f.arrow([(3.25, 1.57), (3.25, 1.44), (x, 1.44), (x, 1.36)], TEAL
150         )
151     f.box(.15, .23, 1.96, 1.09, '三档预算配置',
152         '参数、数据与质量\n预测损失与费用份额\n比较投入结构', PURPLE,
153         9.7)
154     f.box(2.27, .23, 1.96, 1.09, '连续预算扫描',
155         '141 个对数网格点\n质量完成度与费用份额\n定位启动、饱和区间',
156         TEAL, 9.7)
157     f.box(4.39, .23, 1.96, 1.09, '情景与敏感性分析',
158         '三种质量成本\n五档上下文与配比情景\n比较配置与损失变化', GOLD
159         , 9.7)
160     f.text(3.25, .09, '扫描中的失败点不补算; 阶段边界按有效预算网格报
161     告。', 9, GRAY)
162     f.save(out, 'q3-workflow')
163
164 def q4(out):
165     f = Workflow(5.45)
166     rows = [
167         (4.14, '损失与得分的联系', PURPLE,
168         'C6 分层、C5 对照', '区分记录可比程度\n分别整理损失与得分',
169         '分层最小二乘回归', '拟合损失—得分关系\n检查相关性与换算误差',
170         '方向判断', '说明变化方向\n保留换算误差边界'),
171         (2.81, '历史增量的关联分解', TEAL,
172         'C1 与 C3 分别整理', 'C1 字段与许可筛选\nC3 得分一致性检查',
173         '规模、年份回归与分解', '估计规模关联与年份差异\n按长、短窗口作
174         shift-share 分解',
175         '历史增量核算', '规模关联项与剩余项\n分别报告两类窗口'),
176         (1.48, '未来前沿的条件预测', BLUE,
177         '前沿序列与回归系数', 'C3 年度、C1 月度前沿\n设定规模与年增量',
178         '两条预测路径', 'logistic 拟合与重拟合分位范围\n规模与设定年增
179         量的结构情景',
180         '前瞻 12 / 24 个月', '条件预测与分位范围\n结构情景点值'),
181         (.15, '任务瓶颈与宏观参照', GOLD,
182         'C1 / C8 与 C4', '六维得分与原始子任务\n宏观资源和字段覆盖',
183         '任务比较与资源趋势', 'C1—C8 名称匹配与口径核对\nC4 单独统计算
184         力与开放权重',
185         '适用范围说明', '识别评测薄弱项\n说明宏观证据边界'),
186     ]
187     for y, label, color, t1, b1, t2, b2, t3, b3 in rows:
188         f.lane(y, label, color)
189         f.box(.17, y+.09, 1.70, .80, t1, b1, color, 9.25)
190         f.box(2.09, y+.09, 2.39, .80, t2, b2, color, 9.25)
191         f.box(4.71, y+.09, 1.60, .80, t3, b3, color, 9.25)
192         f.arrow([(1.90, y+.46), (2.06, y+.46)], color)
193         f.arrow([(4.51, y+.46), (4.68, y+.46)], color)
194     f.save(out, 'q4-workflow')
195
196 if __name__ == '__main__':

```

```

187     parser = argparse.ArgumentParser(description=__doc__)
188     parser.add_argument('--output-dir', type=Path,
189                         default=Path(__file__).resolve().parents[1] / '
论文' / 'figures')
190     args = parser.parse_args()
191     configure()
192     q3(args.output_dir)
193     q4(args.output_dir)

```

审稿补充诊断的计算程序

程序 23: 质量标尺与线性对照稳健性诊断 (求解/审稿补充/q1_决策与稳健性.py)

```

1 # -*- coding: utf-8 -*-
2 """问题一的独立诊断；不覆盖原评分、拟合、推荐或跨问接口。
3
4 用法: python 求解/审稿补充/q1_决策与稳健性.py --data-dir <
      real_attachments>
5 可用 --section decision 只复核决策, 或 --section quality 只复核标尺。
6 固定既有超参数, 不重新搜索核岭；四档岭参数沿用原敏感性区间。
7 线性对照读取“线性有界配比对照.csv”；正式配方读取“推荐配比调整.csv”。
8 诊断不产生真实语言模型训练结果。实现与说明经 OpenAI Codex 辅助。
9 """
10 from pathlib import Path
11 import argparse
12 import ast
13 import hashlib
14 import json
15 import lzma
16 import platform
17
18 import numpy as np
19 import pandas as pd
20 from scipy.linalg import solve
21 from scipy.optimize import linprog
22 from scipy.spatial.distance import cdist
23 from scipy.stats import spearmanr
24
25 HERE = Path(__file__).resolve().parent
26 ROOT = HERE.parents[1]
27 RESULTS = ROOT / '求解/问题一/结果'
28
29 def digest(path):
30     h = hashlib.sha256()
31     with Path(path).open('rb') as stream:
32         for block in iter(lambda: stream.read(1024 * 1024), b''):
33             h.update(block)
34     return h.hexdigest()
35
36 def pure_definitions(relative, functions, assignments=()):
37     """只载入白名单函数/常量, 绝不执行原求解脚本的顶层流程。"""
38     tree = ast.parse((ROOT / relative).read_text(encoding='utf-8-sig'))
39     nodes = [node for node in tree.body if (
40         isinstance(node, ast.FunctionDef) and node.name in functions
41         ) or (isinstance(node, ast.Assign) and any(

```

```

42         isinstance(target, ast.Name) and target.id in assignments for
target in node.targets))]]
43     namespace = {'np': np, 'cdist': cdist, 'solve': solve}
44     exec(compile(ast.Module(body=nodes, type_ignores=[]), '<ql-pure-
definitions>', 'exec'), namespace)
45     return namespace
46
47 def write_json(out, name, value):
48     (out / name).write_text(json.dumps(value, ensure_ascii=False, indent
=2) + '\n', encoding='utf-8')
49
50 def decision_diagnostics(data, out):
51     folder = data / 'A_data_value/regmix_tables'
52     mix = pd.read_csv(folder / 'train_mixture_1m.csv').sort_values('
index')
53     loss = pd.read_csv(folder / 'train_pile_loss_1m.csv').sort_values('
index')
54     if mix['index'].duplicated().any() or not np.array_equal(mix['index'
], loss['index']):
55         raise ValueError('训练配方与损失的 index 不匹配或重复')
56     mc = [c for c in mix if c.startswith('train_the_pile_')]
57     lc = [c for c in loss if c.endswith('_val_loss')]
58     domains = [c.removeprefix('train_the_pile_') for c in mc]
59     targets = [c.removeprefix('metric/the_pile_').removesuffix('_
_val_loss') for c in lc]
60     x, y = mix[mc].to_numpy(float), loss[lc].to_numpy(float)
61     ref = x.mean(0)
62     saved_recipe = pd.read_csv(RESULTS / '线性有界配比对照.csv').
set_index('domain').loc[domains]
63     rec = saved_recipe.recommended_mixture.to_numpy(float)
64     np.testing.assert_allclose(ref, saved_recipe.reference_mixture, atol
=1e-12, rtol=0)
65     weights = ref[[domains.index(d) for d in targets]].copy()
66     weights /= weights.sum()
67     a0 = pd.read_csv(RESULTS / '混合系数矩阵.csv').set_index('
training_domain').loc[domains, targets].to_numpy(float)
68     b0 = pd.read_csv(RESULTS / '截距.csv').set_index('loss_domain').loc[
targets, 'intercept'].to_numpy(float)
69     ac0 = a0 - a0.mean(0)
70     fixed = np.array([d in {'nih_exporter', 'enron_emails', 'europarl',
'philpapers'} for d in domains])
71     bounds = [(ref[i], ref[i]) if fixed[i] else (0, 2.5 * ref[i]) for i
in range(len(domains))]
72
73     sensitivity, recipes = [], []
74     for lam in [1e-4, 1e-3, 1e-2, 1e-1]:
75         design = np.column_stack([x, np.ones(len(x))])
76         penalty = lam * np.eye(design.shape[1])
77         penalty[-1, -1] = 0
78         fit = np.linalg.solve(design.T @ design + penalty, design.T @ y)
79         a, b = fit[:-1], fit[-1]
80         centered = a - a.mean(0)
81         opt = linprog(a @ weights, A_eq=np.ones((1, len(domains))), b_eq
=[1], bounds=bounds, method='highs')
82         if not opt.success:
83             raise RuntimeError(opt.message)
84         if lam == 1e-3:
85             np.testing.assert_allclose(a, a0, atol=1e-10, rtol=0)

```

```

86         np.testing.assert_allclose(b, b0, atol=1e-10, rtol=0)
87         np.testing.assert_allclose(opt.x, rec, atol=1e-9, rtol=0)
88         sensitivity.append({
89             'lambda': lam,
90             'centered_relative_change': float(np.linalg.norm(centered-
ac0) / np.linalg.norm(ac0)),
91             'centered_sign_flips': int(np.sum(centered * ac0 < 0)),
92             'recipe_L1_change': float(np.abs(opt.x-rec).sum()),
93             'candidate_regret_under_main_model': float((opt.x-rec) @ a0
@ weights),
94             'own_model_reference_minus_candidate': float((ref-opt.x) @ a
@ weights),
95         })
96         recipes.extend({'lambda': lam, 'domain': d, 'mixture': float(v)}
for d, v in zip(domains, opt.x))
97
98     table = pd.read_csv(RESULTS / '非线性代理对照_预测明细.csv')
99     weighted = []
100     for (scope, model), group in table.groupby(['scope', 'model']):
101         yp_frame = group.pivot(index='index', columns='loss_domain',
values='predicted')[targets]
102         yo_frame = group.pivot(index='index', columns='loss_domain',
values='observed')[targets]
103         yp, yo = yp_frame.to_numpy() @ weights, yo_frame.to_numpy() @
weights
104         best = int(np.argmin(yp))
105         weighted.append({
106             'scope': scope, 'model': model, 'n_recipes': len(yp),
107             'weighted_RMSE': float(np.sqrt(np.mean((yp-yo)**2))),
108             'weighted_Spearman': float(spearmanr(yp, yo).statistic),
109             'predicted_best_index': int(yp_frame.index[best]),
110             'observed_rank_of_predicted_best': float(pd.Series(yo).rank(
method='average').iloc[best]),
111             'observed_regret_of_predicted_best': float(yo[best]-yo.min()
),
112         })
113
114     krr = pure_definitions('求解/问题一/非线性代理对照.py', {'kernel', '
centered_kernel', 'cross_center', 'krr_predict'})['krr_predict']
115     config = json.loads((RESULTS / '非线性代理对照_复现清单.json').
read_text(encoding='utf-8-sig'))
116     gamma, krr_lambda = float(config['selected_gamma']), float(config['
selected_lambda'])
117     px = x / x.sum(1, keepdims=True)
118     tx_frame = pd.read_csv(folder / 'test_mixture_lm.csv').sort_values('
index')
119     tx = tx_frame[mc].to_numpy(float)
120     test_pred = krr(px, y, tx / tx.sum(1, keepdims=True), gamma,
krr_lambda)
121     saved_test = table[(table.scope == 'lm') & (table.model == 'sqrt_krr
')].pivot(index='index', columns='loss_domain', values='predicted').
loc[tx_frame['index'], targets].to_numpy()
122     max_error = float(np.max(np.abs(test_pred-saved_test)))
123     np.testing.assert_allclose(test_pred, saved_test, atol=1e-9, rtol=0)
124     kp = krr(px, y, np.vstack([ref/ref.sum(), rec]), gamma, krr_lambda)
@ weights
125     lp = (np.vstack([ref, rec]) @ a0 + b0) @ weights

```

```

126     candidate = [dict(model=model, reference=float(v[0]), candidate=
float(v[1]), reference_minus_candidate=float(v[0]-v[1])) for model, v
in [('linear_ridge', lp), ('sqrt_krr', kp)]]
127 # 给出训练支持域检查; 不把凸包外自动解释成预测无效。
128 n, p = px.shape
129 projection = linprog(np.r_[np.zeros(n), np.ones(2*p)],
130     A_eq=np.r_[np.c_[px.T, np.eye(p), -np.eye(p)], np.r_[np.ones(n),
np.zeros(2*p)])[None, :]],
131     b_eq=np.r_[rec, 1], bounds=(0, None), method='highs')
132 if not projection.success:
133     raise RuntimeError(projection.message)
134 formal = pd.read_csv(RESULTS / '推荐配比调整.csv').set_index('domain
').loc[domains]
135 formal_p = formal.recommended_mixture.to_numpy(float)
136 formal_ref = formal.reference_mixture.to_numpy(float)
137 np.testing.assert_allclose(formal_ref, ref/ref.sum(), atol=1e-12,
rtol=0)
138 certificate = pd.read_csv(RESULTS / '核岭推荐_训练凸组合.csv').
set_index('index')
139 z = certificate.loc[mix['index'], 'convex_weight'].to_numpy(float)
140 reconstruction_error = float(np.max(np.abs(z @ px-formal_p)))
141 residual = max(float(abs(z.sum()-1)), float(max(0, -z.min()))),
142     float(np.max(np.abs(formal_p[fixed]-formal_ref[fixed
]))) ,
143     float(np.maximum(formal_p[~fixed]-2.5*formal_ref[~
fixed], 0).max()))
144 assert reconstruction_error < 1e-10 and residual < 1e-8
145 formal_scores = krr(px, y, np.vstack([formal_ref, formal_p]), gamma,
krr_lambda) @ weights
146 saved_targets = pd.read_csv(RESULTS / '核岭推荐_目标对照.csv').
set_index('mixture')
147 np.testing.assert_allclose(formal_scores, saved_targets.loc[['参考配
比', '问题一有界推荐'], 'predicted_weighted_loss'], atol=1e-9, rtol
=0)
148 starts = pd.read_csv(RESULTS / '核岭推荐_多初值.csv')
149 formal_check = {
150     'recipe_role': '
current_kernel_recommendation_with_training_convex_hull_constraint',
151     'reference': float(formal_scores[0]), 'candidate': float(
formal_scores[1]),
152     'M_p': float(formal_scores[1]-formal_scores[0]),
153     'certificate_reconstruction_max_error': reconstruction_error,
154     'certificate_constraint_residual': residual,
155     'active_training_rows': int(np.sum(z > 1e-8)),
156     'multistart_objective_span': float(starts.objective.max()-starts
.objective.min()),
157     'saved_multistart_all_success': bool(starts.success.all()),
158     'new_vs_old_linear_recipe_L1': float(np.abs(formal_p-rec).sum())
,
159     'interpretation': 'Saved certificate and fixed-model prediction
verified; no optimization rerun, global optimality or real-training
claim.',
160 }
161 write_json(out, 'q1_核岭正式配方核验.json', formal_check)
162 summary = {
163     'status': 'independent_diagnostic_not_new_training',
164     'linear_candidate_source': '线性有界配比如对照.csv',

```

```

165     'formal_candidate_source': '推荐配比调整.csv',
166     'weights_definition': 'A4 raw mean shares restricted to the 13
validation domains, then normalized',
167     'target_weights': dict(zip(targets, weights.tolist())),
168     'reference_raw_sum': float(ref.sum()),
169     'krr_reference_rule': 'row-closed reference; linear baseline
uses original raw shares',
170     'fixed_krr_theta': gamma, 'fixed_krr_lambda': krr_lambda,
171     'saved_lm_krr_max_abs_error': max_error,
172     'candidate_training_convex_hull_L1_distance': float(projection.
fun),
173     'ridge_sensitivity': sensitivity, 'candidate_model_comparison':
candidate,
174     'weighted_recipe_diagnostics': weighted,
175     'formal_kernel_candidate_check': formal_check,
176 }
177 pd.DataFrame(weighted).to_csv(out / 'q1_目标一致检验.csv', index=
False, encoding='utf-8-sig')
178 pd.DataFrame(candidate).to_csv(out / 'q1_线性候选双代理对照.csv',
index=False, encoding='utf-8-sig')
179 pd.DataFrame(sensitivity).to_csv(out / 'q1_正则配置敏感性.csv',
index=False, encoding='utf-8-sig')
180 pd.DataFrame(recipes).to_csv(out / 'q1_正则条件候选配比.csv', index=
False, encoding='utf-8-sig')
181 write_json(out, 'q1_决策诊断.json', summary)
182 return summary
183
184 def conflict_diagnostics(out):
185     grouped = pd.read_csv(RESULTS / '冲突样本裁决汇总.csv')
186     rows = []
187     for domain, group in grouped.groupby('domain'):
188         flagged = group[group.conflict_flag == True]
189         count = lambda label: int(flagged.loc[flagged.decision == label,
'n'].sum())
190         rows.append({'domain': domain, 'n': int(group.n.sum()), 'flagged
': int(flagged.n.sum()),
191                     'flagged_fraction': float(flagged.n.sum()/group.n.
sum()),
192                     'flagged_below_median': count('低于域中位数'),
193                     'flagged_equal_median': count('等于域中位数'),
194                     'flagged_above_median': count('高于域中位数')})
195     pd.DataFrame(rows).to_csv(out / 'q1_冲突标签定位.csv', index=False,
encoding='utf-8-sig')
196     return rows
197
198 def quality_diagnostics(data, out):
199     functions = pure_definitions('求解/问题一/问题一.py', {'_scalarize',
'_softmax_expected'}, {'QUALITY_COLS', 'TARGET_COLS', 'NEGATIVE_COLS
'})
200     cols = functions['QUALITY_COLS']
201     raw = data / 'A_data_value'
202     paths = [(raw / 'slimpajama_quality_signal_sample.jsonl.xz', 'A1',
None)]
203     paths += [(p, 'A2' if p.name.startswith('arxiv') else 'A3', 'arxiv'
if p.name.startswith('arxiv') else 'github') for p in sorted((raw / '
slimpajama_quality_extended').glob('*.xz'))]
204     rows, qr, domains, sources, ids = [], [], [], [], []

```

```

205     for path, source, hint in paths:
206         with lzma.open(path, 'rt', encoding='utf-8') as stream:
207             for line in stream:
208                 item = json.loads(line)
209                 rows.append([functions['_scalarize'](c, item.get(c)) for
c in cols])
210             qr.append(item['qurater'])
211             domains.append(item.get('_source_domain', hint))
212             sources.append(source)
213             ids.append(item['id'])
214             print(f'读取 {path.name}: 累计 {len(rows)} 条', flush=True)
215     x, qr = np.asarray(rows, float), np.asarray(qr, float)
216     domain, source = np.asarray(domains), np.asarray(sources)
217     a1 = source == 'A1'
218     qi = cols.index('qurater')
219
220     def score(mask):
221         q = qr.copy()
222         q = np.where(np.isfinite(q), q, np.nanmedian(np.where(np.
isfinite(q[mask]), q[mask], np.nan), axis=0))
223         lo, hi = np.quantile(q[mask], [.01, .99], axis=0)
224         values = x.copy()
225         values[:, qi] = np.clip((q-lo)/np.maximum(hi-lo, 1e-12), 0, 1).
mean(1)
226         med = np.nanmedian(np.where(np.isfinite(values[mask]), values[
mask], np.nan), axis=0)
227         values = np.where(np.isfinite(values), values, med)
228         centers = {}
229         for name in functions['TARGET_COLS']:
230             j = cols.index(name)
231             z = np.log1p(np.maximum(values[:, j], 0))
232             m = np.median(z[mask])
233             mad = np.median(np.abs(z[mask]-m)) + 1e-9
234             centers[name] = float(np.expml(m))
235             values[:, j] = np.exp(-np.abs(z-m)/(2.5*mad))
236             lo, hi = np.quantile(values[mask], [.01, .99], axis=0)
237             z = np.clip((values-lo)/np.maximum(hi-lo, 1e-12), 0, 1)
238             for name in functions['NEGATIVE_COLS']:
239                 z[:, cols.index(name)] = 1-z[:, cols.index(name)]
240             selected = z[mask]
241             p = (selected+1e-12)/(selected.sum(0)+1e-12)
242             entropy = -(p*np.log(p)).sum(0)/np.log(mask.sum())
243             w = np.maximum(1-entropy, 0)
244             w /= w.sum()
245             return z @ w, w, centers
246
247     full, wf, cf = score(np.ones(len(x), dtype=bool))
248     alternate, wa, ca = score(a1)
249     saved = pd.read_csv(RESULTS / '样本级质量评分.csv')
250     if list(saved.id) != ids or not np.array_equal(saved.source.to_numpy
()), source):
251         raise ValueError('原始记录次序与保存评分不对应')
252     error = float(np.max(np.abs(full-saved.quality_Q.to_numpy())))
253     np.testing.assert_allclose(full, saved.quality_Q, atol=2e-10, rtol
=0)
254     means = pd.DataFrame({'domain': domain, 'pooled_calibration': full,
'A1_calibration': alternate}).groupby('domain').mean().reset_index()
255     summary = {

```

```

256     'status': 'calibration_sensitivity_not_replacement', 'n_records'
: len(x), 'n_A1': int(a1.sum()),
257     'pooled_score_reconstruction_max_error': error,
258     'length_centers_pooled': cf, 'length_centers_A1': ca,
259     'github_fraction_pooled': float(np.mean(domain == 'github')),
260     'github_fraction_A1': float(np.mean(domain[a1] == 'github')),
261     'domain_rank_spearman': float(spearmanr(means.pooled_calibration
, means.A1_calibration).statistic),
262     'domain_means': means.to_dict('records'),
263     'quality_global_mean_pooled': float(full.mean()), '
quality_global_mean_A1_calibrated': float(alternate.mean()),
264     'raw_inputs': [{ 'path': p.relative_to(data).as_posix(), 'sha256'
: digest(p)} for p, _, _ in paths],
265     }
266     means.to_csv(out / 'q1_A1标尺域均分对照.csv', index=False, encoding=
'utf-8-sig')
267     pd.DataFrame({'indicator': cols, 'pooled_weight': wf, '
A1_calibrated_weight': wa}).to_csv(out / 'q1_A1标尺权重对照.csv',
index=False, encoding='utf-8-sig')
268     write_json(out, 'q1_A1标尺诊断.json', summary)
269     return summary
270
271 def main():
272     parser = argparse.ArgumentParser(description=__doc__)
273     parser.add_argument('--data-dir', type=Path, required=True, help='包
含 A_data_value 的原始附件根目录')
274     parser.add_argument('--output-dir', type=Path, default=HERE / 'q1_结
果')
275     parser.add_argument('--section', choices=['all', 'decision', '
quality'], default='all')
276     args = parser.parse_args()
277     data, out = args.data_dir.resolve(), args.output_dir.resolve()
278     if not (data / 'A_data_value/regmix_tables').is_dir():
279         raise FileNotFoundError('附件根目录缺少 A_data_value/
regmix_tables')
280     if out == RESULTS.resolve() or RESULTS.resolve() in out.parents or
out == data or data in out.parents:
281         raise ValueError('诊断结果不能写入原结果目录或原始附件')
282     out.mkdir(parents=True, exist_ok=True)
283     watched = list(RESULTS.glob('*.csv')) + [ROOT / '求解/问题一_关键量.
json']
284     before = {p: digest(p) for p in watched}
285     if args.section in {'all', 'decision'}:
286         decision_diagnostics(data, out)
287     if args.section in {'all', 'quality'}:
288         quality_diagnostics(data, out)
289     conflict_diagnostics(out)
290     if before != {p: digest(p) for p in watched}:
291         raise RuntimeError('原结果或跨问接口在诊断期间发生变化')
292     manifest = {
293         'section': args.section, 'python': platform.python_version(),
294         'description': 'Independent diagnostics; original result hashes
unchanged. No language model training.',
295         'script_sha256': digest(__file__),
296         'method_sources': {p.relative_to(ROOT).as_posix(): digest(p) for
p in

```

```

297         [ROOT / '求解/问题一/问题一.py', ROOT / '求解
    /问题一/非线性代理对照.py',
298         ROOT / '求解/问题一/核岭支持域推荐.py']},
299         'protected_inputs': {p.relative_to(ROOT).as_posix(): h for p, h
    in before.items()},
300         'regmix_inputs': {p.relative_to(data).as_posix(): digest(p) for
    p in sorted((data / 'A_data_value/regmix_tables').glob('*.csv'))},
301         'outputs': {p.name: digest(p) for p in sorted(out.glob('q1_*'))
    if p.is_file() and p.name != 'q1_复现清单.json'},
302     }
303     write_json(out, 'q1_复现清单.json', manifest)
304     print('独立诊断完成; 原结果、推荐与跨间接口的 SHA256 均未改变。')
305
306 if __name__ == '__main__':
307     main()

```

程序 24: 质量尺度与配比收益诊断 (求解/审稿补充/q23_尺度与结构敏感性.py)

```

1 # -*- coding: utf-8 -*-
2 # 本补充程序于 2026-09-27 使用 OpenAI Codex 辅助整理及核验。
3 # 仅增加审稿所需的独立诊断; 不运行原求解入口、不修改原参数或原结果。
4 """Q2/Q3 的评分尺度、质量指数与配比收益敏感性。
5
6 从任意工作目录运行:
7     python q23_尺度与结构敏感性.py --data-dir /path/to/real_attachments
8 可用 --output-dir 指定独立输出目录; 默认写入本脚本旁的 q23_结果。
9 依赖: numpy、pandas、scipy。附件可分层或平铺, 不依赖 _common.py。
10
11 gamma=1 仅作六参数受限拟合; 不重选主模型、不重跑完整预算扫描。
12 Q 剖面在少量预算处显式比较端点和网格定位后的局部极小, 不声明全局证明。
13 """
14 from __future__ import annotations
15
16 import argparse
17 from dataclasses import asdict, dataclass
18 from datetime import datetime, timezone
19 import hashlib
20 import json
21 import os
22 from pathlib import Path
23 import platform
24 import subprocess
25 import sys
26
27 import numpy as np
28 import pandas as pd
29 import scipy
30 from scipy.optimize import brentq, least_squares, minimize_scalar
31 from scipy.stats import spearmanr
32
33 HERE = Path(__file__).resolve().parent
34 PROJECT = HERE.parents[1]
35 K = 6.0 + 2e-4 * 2048.0
36 PARAMETER_NAMES = ('E', 'A', 'alpha', 'B', 'beta', 'C', 'gamma')
37
38 @dataclass(frozen=True)

```

```

39 class Parameters:
40     E: float
41     A: float
42     alpha: float
43     B: float
44     beta: float
45     C: float
46     gamma: float
47
48     def vector(self):
49         return np.array([getattr(self, name) for name in PARAMETER_NAMES
50 ])
51
52 def predict(p, n, d, q):
53     return p.E + p.A * np.asarray(n) ** -p.alpha + p.B * np.asarray(d) *
54         * -p.beta + p.C * (1 - np.asarray(q)) ** p.gamma
55
56 def quality_cost(q):
57     return 5e9 * q ** 4
58
59 def fixed_quality(p, budget, q, q0):
60     """消元 D 后在 N 方向求解，并保留原 N/D 搜索边界。N、D 输出为 B。"""
61     if not 0 <= q0 <= q <= 1:
62         raise ValueError('需要 0 <= q0 <= q <= 1')
63     s = (quality_cost(q) - quality_cost(q0)) / 1e9
64     cb = budget / 1e18
65     n_lo = max(1e-4, (cb / 1e7 - s) / K)
66     n_hi = min(1e6, (cb / 1e-3 - s) / K)
67     if n_hi <= n_lo:
68         raise ValueError('给定质量下无可行规模区间')
69
70     def data(n):
71         return cb / (K * n + s)
72
73     def derivative(log_n):
74         n = np.exp(log_n)
75         d = data(n)
76         return -p.alpha * p.A * n ** -p.alpha + p.beta * p.B * d ** -p.
77         beta * K * n / (K * n + s)
78
79     lo, hi = np.log(n_lo), np.log(n_hi)
80     if derivative(lo) >= 0:
81         log_n = lo
82     elif derivative(hi) <= 0:
83         log_n = hi
84     else:
85         log_n = brentq(derivative, lo, hi, xtol=1e-13)
86     n = float(np.exp(log_n))
87     d = float(data(n))
88     residual = (d * (K * n + s) - cb) / cb
89     if not (1e-4 * (1-1e-10) <= n <= 1e6 * (1+1e-10) and 1e-3 * (1-1e-
90 10) <= d <= 1e7 * (1+1e-10)):
91         raise RuntimeError('规模边界核验失败')
92     if abs(residual) > 1e-10:
93         raise RuntimeError('预算核验失败')
94     return dict(C_budget=float(budget), N_B=n, D_B=d, Q=float(q), L=
95 float(predict(p, n, d, q)), budget_relative_residual=float(residual))

```

```

92 def quality_profile(p, budget, q0):
93     grid = np.unique(np.r_[np.linspace(q0, 1, 121), 1 - 10. ** np.arange
94         (-12., -1.)])
95     grid = grid[grid >= q0]
96     values = [fixed_quality(p, budget, q, q0)['L'] for q in grid]
97     candidates = [fixed_quality(p, budget, q0, q0), fixed_quality(p,
98         budget, 1., q0)]
99     for i in range(1, len(grid) - 1):
100         if values[i] < values[i-1] and values[i] < values[i+1]:
101             result = minimize_scalar(lambda q: fixed_quality(p, budget,
102                 q, q0)['L'], bounds=(grid[i-1], grid[i+1]), method='bounded', options
103                 ={'xatol': 1e-13})
104             if not result.success:
105                 raise RuntimeError('质量剖面局部细化失败')
106             candidates.append(fixed_quality(p, budget, float(result.x),
107                 q0))
108     return min(candidates, key=lambda x: x['L']), candidates
109
110 def robust_objective(residual):
111     return float(np.sum(.05**2 * (np.sqrt(1 + (residual / .05)**2) - 1))
112 )
113
114 def errors(residual):
115     return dict(RMSE=float(np.sqrt(np.mean(residual**2))), MAE=float(np.
116         mean(np.abs(residual))))
117
118 def sha256(path):
119     return hashlib.sha256(path.read_bytes()).hexdigest()
120
121 def attachment(data_dir, name):
122     matches = sorted(data_dir.rglob(name))
123     if len(matches) != 1:
124         raise FileNotFoundError(f'{name} 应有且仅有一个文件，实际匹配 {
125             len(matches)} 个')
126     return matches[0]
127
128 def main():
129     if hasattr(sys.stdout, 'reconfigure'):
130         sys.stdout.reconfigure(encoding='utf-8')
131     parser = argparse.ArgumentParser(description=__doc__,
132         formatter_class=argparse.RawDescriptionHelpFormatter)
133     parser.add_argument('--data-dir', type=Path, default=os.environ.get(
134         'MODELING_DATA_DIR'), help='附件根目录；也可设置 MODELING_DATA_DIR')
135     parser.add_argument('--output-dir', type=Path, default=HERE/'q23_结
136         果', help='独立补充结果目录，不得覆盖原求解结果')
137     args = parser.parse_args()
138     if args.data_dir is None:
139         parser.error('请用 --data-dir 或 MODELING_DATA_DIR 指定附件目录')
140     data_dir, output = args.data_dir.resolve(), args.output_dir.resolve
141     ()
142     if not data_dir.is_dir():
143         parser.error('附件目录不存在')
144     if output.is_relative_to(data_dir):
145         parser.error('补充输出不能写入原附件目录')
146     if output.is_relative_to(PROJECT) and not output.is_relative_to(HERE
147         /'q23_结果'):

```

```

135     parser.error('项目内仅允许写入 求解/审稿补充/q23_结果; 其他输出
请放项目外')
136
137     inputs = {}
138     def read_project(relative, **kwargs):
139         path = PROJECT/relative
140         inputs['project:'+relative] = sha256(path)
141         return pd.read_csv(path, **kwargs)
142
143     def read_attachment(name):
144         path = attachment(data_dir, name)
145         inputs['attachment:'+path.relative_to(data_dir).as_posix()] =
sha256(path)
146         return pd.read_csv(path)
147
148     values = read_project('求解/广义标度律参数.csv').set_index('param').
value
149     p = Parameters(**{name: float(values[name]) for name in
PARAMETER_NAMES})
150     key_path = PROJECT/'求解/问题一_关键量.json'
151     inputs['project:求解/问题一_关键量.json'] = sha256(key_path)
152     key = json.loads(key_path.read_text(encoding='utf-8'))
153     q0 = float(key['Q0_mix_weighted'])
154     qmap = read_project('求解/17域质量映射.csv').set_index('
mixture_domain').quality_Q
155     mix = read_project('求解/问题一/结果/推荐配比调整.csv')
156     mapped = mix.domain.map(qmap)
157     if mapped.isna().any():
158         raise ValueError('候选配方有未映射的质量领域')
159     reference_q = float(mix.reference_mixture @ mapped)
160     candidate_q = float(mix.recommended_mixture @ mapped)
161     # Q3 沿用原始平均份额计费; KRR 使用和为 1 的闭合参考配比。
162     # 分别重建并核验两种口径, 不把闭合后的质量静默写回正式 Q0。
163     raw_mix = read_attachment('train_mixture_lm.csv')
164     mix_columns = [name for name in raw_mix if name.startswith('
train_the_pile_')]
165     raw_reference = raw_mix[mix_columns].mean()
166     raw_reference.index = raw_reference.index.str.removeprefix('
train_the_pile_')
167     raw_reference = raw_reference.reindex(mix.domain)
168     if raw_reference.isna().any() or not np.isfinite(raw_reference).all
():
169         raise ValueError('原始平均配比与推荐领域不能一一对应')
170     raw_mass = float(raw_reference.sum())
171     raw_reference_q = float(raw_reference.to_numpy() @ mapped.to_numpy()
)
172     closed_reference = raw_reference.to_numpy()/raw_mass
173     if abs(raw_reference_q-q0) > 1e-10:
174         raise ValueError('共享外生成成本基线与原始平均份额加权值不一致')
175     if not np.allclose(mix.reference_mixture, closed_reference, rtol=0,
atol=1e-10):
176         raise ValueError('核岭参考配比与原始平均份额的闭合结果不一致')
177     if abs(reference_q-raw_reference_q/raw_mass) > 1e-10:
178         raise ValueError('闭合参考质量口径核验失败')
179     if abs(float(mix.recommended_mixture.sum())-1) > 1e-8 or mix.
recommended_mixture.min() < -1e-10:
180         raise ValueError('候选配比不满足概率单纯形约束')

```

```

181 calibration = read_project('求解/问题二/结果/配比项校准.csv')
182 krr_calibration = read_project('求解/问题一/结果/核岭推荐_目标对照.
csv')
183 if list(calibration.mixture) != list(krr_calibration.mixture) or not
np.allclose(
184     calibration[['predicted_weighted_loss', 'M_p']],
185     krr_calibration[['predicted_weighted_loss', 'M_p']], rtol=0,
atol=1e-12):
186     raise ValueError('Q2 配比修正与 Q1 核岭目标对照不一致')
187 gains = calibration.loc[~np.isclose(calibration.M_p, 0), 'M_p']
188 if len(gains) != 1:
189     raise ValueError('需有一个非零核岭候选配比项')
190 m = float(gains.iloc[0])
191 if m >= 0:
192     raise ValueError('本补充的固定收益幅度算例要求现有 M<0')
193 sweep = read_project('求解/问题三/结果/预算扫略.csv').sort_values('C
')
194 saturated = sweep[sweep.Q >= 1-1e-7]
195 if saturated.empty:
196     raise ValueError('原扫描没有达到上界的有效点')
197 sat_budget = float(saturated.iloc[0].C)
198 previous_budget = float(sweep[sweep.C < sat_budget].iloc[-1].C)
199 b1 = read_attachment('pythia_training_log_existing.csv')
200 b6 = read_attachment('supplementary_NQ_experiment.csv')
201 n = np.r_[b1.N_params_B, b6.N_params_B]
202 d = np.r_[b1.D_tokens_B, b6.D_tokens_B]
203 q = np.r_[np.ones(len(b1)), b6.Q_score]
204 y = np.r_[b1.val_loss, b6.val_loss]
205 residual = predict(p, n, d, q)-y
206
207 scale_rows = []
208 for exponent in [.5, 1., 2.]:
209     h0, hp = q0**exponent, exponent*q0**(exponent-1)
210     benefit = p.C*p.gamma*(1-h0)**(p.gamma-1)*hp
211     ncrit = (p.alpha*p.A*(20e9*q0**3)/(K*1e9*benefit))**(1/(p.alpha
+1))
212     dcrit = (p.beta*p.B/(p.alpha*p.A*ncrit**-p.alpha))**(1/p.beta)
213     scale_rows.append(dict(mapping_power=exponent, Q_B_at_A_baseline
=h0, derivative_h=hp, marginal_quality_benefit=benefit,
local_baseline_release_budget=K*ncrit*dcrit*1e18))
214
215 wide = b6.pivot(index=['N_params_B', 'D_tokens_B'], columns='Q_score'
, values='val_loss')
216 if .1 not in wide or 1. not in wide or wide[.1,1:].isna().any().
any():
217     raise ValueError('B6组内质量端点不完整')
218 contrasts = (wide[.1]-wide[1.]).rename('given_loss_reduction').
reset_index()
219 contrasts['additive_model_reduction'] = p.C*.9**p.gamma
220 contrasts['contrast_residual'] = contrasts.additive_model_reduction-
contrasts.given_loss_reduction
221 contrast_summary = dict(groups=len(contrasts), minimum=float(
contrasts.given_loss_reduction.min()), maximum=float(contrasts.
given_loss_reduction.max()), median=float(contrasts.
given_loss_reduction.median()), additive_model=float(p.C*.9**p.gamma)
, spearman_vs_N=float(spearmanr(contrasts.N_params_B, contrasts.

```

```

given_loss_reduction).statistic), spearman_vs_D=float(spearmanr(
contrasts.D_tokens_B, contrasts.given_loss_reduction).statistic))
222
223     def restricted_residual(vector):
224         alt = Parameters(*vector, 1.)
225         return predict(alt, n, d, q)-y
226     fit = least_squares(restricted_residual, p.vector()[:-1], bounds
=([0,1e-8,.01,1e-8,.01,1e-8], [5,1e3,2,1e3,2,50]), loss='soft_l1',
f_scale=.05, max_nfev=3000)
227     if not fit.success or not np.isfinite(fit.x).all():
228         raise RuntimeError('gamma=1受限拟合未成功')
229     alt = Parameters(*[float(x) for x in fit.x], 1.)
230     alt_residual = restricted_residual(fit.x)
231     fit_rows = [dict(case='原参数', gamma=p.gamma, objective=
robust_objective(residual), **errors(residual)), dict(case='gamma固定
1受限拟合', gamma=1., objective=robust_objective(alt_residual), **
errors(alt_residual))]
232     relative_cost = fit_rows[1]['objective']/fit_rows[0]['objective']-1
233     endpoint_rows, endpoint_candidates = [], {}
234     for name, pp in [('原参数', p), ('gamma固定1受限拟合', alt)]:
235         best, candidates = quality_profile(pp, sat_budget, q0)
236         endpoint_rows.append(dict(case=name, **best))
237         endpoint_candidates[name] = candidates
238
239     profile_rows = []
240     for budget in sorted(set([1e19, previous_budget, sat_budget, 1e22]))
:
241         best, _ = quality_profile(p, budget, q0)
242         original = sweep.iloc[np.argmin(np.abs(np.log(sweep.C/budget)))]
243         if not np.isclose(original.C, budget, rtol=1e-9):
244             raise ValueError('抽查预算不在原扫描中')
245         gap = best['L']-float(original.L)
246         if abs(gap) > 1e-7:
247             raise RuntimeError(f'剖面与保存解的损失差需要复核: {gap}')
248         profile_rows.append(dict(**best, saved_L=float(original.L),
saved_Q=float(original.Q), profile_minus_saved_loss=gap))
249
250     base = fixed_quality(p, 1e22, 1., q0)
251     transfer_rows = []
252     for fraction in [0.,.25,.5,.75,1.]:
253         target = base['L']+fraction*m
254         logc = brentq(lambda logc: fixed_quality(p, np.exp(logc), 1., q0
)['L']-target, np.log(1e21), np.log(1e25))
255         budget = float(np.exp(logc))
256         transfer_rows.append(dict(retained_fraction=fraction,
M_transferred=fraction*m, target_loss=target, equivalent_budget=
budget, budget_multiple=budget/1e22))
257     baseline_rows = [dict(case='正式Q3外生成成本基线', Q0=q0, **base),
258                     dict(case='仅将闭合参考质量作为费用基线的对照', Q0=
reference_q, **fixed_quality(p, 1e22, 1., reference_q)),
259                     dict(case='仅将核岭候选质量作为费用基线的对照', Q0=
candidate_q, **fixed_quality(p, 1e22, 1., candidate_q))]
260     official_transfer = read_project('求解/问题三/结果/配比项算力等价.
csv')
261     official_candidate = official_transfer.loc[official_transfer.M_p.ne
(0)]
262     if len(official_candidate) != 1:

```

```

263         raise ValueError('正式Q3配比表须有且仅有一个非零配比项')
264     saved = official_candidate.iloc[0]
265     official_checks = dict(M=float(saved.M_p), target_loss=float(saved.L
266 ), budget_multiple=float(saved.budget_multiplier),
267         independent_target_loss=transfer_rows[-1]['
target_loss'], independent_budget_multiple=transfer_rows[-1]['
budget_multiple'])
268     if abs(float(saved.M_p)-m)>1e-12 or abs(float(saved.L)-transfer_rows
269 [-1]['target_loss'])>1e-7:
270         raise ValueError('正式Q3配比损失与独立代回不一致')
271     if not np.isclose(saved.budget_multiplier, transfer_rows[-1]['
budget_multiple'], rtol=1e-6, atol=1e-8):
272         raise ValueError('正式Q3等效预算与独立反解不一致')
273
274     classic = read_project('求解/问题二/结果/经典标度律参数.csv').iloc
[0]
275     classic_pred = classic.E + classic.A*b1.N_params_B**-classic.alpha +
276     classic.B*b1.D_tokens_B**-classic.beta
277     metric_rows = [dict(case='经典全量', rows=len(b1), **errors(np.
asarray(classic_pred-b1.val_loss))), dict(case='广义联合', rows=len(y
), **errors(residual))]
278     try:
279         revision = subprocess.check_output(['git','rev-parse','HEAD'],
cwd=PROJECT, stderr=subprocess.DEVNULL).decode().strip()
280     except (OSError, subprocess.CalledProcessError):
281         revision = None
282     summary = dict(source_revision=revision, scope='独立补充诊断, 不替换
主模型参数或正式结果; 不构成统计置信区间', tool_record=dict(assistant
='OpenAI Codex', purpose='审稿补充代码整理与核验', recorded_date='
2026-09-27'), created_utc=datetime.now(timezone.utc).isoformat(),
parameters=asdict(p), fixed_gamma_parameters=asdict(alt),
Q0_reference=q0, Q0_candidate=candidate_q,
quality_scale_counterexample=scale_rows, B6_contrast=contrast_summary
, gamma_restricted=dict(success=bool(fit.success), nfev=int(fit.nfev)
, objective_relative_change=relative_cost, fit_rows=fit_rows,
endpoint_budget=sat_budget, endpoint_profiles=endpoint_rows,
endpoint_candidates=endpoint_candidates), saved_solution_checks=
profile_rows, M_amplitude=transfer_rows, baseline_sensitivity=
baseline_rows, original_model_metrics=metric_rows, input_sha256=
inputs, script_sha256=sha256(Path(__file__)), versions=dict(python=
platform.python_version(), numpy=np.__version__, pandas=pd.
__version__, scipy=scipy.__version__))
283
284     summary['revision_note'] = 'source_revision 是读取时的 Git HEAD; 本
补充脚本可尚未提交, 其精确身份由 script_sha256 标识。'
285     summary['quality_baselines'] = dict(Q0_official_exogenous=q0,
raw_reference_mass=raw_mass,
286         Q_raw_reference=raw_reference_q,
Q_closed_reference=reference_q,
287         Q_krr_candidate=candidate_q,
closed_minus_official=reference_q-q0,
288         policy='正式Q3保持共享外生Q0; 闭
合参考质量与候选质量仅作另列成本基线对照')
289     summary['M_source'] = 'Q1 核岭推荐_目标对照.csv, 经 Q2 配比项校准.
csv 传递'
290     summary['official_transfer_checks'] = official_checks
291     # 输入在本次诊断前后保持不变; 程序只在独立输出目录写入自己的文件。

```

```

289     for spec, digest in inputs.items():
290         kind, relative = spec.split(':', 1)
291         path = (PROJECT if kind=='project' else data_dir)/relative
292         if sha256(path) != digest:
293             raise RuntimeError(f'运行期间输入发生变化: {spec}')
294     output.mkdir(parents=True, exist_ok=True)
295     frames = {'质量尺度反例.csv':pd.DataFrame(scale_rows), 'B6组内质量改善.csv':contrasts, 'gamma固定1_参数.csv':pd.DataFrame({'param':PARAMETER_NAMES, 'original':p.vector(), 'restricted':alt.vector()}), 'gamma固定1_拟合对照.csv':pd.DataFrame(fit_rows), 'gamma固定1_临界预算对照.csv':pd.DataFrame(endpoint_rows), '原解剖面抽查.csv':pd.DataFrame(profile_rows), '配比收益幅度敏感性.csv':pd.DataFrame(transfer_rows), '配比质量基线对照.csv':pd.DataFrame(baseline_rows), '原模型误差复核.csv':pd.DataFrame(metric_rows)}
296     for name, frame in frames.items():
297         frame.to_csv(output/name, index=False, encoding='utf-8-sig')
298     (output/'q23_核验.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding='utf-8')
299     (output/'README.md').write_text(''# Q2/Q3 审稿补充结果
300
301 运行 `python 求解/审稿补充/q23_尺度与结构敏感性.py --data-dir <附件目录>`。
302 可添加 `--output-dir <项目外独立目录>`；在项目内仅允许写本目录。
303
304 - 质量尺度表固定A评分轴上的题设成本，改变损失轴转换；给出基线一阶释放点，不宣称全局转移阈值。
305 - B6表是相同N、D内Q从0.1提高到1的给定损失差，不是新增真实训练实验。
306 - gamma=1仅受限重估其他6个参数，用相同soft-L1目标比较，不替代原参数，也不构成置信区间。
307 - 剖面抽查只核对少量预算的端点与局部候选，不能证明所有预算全局最优。
308 - 配比幅度表在Q=1、现有费用关系下反解预算，幅度比例为假设；不代表实测节约。
309 - 配比基线表区分正式Q3外生Q0、核岭闭合参考质量、核岭候选质量；后两项仅改变费用基线作对照，正式Q3不变。
310 - 原模型误差表只用保存参数代回数据，无额外主模型拟合。
311
312 输入及脚本SHA-256、版本、求解状态和全精度数值见q23_核验.json。原模型、附件、参数和原结果不被写入。
313 '''， encoding='utf-8')
314     print(json.dumps(dict(output=str(output), B6_groups=len(contrasts), restricted_objective_increase_percent=100*relative_cost, Q_at_original_saturation=endpoint_rows[1]['Q'], profile_checks=len(profile_rows), original_metrics=metric_rows, input_files_unchanged=len(inputs)), ensure_ascii=False, indent=2))
315
316 if __name__ == '__main__':
317     main()

```

程序 25: 年份与前沿窗口诊断（求解/审稿补充/q4_时间与前沿验证.py）

1 """问题四的事后时间诊断，不替换论文原模型或原结果。

2

3 用法: python q4_时间与前沿验证.py --data /path/to/real_attachments

```

4 仅读取附件和问题四已有结果；默认写入本目录的 q4 结果。
5 改变前沿点权重/时间窗口仅用于敏感性检查，不是新选定的预测方案。
6 """
7 from pathlib import Path
8 import argparse
9 import hashlib
10 import json
11 import os
12
13 import numpy as np
14 import pandas as pd
15 import scipy
16 from scipy.optimize import curve_fit
17
18 HERE = Path(__file__).resolve().parent
19 PAPER_RESULT = HERE.parent / '问题四' / '结果'
20 TASKS = ['IFEval', 'BBH', 'MATH Lvl 5', 'GPQA', 'MUSR', 'MMLU-PRO']
21 LICENSES = {'apache-2.0', 'mit', 'gpl-3.0', 'cc-by-4.0', 'mpl-2.0',
22             'bsd-3-clause', 'llama2', 'llama3', 'llama3.1', 'llama3.2',
23             'llama3.3', 'gemma', 'cc-by-sa-4.0', 'openrail'}
24
25 def digest(path):
26     return hashlib.sha256(path.read_bytes()).hexdigest()
27
28 def result_hashes():
29     return {str(p.relative_to(PAPER_RESULT)): digest(p)
30             for p in PAPER_RESULT.rglob('*') if p.is_file()}
31
32 def logistic(t, K, r, t0):
33     return K / (1 + np.exp(-r * (t - t0)))
34
35 def fit_curve(t, y):
36     # 沿用原拟合的函数、初值、参数界限；未重新搜索超参数。
37     p, _ = curve_fit(logistic, t, y, p0=[55, .8, 3.5],
38                     bounds=([20, .01, .5], [100, 5, 8]), maxfev=40000)
39     residual = y - logistic(t, *p)
40     r2 = 1 - np.sum(residual**2) / np.sum((y - y.mean())**2)
41     return p, float(r2)
42
43 def ols(df, time_column='year_t', include_type=False):
44     X = np.column_stack([np.ones(len(df)), df.logP, df[time_column]])
45     if include_type:
46         X = np.column_stack([X, pd.get_dummies(df.type_group, drop_first=True, dtype=float)])
47     beta = np.linalg.lstsq(X, df.avg, rcond=None)[0]
48     fitted = X @ beta
49     return {'n': len(df), 'b_P': beta[1], 'time_coefficient': beta[2],
50           'R2': 1 - np.sum((df.avg-fitted)**2)/np.sum((df.avg-df.avg.mean())**2)}
51
52 def prepare(data):
53     src = data / 'C_efficiency_evolution'
54     lb = pd.read_csv(src / 'leaderboard_cleaned.csv')
55     lb['P'] = pd.to_numeric(lb['#Params (B)'], errors='coerce')
56     lb['date'] = pd.to_datetime(lb['Submission Date'], errors='coerce')
57     lb['avg'] = pd.to_numeric(lb['Average \u2b06\ufe0f'], errors='coerce')
58     return lb

```

```

58 lb[TASKS] = lb[TASKS].apply(pd.to_numeric, errors='coerce')
59 good = (lb[TASKS].notna().all(axis=1) & lb.date.notna() & lb.P.notna()
60         & lb['Hub License'].astype(str).str.strip().str.lower().isin(
(LICENSES)))
61 core = lb[good].copy()
62 if not (core.P.gt(0).all() and core.avg.notna().all()):
63     raise ValueError('C1 出现不满足原回归定义参数或均分，须先核对
原样本口径。')
64 core['Year'] = core.date.dt.year
65 core['year_t'] = core.Year - 2019
66 core['continuous_t'] = (core.date-pd.Timestamp('2019-01-01')).dt.
total_seconds()/86400/365.25
67 core['logP'] = np.log10(core.P)
68 pre = ['\U0001f7e2 pretrained', '\U0001f7e9 continuously pretrained'
]
69 chat = ['\U0001f4ac chat models (RLHF, DP0, IFT, ...)', '\U0001f536
fine-tuned on domain-specific datasets']
70 core['type_group'] = np.where(core.Type.isin(pre), 'pretrained',
71                               np.where(core.Type.isin(chat), 'chat/
finetuned', '其他'))
72 core['month'] = core.date.dt.to_period('M')
73 monthly = []
74 for month, g in core.groupby('month', sort=True):
75     if month < pd.Period('2024-01'):
76         continue
77     n_top = max(1, int(len(g)*.01))
78     monthly.append({'source': 'C1', 'period': str(month), 'n_records
': len(g),
79                    'n_top': n_top, 't': month.year-2019+(month.
month-.5)/12,
80                    'raw_frontier': float(g.avg.nlargest(n_top).mean
())})
81 mon = pd.DataFrame(monthly)
82 mon['cumulative_frontier'] = mon.raw_frontier.cummax()
83
84 ts = pd.read_csv(src / 'leaderboard_extended_timeseries.csv')
85 ts['avg'] = pd.to_numeric(ts.Average, errors='coerce')
86 ts['P'] = pd.to_numeric(ts.Params_B, errors='coerce')
87 six = ['IFEval', 'BBH', 'MATH_Lvl5', 'GPQA', 'MUSR', 'MMLU_PRO']
88 ts[six] = ts[six].apply(pd.to_numeric, errors='coerce')
89 ts['gap'] = (ts.avg-ts[six].mean(axis=1)).abs()
90 valid = ts[(ts.gap <= 5) & ts.P.notna() & ts.avg.notna() & ts.P.gt
(0)].copy()
91 valid = valid[np.isfinite(np.log10(valid.P)) & np.isfinite(valid.avg
)]
92 annual = valid.groupby('Year').agg(raw_frontier=('avg', 'max'),
n_records=('avg', 'size')).reset_index()
93 annual['cumulative_frontier'] = annual.raw_frontier.cummax()
94 hist = annual[annual.Year <= 2023].copy()
95 hist['source'] = 'C3'
96 hist['period'] = hist.Year.astype(int).astype(str)
97 hist['n_top'] = 1
98 hist['t'] = hist.Year-2019+.5
99 columns = ['source', 'period', 'n_records', 'n_top', 't', '
raw_frontier', 'cumulative_frontier']
100 frontier = pd.concat([hist[columns], mon[columns]], ignore_index=
True).sort_values('t')

```

```

101     frontier['cumulative_frontier'] = frontier.cumulative_frontier.
cummax()
102     return core, valid, frontier.reset_index(drop=True)
103
104 def verify_scale_pairing(core, c3):
105     """从原始候选集合核对已保存配对及分解，不调用或改写正式求解程序。"""
106     annual = pd.read_csv(PAPER_RESULT/'前沿序列.csv')
107     for row in annual.itertuples(index=False):
108         past = c3[c3.Year <= row.Year]
109         winners = past[past.avg.eq(past.avg.max())].sort_values(['Year',
'P', 'Model'])
110         expected = winners.iloc[0]
111         assert row.frontier_model == expected.Model
112         assert row.frontier_record_year == expected.Year
113         assert np.allclose([row.P, row.cum_avg, row.avg],
114                             [expected.P, expected.avg, c3.loc[c3.Year.eq(
row.Year), 'avg'].max()])
115     # 独立回算横截面规模斜率，并逐列核对新配对定义下的保存结果。
116     X = np.column_stack([np.ones(len(c3)), np.log10(c3.P), c3.Year-
2019])
117     b_long = np.linalg.lstsq(X, c3.avg, rcond=None)[0][1]
118     long_gain = annual.cum_avg.iloc[-1]-annual.cum_avg.iloc[0]
119     long_logp = np.log10(annual.P.iloc[-1]/annual.P.iloc[0])
120     monthly = []
121     for month, group in core.groupby('month', sort=True):
122         selected = group.nlargest(max(1, int(len(group)*.01)), 'avg')
123         monthly.append((str(month), float(selected.avg.mean()), float(np
.log10(selected.P).mean()))))
124     first = monthly[0]
125     peak = max(monthly, key=lambda item: item[1])
126     near_gain, near_logp = peak[1]-first[1], peak[2]-first[2]
127     saved = pd.read_csv(PAPER_RESULT/'规模时间分解.csv')
128     details = {}
129     for i, (key, gain, logp, bp) in enumerate([
130         ('C3', long_gain, long_logp, b_long),
131         ('C1', near_gain, near_logp, ols(core)['b_P']),
132     ]):
133         scale = bp*logp
134         values = [gain, logp, bp, scale, gain-scale, scale/gain*100, (
gain-scale)/gain*100]
135         columns = ['前沿提升分', 'Δlog10P', 'bP', '规模贡献分', '非规模
贡献分', '规模占比%', '非规模占比%']
136         assert np.allclose(saved.iloc[i][columns].astype(float), values,
rtol=1e-9, atol=1e-9)
137         details[key] = dict(zip(columns, map(float, values)))
138     long_saved = pd.read_csv(PAPER_RESULT/'规模时间分解_长周期.csv')
139     assert np.allclose(long_saved[columns].to_numpy(float), saved.loc
[[0], columns].to_numpy(float))
140     return {'annual_record_pairing_verified': True,
141             'C3_rule': '截至当年最高分；并列依次取较早年份、较小参数、模
型名',
142             'C1_rule': '当月前1%同组平均分与平均对数参数量',
143             'saved_decomposition_verified': True, 'decomposition':
details}
144
145 def main():
146     ap = argparse.ArgumentParser(description=__doc__)

```

```

147     ap.add_argument('--data', '--data-dir', dest='data', type=Path,
148     default=os.environ.get('MODELING_DATA_DIR'))
149     ap.add_argument('--output', '--output-dir', dest='output', type=Path
150     , default=HERE/'q4_结果')
151     args = ap.parse_args()
152     if args.data is None:
153         ap.error('请用 --data 或 MODELING_DATA_DIR 指定 real_attachments
154         。')
155     data, output = Path(args.data).resolve(), args.output.resolve()
156     if output.is_relative_to(data) or output.is_relative_to(PAPER_RESULT
157     .resolve()):
158         ap.error('输出目录不能位于原始附件或问题四原结果目录内。')
159     before = result_hashes()
160     core, c3, frontier = prepare(data)
161     scale_pairing = verify_scale_pairing(core, c3)
162     t = frontier.t.to_numpy(float)
163     y = frontier.cumulative_frontier.to_numpy(float)
164     if len(t) != 14 or not np.isclose(t[-1], 2025-2019+(3-.5)/12):
165         raise ValueError('本诊断针对截至2025年3月的14点序列；数据窗口变
166         化后须重新确定留出期与表头。')
167     future = np.array([t[-1]+1, t[-1]+2])
168     original, original_r2 = fit_curve(t, y)
169     saved = pd.read_csv(PAPER_RESULT/'logistic参数.csv').iloc[0]
170     saved_forecast = pd.read_csv(PAPER_RESULT/'前沿预测.csv')
171     if not np.allclose(original, saved[['K','r','t0(2019起)']].astype(
172     float), rtol=2e-5, atol=1e-5):
173         raise ValueError('重建的原曲线与论文结果不一致，停止生成诊断。')
174     if not np.allclose(logistic(future,*original), saved_forecast.
175     frontier_pred, rtol=2e-5, atol=1e-5):
176         raise ValueError('重建的预测时点/原预测值不一致。')
177
178     regressions = []
179     for name, frame, time_col, typed in [
180         ('年份标签 (原规格)', core, 'year_t', False),
181         ('按名称保留首条', core.drop_duplicates('Model'), 'year_t', False),
182         ('按名称保留末条', core.drop_duplicates('Model', keep='last'), '
183         year_t', False),
184         ('年份标签并控制类型', core, 'year_t', True),
185         ('实际提交日连续年数', core, 'continuous_t', False),
186     ]:
187         regressions.append({'diagnostic': name, *ols(frame, time_col, typed
188         )})
189
190     variants = [
191         ('原14点', np.ones(len(t), bool)),
192         ('仅保留C1月度点', frontier.source.eq('C1').to_numpy()),
193         ('去掉末3个月', np.arange(len(t))<len(t)-3),
194         ('相同累计值仅留首次', np.r_[True, np.diff(y)!=0]),
195     ]
196     windows = []
197     for name, mask in variants:
198         p, r2 = fit_curve(t[mask], y[mask])
199         pred = logistic(future,*p)
200         bound = ('K上界' if np.isclose(p[0], 100, atol=1e-6) else
201         'r上界' if np.isclose(p[1], 5, atol=1e-6) else '无')
202         windows.append({'diagnostic': name, 'n': int(mask.sum()), 'K': p[0], '
203         r': p[1], 't0': p[2],

```

```

194         'training_R2':r2,'value_2026_03':pred[0], '
value_2027_03':pred[1],
195         'active_bound':bound})
196
197     cutoff = 2024-2019+(9-.5)/12
198     train = t <= cutoff+1e-9
199     p, _ = fit_curve(t[train],y[train])
200     hold = frontier.loc[~train,['period','t','cumulative_frontier']].
copy()
201     hold['logistic_prediction'] = logistic(t[~train],*p)
202     hold['last_peak_prediction'] = y[train][-1]
203     metrics = []
204     for name,col in [('logistic条件曲线','logistic_prediction'),('保持最后
累计峰值','last_peak_prediction')]:
205         err = hold[col]-hold.cumulative_frontier
206         metrics.append({'method':name,'training_cutoff':'2024-09', '
holdout':'2024-10至2025-03',
207                         'n':len(hold),'RMSE':float(np.sqrt(np.mean(err**
2)))),'MAE':float(np.mean(np.abs(err)))})
208
209     mean_dates = core.groupby('Year').date.mean()
210     summary = {
211         'purpose':'事后诊断，不替代论文原模型、原结果或点预测；单次留出
窗口恰为累计平台',
212         'n_C1':len(core),'n_C3_valid':len(c3),'unique_exact_names':int(
core.Model.nunique()),
213         'names_in_multiple_years':int((core.groupby('Model').Year.
nunique(>1).sum()),
214         'mean_submission_dates':{str(k):str(v) for k,v in mean_dates.
items()},
215         'mean_date_gap_years':float(core.groupby('Year').continuous_t.
mean().diff().iloc[-1]),
216         'origin_observed_cumulative':float(y[-1]),'
origin_logistic_fitted':float(logistic(t[-1],*original)),
217         'original_R2':original_r2,'original_result_verified':True,
218         'scale_pairing':scale_pairing,
219         'curve_inputs':'C3年度累计最高分与C1月度前1%均值累计最大值；未作
历史量表校准',
220         'diagnostic_limits':'删点改变权重和时间窗口；单次留出为事后检
查，不能用于宣称基线普遍优越',
221         'versions':{'numpy':np.__version__,'pandas':pd.__version__,'
scipy':scipy.__version__},
222         'original_result_sha256':before,
223         'source_hashes':{'name:digest(data/'C_efficiency_evolution'/name)
for name in
224                         ['leaderboard_cleaned.csv','
leaderboard_extended_timeseries.csv']},
225     }
226     output.mkdir(parents=True,exist_ok=True)
227     for name,df in [('q4_横截面诊断',pd.DataFrame(regressions)),('q4_窗
口敏感性',pd.DataFrame(windows)),
228                     ('q4_时间留出',pd.DataFrame(metrics)),('q4_时间留出
明细',hold),('q4_前沿诊断序列',frontier)]:
229         df.to_csv(output/(name+'.csv'),index=False,encoding='utf-8-sig')
230     if before != result_hashes():
231         raise RuntimeError('问题四原结果文件发生变化。')
232     summary['original_results_unchanged'] = True

```

```

233     (output/'q4_诊断摘要.json').write_text(json.dumps(summary,
ensure_ascii=False,indent=2),encoding='utf-8')
234     print(pd.DataFrame(regressions).to_string(index=False))
235     print(pd.DataFrame(windows).to_string(index=False))
236     print(pd.DataFrame(metrics).to_string(index=False))
237     print('诊断完成; 原曲线一致, 原结果文件未改动。')
238
239 if __name__ == '__main__':
240     main()

```

程序 26: 总体流程图原生源码 (论文/figures/overall-architecture.tex)

```

1 % 总体架构: 横向箭头对应每问的数据、方法和产出。
2 % 不在此图中表达尚未核实的跨问数值校准关系。
3 \begin{group}
4 \definecolor{archblue}{HTML}{315A79}
5 \definecolor{archlight}{HTML}{EDF3F7}
6 \definecolor{archgray}{HTML}{536572}
7 \begin{tikzpicture}[
8     font=\song\fontsize{10}{13}\selectfont,
9     archbox/.style={draw=archblue!55, line width=0.55pt, rounded corners=2
10         pt,
11         fill=archlight, align=center, inner sep=7pt, minimum height=1.55cm},
12     archdata/.style={archbox, text width=2.55cm},
13     archmodel/.style={archbox, text width=5.1cm},
14     archoutput/.style={archbox, text width=3.3cm, fill=white},
15     archlink/.style={-{Stealth[length=2mm]}}, draw=archblue, line width=0.7
16         pt},
17     archhead/.style={text=archblue, font=\song\bfseries\fontsize{11}{14}\
18         selectfont}
19 ]
20 \node[archhead] at (-5.8,0.65) {数据与条件};
21 \node[archhead] at (-0.8,0.65) {建模任务与方法};
22 \node[archhead] at (4.8,0.65) {主要产出};
23
24 \node[archdata] (a) at (-5.8,-0.75) {附件 A\\质量信号\\配方与验证损失};
25 \node[archmodel, fill=archblue, text=white] (q1) at (-0.8,-0.75)
26 {问题一: 数据质量与领域配比\\熵权评价、冲突分析、岭与核岭};
27 \node[archoutput] (o1) at (4.8,-0.75)
28 {综合质量评分\\配比与损失关系\\核岭支持域推荐};
29
30 \node[archdata] (b) at (-5.8,-3.0) {附件 B\\规模、训练量\\与质量信息};
31 \node[archmodel] (q2) at (-0.8,-3.0)
32 {问题二: 损失与资源的关系\\广义标度律、边际效用与弹性};
33 \node[archoutput] (o2) at (4.8,-3.0)
34 {损失函数与参数\\资源之间的替代关系};
35
36 \node[archdata] (c) at (-5.8,-5.25)
37 {算力预算\\题给成本设定\\C7 上下文信息};
38 \node[archmodel] (q3) at (-0.8,-5.25)
39 {问题三: 算力约束下的配置\\成本建模与约束优化};
40 \node[archoutput] (o3) at (4.8,-5.25)
41 {最优配置  $(N,D,Q)$ \\预算变化下的\\配置特征};
42
43 \node[archdata] (d) at (-5.8,-7.5)
44 {附件 C\\历史模型\\与评测信息};
45 \node[archmodel] (q4) at (-0.8,-7.5)

```

```

43 {问题四：能力演进与前沿预测\\损失与评测桥接、回归与外推};
44 \node[archoutput] (o4) at (4.8,-7.5)
45 {规模关联项与剩余项\\前沿条件预测与分位范围};
46
47 \foreach \archsrc/\archmid/\archdst in {a/q1/o1,b/q2/o2,c/q3/o3,d/q4/o4}
48 {
49 \draw[archlink] (\archsrc.east) -- (\archmid.west);
50 \draw[archlink] (\archmid.east) -- (\archdst.west);
51 }
52 \end{tikzpicture}
53 \endgroup

```

附录 F：问题三图表的计算公式与口径

图21–图23的坐标和费用份额由保存的求解结果计算。下文沿用正文符号： N, D 分别为参数个数和 token 数， C_b 的单位为 FLOPs， $g(Q)$ 的单位为 FLOPs/token； Q_0 为参考配比加权质量基线。图20为方法流程示意，不对应额外的数值推导。

1. 预算与费用份额图

图21在 $L_{\text{ctx}} = 2048$ 、 $M = 0$ 和幂函数型成本下，逐个预算点求解正式式(36)，得到 (N^*, D^*, Q^*) 。图中的质量完成度和质量处理份额分别按

$$\Theta = \frac{Q^* - Q_0}{1 - Q_0}, \quad f_Q = \frac{D^* [g(Q^*) - g(Q_0)]}{C_b} \quad (53)$$

计算。基础训练和注意力份额为

$$f_{\text{base}} = \frac{6N^*D^*}{C_b}, \quad f_{\text{attn}} = \frac{\eta N^*D^*L_{\text{ctx}}}{C_b}. \quad (54)$$

将式(35)的三项成本分别除以预算，即得上述份额，其和等于 C_{tot}/C_b ；当前预算扫描配置用尽预算，因而该和在数值误差范围内等于 1。纵轴按百分数显示 100Θ 及三类份额的 100 倍。“预算扫描.csv”的 141 条记录均有有效配置，图中标记对应这些记录；启动点和饱和点是在该离散预算网格上识别的首次满足 $Q > Q_0 + 10^{-7}$ 和 $Q \geq 1 - 10^{-7}$ 的记录，容差与正式绘图程序一致。

2. 成本形式比较图

图22读取“成本函数对比.csv”中三种成本形式与三档预算的九组求解记录，各组均固定 $L_{\text{ctx}} = 2048$ 、 $M = 0$ 。左图柱高为 Q^* ，质量投入柱高为 $100f_Q$ ；其中 f_Q 仍按式(53)计算。三种成本函数为正文表19中的 $g(Q)$ ，其差异通过预算约束影响数值候选配置。

3. 上下文敏感性图

图23固定幂函数型成本和 $M = 0$ ，在每档预算下分别改变 L_{ctx} 并求解配置。注意力成本与基础训练成本具有相同的 ND 因子，消去后得到

$$\lambda = \frac{C_{\text{attn}}}{C_{\text{base}}} = \frac{\eta ND L_{\text{ctx}}}{6ND} = \frac{\eta L_{\text{ctx}}}{6}, \quad L_{\text{crit}} = \frac{6}{\eta} = 30,000 \text{ tokens}. \quad (55)$$

令 $\lambda = 1$ 即得图中竖线所示的 L_{crit} 。左图预算为 10^{22} FLOPs，使用“上下文长度连续扫描.csv”中的 L_{ctx} 与 $N^*/10^9$ （字段 N_B ，单位为十亿个参数）；右图在 10^{19} 、 10^{20} 、 10^{21} FLOPs 三档预算下，使用“上下文长度与最优质量.csv”中的 L_{ctx} 与 Q^* 。C7 的五个离散档位来自“上下文长度敏感性.csv”，作为方形标记叠加在左图中。当前连续扫描包含 40 个有效点，右图每档预算均有 14 个有效点，五档配置也均有有效输出。折线仅连接计算点以辅助读图，不表示区间内另有求解结果。

上述公式只用于把已保存的求解结果转换为图中坐标或份额，图表不构成新的实验观测；输入文件、生成程序及复现命令见附录 E 的表42和代码清单。